Lectures on Quantum Computing and Completely Positive Maps

A formalisation in Isabelle/HOL of Vern Paulsen's Houston 2011 lecture notes

Dustin Bryant

formal statements and proofs: the theories listed in the table of contents

October 23, 2026

Abstract

This document is generated by Isabelle from the project theories. Every definition and theorem shown here has been checked by Isabelle/HOL (proofs are folded). The commentary restates Paulsen's lecture notes lecture by lecture (Day 1 = 22 August 2011, Day 34 = 18 November 2011): definitions and results in the notes' own language, each followed by the formal statement. The informal proofs of the notes are omitted; the formal proofs replace them. Where the formalisation deviates from the notes, the commentary says so. The companion file `NOTES_AUDIT.md` lists, for every lecture, what is formalised and what is not. Two companion entries build on it: the functional calculus of Hermitian matrices with product formulas, and quantum phase estimation with the HHL algorithm.

Contents

1 Reading guide **3**

2 Conventions **5**

I Lectures 1–10: basic mathematics and the postulates of quantum mechanics **5**

3 Preliminaries: linear-space isomorphisms **6**

4 Day 1 (22 August 2011): the inner product **7**
4.1 The Hilbert–Schmidt inner product on matrices 7
4.2 Properties of the inner product 8

5 Day 2 (24 August 2011): bases, matrix units, trace, projections **9**
5.1 The trace 10
5.2 Matrix-unit algebra and cyclicity of the trace 11
5.3 Orthogonal projections and Parseval's identity 12

6 Day 2: the projection does not depend on the orthonormal basis **15**

7 Day 3 (31 August 2011): unitary matrices, Householder unitaries, Schur's theorem **16**
7.1 Unitary matrices 16
7.2 Householder unitaries 17

8 Day 4 (2 September 2011): Hermitian matrices; positive semidefinite and positive definite matrices **23**
8.1 Hermitian matrices . . . 23
8.2 The spectral theorem for Hermitian matrices . . . 25
8.3 Positive semidefinite and positive definite matrices . . . 26
8.4 The positive square root . . . 29

9 Day 4: the converse directions for Hermitian matrices (D4-1) **30**

10 Day 5 (7 September 2011): tests for positivity; direct sums and partitioned matrices **30**
10.1 Practical tests for positivity . . . 30
10.2 Direct sums of vector spaces and partitioned matrices . . . 31

11 Day 5: Cholesky's algorithm on the notes' two examples (D5-3) **32**
11.1 One step of Cholesky's algorithm . . . 32
11.2 Small matrices entry by entry . . . 32
11.3 Example 1: a matrix that is not positive . . . 33
11.4 Example 2: a positive matrix, in two steps . . . 33

12 Day 5: a basis of $V \oplus W$ (D5-5) **35**

13 Day 6 (9 September 2011): tensor products **35**

14 Day 6: coordinates in a tensor product **37**

15 Day 6: the Schmidt rank (D6-4) **39**

16 Day 6 and Day 15: the universal properties in the Kronecker model **40**

17 Day 7 (12 September 2011): tensor products of Hilbert spaces; the postulates **43**
17.1 Postulates of quantum mechanics . . . 44

18 Day 8 (14 September 2011): the quantum game; projective measurements and expected values **45**
18.1 Distinguishing states . . . 45
18.2 Projective measurements, expected values and self-adjoint operators . . . 47

19 Day 8: non-orthogonal states cannot be distinguished **48**

20 Day 9 (16 September 2011): positive operator-valued measures; composite systems; entanglement **49**
20.1 Positive operator-valued measures . . . 49
20.2 Composite systems . . . 50
20.3 Entanglement . . . 50

21 Day 9: measuring one half of a Bell pair **51**

22 Day 10 (19 September 2011): superdense coding; classical and quantum gates **52**
22.1 One-qubit gates . . . 52
22.2 Two- and three-qubit gates . . . 53
22.3 Superdense coding . . . 54

23 Day 10: superdense coding (D10-1) **55**

II Lectures 11–22: quantum computing, density matrices, completely positive maps **56**

24 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning **56**
24.1 Cloning and no-cloning . . . 57

25 Day 11: SWAP from three CNOTs **58**

26 Day 11: the CCNOT truth table (D11-2) **58**

27 Day 12 (23 September 2011): quantum parallelism **59**

28 Day 12: $H^{\otimes N}$ on $|0\cdots 0\rangle$ and the counting probability (D12-1, D12-4) **60**

29 Day 13 (26 September 2011): ensembles and density matrices **61**
29.1 Ensembles . . . 61
29.2 Von Neumann's density matrix . . . 62
29.3 Probabilities via the trace . . . 63

30 Day 14 (28 September 2011): density matrices, pure states, composite ensembles **64**
30.1 Pure states . . . 65
30.2 Composite ensembles . . . 66
30.3 Partial traces: motivation . . . 66

31 Day 14: composite ensembles (D14-5) **66**

32 Day 15 (30 September 2011): traces, tensor products of operators, partial traces **67**
32.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ . . . 67
32.2 Partial traces . . . 70
32.3 Partial traces and measurements . . . 72

33 Day 15: the partial trace over A (D15-6) **72**

34 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices **73**

35 Day 16: $H_A \otimes H_B \cong H_A^{(p)}$ and $L(H_A \otimes H_B) \cong M_p(L(H_A))$ (D16-2) **74**

36 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps **76**
36.1 Measurement maps . . . 76
36.2 Kraus maps, positive maps, complete positivity . . . 77

37 Day 17: the environment model of noise **81**

38 Day 18 (7 October 2011): the theory of completely positive maps **82**

39 Days 17–18: densities to densities iff positive and trace-preserving (D17-4, D18-2) **83**

40 Day 18: the transpose is positive but not 2-positive (D18-4) **83**

41 Day 19 (10 October 2011): Choi's theorem **84**
41.1 From a positive Choi matrix to a Kraus representation . . . 85

42 Day 19: Choi's theorem with (2) isolated (D19-1) **88**

43 Day 19: trace preservation and unitality in Choi-matrix form (D19-3) **89**

44 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank **90**
44.1 The adjoint of a Kraus map . . . 90
44.2 Complete positivity and the UCP/CPTP duality . . . 92
44.3 Positive functionals are completely positive . . . 92
44.4 Choi rank . . . 93

45 Day 20: Choi's rank theorem $\mathrm{cr}(\Phi) = \mathrm{rank}\, P_\Phi$ (D20-4) **93**
45.1 $\mathrm{rank}(\sum_{l<L} w_l w_l^*) \leq L$ . . . 93
45.2 Independent vectors inside the column space bound the rank from below . . . 93
45.3 The theorem . . . 94

46 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations **94**

47 Day 21: unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6) **95**
47.1 The Gram matrix of an independent list is invertible; dual vectors . . . 95
47.2 D21-3: independent vectors give independent rank-one matrices . . . 96
47.3 Isometric mixing of columns preserves VV^* . . . 96
47.4 D21-5: two rank-one decompositions are related by an isometry . . . 96
47.5 D21-6 (Choi 2): unitary freedom of Kraus representations . . . 97

48 Day 21: ranges of positive matrices (D21-4) **97**
48.1 Day 4 revisited: positive definiteness and spanning (D4-8) . . . 98

49 Day 21: the conjecture on averages of unital maps (D21-2) **98**
49.1 Positive matrices as sums of d rank-one terms . . . 100
49.2 Measurement maps . . . 100
49.3 The case $d = 1$: Horn's lemma . . . 101
49.4 Maps with diagonal values . . . 101
49.5 The notes' form (i) is false as written . . . 101

50 Day 21: the conjecture D21-2 for $n = 2$ **101**
50.1 Completing an orthonormal family to an orthonormal basis . . . 102
50.2 Singular value decomposition . . . 102
50.3 A contraction is the average of two unitaries . . . 102
50.4 Bessel's inequality for rows that are orthonormal or zero . . . 102
50.5 Factoring a wide matrix . . . 103
50.6 The case $n = 2$ . . . 103

51 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP **104**
51.1 Convex sets and extreme points . . . 104
51.2 The convex sets UCP and CPTP . . . 105

52 Day 22: linear independence bounds the number of Kraus operators (D22-6) **106**

53 Day 22: a minimal Kraus representation is linearly independent (D22-1) 107
53.1 Mixing a Kraus list by an isometry does not change the map . . . 107
53.2 A unitary with a prescribed first row . . . 107
53.3 D22-1 . . . 108

54 Day 22: the adjoint map, an affine bijection from UCP onto CPTP (D22-5) 108

55 Day 22: $\mathrm{UCP}(M_n, M_d)$ is compact (D22-4) 110

III Lectures 23–28: extreme points, operator systems, Arveson's theorem, entanglement 111

56 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP 112
56.1 Extreme UCP maps . . . 112
56.2 UCP-Kraus representation . . . 113
56.3 Unital condition from the Kraus list . . . 114
56.4 UCP-Kraus existence . . . 114
56.5 Sanity facts about *kraus-products* . . . 114
56.6 Quadratic Kraus map (perturbation primitive) . . . 115
56.7 Lemma A: PSD coefficient matrix gives a CP perturbation . . . 118
56.8 Small-ε PSD perturbation of the identity . . . 119
56.9 Symmetric / anti-hermitian split of the coefficient matrix . . . 119
56.10 Bridge: ¬ *mat-lin-indep-real* gives a real *A-orig* witness . . . 121
56.11 Auxiliary lemmas for the assembly . . . 122
56.12 Minimality and the distinctness lemma (Lemma B) . . . 123
56.13 CPTP-Kraus representation and extreme CPTP maps . . . 124
56.14 Vectorisation of Kraus operators . . . 125
56.15 Choi matrix of a Kraus map . . . 125
56.16 Choi matrix is linear in the map . . . 125
56.17 The Choi matrix of a *kraus-quad-map* . . . 126
56.18 CP Radon–Nikodym — the dominated-PSD kernel . . . 127
56.18.1 Diagonal pseudo-inverse list and its identity . . . 127
56.19 Linearity of *kraus-quad-map* in its matrix argument . . . 129
56.20 Choi matrix determines a linear CP map on carrier . . . 129
56.21 CP Radon–Nikodym . . . 130
56.22 Choi's Theorem 3 — reverse direction . . . 131
56.23 Trace pairing duality and TP-converse . . . 133
56.24 Choi's Theorem 3 — forward direction . . . 134
56.25 Choi's Theorem 3 (statement) . . . 134
56.26 Sanity / easy facts about extreme UCP . . . 136

57 Day 23: Choi's Theorem 3 for unital maps (D23-1) 137

58 Day 23: extreme and non-extreme maps (D23-4) 138
58.1 Linear dependence tests . . . 138
58.2 Example 1: unitary conjugation is extreme . . . 139
58.3 Example 2: an average of two unitary conjugations . . . 139
58.4 Example 4: the diagonal map . . . 140
58.5 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$ . . . 140

59 Day 24 (24 October 2011): operator systems **141**
59.1 $M_p(OS)$ is an operator system . . . 142
59.2 Every element is a combination of four positives . . . 143

60 Day 25 (26 October 2011): the Hahn–Banach theorem **143**
60.1 Bounded complex-linear functionals on lmat . . . 143
60.2 Real-part functional and linearform/continuous instances . . . 144
60.3 Application of HOL/Hahn_Banach (norm-preserving extension) . . . 144
60.4 Bohnenblust-Sobczyk: complex extension from real extension . . . 144
60.5 The matrix-space Hahn-Banach theorem . . . 146

61 Days 24–25: Arveson's correspondence between maps and functionals **146**
61.1 Finite sums of matrices . . . 147
61.2 $E_{ij} \otimes M$ and the block decomposition . . . 147
61.3 The correspondence . . . 148
61.4 Theorem (Arveson): completely positive iff d-positive iff S_Φ positive . . . 149

62 Day 25: a unital functional is positive iff it has norm one (D25-4) **151**
62.1 Vectors . . . 151
62.2 The operator norm . . . 152
62.3 Unital functionals . . . 152

63 Day 26: Arveson's extension theorem **154**
63.1 Auxiliary facts . . . 154
63.2 Matrices as points of the Euclidean space *lmat* . . . 155
63.3 Extension of positive functionals . . . 155
63.4 Arveson's extension theorem . . . 155

64 Day 26: factoring a CP map through a unital or a trace-preserving one (D26-1) **156**

65 Days 26–28 (28 October – 2 November 2011): separability and entanglement witnesses **157**
65.1 The separable cone (Day 27) . . . 158
65.2 Separable density matrices (Day 26) . . . 158
65.3 The separable cone is closed (Day 28) . . . 158
65.4 Separation (Days 27–28) . . . 158
65.5 Entanglement witnesses (Day 28) . . . 158

66 Days 27–28: positive maps detect entanglement **159**
66.1 Tensor products with a matrix in the first factor . . . 159
66.2 The quadratic form of Φ_f . . . 159
66.3 Proposition (Day 27): $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive . . . 159
66.4 Proposition (Day 27): positive maps keep separable matrices positive . . . 160
66.5 Theorem (Day 28): the positive-map criterion . . . 160

67 Day 27: entangled matrices detected by the transpose (D27-2) **161**

68 Day 28: the Horodecki–Peres criterion for two qubits (D28-5) **162**

IV Lectures 29–34: error correction **163**

69 Day 29 (7 November 2011): classical error detection and correction 163
69.1 Binary strings, Hamming weight and distance . . . 163
69.2 Code distance . . . 164
69.3 The parity-check and repetition codes . . . 164

70 Days 29–30: classical codes over $\mathbb{Z}_2$ 166
70.1 The notes' "Hamming [7,1,3]" code does not correct one error . . . 166
70.2 The Hamming [7,4,3] code corrects one error . . . 167
70.3 Probabilities of undetected and uncorrected errors (Day 30) . . . 167

71 Day 29: Hamming distance and subspace codes (D29-4) 168

72 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code 170
72.1 The Pauli matrices . . . 170
72.2 Pauli magic (Day 32) . . . 171
72.3 The bit-flip channel (Day 31) . . . 171

73 Day 30: the three-qubit bit-flip code and its recovery map (D30-3, D30-4) 172

74 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity 175
74.1 Fidelity . . . 176

75 Day 31: the three-qubit code against independent bit flips 176
75.1 Errors followed by recovery . . . 177
75.2 Minimum fidelities (D31-4) . . . 178

76 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic 178
76.1 The phase-flip code . . . 178
76.2 Projections . . . 180
76.3 More on the repetition code . . . 181
76.4 Algebra of projections; towards Days 33 and 34 . . . 181

77 Day 32: the Pauli expansion identity for unitaries (D32-7) 182

78 Day 32: the code for Y flips 183
78.1 Codes with orthogonal error images . . . 183
78.2 The Y-flip code . . . 184

79 Days 32–33: composition of Kraus maps and the general recovery proposition (D32-4, D33-2) 185
79.1 Algebra of *foldr*-sums of matrices . . . 185
79.2 Composition of Kraus maps . . . 186
79.3 The general recovery proposition (D32-4) and its linear-combination form (D33-2) 187
79.4 Matrix sums acting on vectors . . . 187

80 Days 32–33: the nine-qubit Shor code 190
80.1 Tensor products of matrices . . . 190
80.2 Single-qubit operators on nine qubits, and on a block of three . . . 190
80.3 The block vectors $|m\rangle \pm |7-m\rangle$ . . . 191
80.4 Code words and the images of the errors . . . 192
80.5 The 22 correctable errors . . . 194
80.6 The code space and its projection . . . 195
80.7 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$ . . . 195

80.8 Shor's theorem . . . 196

81 Day 34: the Knill–Laflamme conditions (D34-1, D34-2, D34-3) 197
81.1 Vectors in the range of a projection; rank-one decompositions inside V . . . 197
81.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$ . . . 198
81.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$ . . . 198
81.4 Kraus lists consisting of multiples of one operator . . . 199
81.5 Theorem 2 . . . 199

82 Day 34: sufficiency of the Knill–Laflamme conditions (D34-2) 199

V Beyond the notes, and supporting theories 201

83 Extensions beyond the notes 201
83.1 Rank-one projections . . . 201
83.2 The bit-flip channel at $p = 0$ and $p = 1$ . . . 202
83.3 The phase-flip channel . . . 202
83.4 Unitary-conjugation channels . . . 203
83.5 The bit-phase-flip channel and the identity channel . . . 203
83.6 Composition of channels . . . 204
83.7 Conjugation channels are completely positive . . . 205
83.8 Unitary-conjugation channels are UCP and CPTP . . . 206
83.9 The Pauli channels are UCP and CPTP . . . 206
83.10 Pauli matrices as Kraus operators . . . 207
83.11 Kraus maps of Kraus-operator lists are CPTP . . . 207
83.12 Concatenation of Kraus lists . . . 208
83.13 Fixed points of Pauli channels . . . 208
83.14 The maximally mixed state . . . 209
83.15 Fixed points of unitary-conjugation channels . . . 210
83.16 Channels preserve density matrices . . . 211
83.17 Pauli product identities . . . 211

84 Unitary algebra 212

85 Tensor product infrastructure 213
85.1 Tensor of Identities . . . 213
85.2 Tensor Adjoint . . . 213
85.3 Mixed-Product Property . . . 213
85.4 Corollaries of the Mixed-Product Property . . . 213
85.5 Tensor Unitarity . . . 213

86 Small matrix and vector lemmas shared by the gap-closure theories 214

87 Typed-dimension matrix wrapper 216

88 Typeclass instances 217

89 Real scalar multiplication and Hahn-Banach overloading 218

90 Real-vectorspace locale on the full type 219

91 The Hilbert-Schmidt norm on *(complex, 'm, 'n) lmat* **219**

92 Triangle inequality for the HS norm 219

93 Subspace, seminorm, normed-vectorspace instances 219

94 Complex scalar multiplication on lmat 220

95 Overview 221

96 HS inner-product algebra 222

97 Trace lemmas 222

98 HS norm definition and basic facts 222

99 HS-norm convergence vs entry-wise convergence 223

100 Carathéodory's theorem for conic combinations 226

101 Overview 229

102 Real scalar multiplication and *real-vector* instance 229

103 Auxiliary: pointwise unwrapping (used by later instances) 229

104 Distance, uniformity, open: *metric-space* instance 230

105 Norm and *real-normed-vector* instance 230

106 Real inner product 231

107 Basis: matrix units carrying a basis vector 231

108 Basis set 232

109 Euclidean space instance 233

110 Heine-Borel 233

111 Dimension 233

112 Component-wise bounds via the L2 norm 233

113 Convergence: lmat sequence convergence implies entry convergence 234

114 Overview (Days 27–28 of the notes) 234

115 Typed PSD predicate 235

116 Typed tensor product 235

117 The separable cone 235

118 Step 1: PSD set is closed 236

119 Trace on lmat 236

120 Compactness of the unit-trace PSD slice 237

121 Tensor continuity 237

122 The unit-trace product slice 237

123 Closedness of the separable cone 238

124 The lmat inner product as a real-trace inner product 240

125 Separating hyperplane for non-separable states 241

126 Entanglement witness in real-trace form 241

127 Hermitian entanglement witness 241

128 Hermitian closure of the separable cone 242

129 Main separability characterisation 243

130 Density-matrix specialisation 243

theory *QC-Notes-Overview*

imports

Quantum-Computing-by-VP-3-10.Day01-Inner-Product
Quantum-Computing-by-VP-3-10.Day02-Trace-Projections
Quantum-Computing-by-VP.Day02-Projection-Basis
Quantum-Computing-by-VP-3-10.Day03-Unitary-Matrices
Quantum-Computing-by-VP-3-10.Day04-Hermitian-Matrices
Quantum-Computing-by-VP.Day04-Hermitian-Converse
Quantum-Computing-by-VP-3-10.Day05-Positivity-Tests
Quantum-Computing-by-VP.Day05-Cholesky-Examples
Quantum-Computing-by-VP.Day05-Direct-Sum-Basis
Quantum-Computing-by-VP-3-10.Day06-Tensor-Products
Quantum-Computing-by-VP.Day06-Tensor-Coordinates
Quantum-Computing-by-VP.Day06-Schmidt-Rank
Quantum-Computing-by-VP.Day06-15-Universal
Quantum-Computing-by-VP-3-10.Day07-Postulates
Quantum-Computing-by-VP-3-10.Day08-Measurements
Quantum-Computing-by-VP.Day08-Indistinguishable-States
Quantum-Computing-by-VP-3-10.Day09-POVMs-Entanglement
Quantum-Computing-by-VP.Day09-Bell-Measurement
Quantum-Computing-by-VP-3-10.Day10-Quantum-Gates
Quantum-Computing-by-VP.Day10-Superdense-Coding
Quantum-Computing-by-VP.Day11-Gates-No-Cloning
Quantum-Computing-by-VP.Day11-SWAP-Circuit
Quantum-Computing-by-VP.Day11-CCNOT-Table
Quantum-Computing-by-VP.Day12-Quantum-Parallelism
Quantum-Computing-by-VP.Day12-Counting-Probability
Quantum-Computing-by-VP.Day13-Density-Matrices
Quantum-Computing-by-VP.Day14-Pure-States
Quantum-Computing-by-VP.Day14-Composite-Ensembles
Quantum-Computing-by-VP.Day15-Partial-Traces
Quantum-Computing-by-VP.Day15-Partial-Trace-A
Quantum-Computing-by-VP.Day16-Density-Postulates
Quantum-Computing-by-VP.Day16-Block-Form
Quantum-Computing-by-VP.Day17-Quantum-Noise
Quantum-Computing-by-VP.Day17-Environment-Model
Quantum-Computing-by-VP.Day18-CP-Maps
Quantum-Computing-by-VP.Day18-Positive-Maps
Quantum-Computing-by-VP.Day19-Choi-Theorem
Quantum-Computing-by-VP.Day19-Choi-Positivity
Quantum-Computing-by-VP.Day19-Choi-TP-Unital
Quantum-Computing-by-VP.Day20-Positive-Functionals
Quantum-Computing-by-VP.Day20-Choi-Rank
Quantum-Computing-by-VP.Day21-Kraus-Rank
Quantum-Computing-by-VP.Day21-Unitary-Freedom
Quantum-Computing-by-VP.Day21-Ranges
Quantum-Computing-by-VP.Day21-Average-Conjecture
Quantum-Computing-by-VP.Day21-Average-Two
Quantum-Computing-by-VP.Day22-Convex-Sets
Quantum-Computing-by-VP.Day22-Kraus-Independence
Quantum-Computing-by-VP.Day22-Minimal-Kraus
Quantum-Computing-by-VP.Day22-Adjoint-Maps
Quantum-Computing-by-VP.Day22-UCP-Compact
Quantum-Computing-by-VP.Day23-ChoiExtremeUCP
Quantum-Computing-by-VP.Day23-Choi3-UCP
Quantum-Computing-by-VP.Day23-Extremality-Examples
Quantum-Computing-by-VP.Day24-Operator-Systems

Quantum-Computing-by-VP-3-10.Day25-Hahn-Banach
Quantum-Computing-by-VP.Day25-Arveson-Correspondence
Quantum-Computing-by-VP.Day25-Unital-Functional-Norm
Quantum-Computing-by-VP.Day26-Arveson-Extension
Quantum-Computing-by-VP.Day26-CP-Factorization
Quantum-Computing-by-VP.Day26-Separability
Quantum-Computing-by-VP.Day27-Positive-Map-Criterion
Quantum-Computing-by-VP.Day27-Entangled-Examples
Quantum-Computing-by-VP.Day28-Horodecki-Peres
Quantum-Computing-by-VP.Day29-Error-Correction
Quantum-Computing-by-VP.Day29-Classical-Codes
Quantum-Computing-by-VP.Day29-Subspace-Codes
Quantum-Computing-by-VP.Day30-Error-Probabilities
Quantum-Computing-by-VP.Day30-BitFlip-Code
Quantum-Computing-by-VP.Day31-Bit-Flip-Channel
Quantum-Computing-by-VP.Day31-Code-Fidelity
Quantum-Computing-by-VP.Day32-Phase-Flip-Code
Quantum-Computing-by-VP.Day32-Pauli-Expansion
Quantum-Computing-by-VP.Day32-Y-Code
Quantum-Computing-by-VP.Day32-Recovery-Proposition
Quantum-Computing-by-VP.Day33-Shor-Code
Quantum-Computing-by-VP.Day34-Knill-Laflamme
Quantum-Computing-by-VP.Day34-KL-Sufficiency
Quantum-Computing-by-VP.Quantum-Channels
Quantum-Computing-by-VP-3-10.Unitary-Tensor-Algebra
Quantum-Computing-by-VP.QC-Prelims
Quantum-Computing-by-VP-3-10.Matrix-Typed
Quantum-Computing-by-VP-3-10.Essential-Functional-Analysis
Quantum-Computing-by-VP-3-10-Lmat.Lmat-Euclidean
Quantum-Computing-by-VP-3-10-Lmat.Separable-Cone
begin

1 Reading guide

Paulsen's course had five parts: (I) basic mathematics, (II) an introduction to quantum computing, (III) the theory of completely positive maps, (IV) entanglement and entanglement witnesses, (V) quantum error correction. The theories follow the lectures: lecture n is the theory `Day`nn`_...` that restates the notes, followed by the theories that complete it (they close the gaps recorded by the audits; the remaining unformalised items are listed in `NOTES_AUDIT.md`). A theory completing an early lecture may use results of later ones; it is still printed with its lecture.

Day	Theories
1	Day01_Inner_Product
2	Day02_Trace_Projections, Day02_Projection_Basis
3	Day03_Unitary_Matrices
4	Day04_Hermitian_Matrices, Day04_Hermitian_Converse
5	Day05_Positivity_Tests, Day05_Cholesky_Examples, Day05_Direct_Sum_Basis, Day05_Sylvester_Criterion (separate session)
6	Day06_Tensor_Products, Day06_Tensor_Coordinates, Day06_Schmidt_Rank, Day06_15_Universal
7	Day07_Postulates
8	Day08_Measurements, Day08_Indistinguishable_States
9	Day09_POVMs_Entanglement, Day09_Bell_Measurement
10	Day10_Quantum_Gates, Day10_Superdense_Coding
11	Day11_Gates_No_Cloning, Day11_SWAP_Circuit, Day11_CCNOT_Table
12	Day12_Quantum_Parallelism, Day12_Counting_Probability
13	Day13_Density_Matrices
14	Day14_Pure_States, Day14_Composite_Ensembles
15	Day15_Partial_Traces, Day15_Partial_Trace_A
16	Day16_Density_Postulates, Day16_Block_Form
17	Day17_Quantum_Noise, Day17_Environment_Model
18	Day18_CP_Maps, Day18_Positive_Maps
19	Day19_Choi_Theorem, Day19_Choi_Positivity, Day19_Choi_TP_Unital
20	Day20_Positive_Functionals, Day20_Choi_Rank
21	Day21_Kraus_Rank, Day21_Unitary_Freedom, Day21_Ranges, Day21_Average_Conjecture, Day21_Average_Two
22	Day22_Convex_Sets, Day22_Kraus_Independence, Day22_Minimal_Kraus, Day22_Adjoint_Maps, Day22_UCP_Compact
23	Day23_ChoiExtremeUCP, Day23_Choi3_UCP, Day23_Extremality_Examples
24	Day24_Operator_Systems
25	Day25_Hahn_Banach, Day25_Arveson_Correspondence, Day25_Unital_Functional_Norm
26	Day26_Arveson_Extension, Day26_CP_Factorization, Day26_Separability
27	Day27_Positive_Map_Criterion, Day27_Entangled_Examples
28	Day28_Horodecki_Peres
29	Day29_Error_Correction, Day29_Classical_Codes, Day29_Subspace_Codes
30	Day30_Error_Probabilities, Day30_BitFlip_Code
31	Day31_Bit_Flip_Channel, Day31_Code_Fidelity
32	Day32_Phase_Flip_Code, Day32_Pauli_Expansion, Day32_Y_Code, Day32_Recovery_Proposition
33	Day33_Shor_Code
34	Day34_Knill_Laflamme, Day34_KL_Sufficiency

The last part holds what the lectures build on or lead to: `Quantum_Channels` (standard channels beyond the notes), `Unitary_Tensor_Algebra` (unitary and Kronecker-product algebra), `QC_Prelims` (lemmas shared by the completing theories), `Matrix_Typed`, `Essential_Functional_Analysis` and `Lmat_Euclidean` (typed matrices, Hilbert–Schmidt analysis, Carathéodory), and `Separable_Cone` (the proofs behind `Day26_Separability`). The companion AFP entries *Hermitian Functional Calculus* and *HHL Algorithm* go beyond the notes: the functional calculus and product formulas, quantum phase estimation and the HHL algorithm.

2 Conventions

- Vectors in $\mathbb{C}^n$ are *complex Matrix.vec* with *dim-vec* $v = n$; matrices in $M_{m,n}$ are *complex Matrix.mat* with $A \in$ *carrier-mat* m n (the Jordan-Normal-Form library). Dimensions are hypotheses, not types, except in the typed wrapper *lmat* used for the topological arguments of Part IV.
- The physicists' adjoint $X^\dagger = X^*$ is $A^\dagger$; $\langle y|x\rangle$ is $\langle y|x\rangle$ (conjugate-linear in the first slot, as in the notes); $|v\rangle\langle w|$ is $v \oslash w$; the Hilbert–Schmidt inner product $\langle Y|X\rangle = \mathrm{Tr}(Y^*X)$ is $\langle\langle$ Y | X $\rangle\rangle$; the matrix unit $E_{ij} \in M_{m,n}$ is $E[\![i,j$; $m,n]\!]$.
- Indices start at 0, as the physicists in the notes prefer: the canonical basis of $\mathbb{C}^n$ is $|0\rangle, \ldots, |n-1\rangle$, formally *unit-vec n i*.
- $\sum_i |v_i\rangle\langle v_i|$ for a list of vectors is *outer-sum n vs* $= VV^\dagger$ with V the matrix whose columns are the v_i.
- Tensor products are Kronecker products: $v \otimes w$ and $A \bigotimes B$, with the first factor as the major index.
- Kraus sums $\sum_i W_i X W_i^\dagger$ over a list are *kraus-map-dim d Ws X*.

Verification status. All theories build with *quick-and-dirty* = *false*; there are no *sorry*s and no axioms beyond those of Isabelle/HOL and the imported libraries.

end

Part I
Lectures 1–10: basic mathematics and the postulates of quantum mechanics

theory *Day01-Inner-Product*
  imports
    Quantum-Fourier-Transform.QFT
    Isabelle-Marries-Dirac.No-Cloning
begin

Isabelle_Marries_Dirac names its gates *X*, *Y*, *Z*, *H*, *S*, *T*; hidden here so that these letters are variables. The gates remain *Quantum.X* etc.; the project names are *Pauli-X*, *Pauli-Y*, *Pauli-Z*, *Hadamard*, *Phase-S*. Theories importing this one must not import Isabelle_Marries_Dirac directly, as that undoes the hiding.

hide-const (**open**) *Quantum.X Quantum.Y Quantum.Z Quantum.H Quantum.S Quantum.T*

No-Cloning is imported for *cauchy-schwarz-ineq*; its simp rules (e.g. $v = w$ from inner products) slow simp on every vector equation.

declare *cauchy-schwarz-eq* [*simp del*] *cauchy-schwarz-col* [*simp del*] *eq-from-inner-prod* [*simp del*]

3 Preliminaries: linear-space isomorphisms

Matrix spaces are represented by carrier predicates on the single type *complex Matrix.mat*. The record below packages a carrier with its zero, addition and scalar multiplication, so that statements such as $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ (Day 15) can be made as explicit linear isomorphisms with explicit inverses.

record $('a, 'v)$ *linear-structure* =
  ls-carrier :: $'v$ *set*
  ls-zero :: $'v$
  ls-add :: $'v \Rightarrow 'v \Rightarrow 'v$
  ls-smul :: $'a \Rightarrow 'v \Rightarrow 'v$

definition *linear-structure-map* ::
  $('a, 'v)$ *linear-structure* $\Rightarrow$ $('a, 'w)$ *linear-structure* $\Rightarrow$ $('v \Rightarrow 'w) \Rightarrow$ *bool* **where**
  linear-structure-map $V\ W\ f \equiv$
    $f\ (ls\text{-}zero\ V) = ls\text{-}zero\ W \wedge$
    $(\forall x \in ls\text{-}carrier\ V.\ f\ x \in ls\text{-}carrier\ W) \wedge$
    $(\forall x \in ls\text{-}carrier\ V.\ \forall y \in ls\text{-}carrier\ V.$
      $f\ (ls\text{-}add\ V\ x\ y) = ls\text{-}add\ W\ (f\ x)\ (f\ y)) \wedge$
    $(\forall c\ x.\ x \in ls\text{-}carrier\ V \longrightarrow$
      $f\ (ls\text{-}smul\ V\ c\ x) = ls\text{-}smul\ W\ c\ (f\ x))$

definition *is-isomorphism-between* ::
  $('a, 'v)$ *linear-structure* $\Rightarrow$ $('a, 'w)$ *linear-structure* $\Rightarrow$ $('v \Rightarrow 'w) \Rightarrow$ *bool* **where**
  is-isomorphism-between $V\ W\ f \equiv$
    linear-structure-map $V\ W\ f \wedge$
    $(\exists g.$ *linear-structure-map* $W\ V\ g \wedge$
        $(\forall x \in ls\text{-}carrier\ V.\ g\ (f\ x) = x) \wedge$
        $(\forall y \in ls\text{-}carrier\ W.\ f\ (g\ y) = y))$

definition *is-isomorphic-to* ::
  $('a, 'v)$ *linear-structure* $\Rightarrow$ $('a, 'w)$ *linear-structure* $\Rightarrow$ *bool* **where**
  is-isomorphic-to $V\ W \equiv \exists f.$ *is-isomorphism-between* $V\ W\ f$

lemma *is-isomorphism-betweenD-map*:
  assumes *is-isomorphism-between* $V\ W\ f$
  shows *linear-structure-map* $V\ W\ f$
  ⟨*proof*⟩

lemma *is-isomorphism-betweenD-inverse*:
  assumes *is-isomorphism-between* $V\ W\ f$
  obtains g **where** *linear-structure-map* $W\ V\ g$
    and $\bigwedge x.\ x \in ls\text{-}carrier\ V \Longrightarrow g\ (f\ x) = x$
    and $\bigwedge y.\ y \in ls\text{-}carrier\ W \Longrightarrow f\ (g\ y) = y$
  ⟨*proof*⟩

definition *mat-linear-structure* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
  mat-linear-structure $m\ n \equiv$
    $(\!|$ *ls-carrier* $=$ *carrier-mat* $m\ n$,
      ls-zero $= 0_m\ m\ n$,
      ls-add $= (\lambda x\ y.\ x + y)$,
      ls-smul $= (\lambda c\ x.\ c \cdot_m x)$ $|\!)$

4 Day 1 (22 August 2011): the inner product

Paulsen opens with the dictionary between classical and quantum computing: a bit is an element of $\mathbb{Z}_2$, a qubit a unit vector in $\mathbb{C}^2$; N bits form $\mathbb{Z}_2^N$, N qubits a unit vector in $\mathbb{C}^{2^N} \cong \mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2$; operations are Boolean gates versus completely positive maps on M_{2^N}; information is a probability vector versus a positive semidefinite matrix of trace one. He then fixes the physicists' notation.

Definition (notes). $\mathbb{C}^n$ is the space of complex n-tuples with the inner product

$$\langle y|x\rangle = \sum_{i=1}^{n} \overline{y_i}\, x_i,$$

which is linear in the right-hand slot, conjugate linear in the left-hand slot ("sesquilinear"), and positive definite: $\langle x|x\rangle \geq 0$ with equality only for $x = 0$. The Euclidean length is $\|x\| = \sqrt{\langle x|x\rangle}$. A *Hilbert space* is a complex vector space with a sesquilinear positive definite form, complete for this norm.

The library's inner product $\langle y|x\rangle$ is exactly this form, with indices running from 0 to $n-1$.

lemma *inner-prod-sum*:
assumes *dim-vec u = n* **and** *dim-vec v = n*
shows $\langle u|v\rangle = (\sum\ i \in \{0..< n\}.\ cnj(u\ \$\ i) * (v\ \$\ i))$
⟨*proof*⟩

4.1 The Hilbert–Schmidt inner product on matrices

Definition (notes) (Example)**.** The space $M_{n,k}$ of complex $n \times k$ matrices is a Hilbert space for

$$\langle (y_{ij}) \,|\, (x_{ij})\rangle = \sum_i \sum_j \overline{y_{ij}}\, x_{ij},$$

and $M_{n,k} \cong \mathbb{C}^{nk}$.

We write $\langle\langle y|x\rangle\rangle$ for this Hilbert–Schmidt (Frobenius) inner product; the double brackets distinguish it from the inner product of vectors.

definition *hs-inner-prod* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex* **where**
hs-inner-prod = (λ *y x*.
$\sum$ *i* ∈ {*0* ..< *dim-row x*}.
$\sum$ *j* ∈ {*0* ..< *dim-col x*}.
cnj (*y* $$ (*i*,*j*)) * (*x* $$ (*i*,*j*)))

notation *hs-inner-prod* (⟨⟨ - | - ⟩⟩)

lemma *hs-inner-prod-expand*:
assumes *dim-row x = m* **and** *dim-col x = n*
shows ⟨⟨ *y* | *x* ⟩⟩ =
($\sum$ *i* ∈ {*0* ..< *m*}. $\sum$ *j* ∈ {*0* ..< *n*}. *cnj* (*y* $$ (*i*,*j*)) * (*x* $$ (*i*,*j*)))
⟨*proof*⟩

lemma *hs-inner-prod-square*:
assumes *dim-row x = n* **and** *dim-col x = n*
shows ⟨⟨ *y* | *x* ⟩⟩ =
($\sum$ *i* ∈ {*0* ..< *n*}. $\sum$ *j* ∈ {*0* ..< *n*}. *cnj* (*y* $$ (*i*,*j*)) * (*x* $$ (*i*,*j*)))
⟨*proof*⟩

4.2 Properties of the inner product

The properties listed in the notes, verified for the matrix inner product: conjugate symmetry, linearity in the right-hand slot, conjugate linearity in the left-hand slot, and positive definiteness.

lemma *hs-inner-prod-cnj*:
  assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
  shows ⟨⟨ *x* | *y* ⟩⟩ = *cnj* (⟨⟨ *y* | *x* ⟩⟩)
⟨*proof*⟩

lemma *hs-inner-prod-right-add*:
  assumes *dim-row X1* = *dim-row y* **and** *dim-col X1* = *dim-col y*
    and *dim-row X2* = *dim-row y* **and** *dim-col X2* = *dim-col y*
  shows ⟨⟨ *y* | *X1* + *X2* ⟩⟩ = ⟨⟨ *y* | *X1* ⟩⟩ + ⟨⟨ *y* | *X2* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-right-smult*:
  assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
  shows ⟨⟨ *y* | *c* ·$_m$ *x* ⟩⟩ = *c* ∗ ⟨⟨ *y* | *x* ⟩⟩
  ⟨*proof*⟩

lemma *hs-inner-prod-left-smult*:
  assumes *dim-row x* = *dim-row y* **and** *dim-col x* = *dim-col y*
  shows ⟨⟨ *c* ·$_m$ *y* | *x* ⟩⟩ = *cnj c* ∗ ⟨⟨ *y* | *x* ⟩⟩
  ⟨*proof*⟩

lemma *hs-inner-prod-self-real*:
  shows ⟨⟨ *x* | *x* ⟩⟩ ∈ ℝ
⟨*proof*⟩

lemma *hs-inner-prod-self-nonneg*:
  shows *Re* (⟨⟨ *x* | *x* ⟩⟩) ≥ *0*
⟨*proof*⟩

lemma *hs-inner-prod-eq-zero-imp*:
  assumes ⟨⟨ *x* | *x* ⟩⟩ = *0*
    and *i* < *dim-row x* **and** *j* < *dim-col x*
  shows *x* $$ (*i*,*j*) = *0*
⟨*proof*⟩

end

theory *Day02-Trace-Projections*
  imports *Day01-Inner-Product*
begin

5 Day 2 (24 August 2011): bases, matrix units, trace, projections

Definition (notes) (Examples)**.** $\mathbb{C}^n$ has the orthonormal basis $e_1, \ldots, e_n$ and every x satisfies $x = \sum_i x_i e_i = \sum_i \langle e_i|x\rangle e_i$. The *matrix units* E_{ij} (entry 1 in position (i,j), 0 elsewhere) form an orthonormal basis of $M_{n,m}$, and

$$X = \sum_{i,j} X_{ij} E_{ij} = \sum_{i,j} \langle E_{ij}|X\rangle E_{ij}.$$

Formally $E[\![i,j\ ;\ m,n]\!]$ is the $m \times n$ matrix unit (indices first, dimensions second). The notes also recall the conjugate $\overline{X}$, the transpose X^t, the adjoint $X^* = \overline{X}^t$ (written $X^\dagger$ by physicists, formally $X^\dagger$) and the row-times-column rule for matrix products; these are library notions.

definition *mat-unit* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
mat-unit i j m n = *Matrix.mat m n* (λ(*r*,*c*). *if r* = *i* ∧ *c* = *j then 1 else 0*)

notation *mat-unit* (*E*[![-,- ; -,-]!] [*1000*, *1000*, *1000*, *1000*] *999*)

lemma *mat-unit-carrier* [*simp*]: *E*[![*i*,*j* ; *m*,*n*]!] ∈ *carrier-mat m n*
⟨*proof*⟩

lemma *mat-unit-dim-row* [*simp*]:
fixes *i j m n* :: *nat*
shows *dim-row* (*E*[![*i*,*j* ; *m*,*n*]!]) = *m*
⟨*proof*⟩

lemma *mat-unit-dim-col* [*simp*]:
fixes *i j m n* :: *nat*
shows *dim-col* (*E*[![*i*,*j* ; *m*,*n*]!]) = *n*
⟨*proof*⟩

lemma *mat-unit-index* [*simp*]:
assumes *r* < *m* **and** *c* < *n*
shows (*E*[![*i*,*j* ; *m*,*n*]!]) $$ (*r*,*c*) = (*if r* = *i* ∧ *c* = *j then 1 else 0*)
⟨*proof*⟩

Pairing with E_{ij} extracts the (i,j) entry: the matrix analogue of $\langle e_i|v\rangle = v_i$.

lemma *hs-inner-prod-mat-unit-left*:
assumes *i* < *dim-row x* **and** *j* < *dim-col x*
shows ⟨⟨ *E*[![*i*,*j* ; (*dim-row x*), (*dim-col x*)]!] | *x* ⟩⟩ = *x* $$ (*i*,*j*)
⟨*proof*⟩

lemma *mat-unit-orthonormal*:
assumes *i-lt*: *i* < *m*
and *j-lt*: *j* < *n*
and *k-lt*: *k* < *m*
and *l-lt*: *l* < *n*
shows ⟨⟨ *E*[![*i*,*j* ; *m*,*n*]!] | *E*[![*k*,*l* ; *m*,*n*]!] ⟩⟩ =
(*if i* = *k* ∧ *j* = *l then 1 else 0*)
⟨*proof*⟩

Expansion in the basis $\{E_{ij}\}$, stated entrywise because matrices of different dimensions cannot be added in the library.

lemma *mat-unit-expansion*:

assumes $r < \textit{dim-row}\ x$ **and** $c < \textit{dim-col}\ x$
shows $x\ \$\$\ (r,c) =$
$(\sum i \in \{0..<\textit{dim-row}\ x\}.\ \sum j \in \{0..<\textit{dim-col}\ x\}.$
$x\ \$\$\ (i,j) * (\textit{mat-unit}\ i\ j\ (\textit{dim-row}\ x)\ (\textit{dim-col}\ x))\ \$\$\ (r,c))$
⟨*proof*⟩

lemma *mat-unit-hs-expansion*:
assumes $r < \textit{dim-row}\ x$ **and** $c < \textit{dim-col}\ x$
shows $x\ \$\$\ (r,c) =$
$(\sum i \in \{0..<\textit{dim-row}\ x\}.\ \sum j \in \{0..<\textit{dim-col}\ x\}.$
$\langle\langle\ \textit{mat-unit}\ i\ j\ (\textit{dim-row}\ x)\ (\textit{dim-col}\ x)\ |\ x\ \rangle\rangle *$
$(\textit{mat-unit}\ i\ j\ (\textit{dim-row}\ x)\ (\textit{dim-col}\ x))\ \$\$\ (r,c))$
⟨*proof*⟩

5.1 The trace

Definition (notes). $\mathrm{Tr} : M_{n,n} \to \mathbb{C}$, $\mathrm{Tr}(X) = \sum_{i=1}^{n} x_{ii}$; we sometimes write Tr_n.

The formal *Tr* sums the diagonal of any matrix. On matrix units, $\mathrm{Tr}(E_{ii}) = 1$ and $\mathrm{Tr}(E_{ij}) = 0$ for $i \neq j$, so the trace is the sum of the diagonal Hilbert–Schmidt coefficients.

definition *Tr* :: *complex Matrix.mat* ⇒ *complex* **where**
$\textit{Tr}\ x = (\sum r < \textit{dim-row}\ x.\ \textit{if}\ r < \textit{dim-col}\ x\ \textit{then}\ x\ \$\$\ (r,r)\ \textit{else}\ 0)$

lemma *Tr-mat-unit*:
assumes *i-lt*: $i < m$
and *j-lt*: $j < n$
shows $\textit{Tr}\ (E[\![i,j\ ;\ m,n]\!]) = (\textit{if}\ i = j\ \textit{then}\ 1\ \textit{else}\ 0)$
⟨*proof*⟩

corollary *Tr-mat-unit-diag*:
assumes $i < n$
shows $\textit{Tr}\ (E[\![i,i\ ;\ n,n]\!]) = 1$
⟨*proof*⟩

corollary *Tr-mat-unit-offdiag*:
assumes $i < m$ **and** $j < n$ **and** $i \neq j$
shows $\textit{Tr}\ (E[\![i,j\ ;\ m,n]\!]) = 0$
⟨*proof*⟩

lemma *Tr-hs-diagonal-sum*:
shows $\textit{Tr}\ x =$
$(\sum r < \textit{dim-row}\ x.$
$\textit{if}\ r < \textit{dim-col}\ x$
$\textit{then}\ \langle\langle\ E[\![r,r\ ;\ (\textit{dim-row}\ x),\ (\textit{dim-col}\ x)]\!]\ |\ x\ \rangle\rangle$
$\textit{else}\ 0)$
⟨*proof*⟩

corollary *Tr-hs-diagonal-sum-square*:
assumes *rows*: *dim-row* $x = n$
and *cols*: *dim-col* $x = n$
shows $\textit{Tr}\ x = (\sum r{<}n.\ \langle\langle\ E[\![r,r\ ;\ n,n]\!]\ |\ x\ \rangle\rangle)$
⟨*proof*⟩

The trace is linear.

lemma *Tr-add*:
assumes *rows*: *dim-row x* = *dim-row y*
and *cols*: *dim-col x* = *dim-col y*
shows *Tr* (*x* + *y*) = *Tr x* + *Tr y*
⟨*proof*⟩

lemma *Tr-smult*: *Tr* (*a* ·$_m$ *x*) = *a* ∗ *Tr x*
⟨*proof*⟩

corollary *Tr-uminus*: *Tr* (− *x*) = − *Tr x*
⟨*proof*⟩

corollary *Tr-diff*:
assumes *rows*: *dim-row x* = *dim-row y*
and *cols*: *dim-col x* = *dim-col y*
shows *Tr* (*x* − *y*) = *Tr x* − *Tr y*
⟨*proof*⟩

lemma *Tr-zero-mat* [*simp*]:
shows *Tr* (*Matrix.mat m n* (λ(*r*,*c*). *0*)) = *0*
⟨*proof*⟩

5.2 Matrix-unit algebra and cyclicity of the trace

$E^*_{ij} = E_{ji}$, $E_{ij}E_{kl} = \delta_{jk}E_{il}$, $\mathrm{Tr}(AB) = \mathrm{Tr}(BA)$, $\mathrm{Tr}(U^*AU) = \mathrm{Tr}(A)$ for unitary U, and $\langle A|B\rangle = \mathrm{Tr}(A^*B)$: the facts about the trace that the notes use from Day 3 onwards.

lemma *mat-unit-dagger*:
shows (*E*⟦*i*,*j* ; *m*,*n*⟧)† = *E*⟦*j*,*i* ; *n*,*m*⟧
⟨*proof*⟩

lemma *mat-unit-mult-entry*:
assumes *r-lt*: *r* < *m*
and *c-lt*: *c* < *p*
and *j-lt*: *j* < *n*
shows (*E*⟦*i*,*j* ; *m*,*n*⟧ ∗ *E*⟦*k*,*l* ; *n*,*p*⟧) $$ (*r*,*c*) =
(*if j* = *k* ∧ *r* = *i* ∧ *c* = *l then 1 else 0*)
⟨*proof*⟩

lemma *Tr-cyclic*:
assumes *AB*: *dim-col A* = *dim-row B*
and *BA*: *dim-row A* = *dim-col B*
shows *Tr* (*A* ∗ *B*) = *Tr* (*B* ∗ *A*)
⟨*proof*⟩

lemma *Tr-unitary-conjugation*:
assumes *Uu*: *unitary U*
and *Arow*: *dim-row A* = *dim-row U*
and *Acol*: *dim-col A* = *dim-row U*
shows *Tr* ((*U*†) ∗ *A* ∗ *U*) = *Tr*(*A*)
⟨*proof*⟩

lemma *hs-inner-prod-as-trace*:
assumes *rows*: *dim-row A* = *dim-row B*
and *cols*: *dim-col A* = *dim-col B*
shows ⟨⟨ *A* | *B* ⟩⟩ = *Tr* ((*A*†) ∗ *B*)
⟨*proof*⟩

corollary *hs-inner-prod-self-as-trace*:
shows ⟨⟨ *A* | *A* ⟩⟩ = *Tr* ((*A*†) ∗ *A*)
⟨*proof*⟩

5.3 Orthogonal projections and Parseval's identity

Definition (notes) (Orthogonal projections)**.** Let $V \subseteq \mathbb{C}^n$ be a subspace with orthonormal basis $\{v_1, \ldots, v_k\}$ and put $P = \sum_{i=1}^{k} |v_i\rangle\langle v_i|$. Then $Px = \sum_i \langle v_i|x\rangle v_i \in V$ and $x - Px \perp V$, so every $x \in \mathbb{C}^n$ is uniquely $x = v + w$ with $v \in V$ and $w \in V^{\perp}$. Consequently P does not depend on the orthonormal basis chosen for V, $Pv = v$ for $v \in V$, $P^2 = P$ and $P = P^*$: P is *the* orthogonal projection onto V.

Theorem (notes) (Parseval)**.** If $\{v_1, \ldots, v_k\}$ is an orthonormal basis of V and $v \in V$, then $\|v\|^2 = \sum_{i=1}^{k} |\langle v_i|v\rangle|^2$. For $V = \mathbb{C}^n$ this holds for every v.

Theorem (notes). If $P \in M_n$ satisfies $P^2 = P$ and $P = P^*$, and $V = \mathrm{range}(P)$, then P is the orthogonal projection onto V.

Formally, an orthonormal list *vs* of vectors in $\mathbb{C}^n$ is *orthonormal-list n vs*, the matrix with columns $v_1, \ldots, v_k$ is *mat-of-cols n vs*, and $P = \sum_i |v_i\rangle\langle v_i| = VV^{\dagger}$ is *outer-sum n vs*. The results below are the projection formula, $V^{\dagger}V = I_k$, the completeness relation $VV^{\dagger} = I_n$ for a full basis, $P = P^{\dagger}$, $P^2 = P$, Parseval, the orthogonality of the residual, and the uniqueness of the decomposition. Independence of P from the basis is *outer-sum-basis-independent* in theory Day02_11_Supplement; the characterisation of Hermitian idempotents as orthogonal projections is *projection-annihilates-orthogonal* in theory Quantum_Computing11.

definition *orthonormal-list* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *bool* **where**
orthonormal-list n vs ≡
(∀ *v* ∈ *set vs*. *dim-vec v* = *n*) ∧
(∀ *i* < *length vs*. ∀ *j* < *length vs*.
⟨*vs* ! *i* | *vs* ! *j*⟩ = (*if i* = *j then 1 else 0*))

lemma *orthonormal-listD-dim*:
assumes *orthonormal-list n vs* **and** *i* < *length vs*
shows *dim-vec* (*vs* ! *i*) = *n*
⟨*proof*⟩

lemma *orthonormal-listD-inner*:
assumes *orthonormal-list n vs* **and** *i* < *length vs* **and** *j* < *length vs*
shows ⟨*vs* ! *i* | *vs* ! *j*⟩ = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

lemma *orthonormal-listD-carrier*:
assumes *orthonormal-list n vs*
shows *set vs* ⊆ *carrier-vec n*
⟨*proof*⟩

definition *outer-sum* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.mat* **where**
outer-sum n vs = *mat-of-cols n vs* ∗ ((*mat-of-cols n vs*)†)

lemma *outer-sum-dim-row* [*simp*]: *dim-row* (*outer-sum n vs*) = *n*
⟨*proof*⟩

lemma *outer-sum-dim-col* [*simp*]: *dim-col* (*outer-sum n vs*) = *n*
⟨*proof*⟩

lemma *outer-sum-hermitian*:
shows (*outer-sum n vs*)† = *outer-sum n vs*
⟨*proof*⟩

lemma *cols-mat-dagger-left*:
assumes *orth*: *orthonormal-list n vs*
shows (*mat-of-cols n vs*)† ∗ (*mat-of-cols n vs*) = 1_m (*length vs*)
⟨*proof*⟩

lemma *cols-mat-dagger-right*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
shows (*mat-of-cols n vs*) ∗ ((*mat-of-cols n vs*)†) = 1_m *n*
⟨*proof*⟩

lemma *completeness-relation*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
shows (∑ *l*<*n*. (*vs* ! *l*) $ *i* ∗ *cnj* ((*vs* ! *l*) $ *j*)) = (*if i* = *j then 1 else 0*)
⟨*proof*⟩

lemma *outer-sum-index*:
assumes *orth*: *orthonormal-list n vs*
and *i-lt*: *i* < *n* **and** *j-lt*: *j* < *n*
shows (*outer-sum n vs*) $$ (*i*,*j*) =
(∑ *l* < *length vs*. (*vs* ! *l*) $ *i* ∗ *cnj* ((*vs* ! *l*) $ *j*))
⟨*proof*⟩

lemma *outer-sum-apply*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x* = *n*
and *i-lt*: *i* < *n*
shows (*outer-sum n vs* ∗$_v$ *x*) $ *i* = (∑ *l* < *length vs*. ⟨*vs* ! *l* | *x*⟩ ∗ (*vs* ! *l*) $ *i*)
⟨*proof*⟩

lemma *outer-sum-idempotent*:

assumes *orth*: *orthonormal-list n vs*
shows *outer-sum n vs ∗ outer-sum n vs = outer-sum n vs*
⟨*proof*⟩

lemma *parseval*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs = n*
and *vdim*: *dim-vec v = n*
and *i-lt*: *i < n*
shows *v $ i* = (∑ *l*<*n*. ⟨*vs ! l | v*⟩ ∗ (*vs ! l*) $ *i*)
⟨*proof*⟩

corollary *parseval-norm*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs = n*
and *vdim*: *dim-vec v = n*
shows ‖*v*‖² = (∑ *l*<*n*. (*cmod* (⟨*vs ! l | v*⟩))²)
⟨*proof*⟩

The residual $x - Px$ is orthogonal to every v_l, because $\langle v_l | Px\rangle = \langle v_l | x\rangle$; hence the orthogonal decomposition $x = Px + (x - Px)$.

lemma *inner-prod-outer-sum*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x = n*
and *l-lt*: *l < length vs*
shows ⟨*vs ! l | outer-sum n vs ∗*$_v$ *x*⟩ = ⟨*vs ! l | x*⟩
⟨*proof*⟩

lemma *residual-orthogonal*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x = n*
and *l-lt*: *l < length vs*
shows ⟨*vs ! l | x − outer-sum n vs ∗*$_v$ *x*⟩ = *0*
⟨*proof*⟩

corollary *orthogonal-decomposition*:
assumes *xdim*: *dim-vec x = n*
shows *x = outer-sum n vs ∗*$_v$ *x* + (*x − outer-sum n vs ∗*$_v$ *x*)
⟨*proof*⟩

Uniqueness: if $x = v + w$ with $Pv = v$ and $w \perp v_l$ for all l, then $v = Px$ and $w = x - Px$.

lemma *outer-sum-orthogonal-zero*:
assumes *orth*: *orthonormal-list n vs*
and *wdim*: *dim-vec w = n*
and *ortho*: ∀ *l < length vs*. ⟨*vs ! l | w*⟩ = *0*
shows *outer-sum n vs ∗*$_v$ *w* = *0*$_v$ *n*
⟨*proof*⟩

lemma *outer-sum-add*:

assumes *vdim*: *dim-vec v* = *n* **and** *wdim*: *dim-vec w* = *n*
shows *outer-sum n vs* $*_v$ (*v* + *w*) =
outer-sum n vs $*_v$ *v* + *outer-sum n vs* $*_v$ *w*
⟨*proof*⟩

lemma *orthogonal-decomposition-unique*:
assumes *orth*: *orthonormal-list n vs*
and *xdim*: *dim-vec x* = *n*
and *vdim*: *dim-vec v* = *n*
and *wdim*: *dim-vec w* = *n*
and *decomp*: *x* = *v* + *w*
and *v-fixed*: *outer-sum n vs* $*_v$ *v* = *v*
and *w-ortho*: ∀ *l* < *length vs*. ⟨*vs* ! *l* | *w*⟩ = *0*
shows *v* = *outer-sum n vs* $*_v$ *x* **and** *w* = *x* − *outer-sum n vs* $*_v$ *x*
⟨*proof*⟩

end
theory *Day02-Projection-Basis*
imports *Quantum-Computing-by-VP-3-10.Day03-Unitary-Matrices*
begin

6 Day 2: the projection does not depend on the orthonormal basis

definition *col-span* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec set* **where**
col-span n vs = {*mat-of-cols n vs* $*_v$ *c* | *c*. *dim-vec c* = *length vs*}

lemma *outer-sum-fixes-col-span*:
assumes *orth*: *orthonormal-list n vs* **and** *w*: *w* ∈ *col-span n vs*
shows *outer-sum n vs* $*_v$ *w* = *w*
⟨*proof*⟩

lemma *mem-col-span-self*:
assumes ∀ *v* ∈ *set vs*. *dim-vec v* = *n* **and** *j*: *j* < *length vs*
shows *vs* ! *j* ∈ *col-span n vs*
⟨*proof*⟩

lemma *outer-sum-absorbs*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *sub*: *set ws* ⊆ *col-span n vs*
shows *outer-sum n vs* ∗ *outer-sum n ws* = *outer-sum n ws*
⟨*proof*⟩

D2-8: two orthonormal bases of the same subspace give the same projection.

theorem *outer-sum-basis-independent*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *span*: *col-span n vs* = *col-span n ws*
shows *outer-sum n vs* = *outer-sum n ws*
⟨*proof*⟩

end

theory *Day03-Unitary-Matrices*
imports

Unitary-Tensor-Algebra
Jordan-Normal-Form.Schur-Decomposition
begin

7 Day 3 (31 August 2011): unitary matrices, Householder unitaries, Schur's theorem

7.1 Unitary matrices

Definition (notes). $U \in M_n$ is *unitary* if $U^*U = I$.

Theorem (notes). For $U \in M_n$ the following are equivalent: (a) U is unitary; (b) U is invertible and $U^{-1} = U^*$; (c) $UU^* = I$; (d) U^* is unitary; (e) the columns of U are orthonormal; (f) the rows of U are orthonormal; (g) $\|Ux\| = \|x\|$ for all x (isometry); (h) $\langle Ux|Uy\rangle = \langle x|y\rangle$ for all x, y.

The library predicate *unitary U* bundles (a) and (c). The lemmas below give (a)⇒(b), (d), (e), (f), (g), (h); (e)⇒(a) is *orthonormal-mat-unitary*, (g)⇒(a) and (h)⇒(a) are *unitary-of-isometry* and *unitary-of-inner-prod-preserving* (proved on Day 4, after the polarisation lemma); (d) is self-dual and (f) follows from (e) for U^*.

(b) U is invertible with inverse U^*:

lemma *unitary-invertible*:
  assumes *unitary U*
  shows *invertible-mat U*
⟨*proof*⟩

(d) U^* is unitary:

(e), (f) the columns and the rows of U are orthonormal:

lemma *unitary-cols-orthonormal*:
  assumes *U*: *unitary U*
    and *i*: *i* < *dim-col U*
    and *j*: *j* < *dim-col U*
  shows ⟨*Matrix.col U i* | *Matrix.col U j*⟩ = (*if i* = *j then 1 else 0* :: *complex*)
  ⟨*proof*⟩

lemma *unitary-rows-orthonormal*:
  assumes *U*: *unitary U*
    and *i*: *i* < *dim-row U*
    and *j*: *j* < *dim-row U*
  shows ⟨*Matrix.row U i* | *Matrix.row U j*⟩ = (*if i* = *j then 1 else 0* :: *complex*)
  ⟨*proof*⟩

(g), (h) U is an isometry and preserves inner products:

lemma *unitary-isometry*:
  assumes *unitary U* **and** *dim-vec v* = *dim-col U*
  shows ‖*U* ∗$_v$ *v*‖ = ‖*v*‖
⟨*proof*⟩

lemma *unitary-preserves-inner-prod*:
  assumes *U*: *unitary U*
    and *u*: *dim-vec u* = *dim-col U*
    and *v*: *dim-vec v* = *dim-col U*

shows $\langle U *_v u \mid U *_v v\rangle = \langle u \mid v\rangle$
⟨*proof*⟩

7.2 Householder unitaries

Definition (notes). For $w \in \mathbb{C}^n$ with $\|w\| = 1$ let $P_w = |w\rangle\langle w|$, the projection onto span$\{w\}$, and $U_w = I - 2P_w$. Then $U_w^* U_w = (I - 2P_w)^2 = I - 4P_w + 4P_w^2 = I$, so U_w is unitary: the *Householder unitary* of w.

Remark. Geometrically U_w is the reflection through the hyperplane $\{w\}^\perp$.

Formally *householder w* is defined entrywise as $(\delta_{ij} - 2w_i\overline{w_j})$ and shown to equal $I - 2|w\rangle\langle w|$.

abbreviation *outer-prod* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat*
  (**infixr** ⊘ *75*)
  where $v \oslash w \equiv |v\rangle * \textit{bra-vec}\ w$

lemma *bra-vec-index-0*:
  assumes *j*: $j <$ *dim-vec w*
  shows *bra-vec w* $$ (*0*,*j*) = *cnj* (*w* $ *j*)
  ⟨*proof*⟩

lemma *outer-prod-index*:
  fixes *v w* :: *complex Matrix.vec*
  assumes *i*: $i <$ *dim-vec v*
    and *j*: $j <$ *dim-vec w*
  shows $(v \oslash w)$ $$ $(i,j) = (v \$ i) * \textit{cnj}\ (w \$ j)$
⟨*proof*⟩

lemma *outer-prod-mult-vec-index*:
  fixes *v w x* :: *complex Matrix.vec*
  assumes *xdim*: *dim-vec x* = *dim-vec w*
    and *i*: $i <$ *dim-vec v*
  shows $((v \oslash w) *_v x) \$ i = (v \$ i) * \langle w \mid x\rangle$
⟨*proof*⟩

lemma *outer-prod-mult-vec*:
  fixes *v w x* :: *complex Matrix.vec*
  assumes *xdim*: *dim-vec x* = *dim-vec w*
  shows $(v \oslash w) *_v x = \langle w \mid x\rangle \cdot_v v$
⟨*proof*⟩

lemma *smult-mat-mult-vec*:
  fixes *A* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
  assumes *A*: $A \in$ *carrier-mat nr nc*
    and *x*: $x \in$ *carrier-vec nc*
  shows $(a \cdot_m A) *_v x = a \cdot_v (A *_v x)$
⟨*proof*⟩

lemma *cpx-vec-length-zero-iff*:
  fixes *v* :: *complex Matrix.vec*
  shows $\|v\| = 0 \longleftrightarrow v = 0_v$ (*dim-vec v*)
⟨*proof*⟩

$\langle v|v\rangle = 0$ iff $v = 0$.

lemma *inner-prod-self-eq-zero-iff*:
  fixes *v* :: *complex Matrix.vec*
  shows $\langle v \mid v\rangle = 0 \longleftrightarrow v = 0_v$ (*dim-vec v*)

⟨*proof*⟩

lemma *cpx-vec-length-smult*:
fixes a :: *complex* **and** v :: *complex Matrix.vec*
shows $\|a \cdot_v v\| = cmod\ a * \|v\|$
⟨*proof*⟩

lemma *inner-prod-right-smult'*:
fixes $u\ v$ :: *complex Matrix.vec*
shows $\langle u \mid a \cdot_v v\rangle = a * \langle u \mid v\rangle$
⟨*proof*⟩

lemma *inner-prod-right-add'*:
fixes $u\ v\ w$ :: *complex Matrix.vec*
assumes *vw*: *dim-vec* v = *dim-vec* w
shows $\langle u \mid v + w\rangle = \langle u \mid v\rangle + \langle u \mid w\rangle$
⟨*proof*⟩

lemma *inner-prod-right-minus'*:
fixes $u\ v\ w$ :: *complex Matrix.vec*
assumes *vw*: *dim-vec* v = *dim-vec* w
shows $\langle u \mid v - w\rangle = \langle u \mid v\rangle - \langle u \mid w\rangle$
⟨*proof*⟩

lemma *inner-prod-left-minus'*:
fixes $u\ v\ w$ :: *complex Matrix.vec*
assumes *uv*: *dim-vec* u = *dim-vec* v
and *uw*: *dim-vec* u = *dim-vec* w
shows $\langle u - v \mid w\rangle = \langle u \mid w\rangle - \langle v \mid w\rangle$
⟨*proof*⟩

lemma *dagger-of-smult*:
fixes γ :: *complex*
and U :: *complex Matrix.mat*
shows $(\gamma \cdot_m U)^\dagger = (cnj\ \gamma) \cdot_m (U^\dagger)$
⟨*proof*⟩

The adjoint of an $m \times n$ matrix is $n \times m$.

lemma *dagger-carrier*:
assumes $M \in$ *carrier-mat m n*
shows $(M^\dagger) \in$ *carrier-mat n m*
⟨*proof*⟩

definition *householder* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
householder w ≡
Matrix.mat (*dim-vec w*) (*dim-vec w*)
(λ(*i*,*j*). (*if i* = *j then 1 else 0*) − *2* ∗ (*w* $ *i*) ∗ *cnj* (*w* $ *j*))

notation *householder* (U_H)

lemma *householder-index*:
assumes *i* < *dim-vec w* **and** *j* < *dim-vec w*
shows (*householder w*) $$ (*i*,*j*) =
(*if i* = *j then 1 else 0*) − *2* ∗ (*w* $ *i*) ∗ *cnj* (*w* $ *j*)
⟨*proof*⟩

lemma *householder-outer-prod*:

householder w = 1$_m$ *(dim-vec w) − 2 ·*$_m$ *(w ⊘ w)*
⟨*proof*⟩

lemma *householder-hermitian*:
shows *(householder w)*† *= householder w*
⟨*proof*⟩

lemma *householder-unitary*:
assumes *norm-w*: *‖w‖ = (1 :: real)*
shows *unitary (householder w)*
⟨*proof*⟩

lemma *householder-mult-vec-index*:
fixes *w x :: complex Matrix.vec*
assumes *xdim*: *dim-vec x = dim-vec w*
and *i*: *i < dim-vec w*
shows *(householder w ∗*$_v$ *x) $ i =*
x $ i − 2 ∗ (w $ i) ∗ ⟨w | x⟩
⟨*proof*⟩

lemma *householder-mult-vec*:
fixes *w x :: complex Matrix.vec*
assumes *xdim*: *dim-vec x = dim-vec w*
shows *householder w ∗*$_v$ *x =*
x − (2 ∗ ⟨w | x⟩) ·$_v$ *w*
⟨*proof*⟩

Lemma (notes) (Schur)**.** Given $x, y \in \mathbb{C}^n$ with $\|x\| = \|y\|$ there are $w \in \mathbb{C}^n$ with $\|w\| = 1$ and θ such that $U_w x = e^{i\theta} y$.

The formal statement writes $e^{i\theta}$ as *Complex (cos ϑ) (sin ϑ)*; the case $x = 0$ is treated separately.

lemma *schur-lemma-zero-case*:
fixes *x y :: complex Matrix.vec*
assumes *n-pos*: *n > 0*
and *xdim*: *dim-vec x = n*
and *ydim*: *dim-vec y = n*
and *norm-eq*: *‖x‖ = ‖y‖*
and *x-zero-norm*: *‖x‖ = 0*
shows *∃ w :: complex Matrix.vec. ∃ ϑ :: real.*
dim-vec w = n ∧ ‖w‖ = 1 ∧
householder w ∗$_v$ *x = Complex (cos ϑ) (sin ϑ) ·*$_v$ *y*
⟨*proof*⟩

thm *of-real-divide*[**where** *x = cmod (⟨y | x⟩) ^ 2* **and** *y = cmod (⟨y | x⟩)*, *symmetric*]

lemma *schur-lemma*:
fixes *x y :: complex Matrix.vec*
assumes *n-pos*: *n > 0*
and *xdim*: *dim-vec x = n*
and *ydim*: *dim-vec y = n*
and *norm-eq*: *‖x‖ = ‖y‖*
shows *∃ w :: complex Matrix.vec. ∃ ϑ :: real.*
dim-vec w = n ∧ ‖w‖ = 1 ∧
householder w ∗$_v$ *x = Complex (cos ϑ) (sin ϑ) ·*$_v$ *y*

⟨*proof*⟩

Theorem (notes) (Schur)**.** For every $A \in M_n$ there is a unitary U such that U^*AU is upper triangular.

The notes prove this by induction, peeling off one eigenvector with the Householder unitary of Schur's lemma and recursing on the $(n-1) \times (n-1)$ block. The formal proof follows the same route: *schur-triangular-similar* is the similarity version available from the Jordan-Normal-Form library, and the block-matrix lemmas that follow assemble the unitary step by step.

lemma *schur-triangular-similar*:
  fixes A :: *complex Matrix.mat*
  assumes *sq*: $A \in$ *carrier-mat n n*
  shows $\exists\, T$ :: *complex Matrix.mat*.
        $T \in$ *carrier-mat n n* $\wedge$
        upper-triangular $T \wedge$
        similar-mat A T
⟨*proof*⟩

lemma *cpx-mat-cnj-four-block*:
  assumes *A*: $A \in$ *carrier-mat nr1 nc1*
    and *B*: $B \in$ *carrier-mat nr1 nc2*
    and *C*: $C \in$ *carrier-mat nr2 nc1*
    and *D*: $D \in$ *carrier-mat nr2 nc2*
  shows (*four-block-mat* A B C $D)^\star =$
        four-block-mat $(A^\star)$ $(B^\star)$ $(C^\star)$ $(D^\star)$
⟨*proof*⟩

lemma *dagger-four-block-mat-complex*:
  assumes *A*: $A \in$ *carrier-mat nr1 nc1*
    and *B*: $B \in$ *carrier-mat nr1 nc2*
    and *C*: $C \in$ *carrier-mat nr2 nc1*
    and *D*: $D \in$ *carrier-mat nr2 nc2*
  shows (*four-block-mat* A B C $D)^\dagger =$
        four-block-mat $(A^\dagger)$ $(C^\dagger)$ $(B^\dagger)$ $(D^\dagger)$
⟨*proof*⟩

lemma *dagger-zero-mat*[*simp*]:
  $(0_m\ m\ n$ :: *complex Matrix.mat*$)^\dagger = 0_m\ n\ m$
  ⟨*proof*⟩

lemma *block-diag-mult*:
  fixes A B D E :: *complex Matrix.mat*
  assumes *A*: $A \in$ *carrier-mat n n*
    and *B*: $B \in$ *carrier-mat n n*
    and *D*: $D \in$ *carrier-mat m m*
    and *E*: $E \in$ *carrier-mat m m*
  shows *four-block-mat* A $(0_m\ n\ m)$ $(0_m\ m\ n)$ $D *$
        four-block-mat B $(0_m\ n\ m)$ $(0_m\ m\ n)$ $E =$
        four-block-mat $(A * B)$ $(0_m\ n\ m)$ $(0_m\ m\ n)$ $(D * E)$
⟨*proof*⟩

lemma *block-diag-unitary*:
  fixes U V :: *complex Matrix.mat*
  assumes *U*: $U \in$ *carrier-mat n n*
    and *V*: $V \in$ *carrier-mat m m*
    and *uU*: *unitary* U
    and *uV*: *unitary* V

shows *unitary* (*four-block-mat U* (0_m *n m*) (0_m *m n*) *V*)
⟨*proof*⟩

lemma *complex-matrix-has-eigenvector*:
fixes *A* :: *complex Matrix.mat*
assumes *sq*: *A* ∈ *carrier-mat n n*
and *n-pos*: *n* > *0*
shows ∃ *e v*. *eigenvector A v e*
⟨*proof*⟩

lemma *orthonormal-mat-unitary*:
assumes *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
shows *unitary* (*mat-of-cols n vs*)
⟨*proof*⟩

lemma *eigenvec-normalize*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *sq*: *A* ∈ *carrier-mat n n*
and *vdim*: *v* ∈ *carrier-vec n*
and *vne*: *v* ≠ 0_v *n*
and *ev*: *A* ∗$_v$ *v* = *e* ·$_v$ *v*
defines v_1 ≡ (*1* / ‖*v*‖) ·$_v$ *v*
shows *A* ∗$_v$ v_1 = *e* ·$_v$ v_1 **and** ‖v_1‖ = *1*
⟨*proof*⟩

definition *vec-normalize* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
vec-normalize v = (*if v* = 0_v (*dim-vec v*) *then v else* (*1* / ‖*v*‖) ·$_v$ *v*)

lemma *normalized-vec-norm*:
assumes *vdim*: *v* ∈ *carrier-vec n*
and *vne*: *v* ≠ 0_v *n*
shows ‖*vec-normalize v*‖ = *1*
⟨*proof*⟩

lemma *four-block-mat-mult-left-block-diag*:
fixes *U* :: *complex Matrix.mat*
and *A11* :: *complex Matrix.mat*
and *A12* :: *complex Matrix.mat*
and *A21* :: *complex Matrix.mat*
and *A22* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat n n*
and *A11*: *A11* ∈ *carrier-mat 1 1*
and *A12*: *A12* ∈ *carrier-mat 1 n*
and *A21*: *A21* ∈ *carrier-mat n 1*
and *A22*: *A22* ∈ *carrier-mat n n*
shows
(*four-block-mat* (1_m *1* :: *complex Matrix.mat*)
(0_m *1 n* :: *complex Matrix.mat*)
(0_m *n 1* :: *complex Matrix.mat*)
U)
∗
(*four-block-mat A11 A12 A21 A22*)
=
four-block-mat A11 A12 (*U* ∗ *A21*) (*U* ∗ *A22*)
⟨*proof*⟩

lemma *cscalar-prod-is-inner-prod-swap*:

fixes u v :: *complex Matrix.vec*
assumes $u \in$ *carrier-vec* n **and** $v \in$ *carrier-vec* n
shows $u \cdot_c v = \langle v|u\rangle$
⟨*proof*⟩

lemma *vec-normalize-dim*[*simp*]:
dim-vec (*vec-normalize* v) = *dim-vec* v
⟨*proof*⟩

lemma *vec-normalize-carrier*[*simp*]:
assumes $v \in$ *carrier-vec* n
shows *vec-normalize* $v \in$ *carrier-vec* n
⟨*proof*⟩

lemma *vec-normalize-unit*:
assumes *vdim*: $v \in$ *carrier-vec* n **and** *vnorm*: $\|v\| = 1$
shows *vec-normalize* $v = v$
⟨*proof*⟩

Any unit vector extends to an orthonormal basis (needed for the induction).

lemma *unit-vec-extends-to-onb*:
fixes v :: *complex Matrix.vec*
assumes *vdim*: $v \in$ *carrier-vec* n
and *vnorm*: $\|v\| = 1$
shows $\exists$ *vs*. *orthonormal-list* n *vs* $\wedge$ *length vs* $= n \wedge$ *vs* ! 0 $= v$
⟨*proof*⟩

lemma *schur-induct*:
$\forall A$ :: *complex Matrix.mat*.
$A \in$ *carrier-mat* n $n \longrightarrow$
($\exists U$ :: *complex Matrix.mat*. $\exists T$ :: *complex Matrix.mat*.
$U \in$ *carrier-mat* n $n \wedge$
$T \in$ *carrier-mat* n $n \wedge$
unitary $U \wedge$
upper-triangular $T \wedge$
$(((U^\dagger) * A) * U) = T$)
⟨*proof*⟩

theorem *schur-theorem*:
fixes A :: *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat* n n
shows $\exists U$ :: *complex Matrix.mat*. $\exists T$ :: *complex Matrix.mat*.
$U \in$ *carrier-mat* n $n \wedge$
$T \in$ *carrier-mat* n $n \wedge$
unitary $U \wedge$
upper-triangular $T \wedge$
$(((U^\dagger) * A) * U) = T$
⟨*proof*⟩

Remark. The characteristic polynomial of A is $p_A(t) = \det(tI - A)$, a polynomial of degree n.

Corollary (notes). $\mathrm{Tr}(A) = \lambda_1 + \cdots + \lambda_n$, where the λ_j are the eigenvalues of A.

Formally the eigenvalues are listed as the roots of the characteristic polynomial, $p_A(t) = \prod_j (t - \lambda_j)$, read off the diagonal of the Schur form.

corollary *trace-eq-sum-eigenvalues*:
fixes A :: *complex Matrix.mat*

assumes *sq*: $A \in$ *carrier-mat n n*
obtains *eigs* :: *complex list*
where *length eigs* $= n$
and *Tr A* $=$ *sum-list eigs*
and *char-poly A* $= (\prod e \leftarrow$ *eigs*. $[:-e, 1:])$
⟨*proof*⟩

end

theory *Day04-Hermitian-Matrices*
imports *Day03-Unitary-Matrices*
begin

8 Day 4 (2 September 2011): Hermitian matrices; positive semidefinite and positive definite matrices

8.1 Hermitian matrices

Definition (notes). $H \in M_n$ is *Hermitian* (self-adjoint) if $H = H^*$.

Theorem (notes). The following are equivalent: (a) H is Hermitian; (b) there is a unitary U with $U^*HU = D$ diagonal with real entries; (c) $\mathbb{C}^n$ has an orthonormal basis of eigenvectors of H with real eigenvalues; (d) $\langle x|Hx\rangle \in \mathbb{R}$ for all $x \in \mathbb{C}^n$.

Formally *hermitian H* is $H^\dagger = H$; (a)⇒(b) is the spectral theorem *hermitian-spectral-decomp* below, (a)⇔(d) is *hermitian-iff-real-quad-form*, and (c) is read off from (b): the columns of U are an orthonormal eigenbasis. The proof of (d)⇒(a) needs the following lemma, which the notes note fails over $\mathbb{R}$ (take $T = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$).

definition *hermitian* :: *complex Matrix.mat* ⇒ *bool* **where**
hermitian A ≡ $A^\dagger = A$

lemma *hermitian-square*:
assumes *hermitian A*
shows *dim-row A* $=$ *dim-col A*
⟨*proof*⟩

lemma *hermitian-index*:
assumes *hermitian A*
and $i <$ *dim-row A* **and** $j <$ *dim-col A*
shows *A* $$ (*i*,*j*) $=$ *cnj* (*A* $$ (*j*,*i*))
⟨*proof*⟩

lemma *inner-T-double-sum*:
fixes *T* :: *complex Matrix.mat* **and** *u v* :: *complex Matrix.vec*
assumes *T-carr*: $T \in$ *carrier-mat n n*
and *udim*: *dim-vec u* $= n$ **and** *vdim*: *dim-vec v* $= n$
shows $\langle u \mid T *_v v\rangle =$
$(\sum k = 0..<n. \sum l = 0..<n.$ *cnj* (*u* $ *k*) $*$ *T* $$ (*k*, *l*) $*$ *v* $ *l*)
⟨*proof*⟩

lemma *inner-unit-T-unit*:

fixes T :: *complex Matrix.mat*
assumes *T-carr*: $T \in$ *carrier-mat n n*
and *p*: $p < n$ **and** *q*: $q < n$
shows $\langle$*unit-vec n p* | $T *_v$ *unit-vec n q*$\rangle = T$ $$ (p, q)
⟨*proof*⟩

lemma *sum-indicator-T-indicator*:
fixes T :: *complex Matrix.mat*
assumes *p*: $p < n$ **and** *q*: $q < n$
shows $(\sum k = 0..<n.\ \sum l = 0..<n.$
(*if* $k = p$ *then* (*1* :: *complex*) *else* *0*) $*$ T $$ $(k, l) *$
(*if* $l = q$ *then* (*1* :: *complex*) *else* *0*))
$= T$ $$ (p, q)
⟨*proof*⟩

lemma *polarization-helper*:
fixes T :: *complex Matrix.mat* **and** *a b c d* :: *complex*
assumes *sq*: $T \in$ *carrier-mat n n*
and *i*: $i < n$ **and** *j*: $j < n$
shows $\langle a \cdot_v$ *unit-vec n i* $+ b \cdot_v$ *unit-vec n j*
| $T *_v$ ($c \cdot_v$ *unit-vec n i* $+ d \cdot_v$ *unit-vec n j*)$\rangle$
$=$ *cnj a* $*$ *c* $*$ T $$ $(i, i) +$ *cnj a* $*$ *d* $*$ T $$ (i, j)
$+$ *cnj b* $*$ *c* $*$ T $$ $(j, i) +$ *cnj b* $*$ *d* $*$ T $$ (j, j)
⟨*proof*⟩

Lemma (notes). For $T \in M_n$, if $\langle x|Tx\rangle = 0$ for all $x \in \mathbb{C}^n$, then $T = 0$.

lemma *quadratic-form-zero*:
fixes T :: *complex Matrix.mat*
assumes *sq*: $T \in$ *carrier-mat n n*
and *zero*: $\forall x$ :: *complex Matrix.vec*. *dim-vec* $x = n \longrightarrow \langle x \mid T *_v x\rangle = 0$
shows $T = 0_m$ *n n*
⟨*proof*⟩

lemma *mult-mat-vec-index*:
fixes A :: *'a*::{*comm-ring-1*} *Matrix.mat*
and x :: *'a Matrix.vec*
assumes *A*: $A \in$ *carrier-mat m n*
and *x*: $x \in$ *carrier-vec n*
and *j*: $j < m$
shows $(A *_v x)$ $ $j = (\sum i = 0..<n.\ A$ $$ $(j, i) * x$ $ $i)$
⟨*proof*⟩

The adjoint moves across the inner product, $\langle x|My\rangle = \langle M^\dagger x|y\rangle$: the defining property of $M^\dagger$.

lemma *inner-prod-adjoint-swap*:
fixes M :: *complex Matrix.mat*
and $x\ y$ :: *complex Matrix.vec*
assumes *M*: $M \in$ *carrier-mat n m*
and *x*: *dim-vec* $x = n$
and *y*: *dim-vec* $y = m$
shows $\langle x \mid M *_v y\rangle = \langle M^\dagger *_v x \mid y\rangle$
⟨*proof*⟩

Sums and real multiples of Hermitian matrices are Hermitian; the identity is Hermitian.

lemma *hermitian-plus*:
fixes $A\ B$ :: *complex Matrix.mat*

assumes *Aherm*: *hermitian A* **and** *Bherm*: *hermitian B*
and *Ac*: $A \in$ *carrier-mat k k* **and** *Bc*: $B \in$ *carrier-mat k k*
shows *hermitian* $(A + B)$
⟨*proof*⟩

lemma *hermitian-identity*:
hermitian $(1_m\ k ::$ *complex Matrix.mat*$)$
⟨*proof*⟩

lemma *hermitian-real-scale*:
fixes $A ::$ *complex Matrix.mat*
assumes *hermitian A* $A \in$ *carrier-mat k k*
shows *hermitian* (*complex-of-real d* $\cdot_m$ A)
⟨*proof*⟩

The converses (g)⇒(a) and (h)⇒(a) of the Day 3 theorem on unitaries, postponed until the lemma $\langle x|Tx\rangle = 0\ \forall x \Rightarrow T = 0$ was available: a square matrix that preserves lengths, or inner products, is unitary (the library proves the length-preserving case in its own matrix notation as *length-preserving-is-unitary*).

lemma *unitary-of-isometry*:
fixes $U ::$ *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat n n*
and *iso*: $\bigwedge v ::$ *complex Matrix.vec. dim-vec* $v = n \Longrightarrow \|U *_v v\| = \|v\|$
shows *unitary U*
⟨*proof*⟩

lemma *unitary-of-inner-prod-preserving*:
fixes $U ::$ *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat n n*
and *pres*: $\bigwedge u\ v ::$ *complex Matrix.vec. dim-vec* $u = n \Longrightarrow$ *dim-vec* $v = n \Longrightarrow$
$\langle U *_v u \mid U *_v v\rangle = \langle u \mid v\rangle$
shows *unitary U*
⟨*proof*⟩

lemma *hermitian-iff-real-quad-form*:
assumes *sq*: $A \in$ *carrier-mat n n*
shows *hermitian* $A \longleftrightarrow (\forall x ::$ *complex Matrix.vec. dim-vec* $x = n \longrightarrow \langle x \mid A *_v x\rangle \in \mathbb{R})$
⟨*proof*⟩

8.2 The spectral theorem for Hermitian matrices

(a)⇒(b) of the theorem above: by Schur, $U^*HU = T$ is upper triangular, and $T^* = (U^*HU)^* = T$ forces T to be diagonal with real entries. *diag-mat-of-real D* is the complex diagonal matrix with the real list D on its diagonal.

definition *diag-mat-of-real* :: *real list* ⇒ *complex Matrix.mat* **where**
diag-mat-of-real D = *mat-diag* (*length D*) (λ*i*. *complex-of-real* (*D* ! *i*))

lemma *dim-row-diag-mat-of-real*[*simp*]: *dim-row* (*diag-mat-of-real D*) = *length D*
⟨*proof*⟩

lemma *dim-col-diag-mat-of-real*[*simp*]: *dim-col* (*diag-mat-of-real D*) = *length D*
⟨*proof*⟩

lemma *diag-mat-of-real-carrier*[*simp*]:
diag-mat-of-real D ∈ *carrier-mat* (*length D*) (*length D*)
⟨*proof*⟩

lemma *diag-mat-of-real-index*[*simp*]:
assumes $i < \textit{length } D$ **and** $j < \textit{length } D$
shows *diag-mat-of-real D* $$ $(i, j) =$
$(\textit{if } i = j \textit{ then complex-of-real } (D\ !\ i) \textit{ else } 0)$
⟨*proof*⟩

lemma *diag-mat-of-real-mult-vec-index*:
fixes y :: *complex Matrix.vec*
assumes *ydim*: *dim-vec* $y = \textit{length } D$
and *i*: $i < \textit{length } D$
shows $(\textit{diag-mat-of-real } D *_v y)\ \$\ i = \textit{complex-of-real } (D\ !\ i) * y\ \$\ i$
⟨*proof*⟩

lemma *diag-mat-of-real-quad-Re-nonneg*:
fixes y :: *complex Matrix.vec*
assumes *ydim*: *dim-vec* $y = \textit{length } D$
and *D-nonneg*: $\bigwedge i.\ i < \textit{length } D \Longrightarrow 0 \leq D\ !\ i$
shows $0 \leq \textit{Re } \langle y \mid \textit{diag-mat-of-real } D *_v y \rangle$
⟨*proof*⟩

theorem *hermitian-spectral-decomp*:
fixes H :: *complex Matrix.mat*
assumes *sq*: $H \in \textit{carrier-mat } n\ n$
and *herm*: *hermitian H*
shows $\exists (U :: \textit{complex Matrix.mat})\ (D :: \textit{real list}).$
$U \in \textit{carrier-mat } n\ n \wedge$
unitary U $\wedge$
$\textit{length } D = n \wedge$
$(U^\dagger) * H * U = \textit{diag-mat-of-real } D$
⟨*proof*⟩

corollary *hermitian-real-eigenvalues*:
fixes H :: *complex Matrix.mat*
assumes *sq*: $H \in \textit{carrier-mat } n\ n$
and *herm*: *hermitian H*
and *eig*: $H *_v v = \textit{lam} \cdot_v v$
and *v-nonzero*: $v \neq 0_v\ n$
and *vdim*: *dim-vec* $v = n$
shows *lam* $\in \mathbb{R}$
⟨*proof*⟩

8.3 Positive semidefinite and positive definite matrices

Definition (notes). $P \in M_n$ is *positive semidefinite*, $P \geq 0$, if $\langle x|Px\rangle \geq 0$ for all $x \in \mathbb{C}^n$, and *positive definite*, $P > 0$, if $\langle x|Px\rangle > 0$ for all $x \neq 0$.

By part (d) of the previous theorem, $P \geq 0$ implies $P = P^*$. The formal predicates *psd* and *pd* carry Hermiticity as an explicit conjunct together with the real-part inequality; by *hermitian-iff-real-quad-form* this is equivalent to the notes' definition.

Theorem (notes). $P \geq 0$ if and only if $P = P^*$ and all eigenvalues of P are non-negative.

Corollary (notes). $P > 0$ if and only if $P = P^*$ and all eigenvalues of P are strictly positive.

Theorem (notes). $P \geq 0$ if and only if $P = \sum_{i=1}^{m} |v_i\rangle\langle v_i|$ for some vectors $v_1, \ldots, v_m$.

Key: if $P = \sum_i |v_i\rangle\langle v_i|$ then $\langle x|Px\rangle = \sum_i |\langle v_i|x\rangle|^2$.

Theorem (notes). Let $P = \sum_{i=1}^{m} |v_i\rangle\langle v_i| \in M_n$. Then $P > 0$ if and only if $\text{span}\{v_1, \ldots, v_m\} = \mathbb{C}^n$.

The notes' form of the definition is *psd-iff-quad-form-nonneg*; the corollary on positive definite matrices is *pd-iff-pos-eigenvalues* (proved after the square root); the last theorem is formalised with "the v_i span $\mathbb{C}^n$" phrased as "no nonzero x is orthogonal to every v_i".

definition *psd* :: *complex Matrix.mat* ⇒ *bool* **where**
psd P ≡ *hermitian P* ∧
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *dim-row P* ⟶
Re (⟨*x* | *P* ∗*v x*⟩) ≥ *0*)

definition *pd* :: *complex Matrix.mat* ⇒ *bool* **where**
pd P ≡ *hermitian P* ∧
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *dim-row P* ⟶
x ≠ *0v* (*dim-row P*) ⟶
Re (⟨*x* | *P* ∗*v x*⟩) > *0*)

notation *psd* (- ≥*M* *0*)
notation *pd* (- >*M* *0*)

lemma *psd-hermitian*: *psd P* ⟹ *hermitian P*
⟨*proof*⟩

The notes' definition, without the explicit Hermiticity conjunct: $P \geq 0$ iff every quadratic form $\langle x|Px\rangle$ is a non-negative real number.

lemma *psd-iff-quad-form-nonneg*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *psd P* ⟷
(∀ *x* :: *complex Matrix.vec*. *dim-vec x* = *n* ⟶
⟨*x* | *P* ∗*v x*⟩ ∈ ℝ ∧ *0* ≤ *Re* (⟨*x* | *P* ∗*v x*⟩))
⟨*proof*⟩

lemma *psd-MdagM*:
assumes *M*: *M* ∈ *carrier-mat n m*
shows *psd* (*M*† ∗ *M*)
⟨*proof*⟩

$P \geq 0$ iff P is Hermitian with non-negative eigenvalues:

theorem *psd-iff-nonneg-eigenvalues*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *psd P* ⟷
(*hermitian P* ∧
(∀ *mu* :: *complex*. ∀ *v* :: *complex Matrix.vec*.
v ≠ *0v n* ∧ *dim-vec v* = *n* ∧ *P* ∗*v v* = *mu* ·*v v* ⟶ *Re mu* ≥ *0*))
⟨*proof*⟩

$P \geq 0$ iff $P = \sum_i |v_i\rangle\langle v_i|$. First the direction ⇐:

lemma *outer-sum-psd*:
assumes *len*: *length vs* = *k*
and *vdim*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
shows *psd* (*outer-sum n vs*)
⟨*proof*⟩

lemma *hermitian-spectral-decomp-eq*:
  assumes *Uc*: $U \in$ *carrier-mat n n*
    and *Pcar*: $P \in$ *carrier-mat n n*
    and *unit*: *unitary U*
    and *spec*: $U^\dagger * P * U =$ *diag-mat-of-real D*
  shows $P = U *$ *diag-mat-of-real D* $* (U^\dagger)$
⟨*proof*⟩

lemma *hermitian-spectral-decomp-right*:
  assumes *Uc*: $U \in$ *carrier-mat n n*
    and *Pcar*: $P \in$ *carrier-mat n n*
    and *unit*: *unitary U*
    and *spec*: $U^\dagger * P * U =$ *diag-mat-of-real D*
  shows $P * U = U *$ *diag-mat-of-real D*
  ⟨*proof*⟩

The direction ⇒, from the spectral theorem: $P = \sum_j \lambda_j |u_j\rangle\langle u_j| = \sum_j |\lambda_j^{1/2} u_j\rangle\langle\lambda_j^{1/2} u_j|$.

The spectral theorem for positive matrices: $P = U \operatorname{diag}(D)\, U^\dagger$ with $D \geq 0$.

lemma *psd-spectral-decomp-nonneg*:
  fixes *P* :: *complex Matrix.mat*
  assumes *sq*: $P \in$ *carrier-mat n n*
    and *pos*: *psd P*
  shows ∃ (*U* :: *complex Matrix.mat*) (*D* :: *real list*).
      $U \in$ *carrier-mat n n* ∧ *unitary U* ∧ *length D* $= n$ ∧
      (∀ *i* < *n*. 0 ≤ *D* ! *i*) ∧
      $P = U *$ *diag-mat-of-real D* $* (U^\dagger)$
⟨*proof*⟩

theorem *psd-kraus-decomp*:
  fixes *P* :: *complex Matrix.mat*
  assumes *sq*: $P \in$ *carrier-mat n n*
    and *pos*: *psd P*
  shows ∃ *vs* :: *complex Matrix.vec list*.
      length vs $= n$ ∧
      (∀ *v* ∈ *set vs*. *dim-vec v* $= n$) ∧
      outer-sum n vs $= P$
⟨*proof*⟩

$P = \sum_i |v_i\rangle\langle v_i|$ is positive definite iff the v_i span $\mathbb{C}^n$:

Entries of $\sum_i |v_i\rangle\langle v_i|$ for an arbitrary list of vectors.

lemma *outer-sum-entry-sum-list*:
  assumes *vs-dim*: ∀ *v* ∈ *set vs*. *dim-vec v* $= n$
    and *i-lt*: $i < n$ **and** *j-lt*: $j < n$
  shows *outer-sum n vs* $\$\$ (i, j) =$
      sum-list (*map* (λ*v*. *v* $\$ i *$ *cnj* (*v* $\$ j$)) *vs*)
⟨*proof*⟩

lemma *mat-of-cols-dagger-mult-vec-index*:
  assumes *xdim*: *dim-vec x* $= n$
    and *vs-dim*: ∀ *v* ∈ *set vs*. *dim-vec v* $= n$
    and *i*: *i* < *length vs*
  shows $(((\textit{mat-of-cols } n \textit{ vs})^\dagger) *_v x)\ \$\ i = \langle vs\ !\ i \mid x\rangle$
⟨*proof*⟩

lemma *inner-prod-outer-sum-general*:

assumes *xdim*: *dim-vec x = n*
and *vs-dim*: $\forall\, v \in$ *set vs. dim-vec v = n*
shows $\langle x \mid$ *outer-sum n vs* $*_v\ x\rangle =$
$\langle((\textit{mat-of-cols}\ n\ \textit{vs})^\dagger)\ *_v\ x \mid$
$((\textit{mat-of-cols}\ n\ \textit{vs})^\dagger)\ *_v\ x\rangle$
⟨*proof*⟩

lemma *diag-mat-of-real-hermitian*:
shows *hermitian* (*diag-mat-of-real D*)
⟨*proof*⟩

$U\,\mathrm{diag}(D)\,U^\dagger$ is positive whenever the real entries of D are non-negative.

lemma *psd-U-diag-Udag*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
assumes *Uc*: *U* ∈ *carrier-mat n n*
and *Dlen*: *length D = n*
and *Dnn*: $\bigwedge i.\ i < n \Longrightarrow 0 \le D\,!\,i$
shows *psd* (*U* ∗ *diag-mat-of-real D* ∗ ($U^\dagger$))
⟨*proof*⟩

lemma *diag-mat-of-real-sqrt-square*:
assumes *D-nonneg*: $\bigwedge i.\ i <$ *length D* $\Longrightarrow 0 \le D\,!\,i$
shows *diag-mat-of-real* (*map sqrt D*) ∗ *diag-mat-of-real* (*map sqrt D*) = *diag-mat-of-real D*
⟨*proof*⟩

theorem *pd-iff-spanning*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
and *decomp*: *P = outer-sum n vs*
and *vs-dim*: $\forall\, v \in$ *set vs. dim-vec v = n*
shows *pd P* ⟷
(∀ *x* :: *complex Matrix.vec. dim-vec x = n* ⟶ *x* ≠ 0_v *n* ⟶
(∃ *v* ∈ *set vs.* $\langle v \mid x\rangle \neq 0$))
⟨*proof*⟩

8.4 The positive square root

Remark (Day 8)**.** Given $P \ge 0$ with $U^*PU = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, $\lambda_i \ge 0$, put $P^{1/2} = U\,\mathrm{diag}(\lambda_1^{1/2}, \ldots, \lambda_n^{1/2})U^*$. Then $P^{1/2} \ge 0$ and $(P^{1/2})^2 = P$.

psd-sqrt P is defined as a choice of such a square root; *psd-sqrt-exists* is the existence statement and *psd-sqrt-props* the defining properties.

definition *psd-sqrt* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
psd-sqrt P ≡ (*SOME Q. Q* ∈ *carrier-mat* (*dim-row P*) (*dim-row P*) ∧ *psd Q* ∧ *Q* ∗ *Q* = *P*)

theorem *psd-sqrt-exists*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
and *pos*: *psd P*
shows ∃ *Q* :: *complex Matrix.mat.*
Q ∈ *carrier-mat n n* ∧ *psd Q* ∧ *Q* ∗ *Q* = *P*
⟨*proof*⟩
lemma *psd-sqrt-props*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
and *pos*: *psd P*
shows *psd-sqrt P* ∈ *carrier-mat n n*

and *psd* (*psd-sqrt P*)
and *psd-sqrt P* ∗ *psd-sqrt P* = *P*
⟨*proof*⟩

The corollary of Day 4: $P > 0$ iff $P = P^*$ and all eigenvalues are strictly positive.

lemma *pd-iff-pos-eigenvalues*:
fixes *P* :: *complex Matrix.mat*
assumes *sq*: *P* ∈ *carrier-mat n n*
shows *pd P* ⟷
(*hermitian P* ∧
(∀ *mu* :: *complex*. ∀ *v* :: *complex Matrix.vec*.
v ≠ 0_v *n* ∧ *dim-vec v* = *n* ∧ *P* ∗$_v$ *v* = *mu* ·$_v$ *v* ⟶ 0 < *Re mu*))
⟨*proof*⟩

end
theory *Day04-Hermitian-Converse*
imports *Day18-CP-Maps*
begin

9 Day 4: the converse directions for Hermitian matrices (D4-1)

lemma *unitary-real-diag-imp-hermitian*:
assumes *U*: *U* ∈ *carrier-mat n n* **and** *u*: *unitary U* **and** *H*: *H* ∈ *carrier-mat n n*
and *D*: *length D* = *n* **and** *spec*: $U^\dagger$ ∗ *H* ∗ *U* = *diag-mat-of-real D*
shows *hermitian H*
⟨*proof*⟩

lemma *eigenbasis-real-imp-hermitian*:
assumes *H*: *H* ∈ *carrier-mat n n* **and** *orth*: *orthonormal-list n vs* **and** *len*: *length vs* = *n*
and *eig*: ⋀*i*. *i* < *n* ⟹ *H* ∗$_v$ *vs* ! *i* = *complex-of-real* (*lam* ! *i*) ·$_v$ *vs* ! *i* **and** *lam*: *length lam* = *n*
shows *hermitian H*
⟨*proof*⟩

end

theory *Day05-Positivity-Tests*
imports *Day04-Hermitian-Matrices*
begin

10 Day 5 (7 September 2011): tests for positivity; direct sums and partitioned matrices

10.1 Practical tests for positivity

Given $P = P^*$, how does one decide whether $P \geq 0$ or $P > 0$? The notes give two tests. (1) Sylvester's criterion, in terms of the leading principal submatrices $P_k = (p_{ij})_{1 \leq i,j \leq k}$ (the upper left $k \times k$ corners):

Theorem (notes) (Sylvester's criterion)**.** Let $P = P^* \in M_n$. Then $P > 0$ if and only if $\det P_k > 0$ for $k = 1, \ldots, n$.

Each $\det P_k$ is real, as P_k is Hermitian. The same test with ≥ 0 does *not* characterise $P \geq 0$: $\begin{pmatrix} 0 & 0 \\ 0 & -1 \end{pmatrix}$ has leading minors 0 and 0 but is not positive. For $n = 2$ the criterion says

$\begin{pmatrix} a & b \\ \bar{b} & c \end{pmatrix} > 0$ iff $a > 0$ and $ac - |b|^2 > 0$. The notes state it without proof. It is not reproved in this development: it is imported from the AFP entry *Two Hermitian Results* and restated for *pd* and *hermitian* as `pd_iff_leading_minors_pos` (for $P \in M_n$ Hermitian, $P >_M 0$ iff $\det P_k > 0$ for all $k \leq n$) in the theory `Day05_Sylvester_Criterion`. That theory forms the separate session `Quantum_Computing_by_VP_Day05` with its own document (`Quantum_Computing_by_VP_Day05.pdf`), because the AFP entry builds on HOL-Analysis, HOL-Probability and QHLProver, which this document's sessions do not load. (2) Cholesky's algorithm:

Theorem (notes). If $P = (p_{ij})$ is $n \times n$ with $P = P^*$, then $P \geq 0$ if and only if $P - \left(\frac{p_{i1}p_{1j}}{p_{11}}\right)_{ij} \geq 0$.

The subtracted matrix agrees with P in the first row and column, so the difference is really an $(n-1) \times (n-1)$ matrix and the test can be repeated; when $P \geq 0$ the algorithm also writes P as a sum of rank-one matrices. The formal theorem adds the hypothesis $p_{11} > 0$, which the notes leave implicit (for $p_{11} = 0$ the quotient is undefined).

theorem *cholesky-reduction*:
  fixes *P* :: *complex Matrix.mat*
  assumes *sq*: *P* ∈ *carrier-mat n n*
    and *herm*: *hermitian P*
    and *n-pos*: *0* < *n*
    and *p11-pos*: *P $$ (0,0)* > *0*
  shows *psd P* ⟷
      psd (*Matrix.mat* (*n*−*1*) (*n*−*1*)
        (λ(*i*,*j*). *P $$ (i*+*1*, *j*+*1*) −
          P $$ (i+*1*, *0*) ∗ *cnj* (*P $$ (j*+*1*, *0*)) / *P $$ (0*,*0*)))
⟨*proof*⟩

10.2 Direct sums of vector spaces and partitioned matrices

Definition (notes). The *direct sum* of vector spaces V, W is $V \oplus W = \{(v, w) : v \in V, w \in W\}$ with componentwise operations and inner product $\langle (v_1, w_1) | (v_2, w_2) \rangle = \langle v_1 | v_2 \rangle_V + \langle w_1 | w_2 \rangle_W$.

Proposition (notes). If $v_1, \ldots, v_n$ is a basis of V and $w_1, \ldots, w_k$ a basis of W, then $(v_1, 0), \ldots, (v_n, 0), (0, w_1), \ldots$ is a basis of $V \oplus W$; hence $\dim(V \oplus W) = \dim V + \dim W$.

Key examples: $\mathbb{C}^{n+k} = \mathbb{C}^n \oplus \mathbb{C}^k$, and a linear map $T : \mathbb{C}^{n_1+k_1} \to \mathbb{C}^{n_2+k_2}$ has the block form $T = \begin{pmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{pmatrix}$ with $T\binom{v}{w} = \binom{T_{11}v + T_{12}w}{T_{21}v + T_{22}w}$. Formally the direct sum of vectors is the concatenation $v \oplus_v w$, and block matrices are the library's *four-block-mat*.

The direct sum of vectors is JNF's concatenation $@_v$.

abbreviation *direct-sum-vec* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* (**infixr** ⊕*v* *65*) **where**
  v ⊕*v* *w* ≡ *v* @*v* *w*

lemma *direct-sum-vec-left* [*simp*]:
  i < *dim-vec v* ⟹ (*v* ⊕*v* *w*) $ *i* = *v* $ *i*
  ⟨*proof*⟩

lemma *direct-sum-vec-right* [*simp*]:
  i < *dim-vec w* ⟹ (*v* ⊕*v* *w*) $ (*dim-vec v* + *i*) = *w* $ *i*
  ⟨*proof*⟩

lemma *direct-sum-inner-prod*:
assumes *vd1*: *dim-vec v1 = m* **and** *vd2*: *dim-vec v2 = m*
and *wd1*: *dim-vec w1 = p* **and** *wd2*: *dim-vec w2 = p*
shows $\langle v1 \oplus_v w1 \mid v2 \oplus_v w2\rangle = \langle v1 \mid v2\rangle + \langle w1 \mid w2\rangle$
⟨*proof*⟩

lemma *block-mat-apply*:
assumes *dim-vec v = k1* **and** *dim-vec w = k2*
and *T11* ∈ *carrier-mat n1 k1* **and** *T12* ∈ *carrier-mat n1 k2*
and *T21* ∈ *carrier-mat n2 k1* **and** *T22* ∈ *carrier-mat n2 k2*
shows *four-block-mat T11 T12 T21 T22* $*_v$ $(v \oplus_v w) =$
$(T11 *_v v + T12 *_v w) \oplus_v (T21 *_v v + T22 *_v w)$
⟨*proof*⟩

end
theory *Day05-Cholesky-Examples*
imports *QC-Prelims*
begin

11 Day 5: Cholesky's algorithm on the notes' two examples (D5-3)

11.1 One step of Cholesky's algorithm

For $P = (p_{ij})$ with $p_{11} \neq 0$ the notes subtract $R = (\frac{p_{i1}p_{1j}}{p_{11}})_{ij}$; $P - R$ vanishes in the first row and column, and what remains is the $(n-1) \times (n-1)$ matrix *chol-next P*.

definition *chol-R* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
chol-R P = Matrix.mat (*dim-row P*) (*dim-row P*) (λ(*i*, *j*). *P* $$ (*i*, *0*) ∗ *P* $$ (*0*, *j*) / *P* $$ (*0*, *0*))

definition *chol-next* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
chol-next P = Matrix.mat (*dim-row P − 1*) (*dim-row P − 1*) (λ(*i*, *j*). (*P − chol-R P*) $$ (*i + 1*, *j + 1*))

lemma *chol-R-carrier* [*simp*]: *chol-R P* ∈ *carrier-mat* (*dim-row P*) (*dim-row P*)
⟨*proof*⟩

lemma *hermitian-entry*:
assumes *hermitian P P* ∈ *carrier-mat n n i < n j < n*
shows *P* $$ (*i*, *j*) = *cnj* (*P* $$ (*j*, *i*))
⟨*proof*⟩

The test: for $P = P^*$ with $p_{11} > 0$, $P \geq 0$ iff the next matrix is ≥ 0 (from *cholesky-reduction*).

lemma *chol-test*:
assumes *P*: *P* ∈ *carrier-mat n n* **and** *h*: *hermitian P* **and** *n*: *0 < n* **and** *p*: *P* $$ (*0*, *0*) > *0*
shows *psd P* ⟷ *psd* (*chol-next P*)
⟨*proof*⟩

11.2 Small matrices entry by entry

lemma *mat3-eqI*:
assumes *A* ∈ *carrier-mat 3 3 B* ∈ *carrier-mat 3 3*
and ⋀*i j*. *i* ∈ {*0*, *1*, *2*} ⟹ *j* ∈ {*0*, *1*, *2*} ⟹ *A* $$ (*i*, *j*) = *B* $$ (*i*, *j*)
shows *A = B*
⟨*proof*⟩

lemma *mat2-eqI*:
assumes $A \in$ *carrier-mat 2 2* $B \in$ *carrier-mat 2 2*
and $\bigwedge i\ j.\ i \in \{0, 1\} \Longrightarrow j \in \{0, 1\} \Longrightarrow A\ \$\$\ (i, j) = B\ \$\$\ (i, j)$
shows $A = B$
⟨*proof*⟩

11.3 Example 1: a matrix that is not positive

The notes: for $P = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 7 \end{pmatrix}$, $R = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$ and $P - R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & -1 & -2 \end{pmatrix}$, so $P \not\geq 0$. Each matrix is checked below; the last step is the test vector $(1, 1)$ for the remaining 2×2 block.

definition *notes-P1* :: *complex Matrix.mat* **where**
notes-P1 = mat-of-rows-list 3 [[*1*, *2*, *3*], [*2*, *4*, *5*], [*3*, *5*, *7*]]

lemma *notes-P1-carrier*: *notes-P1 ∈ carrier-mat 3 3*
⟨*proof*⟩

lemma *notes-P1-hermitian*: *hermitian notes-P1*
⟨*proof*⟩

lemma *notes-P1-R*: *chol-R notes-P1 = mat-of-rows-list 3* [[*1*, *2*, *3*], [*2*, *4*, *6*], [*3*, *6*, *9*]]
⟨*proof*⟩

lemma *notes-P1-diff*: *notes-P1 − chol-R notes-P1 = mat-of-rows-list 3* [[*0*, *0*, *0*], [*0*, *0*, *−1*], [*0*, *−1*, *−2*]]
⟨*proof*⟩

lemma *notes-P1-next*: *chol-next notes-P1 = mat-of-rows-list 2* [[*0*, *−1*], [*−1*, *−2*]]
⟨*proof*⟩

lemma *not-psd-witness*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *x*: *dim-vec x = n* **and** *neg*: *Re* (*Quantum.inner-prod x* ($A *_v$ *x*)) *< 0*
shows ¬ *psd A*
⟨*proof*⟩

$\begin{pmatrix} 0 & -1 \\ -1 & -2 \end{pmatrix}$ has the value -4 at $(1, 1)$, so it is not positive.

lemma *notes-P1-next-not-psd*: ¬ *psd* (*mat-of-rows-list 2* [[*0*, *−1*], [*−1*, *−2*]])
⟨*proof*⟩

theorem *notes-example-1*: *hermitian notes-P1* ∧ ¬ *psd notes-P1*
⟨*proof*⟩

11.4 Example 2: a positive matrix, in two steps

The notes: for $P = \begin{pmatrix} 4 & 2 & 3 \\ 2 & 5 & 6 \\ 3 & 6 & 8 \end{pmatrix}$, $R = \frac{1}{4}\begin{pmatrix} 16 & 8 & 12 \\ 8 & 4 & 6 \\ 12 & 6 & 9 \end{pmatrix}$ and $P - R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & \frac{9}{2} \\ 0 & \frac{9}{2} & \frac{23}{4} \end{pmatrix}$. Repeating on $Q = \begin{pmatrix} 4 & \frac{9}{2} \\ \frac{9}{2} & \frac{23}{4} \end{pmatrix}$: $Q - \frac{1}{4}\begin{pmatrix} 4 \\ \frac{9}{2} \end{pmatrix}\begin{pmatrix} 4 & \frac{9}{2} \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & \frac{11}{16} \end{pmatrix}$, so $P \geq 0$.

definition *notes-P2* :: *complex Matrix.mat* **where**
notes-P2 = mat-of-rows-list 3 [[*4*, *2*, *3*], [*2*, *5*, *6*], [*3*, *6*, *8*]]

lemma *notes-P2-carrier*: *notes-P2 ∈ carrier-mat 3 3*
⟨*proof*⟩

lemma *notes-P2-hermitian*: *hermitian notes-P2*
⟨*proof*⟩

lemma *notes-P2-R*: *chol-R notes-P2 = (1/4) ·$_m$ mat-of-rows-list 3 [[16, 8, 12], [8, 4, 6], [12, 6, 9]]*
⟨*proof*⟩

lemma *notes-P2-diff*: *notes-P2 − chol-R notes-P2 = mat-of-rows-list 3 [[0, 0, 0], [0, 4, 9/2], [0, 9/2, 23/4]]*
⟨*proof*⟩

definition *notes-Q2* :: *complex Matrix.mat* **where**
notes-Q2 = mat-of-rows-list 2 [[4, 9/2], [9/2, 23/4]]

lemma *notes-P2-next*: *chol-next notes-P2 = notes-Q2*
⟨*proof*⟩

lemma *notes-Q2-carrier*: *notes-Q2 ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *notes-Q2-hermitian*: *hermitian notes-Q2*
⟨*proof*⟩

lemma *notes-Q2-R*: *chol-R notes-Q2 = (1/4) ·$_m$ mat-of-rows-list 2 [[16, 18], [18, 81/4]]*
⟨*proof*⟩

lemma *notes-Q2-diff*: *notes-Q2 − chol-R notes-Q2 = mat-of-rows-list 2 [[0, 0], [0, 11/16]]*
⟨*proof*⟩

The last matrix is $(\frac{11}{16}) = \frac{11}{16}\,|e\rangle\langle e|$, which is positive.

lemma *notes-Q2-next*: *chol-next notes-Q2 = complex-of-real (11/16) ·$_m$ (Matrix.vec 1 (λ-. 1) ⊘ Matrix.vec 1 (λ-. 1))*
⟨*proof*⟩

lemma *notes-Q2-next-psd*: *psd (chol-next notes-Q2)*
⟨*proof*⟩

theorem *notes-example-2*: *hermitian notes-P2 ∧ psd notes-P2*
⟨*proof*⟩

The notes' remark: the algorithm writes P as a sum of rank-one matrices, $P = \frac{1}{4}|v_1\rangle\langle v_1| + \frac{1}{4}|v_2\rangle\langle v_2| + \frac{11}{16}|e_3\rangle\langle e_3|$ with $v_1 = (4,2,3)$, $v_2 = (0,4,\frac{9}{2})$, $e_3 = (0,0,1)$ (the first columns of P, of $P - R$ and of the last step).

lemma *outer-prod-of-list*:
*vec-of-list l ⊘ vec-of-list l = Matrix.mat (length l) (length l) (λ(i, j). l ! i * cnj (l ! j))*
⟨*proof*⟩

lemma *notes-P2-rank-one-sum*:
notes-P2 = complex-of-real (1/4) ·$_m$ (vec-of-list [4, 2, 3] ⊘ vec-of-list [4, 2, 3])
+ complex-of-real (1/4) ·$_m$ (vec-of-list [0, 4, 9/2] ⊘ vec-of-list [0, 4, 9/2])
+ complex-of-real (11/16) ·$_m$ (vec-of-list [0, 0, 1] ⊘ vec-of-list [0, 0, 1])
⟨*proof*⟩

end
theory *Day05-Direct-Sum-Basis*
imports *Quantum-Computing-by-VP-3-10.Day05-Positivity-Tests*

begin

12 Day 5: a basis of $V \oplus W$ (D5-5)

A basis of $\mathbb{C}^n$ is a list of n vectors whose column matrix is invertible. The notes: if $v_1, \ldots, v_n$ and $w_1, \ldots, w_k$ are bases of V and W then $(v_i, 0)$, $(0, w_j)$ is a basis of $V \oplus W$, so $\dim(V \oplus W) = \dim V + \dim W$. The column matrix is block diagonal.

definition *direct-sum-basis* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* **where**
direct-sum-basis n k vs ws = *map* (λ*v*. *v* ⊕$_v$ 0$_v$ *k*) *vs* @ *map* (λ*w*. 0$_v$ *n* ⊕$_v$ *w*) *ws*

lemma *direct-sum-basis-block*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n length vs* = *n* **and** *ws*: ∀ *w* ∈ *set ws*. *dim-vec w* = *k length ws* = *k*
shows *mat-of-cols* (*n* + *k*) (*direct-sum-basis n k vs ws*)
= *four-block-mat* (*mat-of-cols n vs*) (0$_m$ *n k*) (0$_m$ *k n*) (*mat-of-cols k ws*)
⟨*proof*⟩

lemma *basis-inverse*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
obtains *G* **where** *G* ∈ *carrier-mat m m G* ∗ *mat-of-cols m fs* = 1$_m$ *m mat-of-cols m fs* ∗ *G* = 1$_m$ *m*
⟨*proof*⟩

theorem *direct-sum-basis-is-basis*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n length vs* = *n* **and** *iv*: *invertible-mat* (*mat-of-cols n vs*)
and *ws*: ∀ *w* ∈ *set ws*. *dim-vec w* = *k length ws* = *k* **and** *iw*: *invertible-mat* (*mat-of-cols k ws*)
shows ∀ *u* ∈ *set* (*direct-sum-basis n k vs ws*). *dim-vec u* = *n* + *k*
and *length* (*direct-sum-basis n k vs ws*) = *n* + *k*
and *invertible-mat* (*mat-of-cols* (*n* + *k*) (*direct-sum-basis n k vs ws*))
⟨*proof*⟩

end

theory *Day06-Tensor-Products*
imports *Day05-Positivity-Tests*
begin

13 Day 6 (9 September 2011): tensor products

Definition (notes). $B : X \times Y \to Z$ is *bilinear* if $B(x_1 + x_2, y) = B(x_1, y) + B(x_2, y)$, $B(x, y_1 + y_2) = B(x, y_1) + B(x, y_2)$ and $B(\lambda x, y) = B(x, \lambda y) = \lambda B(x, y)$.

The tensor product $X \otimes Y$ is the span of elementary tensors $x \otimes y$ with the universal property: every bilinear B factors as $B(x, y) = L_B(x \otimes y)$ for a unique linear L_B. Key relations: $(x_1 + x_2) \otimes y = x_1 \otimes y + x_2 \otimes y$, $x \otimes (y_1 + y_2) = x \otimes y_1 + x \otimes y_2$, $\lambda(x \otimes y) = (\lambda x) \otimes y = x \otimes (\lambda y)$.

Theorem (notes). If $\{e_1, \ldots, e_k\}$ and $\{f_1, \ldots, f_m\}$ are bases of X and Y, then $\{e_i \otimes f_j\}$ is a basis of $X \otimes Y$; hence $\dim(X \otimes Y) = \dim X \cdot \dim Y$.

Proposition (notes). With bases as above, every $u \in X \otimes Y$ is uniquely $u = \sum_j x_j \otimes f_j$ with $x_j \in X$, and uniquely $u = \sum_i e_i \otimes y_i$ with $y_i \in Y$; consequently $X \otimes Y \cong X \oplus \cdots \oplus X$ (m copies).

Definition (notes). The *Schmidt rank* of $u \in X \otimes Y$ is the least number of elementary tensors in an expression for u.

The formalisation uses the concrete model: $\mathbb{C}^n \otimes \mathbb{C}^m = \mathbb{C}^{nm}$ with $(v \otimes w)_{im+j} = v_i w_j$ (*tensor-vec* v w from the library). The three key relations are the bilinearity lemmas below; the product basis theorem is *tensor-onb* (Day 7); the unique decomposition along a basis of Y is not stated for vectors (its operator analogue is in theory Quantum_Computing11).

lemma *tensor-vec-dim*:
  assumes *vd*: *dim-vec v = n dim-vec w = m*
  shows *dim-vec (tensor-vec v w) = n * m*
⟨*proof*⟩

lemma *tensor-vec-index*:
  assumes *dim-vec v = n dim-vec w = m i < n * m*
  shows *tensor-vec v w $ i = v $ (i div m) * w $ (i mod m)*
⟨*proof*⟩

lemma *tensor-vec-bilinear-left*:
  assumes *vd*: *dim-vec v1 = n dim-vec v2 = n dim-vec w = m*
  shows *tensor-vec (v1 + v2) w = tensor-vec v1 w + tensor-vec v2 w*
⟨*proof*⟩

lemma *tensor-vec-bilinear-right*:
  assumes *vd*: *dim-vec v = n dim-vec w1 = m dim-vec w2 = m*
  shows *tensor-vec v (w1 + w2) = tensor-vec v w1 + tensor-vec v w2*
⟨*proof*⟩

lemma *tensor-vec-smult-left*:
  assumes *vd*: *dim-vec v = n dim-vec w = m*
  shows *tensor-vec (α ·$_v$ v) w = α ·$_v$ tensor-vec v w*
⟨*proof*⟩

lemma *tensor-vec-smult-right*:
  assumes *vd*: *dim-vec v = n dim-vec w = m*
  shows *tensor-vec v (α ·$_v$ w) = α ·$_v$ tensor-vec v w*
⟨*proof*⟩

The Schmidt rank, the least length of a representation by elementary tensors. It is *LEAST*, not *Min* of the set of lengths: that set is infinite (zero terms can be added), and *Min* is unspecified on infinite sets. Its properties (rank 1 iff a product state) are in theory Day06_Schmidt_Rank.

definition *schmidt-rank* :: *complex Matrix.vec ⇒ nat ⇒ nat* **where**
  schmidt-rank u m ≡
    LEAST k. ∃ vs :: complex Matrix.vec list. ∃ ws :: complex Matrix.vec list.
        length vs = k ∧ length ws = k ∧
        (∀ i < k. dim-vec (vs ! i) = dim-vec u div m) ∧
        (∀ i < k. dim-vec (ws ! i) = m) ∧
        u = foldr (λ(v,w) acc. tensor-vec v w + acc)
              (zip vs ws) (0$_v$ (dim-vec u))

end
theory *Day06-Tensor-Coordinates*
  imports
    Day05-Direct-Sum-Basis
    Quantum-Computing-by-VP-3-10.Day06-Tensor-Products

begin

14 Day 6: coordinates in a tensor product

$\sum_j x_j \otimes y_j$ in $\mathbb{C}^N$, written as in *schmidt-rank*.

definition *tensor-combination* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec* **where**
tensor-combination N xs ys = *foldr* (λ(*x*,*y*) *acc*. *tensor-vec x y* + *acc*) (*zip xs ys*) (*0*$_v$ *N*)

lemma *tensor-combination-dim* [*simp*]: *dim-vec* (*tensor-combination N xs ys*) = *N*
⟨*proof*⟩

lemma *tensor-combination-index*:
assumes ∀ *x* ∈ *set xs*. *dim-vec x* = *k* ∀ *y* ∈ *set ys*. *dim-vec y* = *m length xs* = *length ys p* < *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) *xs ys* $ *p* = (∑*j*<*length xs*. *xs* ! *j* $ (*p div m*) ∗ *ys* ! *j* $ (*p mod m*))
⟨*proof*⟩

A vector $u \in \mathbb{C}^k \otimes \mathbb{C}^m$ has $k\,m$ entries: the coefficient of $e_i \otimes e_l$ sits at position $i\,m + l$. Read row by row, u is a $k \times m$ matrix; for $k = 2$, $m = 3$,

$$u = (u_0, u_1, u_2, u_3, u_4, u_5) \quad \longmapsto \quad \begin{pmatrix} u_0 & u_1 & u_2 \\ u_3 & u_4 & u_5 \end{pmatrix}.$$

The columns are the parts of u along the basis of the second factor, the rows those along the basis of the first:

$$u = (u_0, u_3) \otimes e_0 + (u_1, u_4) \otimes e_1 + (u_2, u_5) \otimes e_2 = e_0 \otimes (u_0, u_1, u_2) + e_1 \otimes (u_3, u_4, u_5).$$

definition *vec-to-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
vec-to-mat k m u = *Matrix.mat k m* (λ(*i*, *l*). *u* $ (*i* ∗ *m* + *l*))

lemma *vec-to-mat-carrier* [*simp*]: *vec-to-mat k m u* ∈ *carrier-mat k m*
⟨*proof*⟩

lemma *vec-to-mat-dims* [*simp*]: *dim-row* (*vec-to-mat k m u*) = *k dim-col* (*vec-to-mat k m u*) = *m*
⟨*proof*⟩

lemma *vec-to-mat-index* [*simp*]: *i* < *k* ⟹ *l* < *m* ⟹ *vec-to-mat k m u* $$ (*i*, *l*) = *u* $ (*i* ∗ *m* + *l*)
⟨*proof*⟩

lemma *col-vec-to-mat*: *l* < *m* ⟹ *Matrix.col* (*vec-to-mat k m u*) *l* = *Matrix.vec k* (λ*i*. *u* $ (*i* ∗ *m* + *l*))
⟨*proof*⟩

For other bases the matrix is multiplied by the inverse basis matrix. If $f_1, \ldots, f_m$ is a basis of $\mathbb{C}^m$ with column matrix F and $G = F^{-1}$, then $u = \sum_j x_j \otimes f_j$ says $U = XF^T$, where U is the matrix of u and X has the columns x_j; so the x_j are the columns of UG^T. In the first factor, with H the inverse of the column matrix of $e_1, \ldots, e_k$, $u = \sum_i e_i \otimes y_i$ has y_i the rows of HU.

lemma *right-coordinates-index*:
assumes *G*: *G* ∈ *carrier-mat m m* **and** *i* < *k j* < *m*
shows *Matrix.col* (*vec-to-mat k m u* ∗ *G*^{T}) *j* $ *i* = (∑ *l*<*m*. *u* $ (*i* ∗ *m* + *l*) ∗ *G* $$ (*j*,*l*))
⟨*proof*⟩

lemma *left-coordinates-index*:
assumes *H*: *H* ∈ *carrier-mat k k* **and** *i* < *k l* < *m*

shows *Matrix.row* (*H* ∗ *vec-to-mat k m u*) *i* $ *l* = (∑ *r*<*k*. *H* $$ (*i*,*r*) ∗ *u* $ (*r* ∗ *m* + *l*))
⟨*proof*⟩

lemma *inv-entry-left*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *GF*: *G* ∗ *mat-of-cols m fs* = 1_m *m* **and** *j*: *j* < *m* **and** *j'*: *j'* < *m*
shows (∑ *l*<*m*. *fs* ! *j'* $ *l* ∗ *G* $$ (*j*,*l*)) = (*if j* = *j'* *then 1 else 0*)
⟨*proof*⟩

lemma *inv-entry-right*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* ∗ *G* = 1_m *m* **and** *r*: *r* < *m* **and** *l*: *l* < *m*
shows (∑ *j*<*m*. *fs* ! *j* $ *r* ∗ *G* $$ (*j*,*l*)) = (*if r* = *l* *then 1 else 0*)
⟨*proof*⟩

lemma *right-coordinates-unique*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *GF*: *G* ∗ *mat-of-cols m fs* = 1_m *m*
and *xs*: *length xs* = *m* ∀ *x* ∈ *set xs*. *dim-vec x* = *k* **and** *j*: *j* < *m*
shows *Matrix.col* (*vec-to-mat k m* (*tensor-combination* (*k* ∗ *m*) *xs fs*) ∗ G^T) *j* = *xs* ! *j*
⟨*proof*⟩

lemma *right-coordinates-exist*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* ∗ *G* = 1_m *m* **and** *u*: *dim-vec u* = *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) (*cols* (*vec-to-mat k m u* ∗ G^T)) *fs* = *u*
⟨*proof*⟩

D6-3 (i): for a basis $f_1, \ldots, f_m$ of $\mathbb{C}^m$, every $u \in \mathbb{C}^k \otimes \mathbb{C}^m$ is $\sum_j x_j \otimes f_j$ for unique $x_1, \ldots, x_m \in \mathbb{C}^k$.

theorem *tensor-right-coordinates*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
and *u*: *dim-vec u* = *k* ∗ *m*
shows ∃!*xs*. *length xs* = *m* ∧ (∀ *x* ∈ *set xs*. *dim-vec x* = *k*) ∧ *u* = *tensor-combination* (*k* ∗ *m*) *xs fs*
⟨*proof*⟩

lemma *left-coordinates-unique*:
assumes *ed*: ∀ *e* ∈ *set es*. *dim-vec e* = *k length es* = *k* **and** *Hm*: *Hm* ∈ *carrier-mat k k*
and *HE*: *Hm* ∗ *mat-of-cols k es* = 1_m *k*
and *ys*: *length ys* = *k* ∀ *y* ∈ *set ys*. *dim-vec y* = *m* **and** *i*: *i* < *k*
shows *Matrix.row* (*Hm* ∗ *vec-to-mat k m* (*tensor-combination* (*k* ∗ *m*) *es ys*)) *i* = *ys* ! *i*
⟨*proof*⟩

lemma *left-coordinates-exist*:
assumes *ed*: ∀ *e* ∈ *set es*. *dim-vec e* = *k length es* = *k* **and** *Hm*: *Hm* ∈ *carrier-mat k k*
and *EH*: *mat-of-cols k es* ∗ *Hm* = 1_m *k* **and** *u*: *dim-vec u* = *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) *es* (*rows* (*Hm* ∗ *vec-to-mat k m u*)) = *u*
⟨*proof*⟩

D6-3 (ii): likewise $u = \sum_i e_i \otimes y_i$ for unique $y_1, \ldots, y_k \in \mathbb{C}^m$.

theorem *tensor-left-coordinates*:
assumes *ed*: ∀ *e* ∈ *set es*. *dim-vec e* = *k length es* = *k* **and** *inv*: *invertible-mat* (*mat-of-cols k es*)
and *u*: *dim-vec u* = *k* ∗ *m*
shows ∃!*ys*. *length ys* = *k* ∧ (∀ *y* ∈ *set ys*. *dim-vec y* = *m*) ∧ *u* = *tensor-combination* (*k* ∗ *m*) *es ys*
⟨*proof*⟩

Corollary: $\mathbb{C}^k \otimes \mathbb{C}^m \cong \mathbb{C}^k \oplus \cdots \oplus \mathbb{C}^k$ (m times) by the linear bijection $(x_1, \ldots, x_m) \mapsto \sum_j x_j \otimes f_j$,

and $\cong \mathbb{C}^m \oplus \cdots \oplus \mathbb{C}^m$ (k times) likewise; the identification depends on the chosen basis.

lemma *tensor-combination-add-left*:
assumes *l*: *length xs* = *length fs length xs′* = *length fs*
and *d*: ∀ *x* ∈ *set xs*. *dim-vec x* = *k* ∀ *x* ∈ *set xs′*. *dim-vec x* = *k* **and** *fd*: ∀ *y* ∈ *set fs*. *dim-vec y* = *m*
shows *tensor-combination* (*k* ∗ *m*) (*map2* (+) *xs xs′*) *fs* = *tensor-combination* (*k* ∗ *m*) *xs fs* + *tensor-combination* (*k* ∗ *m*) *xs′ fs*
⟨*proof*⟩

lemma *tensor-combination-smult-left*:
assumes *l*: *length xs* = *length fs* **and** *d*: ∀ *x* ∈ *set xs*. *dim-vec x* = *k* **and** *fd*: ∀ *y* ∈ *set fs*. *dim-vec y* = *m*
shows *tensor-combination* (*k* ∗ *m*) (*map* (λ*x*. *a* ·$_v$ *x*) *xs*) *fs* = *a* ·$_v$ *tensor-combination* (*k* ∗ *m*) *xs fs*
⟨*proof*⟩

corollary *tensor-iso-direct-sum*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
shows *bij-betw* (λ*xs*. *tensor-combination* (*k* ∗ *m*) *xs fs*)
{*xs*. *length xs* = *m* ∧ (∀ *x* ∈ *set xs*. *dim-vec x* = *k*)} (*carrier-vec* (*k* ∗ *m*))
⟨*proof*⟩

end
theory *Day06-Schmidt-Rank*
imports *Day06-Tensor-Coordinates Quantum-Computing-by-VP-3-10.Day09-POVMs-Entanglement*
begin

15 Day 6: the Schmidt rank (D6-4)

The notes define the Schmidt rank $\mathrm{rank}_S(u)$ of $u \in X \otimes Y$ as the least number of elementary tensors in an expression for u, and use on Day 10 that a two-photon state ψ is a product $\psi_1 \otimes \psi_2$ iff $\mathrm{rank}_S(\psi) = 1$; otherwise it is entangled. Here: every $u \in \mathbb{C}^n \otimes \mathbb{C}^m$ has an expression of length n, so the least length exists; $\mathrm{rank}_S(u) = 0$ iff $u = 0$; for $u \neq 0$, $\mathrm{rank}_S(u) = 1$ iff u is a product and u is entangled iff $\mathrm{rank}_S(u) > 1$; the Bell state has Schmidt rank 2.

A representation of $u \in \mathbb{C}^n \otimes \mathbb{C}^m$ by k elementary tensors.

definition *schmidt-repr* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *bool* **where**
schmidt-repr n m u k ⟷ (∃ *vs ws*. *length vs* = *k* ∧ *length ws* = *k* ∧
(∀ *i* < *k*. *dim-vec* (*vs* ! *i*) = *n*) ∧ (∀ *i* < *k*. *dim-vec* (*ws* ! *i*) = *m*) ∧
u = *tensor-combination* (*n* ∗ *m*) *vs ws*)

lemma *schmidt-rank-eq*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m*
shows *schmidt-rank u m* = (*LEAST k*. *schmidt-repr n m u k*)
⟨*proof*⟩

Every u is a sum of n elementary tensors, $u = \sum_i e_i \otimes u_i$ with $u_i = (u_{im+j})_j$, so the least length exists.

lemma *schmidt-repr-exists*:
assumes *u*: *dim-vec u* = *n* ∗ *m*
shows *schmidt-repr n m u n*
⟨*proof*⟩

lemma *schmidt-rank-repr*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m*
shows *schmidt-repr n m u* (*schmidt-rank u m*) **and** ⋀*k*. *schmidt-repr n m u k* ⟹ *schmidt-rank u m* ≤ *k*
⟨*proof*⟩

lemma *schmidt-repr-0*: *schmidt-repr n m u 0* $\longleftrightarrow$ *u* = 0_v (*n* ∗ *m*)
⟨*proof*⟩

lemma *schmidt-repr-1*:
schmidt-repr n m u 1 $\longleftrightarrow$ (∃ *v w*. *dim-vec v* = *n* ∧ *dim-vec w* = *m* ∧ *u* = *tensor-vec v w*)
⟨*proof*⟩

theorem *schmidt-rank-zero-iff*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m*
shows *schmidt-rank u m* = *0* $\longleftrightarrow$ *u* = 0_v (*n* ∗ *m*)
⟨*proof*⟩

The notes: $\psi = \psi_1 \otimes \psi_2$ iff $\text{rank}_S(\psi) = 1$ (for $\psi \neq 0$).

theorem *schmidt-rank-one-iff-product*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m* **and** *u0*: *u* ≠ 0_v (*n* ∗ *m*)
shows *schmidt-rank u m* = *1* $\longleftrightarrow$ (∃ *v w*. *dim-vec v* = *n* ∧ *dim-vec w* = *m* ∧ *u* = *tensor-vec v w*)
⟨*proof*⟩

corollary *entangled-iff-schmidt-rank*:
assumes *u*: *dim-vec u* = *n* ∗ *m* **and** *m*: *0* < *m* **and** *u0*: *u* ≠ 0_v (*n* ∗ *m*)
shows *entangled n m u* $\longleftrightarrow$ *1* < *schmidt-rank u m*
⟨*proof*⟩

corollary *schmidt-rank-bell-state*: *schmidt-rank bell-state 2* = *2*
⟨*proof*⟩

end
theory *Day06-15-Universal*
imports *Day06-Tensor-Coordinates Day15-Partial-Traces*
begin

16 Day 6 and Day 15: the universal properties in the Kronecker model

D6-1: for $\mathbb{C}^n \otimes \mathbb{C}^m = \mathbb{C}^{nm}$ with $(v \otimes w)_{im+j} = v_i w_j$, every bilinear map $\beta : \mathbb{C}^n \times \mathbb{C}^m \to \mathbb{C}^p$ factors as $\beta(v, w) = L(v \otimes w)$ for a unique linear L. First for bilinear forms ($p = 1$), then coordinatewise.

definition *bilinear-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex*) ⇒ *bool* **where**
bilinear-on n m β $\longleftrightarrow$
(∀ *v1 v2 w*. *dim-vec v1* = *n* $\longrightarrow$ *dim-vec v2* = *n* $\longrightarrow$ *dim-vec w* = *m* $\longrightarrow$ *β* (*v1* + *v2*) *w* = *β v1 w* + *β v2 w*)
∧ (∀ *v w1 w2*. *dim-vec v* = *n* $\longrightarrow$ *dim-vec w1* = *m* $\longrightarrow$ *dim-vec w2* = *m* $\longrightarrow$ *β v* (*w1* + *w2*) = *β v w1* + *β v w2*)
∧ (∀ *a v w*. *dim-vec v* = *n* $\longrightarrow$ *dim-vec w* = *m* $\longrightarrow$ *β* (*a* $\cdot_v$ *v*) *w* = *a* ∗ *β v w* ∧ *β v* (*a* $\cdot_v$ *w*) = *a* ∗ *β v w*)

definition *linear-on* :: *nat* ⇒ (*complex Matrix.vec* ⇒ *complex*) ⇒ *bool* **where**
linear-on K L $\longleftrightarrow$ (∀ *x y*. *dim-vec x* = *K* $\longrightarrow$ *dim-vec y* = *K* $\longrightarrow$ *L* (*x* + *y*) = *L x* + *L y*)
∧ (∀ *a x*. *dim-vec x* = *K* $\longrightarrow$ *L* (*a* $\cdot_v$ *x*) = *a* ∗ *L x*)

definition *trunc-vec* :: *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
trunc-vec v k = *Matrix.vec* (*dim-vec v*) (λ*i*. *if i* < *k then v* $ *i else 0*)

lemma *dim-trunc-vec* [*simp*]: *dim-vec* (*trunc-vec v k*) = *dim-vec v* ⟨*proof*⟩

lemma *trunc-vec-full*: *trunc-vec v* (*dim-vec v*) = *v* ⟨*proof*⟩

lemma *trunc-vec-Suc*: *k* < *dim-vec v* ⟹ *trunc-vec v* (*Suc k*) = *trunc-vec v k* + *v* $ *k* ·$_v$ *unit-vec* (*dim-vec v*) *k*
⟨*proof*⟩

lemma *trunc-vec-zero*: *trunc-vec v 0* = *0*$_v$ (*dim-vec v*) ⟨*proof*⟩

A linear map is determined by its values on the unit vectors.

lemma *linear-on-zero*: *linear-on K L* ⟹ *L* (*0*$_v$ *K*) = *0*
⟨*proof*⟩

lemma *linear-on-expand*:
assumes *l*: *linear-on K L* **and** *x*: *dim-vec x* = *K*
shows *L x* = (∑ *k*<*K*. *x* $ *k* ∗ *L* (*unit-vec K k*))
⟨*proof*⟩

A bilinear map is determined by its values on pairs of unit vectors.

lemma *bilinear-on-expand*:
assumes *b*: *bilinear-on n m β* **and** *v*: *dim-vec v* = *n* **and** *w*: *dim-vec w* = *m*
shows *β v w* = (∑ *i*<*n*. ∑ *j*<*m*. (*v* $ *i* ∗ *w* $ *j*) ∗ *β* (*unit-vec n i*) (*unit-vec m j*))
⟨*proof*⟩

lemma *sum-div-mod*:
fixes *n m* :: *nat* **and** *g* :: *nat* ⇒ *nat* ⇒ *'a*::*comm-monoid-add*
shows (∑ *k*<*n* ∗ *m*. *g* (*k div m*) (*k mod m*)) = (∑ *i*<*n*. ∑ *j*<*m*. *g i j*)
⟨*proof*⟩

theorem *tensor-universal-property*:
assumes *b*: *bilinear-on n m β*
shows ∃ *L*. *linear-on* (*n* ∗ *m*) *L* ∧ (∀ *v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *L* (*tensor-vec v w*) = *β v w*)
∧ (∀ *L'*. *linear-on* (*n* ∗ *m*) *L'* ∧ (∀ *v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *L'* (*tensor-vec v w*) = *β v w*)
⟶ (∀ *x*. *dim-vec x* = *n* ∗ *m* ⟶ *L' x* = *L x*))
⟨*proof*⟩

definition *bilinear-vec-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec*) ⇒ *bool* **where**
bilinear-vec-on n m p β ⟷
(∀ *v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *dim-vec* (*β v w*) = *p*)
∧ (∀ *v1 v2 w*. *dim-vec v1* = *n* ⟶ *dim-vec v2* = *n* ⟶ *dim-vec w* = *m* ⟶ *β* (*v1* + *v2*) *w* = *β v1 w* + *β v2 w*)
∧ (∀ *v w1 w2*. *dim-vec v* = *n* ⟶ *dim-vec w1* = *m* ⟶ *dim-vec w2* = *m* ⟶ *β v* (*w1* + *w2*) = *β v w1* + *β v w2*)
∧ (∀ *a v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *β* (*a* ·$_v$ *v*) *w* = *a* ·$_v$ *β v w* ∧ *β v* (*a* ·$_v$ *w*) = *a* ·$_v$ *β v w*)

definition *linear-vec-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec*) ⇒ *bool* **where**
linear-vec-on K p L ⟷ (∀ *x*. *dim-vec x* = *K* ⟶ *dim-vec* (*L x*) = *p*)
∧ (∀ *x y*. *dim-vec x* = *K* ⟶ *dim-vec y* = *K* ⟶ *L* (*x* + *y*) = *L x* + *L y*)
∧ (∀ *a x*. *dim-vec x* = *K* ⟶ *L* (*a* ·$_v$ *x*) = *a* ·$_v$ *L x*)

lemma *bilinear-vec-coord*: *bilinear-vec-on n m p β* ⟹ *r* < *p* ⟹ *bilinear-on n m* (λ*v w*. *β v w* $ *r*)
⟨*proof*⟩

lemma *linear-vec-coord*: *linear-vec-on K p L* $\Longrightarrow$ *r* $<$ *p* $\Longrightarrow$ *linear-on K* ($\lambda x.$ *L x* \$ *r*)
⟨*proof*⟩

theorem *tensor-universal-property-vec*:
assumes *b*: *bilinear-vec-on n m p β*
shows $\exists$ *L. linear-vec-on* (*n* $*$ *m*) *p L* $\wedge$ ($\forall$ *v w. dim-vec v* $=$ *n* $\longrightarrow$ *dim-vec w* $=$ *m* $\longrightarrow$ *L* (*tensor-vec v w*) $=$ *β v w*)
$\wedge$ ($\forall$ *L′. linear-vec-on* (*n* $*$ *m*) *p L′* $\wedge$ ($\forall$ *v w. dim-vec v* $=$ *n* $\longrightarrow$ *dim-vec w* $=$ *m* $\longrightarrow$ *L′* (*tensor-vec v w*) $=$ *β v w*)
$\longrightarrow$ ($\forall$ *x. dim-vec x* $=$ *n* $*$ *m* $\longrightarrow$ *L′ x* $=$ *L x*))
⟨*proof*⟩

D15-3: in the coordinate model $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) = M_{m^2 \times p^2}$ the elementary tensor $R \otimes T$ is the matrix $(R_{ij} T_{kl})_{(im+j,\, kp+l)}$. The isomorphism Γ of *operator-tensor-space-isomorphism* sends it to the Kronecker product $R \otimes T$, and Γ is the only linear map that does.

definition *op-elem* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat* **where**
op-elem m p Ra Tb = *Matrix.mat* (*m* $*$ *m*) (*p* $*$ *p*) (λ(*a*, *b*). *Ra* \$\$ (*a div m*, *a mod m*) $*$ *Tb* \$\$ (*b div p*, *b mod p*))

lemma *op-elem-carrier* [*simp*]: *op-elem m p Ra Tb* $\in$ *carrier-mat* (*m* $*$ *m*) (*p* $*$ *p*) ⟨*proof*⟩

theorem *gamma-elementary*:
assumes *Ra*: *Ra* $\in$ *carrier-mat m m* **and** *Tb*: *Tb* $\in$ *carrier-mat p p*
shows *operator-tensor-iso-map m p* (*op-elem m p Ra Tb*) = *Ra* $\bigotimes$ *Tb*
⟨*proof*⟩

definition *unit-mat* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat* **where**
unit-mat K1 K2 a b = *Matrix.mat K1 K2* (λ(*i*, *j*). *if i* $=$ *a* $\wedge$ *j* $=$ *b then 1 else 0*)

lemma *unit-mat-carrier* [*simp*]: *unit-mat K1 K2 a b* $\in$ *carrier-mat K1 K2* ⟨*proof*⟩

The unit matrices of $M_{m^2 \times p^2}$ are elementary tensors of unit matrices.

lemma *unit-mat-op-elem*:
assumes *a*: *a* $<$ *m* $*$ *m* **and** *b*: *b* $<$ *p* $*$ *p*
shows *unit-mat* (*m* $*$ *m*) (*p* $*$ *p*) *a b* = *op-elem m p* (*unit-mat m m* (*a div m*) (*a mod m*)) (*unit-mat p p* (*b div p*) (*b mod p*))
⟨*proof*⟩

A linear map on $M_{K_1 \times K_2}$ is determined by its values on the unit matrices.

definition *linear-mat-on* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ (*complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*) $\Rightarrow$ *bool* **where**
linear-mat-on K1 K2 Φ $\longleftrightarrow$
($\forall$ *x y. x* $\in$ *carrier-mat K1 K2* $\longrightarrow$ *y* $\in$ *carrier-mat K1 K2* $\longrightarrow$ Φ (*x* $+$ *y*) $=$ Φ *x* $+$ Φ *y*)
$\wedge$ ($\forall$ *c x. x* $\in$ *carrier-mat K1 K2* $\longrightarrow$ Φ (*c* $\cdot_m$ *x*) $=$ *c* $\cdot_m$ Φ *x*)

Reading a matrix row by row gives a vector, and *vec-to-mat* undoes this:

$$\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \quad \longmapsto \quad (a, b, c, d, e, f).$$

definition *mat-to-vec* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.vec* **where**
mat-to-vec c = *Matrix.vec* (*dim-row c* $*$ *dim-col c*) (λ*k*. *c* \$\$ (*k div dim-col c*, *k mod dim-col c*))

lemma *vec-to-mat-mat-to-vec*: *c* $\in$ *carrier-mat K1 K2* $\Longrightarrow$ *vec-to-mat K1 K2* (*mat-to-vec c*) $=$ *c*
⟨*proof*⟩

lemma *dim-mat-to-vec*: *c* $\in$ *carrier-mat K1 K2* $\Longrightarrow$ *dim-vec* (*mat-to-vec c*) $=$ *K1* $*$ *K2* ⟨*proof*⟩

lemma *vec-to-mat-unit*:
assumes *k*: $k < K1 * K2$
shows *vec-to-mat K1 K2* (*unit-vec* (*K1* $*$ *K2*) *k*) = *unit-mat K1 K2* (*k div K2*) (*k mod K2*)
⟨*proof*⟩

lemma *linear-mat-unique*:
assumes *l1*: *linear-mat-on K1 K2* Φ **and** *l2*: *linear-mat-on K1 K2* Ψ
and *dims*: $\bigwedge x.\ x \in$ *carrier-mat K1 K2* $\Longrightarrow$ *dim-row* (Φ *x*) = *Q1* $\wedge$ *dim-col* (Φ *x*) = *Q2* $\wedge$ *dim-row* (Ψ *x*) = *Q1* $\wedge$ *dim-col* (Ψ *x*) = *Q2*
and *e*: $\bigwedge a\ b.\ a < K1 \Longrightarrow b < K2 \Longrightarrow$ Φ (*unit-mat K1 K2 a b*) = Ψ (*unit-mat K1 K2 a b*)
and *c*: *c* ∈ *carrier-mat K1 K2*
shows Φ *c* = Ψ *c*
⟨*proof*⟩

theorem *gamma-unique*:
assumes *p0*: *0* < *p* **and** *l*: *linear-mat-on* (*m* $*$ *m*) (*p* $*$ *p*) Φ
and *dims*: $\bigwedge x.\ x \in$ *carrier-mat* (*m* $*$ *m*) (*p* $*$ *p*) $\Longrightarrow$ Φ *x* ∈ *carrier-mat* (*m* $*$ *p*) (*m* $*$ *p*)
and *e*: $\bigwedge Ra\ Tb.\ Ra \in$ *carrier-mat m m* $\Longrightarrow$ *Tb* ∈ *carrier-mat p p* $\Longrightarrow$ Φ (*op-elem m p Ra Tb*) = *Ra* $\bigotimes$ *Tb*
and *c*: *c* ∈ *carrier-mat* (*m* $*$ *m*) (*p* $*$ *p*)
shows Φ *c* = *operator-tensor-iso-map m p c*
⟨*proof*⟩

end

theory *Day07-Postulates*
imports *Day06-Tensor-Products*
begin

17 Day 7 (12 September 2011): tensor products of Hilbert spaces; the postulates

Remark. For Hilbert spaces $\mathcal{H}, \mathcal{K}$ set $\langle h_1 \otimes k_1 | h_2 \otimes k_2 \rangle = \langle h_1 | h_2 \rangle_{\mathcal{H}} \langle k_1 | k_2 \rangle_{\mathcal{K}}$.

Theorem (notes). This extends to a sesquilinear form on $\mathcal{H} \otimes \mathcal{K}$ with $\langle u | u \rangle = 0$ only for $u = 0$, hence to an inner product; in finite dimensions $\mathcal{H} \otimes \mathcal{K}$ with this inner product is the Hilbert space tensor product (in infinite dimensions one completes).

Proposition (notes). If $\{h_i\}$ and $\{k_j\}$ are orthonormal bases of $\mathcal{H}$ and $\mathcal{K}$, then $\{h_i \otimes k_j\}$ is an orthonormal basis of $\mathcal{H} \otimes \mathcal{K}$.

In physics notation $e_i \otimes e_j = |ij\rangle$, and for N qubits $e_{i_1} \otimes \cdots \otimes e_{i_N} = |i_1 \cdots i_N\rangle$ with the multi-index $I = (i_1, \ldots, i_N) \in \mathbb{Z}_2^N$. In the concrete model positivity of the form is automatic; the multiplicativity of the inner product and the orthonormal product basis are proved below.

lemma *tensor-inner-prod*:
assumes *dim-vec h1 = n dim-vec h2 = n*
dim-vec k1 = m dim-vec k2 = m
shows ⟨*tensor-vec h1 k1* | *tensor-vec h2 k2*⟩ =
⟨*h1* | *h2*⟩ $*$ ⟨*k1* | *k2*⟩
⟨*proof*⟩

lemma *length-concat-map-upt*:
length (*concat* (*map* (λ*i*. *map* (λ*j*. *f i j*) [*0*..<*m*]) [*0*..<*n*])) = *n* $*$ *m*

⟨*proof*⟩

lemma *nth-concat-map-upt*:
assumes *p-lt*: $p < n * m$
shows *concat* (*map* (λ*i*. *map* (λ*j*. *f i j*) [*0*..<*m*]) [*0*..<*n*]) ! *p* =
f (*p div m*) (*p mod m*)
⟨*proof*⟩

lemma *tensor-onb*:
assumes *hs-onb*: *orthonormal-list n hs*
and *hs-len*: *length hs* = *n*
and *ks-onb*: *orthonormal-list m ks*
and *ks-len*: *length ks* = *m*
shows *orthonormal-list* (*n*∗*m*)
[*tensor-vec* (*hs* ! *i*) (*ks* ! *j*). *i* ← [*0*..<*n*], *j* ← [*0*..<*m*]]
⟨*proof*⟩

17.1 Postulates of quantum mechanics

Postulate (notes) (1)**.** To each isolated physical system corresponds a Hilbert space, the *state space*; the state of the system is completely described by a unit vector in it, the *state vector*.

Postulate (notes) (2)**.** The time evolution of a closed system from t_1 to t_2 is described by a unitary $U : \mathcal{H} \to \mathcal{H}$: a system in state ψ at t_1 is in state $U\psi$ at t_2.

Postulate (notes) (3)**.** Quantum measurements are described by a collection $\{M_m\}$ of *measurement operators* on $\mathcal{H}$. If the system is in state ψ, the probability of observing m is $p_m(\psi) = \langle\psi|M_m^* M_m\psi\rangle = \|M_m\psi\|^2$; since the probabilities sum to one, $\sum_m M_m^* M_m = I$. After observing m the state is $M_m\psi/p_m(\psi)^{1/2}$.

Formally a state is *quantum-state n ψ* (a unit vector in $\mathbb{C}^n$), evolution is *evolve U ψ*, a measurement system is a list *Ms* with *measurement-system n Ms* (all $n \times n$ with $\sum_m M_m^\dagger M_m = I$), and $p_m(\psi)$ is *meas-prob M ψ*.

definition *quantum-state* :: *nat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
quantum-state n ψ ≡ *dim-vec ψ* = *n* ∧ ‖*ψ*‖ = *1*

lemma *quantum-state-norm1*:
assumes *quantum-state n ψ*
shows ‖*ψ*‖ = *1*
⟨*proof*⟩

lemma *quantum-state-dim*:
assumes *quantum-state n ψ*
shows *dim-vec ψ* = *n*
⟨*proof*⟩

definition *evolve* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
evolve U ψ ≡ *U* ∗$_v$ *ψ*

lemma *unitary-preserves-state*:
assumes *unitary U* **and** *quantum-state n ψ* **and** *dim-col U* = *n*
shows *quantum-state* (*dim-row U*) (*evolve U ψ*)
⟨*proof*⟩

definition *measurement-system* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**

measurement-system n Ms ≡
(∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*) ∧
foldr (λ*M acc*. ($M^\dagger$) ∗ *M* + *acc*) *Ms* (0_m *n n*) = 1_m *n*

definition *meas-prob* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *real* **where**
meas-prob M ψ ≡ $\|M *_v \psi\|^2$

lemma *measurement-probs-sum-one*:
assumes *ms*: *measurement-system n Ms*
and *st*: *quantum-state n ψ*
shows ($\sum M \leftarrow Ms$. *meas-prob M ψ*) = *1*
⟨*proof*⟩

$p_m(\psi) = \langle\psi|M_m^* M_m\psi\rangle$, the form in which the notes state Postulate 3.

lemma *meas-prob-quad-form*:
fixes *M* :: *complex Matrix.mat* **and** *ψ* :: *complex Matrix.vec*
assumes *M*: *M* ∈ *carrier-mat n n* **and** *ψ*: *dim-vec ψ* = *n*
shows *meas-prob M ψ* = *Re* ⟨*ψ* | (*M*† ∗ *M*) ∗$_v$ *ψ*⟩
⟨*proof*⟩

After outcome m the state becomes $M_m\psi/\|M_m\psi\|$:

definition *post-meas-state* :: *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
post-meas-state M ψ ≡ (*1* / *complex-of-real* ($\|M *_v \psi\|$)) ·$_v$ (*M* ∗$_v$ *ψ*)

Postulate (notes) (4, Day 9)**.** The state space of a composite system is the tensor product of the state spaces of its components; if the i-th component is in state ψ_i, the system is in state $\psi_1 \otimes \cdots \otimes \psi_n$.

lemma *tensor-of-states-is-state*:
assumes *quantum-state n ψ1* **and** *quantum-state m ψ2*
shows *quantum-state* (*n*∗*m*) (*tensor-vec ψ1 ψ2*)
⟨*proof*⟩

end

theory *Day08-Measurements*
imports *Day07-Postulates*
begin

18 Day 8 (14 September 2011): the quantum game; projective measurements and expected values

18.1 Distinguishing states

Alice has two states ψ_1, ψ_2 known to Bob; she sends one. Can Bob build a measurement system that decides with certainty which one he received? Formally one wants M_1, M_2 with $\|M_1\psi_2\| = 0$, $\|M_1\psi_1\|^2 = 1$, $\|M_2\psi_1\| = 0$, $\|M_2\psi_2\|^2 = 1$.

Case I, $\psi_1 \perp \psi_2$: take $M_1 = |\psi_1\rangle\langle\psi_1|$, $M_2 = |\psi_2\rangle\langle\psi_2|$ and $M_3 = I - M_1 - M_2$; this distinguishes with certainty (formalised below).

Case II, $\psi_1 \not\perp \psi_2$: write $\psi_2 = \alpha\psi_1 + \beta\gamma$ with $\gamma \perp \psi_1$ a unit vector, so $\alpha \neq 0$; any measurement system with $\|M_1\psi_1\| = 1$ satisfies $|\alpha|^2 \le \|M_1\psi_2\|^2$, so outcome 1 also occurs with probability at least $|\alpha|^2$ when ψ_2 is sent (*nonorthogonal-outcome-prob* in theory Day02_11_Supplement).

Theorem (notes). Given linearly independent states $\psi_1, \ldots, \psi_n$ there is a measurement system $M_0, \ldots, M_n$ such that if outcome $i \geq 1$ occurs then ψ_i was received.

The notes' proof takes E_i the projection onto $\mathrm{span}\{\psi_j : j \neq i\}^\perp$, $M_i = E_i/\sqrt{n}$, and $M_0 = (I - \sum_i M_i^* M_i)^{1/2}$; the remark that follows observes that the inconclusive outcome 0 then has probability at least $(n-1)/n$. Formally this is *lin-indep-states-unambiguous* below.

lemma *outer-proj-self*:
  assumes *q*: *quantum-state n ψ*
  shows *outer-prod ψ ψ *$_v$ *ψ = ψ*
⟨*proof*⟩

lemma *outer-proj-orth-zero*:
  assumes *q1*: *quantum-state n ψ1*
    and *q2*: *quantum-state n ψ2*
    and *orth*: *⟨ψ1 | ψ2⟩ = 0*
  shows *outer-prod ψ1 ψ1 *$_v$ *ψ2 = 0*$_v$ *n*
⟨*proof*⟩

lemma *outer-proj-idem*:
  assumes *q*: *quantum-state n ψ*
  shows (*outer-prod ψ ψ*) * (*outer-prod ψ ψ*) = *outer-prod ψ ψ*
⟨*proof*⟩

lemma *outer-proj-hermitian*:
  assumes *dim-vec ψ = n*
  shows (*outer-prod ψ ψ*)† = *outer-prod ψ ψ*
⟨*proof*⟩

lemma *outer-proj-cross-zero*:
  assumes *q1*: *quantum-state n ψ1*
    and *q2*: *quantum-state n ψ2*
    and *orth*: *⟨ψ1 | ψ2⟩ = 0*
  shows (*outer-prod ψ1 ψ1*) * (*outer-prod ψ2 ψ2*) = *0*$_m$ *n n*
⟨*proof*⟩

Case I: orthogonal states can be distinguished perfectly.

lemma *orthogonal-states-distinguishable*:
  fixes *ψ1 ψ2* :: *complex Matrix.vec*
  assumes *q1*: *quantum-state n ψ1*
    and *q2*: *quantum-state n ψ2*
    and *orth*: *⟨ψ1 | ψ2⟩ = 0*
  shows ∃ *Ms* :: *complex Matrix.mat list*.
      measurement-system n Ms ∧
      (∃ *M1 M2*. *M1* ∈ *set Ms* ∧ *M2* ∈ *set Ms* ∧
        meas-prob M1 ψ1 = 1 ∧ *meas-prob M2 ψ1 = 0* ∧
        meas-prob M1 ψ2 = 0 ∧ *meas-prob M2 ψ2 = 1*)
⟨*proof*⟩

The theorem on linearly independent states. Linear independence of a list of vectors is injectivity of the matrix with those columns; the formal statement produces the measurement system $M_0, \ldots, M_k$ in the order $M_1, \ldots, M_k, M_0$ (outcome $i < k$ identifies ψ_i, outcome k is inconclusive). The proof follows the notes with one change: instead of the projections E_i onto $\{\psi_j : j \neq i\}^\perp$ it uses the dual vectors ϕ_i with $\langle \phi_i | \psi_j \rangle = \delta_{ij}$ (columns of $V(V^*V)^{-1}$) and the operators $M_i = \sqrt{c}\, |e_0\rangle\langle\phi_i|$, where $c > 0$ makes $I - c\sum_i |\phi_i\rangle\langle\phi_i|$ positive; M_0 is its square root.

definition *lin-indep-vecs* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *bool* **where**
  lin-indep-vecs n vs ≡ (∀ *v* ∈ *set vs*. *dim-vec v = n*) ∧

$(\forall\, x ::$ *complex Matrix.vec*. $x \in$ *carrier-vec* (*length vs*) $\longrightarrow$
mat-of-cols n vs $*_v$ $x = 0_v$ *n* $\longrightarrow$ $x = 0_v$ (*length vs*))

lemma *outer-prod-carrier*:
fixes *v w* :: *complex Matrix.vec*
assumes *dim-vec v* = *n* **and** *dim-vec w* = *m*
shows *v* $\oslash$ *w* $\in$ *carrier-mat n m*
⟨*proof*⟩

lemma *outer-prod-dagger*:
fixes *v w* :: *complex Matrix.vec*
shows $(v \oslash w)^\dagger = w \oslash v$
⟨*proof*⟩

lemma *foldr-add-mat-carrier*:
fixes *f* :: *'a* $\Rightarrow$ *complex Matrix.mat*
assumes $\forall\, x \in$ *set xs*. *f x* $\in$ *carrier-mat n n*
shows *foldr* (λx *acc*. *f x* + *acc*) *xs* (0_m *n n*) $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *foldr-add-mat-index*:
fixes *f* :: *'a* $\Rightarrow$ *complex Matrix.mat*
assumes $\forall\, x \in$ *set xs*. *f x* $\in$ *carrier-mat n n* **and** *i* < *n* **and** *j* < *n*
shows *foldr* (λx *acc*. *f x* + *acc*) *xs* (0_m *n n*) $$ (*i*, *j*) = *sum-list* (*map* (λx. *f x* $$ (*i*, *j*)) *xs*)
⟨*proof*⟩

lemma *foldr-MdagM-shift*:
assumes *Ms-c*: $\forall\, M \in$ *set Ms*. *M* $\in$ *carrier-mat n n* **and** *z*: *z* $\in$ *carrier-mat n n*
shows *foldr* (λM *acc*. ($M^\dagger$) $*$ *M* + *acc*) *Ms z*
= *foldr* (λM *acc*. ($M^\dagger$) $*$ *M* + *acc*) *Ms* (0_m *n n*) + *z*
⟨*proof*⟩

theorem *lin-indep-states-unambiguous*:
fixes ψs :: *complex Matrix.vec list*
assumes *qs*: $\forall\, \psi \in$ *set* ψs. *quantum-state n* ψ
and *li*: *lin-indep-vecs n* ψs
and *k*: *length* ψs = *k* **and** *kpos*: *0* < *k*
shows $\exists$ *Ms* :: *complex Matrix.mat list*.
measurement-system n Ms $\wedge$ *length Ms* = *k* + *1* $\wedge$
($\forall\, i$<*k*. $\forall\, j$<*k*. *i* $\neq$ *j* $\longrightarrow$ *meas-prob* (*Ms* ! *i*) (ψs ! *j*) = *0*) $\wedge$
($\forall\, i$<*k*. *0* < *meas-prob* (*Ms* ! *i*) (ψs ! *i*))
⟨*proof*⟩

18.2 Projective measurements, expected values and self-adjoint operators

If a measurement $\{M_m\}$ has real outcomes λ_m, the expected value of the outcome in state ψ is $E = \sum_m p_m(\psi)\lambda_m = \langle\psi|(\sum_m \lambda_m M_m^* M_m)\psi\rangle = \langle\psi|H\psi\rangle$ with $H = \sum_m \lambda_m M_m^* M_m$ self-adjoint: expected values of observables are quadratic forms of Hermitian matrices (*expected-value-eq-quad-form* below).

Formally: a measurement whose operator is the rank-one projection $|\gamma\rangle\langle\gamma|$ has $p(\psi) = |\langle\gamma|\psi\rangle|^2$ (the proposition of Day 12); an *observable* with outcomes λ_m attached to the measurement operators M_m is $H = \sum_m \lambda_m M_m^* M_m$, which is Hermitian, and the expected outcome $\sum_m \lambda_m p_m(\psi)$ equals $\langle\psi|H\psi\rangle$.

lemma *rank-one-meas-prob*:
fixes γ ψ :: *complex Matrix.vec*
assumes *g*: *quantum-state n* γ **and** ψ: *dim-vec* ψ = *n*

shows *meas-prob* $(\gamma \oslash \gamma)\ \psi = (cmod\ \langle\gamma \mid \psi\rangle)^2$
⟨*proof*⟩

definition *observable* :: *nat* ⇒ (*real* × *complex Matrix.mat*) *list* ⇒ *complex Matrix.mat* **where**
observable n lMs ≡
foldr (λ(*l*, *M*) *acc*. *complex-of-real l* ·$_m$ ((*M*†) ∗ *M*) + *acc*) *lMs* (*0*$_m$ *n n*)

definition *expected-value* :: (*real* × *complex Matrix.mat*) *list* ⇒ *complex Matrix.vec* ⇒ *real* **where**
expected-value lMs ψ ≡ (∑ (*l*, *M*) ← *lMs*. *l* ∗ *meas-prob M ψ*)

lemma *observable-Nil* [*simp*]: *observable n* [] = *0*$_m$ *n n*
⟨*proof*⟩

lemma *observable-Cons* [*simp*]:
observable n ((*l*, *M*) # *lMs*) = *complex-of-real l* ·$_m$ ((*M*†) ∗ *M*) + *observable n lMs*
⟨*proof*⟩

lemma *observable-carrier*:
assumes ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *observable n lMs* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *observable-hermitian*:
assumes ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *hermitian* (*observable n lMs*)
⟨*proof*⟩

lemma *expected-value-eq-quad-form*:
fixes *ψ* :: *complex Matrix.vec*
assumes *ψ*: *dim-vec ψ* = *n*
and *lMs*: ∀ (*l*, *M*) ∈ *set lMs*. *M* ∈ *carrier-mat n n*
shows *expected-value lMs ψ* = *Re* ⟨*ψ* | *observable n lMs* ∗$_v$ *ψ*⟩
⟨*proof*⟩

end
theory *Day08-Indistinguishable-States*
imports *Quantum-Computing-by-VP-3-10.Day07-Postulates*
begin

19 Day 8: non-orthogonal states cannot be distinguished

lemma *meas-sum-norm*:
assumes *ms*: *measurement-system n Ms* **and** *v*: *dim-vec v* = *n*
shows (∑ *l*<*length Ms*. ‖*Ms* ! *l* ∗$_v$ *v*‖²) = ‖*v*‖²
⟨*proof*⟩

lemma *inner-minus-smult*:
assumes *d1*: *dim-vec u* = *n* **and** *d2*: *dim-vec w* = *n*
shows ⟨*w* − *a* ·$_v$ *u* | *w* − *a* ·$_v$ *u*⟩ = ⟨*w*|*w*⟩ − *a* ∗ ⟨*w*|*u*⟩ − *cnj a* ∗ ⟨*u*|*w*⟩ + *a* ∗ *cnj a* ∗ ⟨*u*|*u*⟩
⟨*proof*⟩

lemma *norm-sq-minus-component*:
assumes *d1*: *dim-vec u* = *n* **and** *d2*: *dim-vec w* = *n* **and** *u*: ‖*u*‖ = *1*
shows ‖*w* − ⟨*u*|*w*⟩ ·$_v$ *u*‖² = ‖*w*‖² − (*cmod* ⟨*u*|*w*⟩)²
⟨*proof*⟩

D8-2: if outcome i certainly occurs on ψ_1, it occurs on ψ_2 with probability at least $|\langle\psi_1|\psi_2\rangle|^2$.

theorem *nonorthogonal-outcome-prob*:
assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *i*: $i < length\ Ms$ **and** *p1*: *meas-prob* (*Ms* ! *i*) *ψ1* = *1*
shows $(cmod\ \langle \psi 1|\psi 2\rangle)^2 \leq meas\text{-}prob\ (Ms\ !\ i)\ \psi 2$
⟨*proof*⟩

corollary *nonorthogonal-not-distinguishable*:
assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *no*: $\langle \psi 1|\psi 2\rangle \neq 0$ **and** *i*: $i < length\ Ms$ **and** *p1*: *meas-prob* (*Ms* ! *i*) *ψ1* = *1*
shows *meas-prob* (*Ms* ! *i*) *ψ2* > *0*
⟨*proof*⟩

end

theory *Day09-POVMs-Entanglement*
imports *Day08-Measurements*
begin

20 Day 9 (16 September 2011): positive operator-valued measures; composite systems; entanglement

20.1 Positive operator-valued measures

If two measurement systems satisfy $M_m^* M_m = \tilde{M}_m^* \tilde{M}_m$ for all m then $p_m = \tilde{p}_m$: only the operators $M_m^* M_m$ matter, and $\sum_m M_m^* M_m = I$. Conversely if $P_m \geq 0$ with $\sum_m P_m = I$ then $M_m = P_m^{1/2}$ is a measurement system.

Definition (notes). A *positive operator-valued measure* (POVM) is a set $\{P_m\}$ of positive operators with $\sum_m P_m = I$.

definition *povm* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
povm n Ps ≡
(∀ *P* ∈ *set Ps*. *P* ∈ *carrier-mat n n* ∧ *psd P*) ∧
foldr (λ*P acc*. *P* + *acc*) *Ps* (*0*$_m$ *n n*) = *1*$_m$ *n*

lemma *measurement-induces-povm*:
assumes *ms*: *measurement-system n Ms*
shows *povm n* (*map* (λ*M*. (*M*†) ∗ *M*) *Ms*)
⟨*proof*⟩

lemma *psd-sqrt-measurement-factor*:
assumes *Pcar*: *P* ∈ *carrier-mat n n*
and *Ppsd*: *psd P*
shows ∃ *M*. *M* ∈ *carrier-mat n n* ∧ *M*† ∗ *M* = *P*
⟨*proof*⟩

lemma *psd-sqrt-measurement-factors*:
assumes *Ps*: ∀ *P* ∈ *set Ps*. *P* ∈ *carrier-mat n n* ∧ *psd P*
shows ∃ *Ms*. (∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*) ∧
map (λ*M*. *M*† ∗ *M*) *Ms* = *Ps*
⟨*proof*⟩

lemma *povm-from-sqrt*:
assumes *pv*: *povm n Ps*

shows $\exists\, Ms.$ *measurement-system n Ms* $\wedge$
map $(\lambda M.\ M^\dagger * M)$ *Ms* $=$ *Ps*
⟨*proof*⟩

lemma *povm-prob*:
assumes *pv*: *povm n Ps* **and** *st*: *quantum-state n* ψ
and *Pin*: $P \in$ *set Ps*
shows *Re* $(\langle\psi \mid P *_v \psi\rangle) \geq 0$
⟨*proof*⟩

20.2 Composite systems

Postulate 4 was stated with Day 7. The notes' example: photons in states $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$ form the pair $\frac{1}{2}(|00\rangle + i|01\rangle + |10\rangle + i|11\rangle) \in \mathbb{C}^2 \otimes \mathbb{C}^2$, a vector of Schmidt rank 1. Two photons described by a unit vector $\psi \in \mathbb{C}^2 \otimes \mathbb{C}^2$ can be regarded as two separate subsystems iff $\psi = \psi_1 \otimes \psi_2$ iff the Schmidt rank of ψ is 1; otherwise ψ is *entangled*, which does happen in nature.

Tensor products of operators: for $R : \mathcal{H} \to \mathcal{H}$ and $T : \mathcal{K} \to \mathcal{K}$ there is a unique $R \otimes T$ with $(R \otimes T)(\sum_l h_l \otimes k_l) = \sum_l Rh_l \otimes Tk_l$, and (1) $(R_1 \otimes T_1)(R_2 \otimes T_2) = R_1R_2 \otimes T_1T_2$, (2) $(R \otimes T)^* = R^* \otimes T^*$ (both proved in Unitary_Tensor_Algebra for the Kronecker product).

Remark. If lab A has the measurement system $\{M_m\}$ on $\mathcal{H}$ and lab B has state space $\mathcal{K}$, the measurement system of the composite is $\{M_m \otimes I_\mathcal{K}\}$ and $p_m^{AB}(\psi \otimes \phi) = \|(M_m \otimes I)(\psi \otimes \phi)\|^2 = p_m^A(\psi)$.

lemma *tensor-mat-vec-mult*:
assumes *A*: $A \in$ *carrier-mat nr nc*
and *B*: $B \in$ *carrier-mat mr mc*
and *v*: *dim-vec v* $=$ *nc*
and *w*: *dim-vec w* $=$ *mc*
shows $(A \bigotimes B) *_v$ *tensor-vec v w* $=$ *tensor-vec* $(A *_v v)\ (B *_v w)$
⟨*proof*⟩

lemma *tensor-mat-vec-mult-right-id*:
assumes *A*: $A \in$ *carrier-mat n n*
and *v*: *dim-vec v* $=$ *n*
and *w*: *dim-vec w* $=$ *m*
shows $(A \bigotimes 1_m\ m) *_v$ *tensor-vec v w* $=$ *tensor-vec* $(A *_v v)\ w$
⟨*proof*⟩

lemma *composite-measurement-prob*:
fixes *M* :: *complex Matrix.mat* **and** ψ :: *complex Matrix.vec* **and** φ :: *complex Matrix.vec*
assumes *psi-st*: *quantum-state n* ψ **and** *phi-st*: *quantum-state m* φ
and *M-sq*: $M \in$ *carrier-mat n n*
shows *meas-prob* $(M \bigotimes 1_m\ m)$ *(tensor-vec* $\psi\ \varphi$*)* $=$ *meas-prob M* ψ
⟨*proof*⟩

20.3 Entanglement

A state $\psi \in \mathcal{H}_A \otimes \mathcal{H}_B$ is *entangled* if it is not a product state $u \otimes v$. The Bell state $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is entangled. The notes' first application of entanglement (A and B share γ; A measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ and gets each outcome with probability $\frac{1}{2}$, after which B's measurement is determined) is *bell-measurement* in theory Day02_11_Supplement.

definition *entangled* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
entangled n m ψ ≡
dim-vec ψ = *n* ∗ *m* ∧
¬ (∃ *u v*. *dim-vec u* = *n* ∧ *dim-vec v* = *m* ∧ *ψ* = *tensor-vec u v*)

definition *bell-state* :: *complex Matrix.vec* **where**
bell-state ≡ (*1* / *csqrt 2*) ·$_v$
(*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*))

lemma *bell-state-norm*:
‖*bell-state*‖ = *1*
⟨*proof*⟩

lemma *bell-state-is-quantum-state*:
quantum-state 4 bell-state
⟨*proof*⟩

lemma *bell-state-entangled*:
entangled 2 2 bell-state
⟨*proof*⟩

end
theory *Day09-Bell-Measurement*
imports *Quantum-Computing-by-VP-3-10.Day09-POVMs-Entanglement*
begin

21 Day 9: measuring one half of a Bell pair

definition *e0* :: *complex Matrix.vec* **where** *e0* = *unit-vec 2 0*
definition *e1* :: *complex Matrix.vec* **where** *e1* = *unit-vec 2 1*
definition *proj0* :: *complex Matrix.mat* **where** *proj0* = *e0* ⊘ *e0*
definition *proj1* :: *complex Matrix.mat* **where** *proj1* = *e1* ⊘ *e1*

lemma *e-dims* [*simp*]: *dim-vec e0* = *2 dim-vec e1* = *2*
⟨*proof*⟩

lemma *e-inner* [*simp*]: ⟨*e0*|*e0*⟩ = *1* ⟨*e0*|*e1*⟩ = *0* ⟨*e1*|*e0*⟩ = *0* ⟨*e1*|*e1*⟩ = *1*
⟨*proof*⟩

lemma *proj-carrier* [*simp*]: *proj0* ∈ *carrier-mat 2 2 proj1* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *proj-apply* [*simp*]:
proj0 ∗$_v$ *e0* = *e0 proj0* ∗$_v$ *e1* = *0*$_v$ *2 proj1* ∗$_v$ *e0* = *0*$_v$ *2 proj1* ∗$_v$ *e1* = *e1*
⟨*proof*⟩

lemma *bell-state-e*: *bell-state* = (*1* / *csqrt 2*) ·$_v$ (*tensor-vec e0 e0* + *tensor-vec e1 e1*)
⟨*proof*⟩

lemma *tensor-zero-left*: *dim-vec w* = *2* ⟹ *tensor-vec* (*0*$_v$ *2*) *w* = *0*$_v$ *4*
⟨*proof*⟩

lemma *tensor-zero-right*: *dim-vec v* = *2* ⟹ *tensor-vec v* (*0*$_v$ *2*) = *0*$_v$ *4*
⟨*proof*⟩

lemma *A-meas-bell*:

(*proj0* ⨂ 1_m *2*) $*_v$ *bell-state* = (*1* / *csqrt 2*) $\cdot_v$ *tensor-vec e0 e0*
(*proj1* ⨂ 1_m *2*) $*_v$ *bell-state* = (*1* / *csqrt 2*) $\cdot_v$ *tensor-vec e1 e1*
⟨*proof*⟩

lemma *tensor-e-norm*: ‖*tensor-vec e0 e0*‖ = *1* ‖*tensor-vec e1 e1*‖ = *1*
⟨*proof*⟩

D9-6: A measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ on the shared state γ: each outcome has probability $\frac{1}{2}$, and γ collapses to $e_0 \otimes e_0$ resp. $e_1 \otimes e_1$, after which B's measurement $I \otimes M_m$ gives A's outcome with probability 1.

theorem *bell-measurement*:
meas-prob (*proj0* ⨂ 1_m *2*) *bell-state* = *1/2*
meas-prob (*proj1* ⨂ 1_m *2*) *bell-state* = *1/2*
post-meas-state (*proj0* ⨂ 1_m *2*) *bell-state* = *tensor-vec e0 e0*
post-meas-state (*proj1* ⨂ 1_m *2*) *bell-state* = *tensor-vec e1 e1*
meas-prob (1_m *2* ⨂ *proj0*) (*tensor-vec e0 e0*) = *1*
meas-prob (1_m *2* ⨂ *proj1*) (*tensor-vec e1 e1*) = *1*
⟨*proof*⟩

lemma *proj-explicit*:
proj0 = *Matrix.mat 2 2* (λ(*i*,*j*). *if i* = *0* ∧ *j* = *0 then 1 else 0*)
proj1 = *Matrix.mat 2 2* (λ(*i*,*j*). *if i* = *1* ∧ *j* = *1 then 1 else 0*)
⟨*proof*⟩

lemma *proj-measurement-system*: *measurement-system 2* [*proj0*, *proj1*]
⟨*proof*⟩

end

theory *Day10-Quantum-Gates*
imports *Day09-POVMs-Entanglement*
begin

22 Day 10 (19 September 2011): superdense coding; classical and quantum gates

22.1 One-qubit gates

The one-qubit gates of the notes:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \text{ (NOT)}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad H = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}.$$

H is the Hadamard gate; the notes write "$H^2 = X$" here and correct it to $H^2 = I$ on Day 11, which is what is proved below. H and S together generate all 2×2 unitaries (not formalised). Formally H is *Hadamard* (the library's *Quantum.H*) and S is *Phase-S*.

definition *Hadamard* :: *complex Matrix.mat* **where**
Hadamard ≡ *Quantum.H*

lemma *csqrt-two*: *csqrt* (*2*::*complex*) = (*sqrt 2* :: *complex*)
⟨*proof*⟩

lemma *hadamard-squared*:
Hadamard ∗ *Hadamard* = 1_m *2*

⟨*proof*⟩

lemma *hadamard-e0*:
Hadamard $*_v$ *unit-vec 2 0* =
(*1 / csqrt 2*) $\cdot_v$ (*unit-vec 2 0* + *unit-vec 2 1*)
⟨*proof*⟩

lemma *hadamard-e1*:
Hadamard $*_v$ *unit-vec 2 1* =
(*1 / csqrt 2*) $\cdot_v$ (*unit-vec 2 0* − *unit-vec 2 1*)
⟨*proof*⟩

The phase gate $S = \mathrm{diag}(1, i)$ is unitary.

definition *Phase-S* :: *complex Matrix.mat* **where**
Phase-S ≡ *Quantum.S*

lemma *Phase-S-carrier* [*simp*]: *Phase-S* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Phase-S-dagger-S*: *Phase-S*† ∗ *Phase-S* = 1_m *2*
⟨*proof*⟩

lemma *Phase-S-S-dagger*: *Phase-S* ∗ (*Phase-S*†) = 1_m *2*
⟨*proof*⟩

lemma *Phase-S-unitary*: *unitary Phase-S*
⟨*proof*⟩

22.2 Two- and three-qubit gates

The notes tabulate the classical gates AND, OR, XOR, NAND, NOR (Peirce: NAND generates all Boolean operations; Sheffer: so does NOR). No two-bit classical gate is a quantum gate, since it maps two dimensions to one and is not reversible. The controlled NOT acts by $|a, b\rangle \mapsto |a, b \oplus a\rangle$:

$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

which is unitary, and Toffoli's CCNOT (identity on the first six basis vectors, swapping $|110\rangle$ and $|111\rangle$) is a reversible gate generating all Boolean operations; it is defined in theory Quantum_Computing11.

lemma *CNOT-unitary*: *unitary CNOT*
⟨*proof*⟩

lemma *CNOT-00*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*)) =
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 0*)
⟨*proof*⟩

lemma *CNOT-10*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 0*)) =
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)
⟨*proof*⟩

lemma *CNOT-01*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 1*)) =
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*)
⟨*proof*⟩

lemma *CNOT-11*:
CNOT $*_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 1*)) =
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*)
⟨*proof*⟩

lemma *tensor-unit-vec-2-00*:
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 0*) = *unit-vec 4 0*
⟨*proof*⟩

lemma *tensor-unit-vec-2-01*:
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*) = *unit-vec 4 1*
⟨*proof*⟩

lemma *tensor-unit-vec-2-10*:
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*) = *unit-vec 4 2*
⟨*proof*⟩

lemma *tensor-unit-vec-2-11*:
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*) = *unit-vec 4 3*
⟨*proof*⟩

22.3 Superdense coding

A and B share $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Applying nothing, $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ or $\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$ to its qubit, A turns γ into one of the four orthonormal Bell states $\frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$, $\frac{1}{\sqrt{2}}(|10\rangle \pm |01\rangle)$, so by sending one qubit A communicates two bits; with $2m$ shared qubits, 4^m messages. Below: the four Bell states and their orthonormality (the action table itself is not formalised).

definition *bell-basis* :: *complex Matrix.vec list* **where**
bell-basis ≡
[(*1/csqrt 2*) $\cdot_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)),
(*1/csqrt 2*) $\cdot_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 0*) −
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 1*)),
(*1/csqrt 2*) $\cdot_v$ (*tensor-vec* (*unit-vec 2 1*) (*unit-vec 2 0*) +
tensor-vec (*unit-vec 2 0*) (*unit-vec 2 1*)),
(*1/csqrt 2*) $\cdot_v$ (*tensor-vec* (*unit-vec 2 0*) (*unit-vec 2 1*) −
tensor-vec (*unit-vec 2 1*) (*unit-vec 2 0*))]

lemma *bell-basis-orthonormal*: *orthonormal-list 4 bell-basis*
⟨*proof*⟩

lemma *psd-smult-nonneg*:
assumes *c*: $0 \le c$
and *A*: *psd A* **and** *Ac*: $A \in$ *carrier-mat n n*
shows *psd* (*complex-of-real c* $\cdot_m$ *A*)
⟨*proof*⟩

lemma *tensor-unit-vec*:
assumes *i*: $i < n$ **and** *j*: $j < m$

shows *tensor-vec* (*unit-vec n i*) (*unit-vec m j*) = (*unit-vec* (*n* ∗ *m*) (*i* ∗ *m* + *j*) :: *complex Matrix.vec*)
⟨*proof*⟩

lemma *Tr-dagger*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
shows *Tr* (*M*†) = *cnj* (*Tr M*)
⟨*proof*⟩

end
theory *Day10-Superdense-Coding*
imports *Day30-Error-Probabilities*
begin

23 Day 10: superdense coding (D10-1)

Lab A and Lab B share $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, Lab A holding the first qubit. Lab A multiplies its qubit by I, $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ or $\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$ and sends it to Lab B, which then holds $\frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$ or $\frac{1}{\sqrt{2}}(|01\rangle \pm |10\rangle)$: four orthonormal states, so one qubit carries two bits. The four matrices are I, Z, X and ZX (*sd-op-paulis*).

lemma *less4*: *i* < (*4*::*nat*) ⟷ *i* = *0* ∨ *i* = *1* ∨ *i* = *2* ∨ *i* = *3* ⟨*proof*⟩

lemma *tensor-I2-apply*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *A*: *A* ∈ *carrier-mat 2 2* **and** *v*: *dim-vec v* = *4* **and** *i*: *i* < *4*
shows (*Tensor.tensor-mat A* (1$_m$ *2*) ∗$_v$ *v*) $ *i* =
(∑*j*<*4*. *A* $$ (*i div 2*, *j div 2*) ∗ (*if i mod 2* = *j mod 2 then 1 else 0*) ∗ *v* $ *j*)
⟨*proof*⟩

definition *superdense-gamma* :: *complex Matrix.vec* **where**
superdense-gamma = *bell-basis* ! *0*

Lab A applies one of the notes' four matrices to its qubit.

definition *sd-op* :: *nat* ⇒ *complex Matrix.mat* **where**
sd-op k = (*if k* = *0 then mat-of-rows-list 2* [[*1*, *0*], [*0*, *1*]]
else if k = *1 then mat-of-rows-list 2* [[*1*, *0*], [*0*, −*1*]]
else if k = *2 then mat-of-rows-list 2* [[*0*, *1*], [*1*, *0*]]
else mat-of-rows-list 2 [[*0*, *1*], [−*1*, *0*]])

lemma *sd-op-carrier*: *sd-op k* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *sd-op-paulis*:
sd-op 0 = 1$_m$ *2 sd-op 1* = *Pauli-Z sd-op 2* = *Pauli-X sd-op 3* = *Pauli-Z* ∗ *Pauli-X*
⟨*proof*⟩

The notes' table: the four results are the four Bell states.

lemma *superdense-coding-0*: *Tensor.tensor-mat* (*sd-op 0*) (1$_m$ *2*) ∗$_v$ *superdense-gamma* = *bell-basis* ! *0*
⟨*proof*⟩

lemma *superdense-coding-1*: *Tensor.tensor-mat* (*sd-op 1*) (1$_m$ *2*) ∗$_v$ *superdense-gamma* = *bell-basis* ! *1*

⟨*proof*⟩

lemma *superdense-coding-2*: *Tensor.tensor-mat* (*sd-op 2*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *2*
⟨*proof*⟩

lemma *superdense-coding-3*: *Tensor.tensor-mat* (*sd-op 3*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *3*
⟨*proof*⟩

theorem *superdense-coding*:
assumes *k*: *k* < *4*
shows *Tensor.tensor-mat* (*sd-op k*) (*1*$_m$ *2*) $*_v$ *superdense-gamma* = *bell-basis* ! *k*
⟨*proof*⟩

corollary *superdense-coding-orthonormal*:
orthonormal-list 4 (*map* (λ*k*. *Tensor.tensor-mat* (*sd-op k*) (*1*$_m$ *2*) $*_v$ *superdense-gamma*) [*0*..<*4*])
⟨*proof*⟩

end

Part II

Lectures 11–22: quantum computing, density matrices, completely positive maps

theory *Day11-Gates-No-Cloning*
imports *Quantum-Computing-by-VP-3-10.Day10-Quantum-Gates*
begin

24 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning

The lecture opens with two corrections: $H^2 = I$ (not X), with $He_0 = (e_0 + e_1)/\sqrt{2}$, $Xe_0 = e_1$, $Xe_1 = e_0$, $He_1 = (e_0 - e_1)/\sqrt{2}$, so that X and H are reflections; and the truth tables of CNOT ($|a, b\rangle \mapsto |a, b + a\rangle$, arithmetic in $\mathbb{Z}_2$) and of CCNOT ($|a, b, c\rangle \mapsto |a, b, c + ab\rangle$). Circuit diagrams are introduced; the CNOT with control on the second wire has matrix $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$, and three alternating CNOTs compose to the swap of the two wires (*SWAP-three-CNOTs* in theory Day02_11_Supplement).

Definition (notes). The Toffoli gate CCNOT is the 8×8 permutation matrix that fixes the first six basis vectors and exchanges $|110\rangle$ and $|111\rangle$.

definition *CCNOT* :: *complex Matrix.mat* **where**
CCNOT ≡ *Matrix.mat 8 8*
(λ(*i*,*j*). *if* (*i* = *6* ∧ *j* = *7*) ∨ (*i* = *7* ∧ *j* = *6*)
then 1
else if i = *j* ∧ *i* < *6 then 1 else 0*)

lemma *CCNOT-dagger* [*simp*]:
CCNOT† = *CCNOT*
⟨*proof*⟩

lemma *CCNOT-dim-row* [*simp*]:
dim-row CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-dim-col* [*simp*]:
dim-col CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-inv* [*simp*]:
CCNOT ∗ *CCNOT* = *1*$_m$ *8*
⟨*proof*⟩

lemma *CCNOT-unitary*: *unitary CCNOT*
⟨*proof*⟩

24.1 Cloning and no-cloning

CNOT clones the computational basis: $U(e_a \otimes e_0) = e_a \otimes e_a$, and more generally $|i_1 \cdots i_n\rangle \otimes |0 \cdots 0\rangle \mapsto |i_1 \cdots i_n\rangle \otimes |i_1 \cdots i_n\rangle$ for some unitary. But for $\psi = \alpha e_0 + \beta e_1$ with $\alpha\beta \neq 0$, $U(\psi \otimes e_0) \neq \psi \otimes \psi$.

Theorem (notes) (No-cloning)**.** There is no state ϕ and unitary U on $\mathcal{H} \otimes \mathcal{H}$ with $U(\psi \otimes \phi) = \psi \otimes \psi$ for all $\psi \in \mathcal{H}$.

The notes argue with $-\psi$; the formal proof (for $\dim \mathcal{H} = n \geq 2$ and unit vectors ψ) uses two states with inner product neither 0 nor 1.

definition *is-cloner* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
is-cloner n U φ ≡
U ∈ *carrier-mat* (*n*^*2*) (*n*^*2*) ∧ *unitary U* ∧ *dim-vec φ* = *n* ∧
(∀ *ψ*. *quantum-state n ψ* ⟶
U ∗$_v$ *tensor-vec ψ φ* = *tensor-vec ψ ψ*)

Two states whose inner product is neither 0 nor 1 exist once $n \geq 2$.

lemma *two-states-nontrivial-inner-prod*:
assumes *n*: *2* ≤ *n*
obtains *ψ*$_1$ *ψ*$_2$ :: *complex Matrix.vec*
where *quantum-state n ψ*$_1$ **and** *quantum-state n ψ*$_2$
and ⟨*ψ*$_1$ | *ψ*$_2$⟩ ≠ (*0* :: *real*) **and** ⟨*ψ*$_1$ | *ψ*$_2$⟩ ≠ (*1* :: *real*)
⟨*proof*⟩

theorem *no-cloning*:
assumes *n2*: *2* ≤ *n*
shows ¬ (∃ *U φ*. *is-cloner n U φ*)
⟨*proof*⟩

end
theory *Day11-SWAP-Circuit*
imports *Quantum-Computing-by-VP-3-10.Day06-Tensor-Products*
begin

25 Day 11: SWAP from three CNOTs

lemma *less4-cases*: $(i::nat) < 4 \longleftrightarrow i = 0 \vee i = 1 \vee i = 2 \vee i = 3$
⟨*proof*⟩

CNOT′ is CNOT controlled by the second qubit.

definition *CNOT′* :: *complex Matrix.mat* **where**
CNOT′ = *Matrix.mat 4 4* (λ(*i*,*j*). *if* (*i*,*j*) ∈ {(*0*,*0*),(*1*,*3*),(*2*,*2*),(*3*,*1*)} *then 1 else 0*)

D11-3: SWAP = CNOT · CNOT′ · CNOT (*SWAP* is the AFP's).

theorem *SWAP-three-CNOTs*: *SWAP* = *CNOT* ∗ *CNOT′* ∗ *CNOT*
⟨*proof*⟩

lemma *SWAP-basis*:
assumes *a* < *2 b* < *2*
shows *SWAP* ∗$_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 a*)
⟨*proof*⟩

lemma *CNOT′-basis*:
assumes *a* < *2 b* < *2*
shows *CNOT′* ∗$_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2* ((*a* + *b*) *mod 2*)) (*unit-vec 2 b*)
⟨*proof*⟩

end
theory *Day11-CCNOT-Table*
imports *Day11-Gates-No-Cloning*
begin

26 Day 11: the CCNOT truth table (D11-2)

lemma *CCNOT-unit-vec*:
assumes *k*: *k* < *8*
shows *CCNOT* ∗$_v$ *unit-vec 8 k* = *unit-vec 8* (*if k = 6 then 7 else if k = 7 then 6 else k*)
⟨*proof*⟩

On basis states $|a, b, c\rangle = e_a \otimes e_b \otimes e_c$: when $a = 0$ nothing happens; when $a = 1$, CNOT acts on b, c.

theorem *CCNOT-basis*:
assumes *a*: *a* < *2* **and** *b*: *b* < *2* **and** *c*: *c* < *2*
shows *CCNOT* ∗$_v$ *tensor-vec* (*unit-vec 2 a*) (*tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 c*)) =
tensor-vec (*unit-vec 2 a*) (*tensor-vec* (*unit-vec 2 b*) (*unit-vec 2* (*if a = 1* ∧ *b = 1 then 1* − *c else c*)))
⟨*proof*⟩

corollary *CCNOT-truth-table*:
fixes *e* :: *nat* ⇒ *complex Matrix.vec*
defines *e* ≡ λ*a*. *unit-vec 2 a*
shows *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 0*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 0*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 1*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 0*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 0*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 1*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 0*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 0*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 1*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 0*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 1*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 0*))

⟨*proof*⟩

end

theory *Day12-Quantum-Parallelism*
imports *Day11-Gates-No-Cloning*
begin

27 Day 12 (23 September 2011): quantum parallelism

Since $He_0 = (e_0 + e_1)/\sqrt{2}$,

$$H^{\otimes N}(e_0 \otimes \cdots \otimes e_0) = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_{J \in \mathbb{Z}_2^N} e_J = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_J |J\rangle.$$

Only the case $N = 1$ is stated here; the general identity is *hadamard-uniform-N* in *Day12-Counting-Probability* (with *tensor-pow-mat* for $H^{\otimes N}$).

Definition (notes) (Oracle)**.** For $f : \mathbb{Z}_2^N \to \mathbb{Z}_2$, the vectors $e_J \otimes e_{i+f(J)}$ form an orthonormal basis of $(\mathbb{C}^2)^{\otimes N} \otimes \mathbb{C}^2$, so there is a unitary U_f with $U_f(e_J \otimes e_i) = e_J \otimes e_{i+f(J)}$, the *oracle* for f.

Proposition (notes). For a unit vector γ, the measurement $P_0 = |\gamma\rangle\langle\gamma|$, $P_1 = I - P_0$ has $p_0(\psi) = |\langle\gamma|\psi\rangle|^2$ and $p_1(\psi) = 1 - |\langle\gamma|\psi\rangle|^2$.

Application: with $x = (He_0)^{\otimes N}$, $U_f(x \otimes e_0) = 2^{-N/2} \sum_J e_J \otimes e_{f(J)}$, and measuring against $\gamma = x \otimes e_0$ gives $\langle\gamma|U_f(x \otimes e_0)\rangle = M/2^N$ with $M = \#\{J : f(J) = 0\}$, hence $p_0 = (M/2^N)^2$: repeating the experiment estimates M (a Bernoulli trial). The existence of the oracle is formalised below; the proposition is *rank-one-meas-prob* (Day 8); the counting identity is *counting-probability* in *Day12-Counting-Probability*.

definition *oracle-unitary* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *bool* **where**
oracle-unitary f N ≡
∃ *U* :: *complex Matrix.mat*.
U ∈ *carrier-mat* (*2^(N+1)*) (*2^(N+1)*) ∧
unitary U ∧
(∀ *J* < *2^N*. ∀ *i* < *2*.
U ∗$_v$ (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*)))

theorem *oracle-exists*:
fixes *f* :: *nat* ⇒ *nat*
assumes *f-range*: ∀ *J* < *2^*(*N* :: *nat*). *f J* < *2*
shows *oracle-unitary f N*
⟨*proof*⟩

$M = \#\{J < 2^N : f(J) = 0\}$, the quantity estimated by the counting experiment.

definition *count-zeros* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *nat* **where**
count-zeros f N ≡ *card* {*J*. *J* < *2^N* ∧ *f J* = *0*}

end
theory *Day12-Counting-Probability*
imports *QC-Prelims*
begin

28 Day 12: $H^{\otimes N}$ on $|0\cdots 0\rangle$ and the counting probability (D12-1, D12-4)

N-fold tensor powers (last factor least significant, matching $(\otimes)$).

primrec *tensor-pow-mat* :: *complex Matrix.mat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
  tensor-pow-mat A 0 = 1$_m$ *1*
| *tensor-pow-mat A* (*Suc k*) = *Tensor.tensor-mat* (*tensor-pow-mat A k*) *A*

primrec *tensor-pow-vec* :: *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
  tensor-pow-vec v 0 = Matrix.vec 1 (λ-. *1*)
| *tensor-pow-vec v* (*Suc k*) = *tensor-vec* (*tensor-pow-vec v k*) *v*

lemma *tensor-pow-mat-carrier*:
  assumes *A* ∈ *carrier-mat 2 2*
  shows *tensor-pow-mat A k* ∈ *carrier-mat* (*2^k*) (*2^k*)
  ⟨*proof*⟩

lemma *tensor-pow-vec-dim*:
  assumes *dim-vec v = 2*
  shows *dim-vec* (*tensor-pow-vec v k*) = *2^k*
  ⟨*proof*⟩

lemma *tensor-pow-mat-vec*:
  assumes *Ac*: *A* ∈ *carrier-mat 2 2* **and** *vd*: *dim-vec v = 2*
  shows *tensor-pow-mat A k* ∗$_v$ *tensor-pow-vec v k = tensor-pow-vec* (*A* ∗$_v$ *v*) *k*
⟨*proof*⟩

lemma *tensor-pow-vec-const*:
  tensor-pow-vec (*Matrix.vec 2* (λ-. *c*)) *k = Matrix.vec* (*2^k*) (λ-. *c^k*)
⟨*proof*⟩

definition *uniform-vec* :: *nat* ⇒ *complex Matrix.vec* **where**
  uniform-vec N = Matrix.vec (*2^N*) (λ-. *complex-of-real* (*1 / sqrt 2*) ^ *N*)

lemma *uniform-vec-dim* [*simp*]: *dim-vec* (*uniform-vec N*) = *2^N*
  ⟨*proof*⟩

lemma *hadamard-e0-const*: *Hadamard* ∗$_v$ *unit-vec 2 0 = Matrix.vec 2* (λ-. *complex-of-real* (*1 / sqrt 2*))
⟨*proof*⟩

theorem *hadamard-uniform-N*:
  tensor-pow-mat Hadamard N ∗$_v$ *tensor-pow-vec* (*unit-vec 2 0*) *N = uniform-vec N*
⟨*proof*⟩

lemma *inner-prod-unit-vec-2*:
  assumes *k < 2*
  shows ⟨*unit-vec 2 0* | *unit-vec 2 k*⟩ = (*if k = 0 then 1 else 0*)
⟨*proof*⟩

lemma *inner-prod-uniform-unit*:
  assumes *J < 2^N*
  shows ⟨*uniform-vec N* | *unit-vec* (*2^N*) *J*⟩ = *complex-of-real* (*1 / sqrt 2*) ^ *N*
  ⟨*proof*⟩

lemma *half-sqrt2-sq*: (*1 / sqrt 2* :: *real*) ∗ (*1 / sqrt 2*) = *1 / 2*
⟨*proof*⟩

lemma *uniform-coeff-sq*:
*complex-of-real (1 / sqrt 2) ^ N * complex-of-real (1 / sqrt 2) ^ N = complex-of-real ((1/2) ^ N)*
⟨*proof*⟩

lemma *uniform-vec-self*: ⟨*uniform-vec N* | *uniform-vec N*⟩ = *1*
⟨*proof*⟩

lemma *uniform-tensor-e0-as-sum*:
tensor-vec (uniform-vec N) (unit-vec 2 0) =
vec-sum (2^(N+1)) {J. J < 2^N}
(λJ. complex-of-real (1 / sqrt 2) ^ N ·v tensor-vec (unit-vec (2^N) J) (unit-vec 2 0))
⟨*proof*⟩

lemma *uniform-e0-state*: *quantum-state (2^(N+1)) (tensor-vec (uniform-vec N) (unit-vec 2 0))*
⟨*proof*⟩

The notes' computation: with $x = H^{\otimes N}|0\cdots0\rangle$ and $\gamma = x \otimes e_0$, $p_0(U_f(x \otimes e_0)) = (M/2^N)^2$ where $M = \#\{J : f(J) = 0\}$.

theorem *counting-probability*:
fixes *f* :: *nat* ⇒ *nat* **and** *U* :: *complex Matrix.mat*
assumes *f-range*: ∀ *J* < *2^N*. *f J* < *2*
and *Uc*: *U* ∈ *carrier-mat (2^(N+1)) (2^(N+1))*
and *orac*: ∀ *J* < *2^N*. ∀ *i* < *2*.
U ∗*v* (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*))
shows *meas-prob* (*tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*) ⊘ *tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
(*U* ∗*v* *tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
= (*real* (*count-zeros f N*) / *2^N*)^2
⟨*proof*⟩

end

theory *Day13-Density-Matrices*
imports *Day12-Quantum-Parallelism*
begin

29 Day 13 (26 September 2011): ensembles and density matrices

29.1 Ensembles

Motivation: measuring $\psi = a|0\rangle + b|1\rangle$ with $M_0 = |0\rangle\langle 0|$, $M_1 = |1\rangle\langle 1|$ leaves the state $\frac{a}{|a|}|0\rangle$ with probability $|a|^2$ and $\frac{b}{|b|}|1\rangle$ with probability $|b|^2$; a later measurement $\{\tilde{M}_\alpha\}$ then has outcome probabilities $p_\alpha = |a|^2\|\tilde{M}_\alpha \frac{a}{|a|}|0\rangle\|^2 + |b|^2\|\tilde{M}_\alpha \frac{b}{|b|}|1\rangle\|^2$.

Definition (notes). An *ensemble* $\{p_i, \psi_i\}_{i=1}^L$ is a set of states with probabilities $p_i \geq 0$, $\sum_i p_i = 1$. For a measurement system $\{M_\alpha\}$ the probability of outcome α is $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2$.

A second source of ensembles: a unit vector $\psi = \sum_l \phi_l \otimes f_l \in \mathcal{H}_A \otimes \mathcal{H}_E$ (environment $\mathcal{H}_E$ with orthonormal basis f_l) behaves, for measurements $M_\alpha \otimes I$, exactly like the ensemble $\{\|\phi_l\|^2, \phi_l/\|\phi_l\|\}$.

definition *ensemble* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *bool* **where**

```
ensemble n E ≡
  (∀(p, ψ) ∈ set E. p ≥ 0 ∧ quantum-state n ψ) ∧
  (∑(p, -) ← E. p) = 1
```

29.2 Von Neumann's density matrix

Measurements cannot distinguish ψ from $e^{i\theta}\psi$, and $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi| = (\alpha_i\overline{\alpha_j})$: the object attached to a state is the projection, not the vector. For any γ, $\|\gamma\|^2 = \operatorname{Tr}|\gamma\rangle\langle\gamma|$, and

$$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \operatorname{Tr}\big((M_\alpha^* M_\alpha)(\psi\psi^*)\big), \qquad p_\alpha(\{p_l,\psi_l\}) = \operatorname{Tr}\Big((M_\alpha^* M_\alpha)\sum_l p_l\psi_l\psi_l^*\Big).$$

Definition (notes). $P = \sum_l p_l|\psi_l\rangle\langle\psi_l|$ is the *density matrix* of the ensemble.

Formally *pure-density* ψ is $|\psi\rangle\langle\psi|$, *ensemble-density* n E is P, and *density-matrix* n ϱ is the Day 14 characterisation ($\rho \geq 0$, $\operatorname{Tr}\rho = 1$).

```
definition density-matrix :: nat ⇒ complex Matrix.mat ⇒ bool where
  density-matrix n ϱ ≡ ϱ ∈ carrier-mat n n ∧ psd ϱ ∧ Tr ϱ = 1
```

```
definition pure-density :: complex Matrix.vec ⇒ complex Matrix.mat where
  pure-density ψ ≡
    let n = dim-vec ψ in
    outer-sum n [ψ]
```

```
lemma Tr-outer-sum-single:
  assumes dim: dim-vec ψ = n
  shows Tr (outer-sum n [ψ]) = ⟨ψ | ψ⟩
⟨proof⟩
```

```
lemma pure-density-is-density-matrix:
  assumes quantum-state n ψ
  shows density-matrix n (pure-density ψ)
⟨proof⟩
```

```
definition ensemble-density :: nat ⇒ (real × complex Matrix.vec) list ⇒ complex Matrix.mat where
  ensemble-density n E ≡
    foldr (λ(p,ψ) acc. complex-of-real p ·_m outer-sum n [ψ] + acc)
          E (0_m n n)
```

```
lemma psd-zero-mat [simp]:
  psd (0_m n n)
⟨proof⟩
```

```
lemma psd-add-psd:
  assumes A: psd A and B: psd B
     and Ac: A ∈ carrier-mat n n and Bc: B ∈ carrier-mat n n
  shows psd (A + B)
⟨proof⟩
```

```
lemma ensemble-density-Nil [simp]:
  ensemble-density n [] = 0_m n n
  ⟨proof⟩
```

lemma *ensemble-density-Cons*:
ensemble-density n ((p, ψ) # E) =
complex-of-real p ·$_m$ outer-sum n [ψ] + ensemble-density n E
⟨*proof*⟩

lemma *ensemble-density-carrier*:
assumes ∀(*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows *ensemble-density n E* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *ensemble-density-psd-aux*:
assumes ∀(*p*, *ψ*) ∈ *set E*. *p* ≥ *0* ∧ *quantum-state n ψ*
shows *psd* (*ensemble-density n E*)
⟨*proof*⟩

lemma *Tr-zero-mat′* [*simp*]:
Tr (*0$_m$ n n*) = *0*
⟨*proof*⟩

lemma *ensemble-density-Tr*:
assumes ∀(*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows *Tr* (*ensemble-density n E*) = *complex-of-real* (∑(*p*, -) ← *E*. *p*)
⟨*proof*⟩

theorem *ensemble-density-is-dm*:
assumes *E*: *ensemble n E*
shows *density-matrix n* (*ensemble-density n E*)
⟨*proof*⟩

29.3 Probabilities via the trace

$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha |\psi\rangle\langle\psi|)$. For an ensemble with density matrix ρ, $p_\alpha = \mathrm{Tr}(M_\alpha^* M_\alpha \rho) = \langle M_\alpha^* M_\alpha | \rho \rangle$ (*ensemble-prob-trace-eq* below).

lemma *pure-state-prob-trace*:
fixes *M* :: *complex Matrix.mat* **and** *ψ* :: *complex Matrix.vec*
assumes *st*: *quantum-state n ψ*
and *Msq*: *M* ∈ *carrier-mat n n*
shows *meas-prob M ψ = Re* (*Tr* ((*M*†) ∗ *M* ∗ *outer-sum n* [*ψ*]))
⟨*proof*⟩

For an ensemble: $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha \rho)$.

lemma *ensemble-prob-trace-eq*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat n n*
and *E*: ∀(*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows (∑(*p*, *ψ*) ← *E*. *p* ∗ *meas-prob M ψ*)
= *Re* (*Tr* (((*M*†) ∗ *M*) ∗ *ensemble-density n E*))
⟨*proof*⟩

lemma *Tr-psd-Re-nonneg*:
assumes *P*: *psd P* **and** *Pc*: *P* ∈ *carrier-mat n n*
shows *0* ≤ *Re* (*Tr P*)
⟨*proof*⟩

lemma *ensemble-prob-trace*:
fixes *Ms* :: *complex Matrix.mat list*

assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n ϱ*
and *Min*: *M ∈ set Ms*
shows *Re (Tr ((M†) ∗ M ∗ ϱ)) ≥ 0*
⟨*proof*⟩

end

theory *Day14-Pure-States*
imports *Day13-Density-Matrices*
begin

30 Day 14 (28 September 2011): density matrices, pure states, composite ensembles

Proposition (notes). $\rho \in L(\mathcal{H}_A)$ is the density matrix of some ensemble if and only if $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$.

Definition (notes). ρ is a *density matrix* when $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$. The ensemble $\{\lambda_i, \psi_i\}$ of eigenvalues and an orthonormal eigenbasis is the *spectral ensemble*.

Many ensembles have the same density matrix: writing $\rho = XX^* = \sum_i c_i c_i^*$ with columns c_i gives the ensemble $\{\|c_i\|^2, c_i/\|c_i\|\}$.

theorem *density-matrix-characterisation*:
fixes *ϱ* :: *complex Matrix.mat*
assumes *sq*: *ϱ ∈ carrier-mat n n*
shows *density-matrix n ϱ ⟷ psd ϱ ∧ Tr ϱ = 1*
⟨*proof*⟩

lemma *ensemble-density-carrier-dim*:
shows *ensemble-density n E ∈ carrier-mat n n*
⟨*proof*⟩

lemma *outer-sum-singleton-index*:
assumes *r*: *r < n* **and** *c*: *c < n*
shows *outer-sum n [ψ] $$ (r, c) = ψ $ r ∗ cnj (ψ $ c)*
⟨*proof*⟩

Day 13: $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi|$; the density matrix forgets the phase.

lemma *pure-density-smult-unit*:
fixes *ψ* :: *complex Matrix.vec* **and** *c* :: *complex*
assumes *c*: *cmod c = 1*
shows *pure-density (c ·v ψ) = pure-density ψ*
⟨*proof*⟩

lemma *ensemble-density-index*:
assumes *r*: *r < n* **and** *c*: *c < n*
shows *ensemble-density n E $$ (r, c) =*
(∑ (p, ψ) ← E. complex-of-real p ∗ ψ $ r ∗ cnj (ψ $ c))
⟨*proof*⟩

The spectral ensemble: every density matrix comes from an ensemble.

theorem *spectral-ensemble-exists*:
fixes *ϱ* :: *complex Matrix.mat*

assumes *dm*: *density-matrix n ϱ*
shows ∃ *E*. *ensemble n E* ∧ *ensemble-density n E* = *ϱ*
⟨*proof*⟩

corollary *density-matrix-iff-ensemble*:
fixes *ϱ* :: *complex Matrix.mat*
shows *density-matrix n ϱ* ⟷
(∃ *E*. *ensemble n E* ∧ *ensemble-density n E* = *ϱ*)
⟨*proof*⟩

30.1 Pure states

If $\rho = |\psi\rangle\langle\psi| = \sum_i p_i|\psi_i\rangle\langle\psi_i|$ then for $\gamma \perp \psi$, $0 = \langle\gamma|\rho\gamma\rangle = \sum_i p_i|\langle\psi_i|\gamma\rangle|^2$, so every ψ_i with $p_i > 0$ is parallel to ψ: $\psi_i = \beta_i\psi$ with $|\beta_i| = 1$ and $|\psi_i\rangle\langle\psi_i| = |\psi\rangle\langle\psi|$. These density matrices are the *pure states*.

definition *pure-state-density* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
pure-state-density n ϱ ≡
density-matrix n ϱ ∧
(∃ *ψ*. *quantum-state n ψ* ∧ *ϱ* = *pure-density ψ*)

lemma *pure-state-density-is-density*:
assumes *pure-state-density n ϱ*
shows *density-matrix n ϱ*
⟨*proof*⟩

lemma *pure-density-is-pure-state-density*:
assumes *quantum-state n ψ*
shows *pure-state-density n* (*pure-density ψ*)
⟨*proof*⟩

A companion fact not in the notes: a density matrix is pure iff it is idempotent.

theorem *pure-density-iff-idempotent*:
fixes *ϱ* :: *complex Matrix.mat*
assumes *dm*: *density-matrix n ϱ*
shows *pure-state-density n ϱ* ⟷ *ϱ* ∗ *ϱ* = *ϱ*
⟨*proof*⟩

lemma *outer-sum-singleton-mult-vec*:
assumes *ψ-dim*: *dim-vec ψ* = *n* **and** *γ-dim*: *dim-vec γ* = *n*
shows *outer-sum n* [*ψ*] ∗*v γ* = ⟨*ψ*|*γ*⟩ ·*v ψ*
⟨*proof*⟩

lemma *ensemble-density-quad-form*:
assumes *E-dim*: ∀ (*p*, *ψ*) ∈ *set E*. *dim-vec ψ* = *n*
and *γ-dim*: *dim-vec γ* = *n*
shows ⟨*γ* | *ensemble-density n E* ∗*v γ*⟩ =
(∑ (*p*, *ψ*) ← *E*. *complex-of-real p* ∗ ⟨*ψ*|*γ*⟩ ∗ *cnj* ⟨*ψ*|*γ*⟩)
⟨*proof*⟩

theorem *pure-ensemble-parallel*:
fixes *ψ* :: *complex Matrix.vec*
assumes *st*: *quantum-state n ψ*
and *ens*: *ensemble n E*
and *dens*: *ensemble-density n E* = *pure-density ψ*

and *mem*: $(p, \psi_i) \in$ *set* E
and *ppos*: $0 < p$
shows $\exists\, \beta.\ \psi_i = \beta \cdot_v \psi$
⟨*proof*⟩

30.2 Composite ensembles

For ensembles $\{p_i, \psi_i\}$ in lab A and $\{q_l, \phi_l\}$ in lab B the composite is the ensemble $\{p_i q_l, \psi_i \otimes \phi_l\}$ with density matrix $\rho = \sum_{i,l} p_i q_l |\psi_i\rangle\langle\psi_i| \otimes |\phi_l\rangle\langle\phi_l|$, because $|\psi \otimes \phi\rangle\langle\psi \otimes \phi| = |\psi\rangle\langle\psi| \otimes |\phi\rangle\langle\phi|$ (checked on elementary tensors $h \otimes k$). The identity is formalised below; that $\rho_1 \otimes \rho_2$ is again a density matrix is *density-matrix-tensor* (Day 15).

lemma *tensor-pure-density*:
fixes $\psi\ \varphi$ :: *complex Matrix.vec*
assumes ψ-*dim*: *dim-vec* $\psi = m$
and φ-*dim*: *dim-vec* $\varphi = p$
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows *pure-density* (*tensor-vec* $\psi\ \varphi$) =
(*pure-density* ψ) $\bigotimes$ (*pure-density* φ)
⟨*proof*⟩

30.3 Partial traces: motivation

For a density matrix $\rho \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ and measurements $\{M_\alpha\}$ in lab A, which act as $M_\alpha \otimes I$, $p_\alpha(\rho) = \mathrm{Tr}((M_\alpha^* M_\alpha \otimes I)\rho)$. The notes announce a density matrix $\rho_A \in L(\mathcal{H}_A)$ with $\mathrm{Tr}_{\mathcal{H}_A \otimes \mathcal{H}_B}((M^*M \otimes I)\rho) = \mathrm{Tr}_{\mathcal{H}_A}(M^* M \rho_A)$ for all M; it is constructed on Day 15.

end
theory *Day14-Composite-Ensembles*
imports *Day23-ChoiExtremeUCP*
begin

31 Day 14: composite ensembles (D14-5)

$|\psi \otimes \varphi\rangle\langle\psi \otimes \varphi| = |\psi\rangle\langle\psi| \otimes |\varphi\rangle\langle\varphi|$.

lemma *outer-sum-tensor*:
assumes ψ: *dim-vec* $\psi = n$ **and** φ: *dim-vec* $\varphi = m$
shows *outer-sum* $(n * m)$ [*tensor-vec* $\psi\ \varphi$] = *outer-sum* n [ψ] $\bigotimes$ *outer-sum* m [φ]
⟨*proof*⟩

For ensembles $\{p_i, \psi_i\}$ on $\mathcal{H}_A$ and $\{q_j, \varphi_j\}$ on $\mathcal{H}_B$, the composite ensemble is $\{p_i q_j, \psi_i \otimes \varphi_j\}$.

definition *product-ensemble* :: (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* **where**
product-ensemble EA EB = *map* (λ((*p*, ψ), (*q*, φ)). (*p* ∗ *q*, *tensor-vec* ψ φ)) (*List.product EA EB*)

lemma *sum-list-product*:
fixes f :: $'a \Rightarrow$ *real* **and** g :: $'b \Rightarrow$ *real*
shows $(\sum (x,\ y) \leftarrow$ *List.product xs ys*. $f\ x * g\ y) = (\sum x \leftarrow xs.\ f\ x) * (\sum y \leftarrow ys.\ g\ y)$
⟨*proof*⟩

lemma *product-ensemble-ensemble*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *ensemble* $(n * m)$ (*product-ensemble EA EB*)
⟨*proof*⟩

The notes' composite density $\sum_{i,j} p_i q_j\, |\psi_i\rangle\langle\psi_i| \otimes |\varphi_j\rangle\langle\varphi_j|$ is the density of the composite ensemble, hence a density matrix.

definition *composite-density* :: *nat* ⇒ *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *complex Matrix.mat* **where**
composite-density n m EA EB = *foldr* (λ((*p*, *ψ*), (*q*, *φ*)) *acc*.
complex-of-real (*p* ∗ *q*) ·$_m$ (*outer-sum n* [*ψ*] ⨂ *outer-sum m* [*φ*]) + *acc*) (*List.product EA EB*) (0$_m$ (*n* ∗ *m*) (*n* ∗ *m*))

lemma *composite-density-eq*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *composite-density n m EA EB* = *ensemble-density* (*n* ∗ *m*) (*product-ensemble EA EB*)
⟨*proof*⟩

theorem *composite-density-is-dm*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *density-matrix* (*n* ∗ *m*) (*composite-density n m EA EB*)
⟨*proof*⟩

end

theory *Day15-Partial-Traces*
imports *Day14-Pure-States*
begin

32 Day 15 (30 September 2011): traces, tensor products of operators, partial traces

Proposition (notes). For any orthonormal basis $\{u_1, \ldots, u_n\}$ of $\mathbb{C}^n$, $\mathrm{tr}(A) = \sum_i \langle u_i | A u_i \rangle$.

theorem *trace-via-onb*:
fixes *A* :: *complex Matrix.mat*
assumes *sq*: *A* ∈ *carrier-mat n n*
and *orth*: *orthonormal-list n vs*
and *full*: *length vs* = *n*
shows *Tr A* = (∑ *i*<*n*. ⟨*vs* ! *i* | *A* ∗$_v$ *vs* ! *i*⟩)
⟨*proof*⟩

Proposition (notes). $\mathrm{tr}(R \otimes T) = \mathrm{tr}(R)\,\mathrm{tr}(T)$.

theorem *trace-tensor*:
fixes *R T* :: *complex Matrix.mat*
assumes *Rsq*: *R* ∈ *carrier-mat m m*
and *Tsq*: *T* ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *Tr* (*R* ⨂ *T*) = *Tr R* ∗ *Tr T*
⟨*proof*⟩

32.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$

Proposition (notes). In finite dimensions the map $\Gamma : L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \to L(\mathcal{H}_A \otimes \mathcal{H}_B)$, $R \otimes T \mapsto R \otimes T$, is a vector-space isomorphism (both sides have dimension $m^2 p^2$ and Γ is injective).

Formally, $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is represented in coordinates as M_{m^2,p^2} and Γ as the reindexing $M_{m^2,p^2} \to M_{mp}$; it is an explicit linear isomorphism with explicit inverse.

Postulate $4'$ (Day 16) and the composite ensembles of Day 14: tensor products of positive matrices are positive, and tensor products of density matrices are density matrices.

lemma *psd-tensor*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat m m* **and** *B*: *B* ∈ *carrier-mat p p*
and *pA*: *psd A* **and** *pB*: *psd B*
shows *psd* (*A* ⨂ *B*)
⟨*proof*⟩

lemma *density-matrix-tensor*:
fixes *A B* :: *complex Matrix.mat*
assumes *dA*: *density-matrix m A* **and** *dB*: *density-matrix p B*
and *m*: *0* < *m* **and** *p*: *0* < *p*
shows *density-matrix* (*m* ∗ *p*) (*A* ⨂ *B*)
⟨*proof*⟩

proposition *L-tensor-dim*:
fixes *m p* :: *nat*
shows *m*^2 ∗ *p*^2 = (*m* ∗ *p*)^2
⟨*proof*⟩

definition *operator-tensor-iso-map* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-map m p c ≡
Matrix.mat (*m*∗*p*) (*m*∗*p*)
(λ(*r*,*s*). *c* $$ ((*r div p*) ∗ *m* + (*s div p*), (*r mod p*) ∗ *p* + (*s mod p*)))

lemma *operator-tensor-iso-map-carrier* [*simp*]:
operator-tensor-iso-map m p c ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-map-entry*:
assumes *r* < *m*∗*p* **and** *s* < *m*∗*p*
shows *operator-tensor-iso-map m p c* $$ (*r*,*s*) =
c $$ ((*r div p*) ∗ *m* + (*s div p*), (*r mod p*) ∗ *p* + (*s mod p*))
⟨*proof*⟩

lemma *operator-tensor-iso-map-row-bound*:
fixes *m p r s* :: *nat*
assumes *p-pos*: *0* < *p* **and** *r* < *m*∗*p* **and** *s* < *m*∗*p*
shows (*r div p*) ∗ *m* + (*s div p*) < *m*∗*m*
⟨*proof*⟩

lemma *operator-tensor-iso-map-col-bound*:
fixes *p r s* :: *nat*
assumes *p-pos*: *0* < *p*
shows (*r mod p*) ∗ *p* + (*s mod p*) < *p*∗*p*
⟨*proof*⟩

definition *operator-tensor-iso-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-inv m p x ≡
Matrix.mat (*m*∗*m*) (*p*∗*p*)
(λ(*a*,*b*). *x* $$ ((*a div m*) ∗ *p* + (*b div p*), (*a mod m*) ∗ *p* + (*b mod p*)))

definition *operator-tensor-coordinate-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
operator-tensor-coordinate-space m p ≡ *mat-linear-structure* (*m*∗*m*) (*p*∗*p*)

definition *bipartite-operator-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
bipartite-operator-space m p ≡ *mat-linear-structure* (*m*∗*p*) (*m*∗*p*)

lemma *operator-tensor-iso-inv-carrier* [*simp*]:
operator-tensor-iso-inv m p x ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-inv-entry*:
assumes *a* < *m*∗*m* **and** *b* < *p*∗*p*
shows *operator-tensor-iso-inv m p x* $$ (*a*,*b*) =
x $$ ((*a div m*) ∗ *p* + (*b div p*), (*a mod m*) ∗ *p* + (*b mod p*))
⟨*proof*⟩

lemma *operator-tensor-iso-inv-row-bound*:
fixes *m p a b* :: *nat*
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p* **and** *a* < *m*∗*m* **and** *b* < *p*∗*p*
shows (*a div m*) ∗ *p* + (*b div p*) < *m*∗*p*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-col-bound*:
fixes *m p a b* :: *nat*
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows (*a mod m*) ∗ *p* + (*b mod p*) < *m*∗*p*
⟨*proof*⟩

lemma *operator-tensor-iso-map-add*:
assumes *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
and *y*: *y* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
shows *operator-tensor-iso-map m p* (*x* + *y*) =
operator-tensor-iso-map m p x + *operator-tensor-iso-map m p y*
⟨*proof*⟩

lemma *operator-tensor-iso-map-zero*:
assumes *p-pos*: *0* < *p*
shows *operator-tensor-iso-map m p* (*0*$_m$ (*m*∗*m*) (*p*∗*p*)) = *0*$_m$ (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-map-smult*:
assumes *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
shows *operator-tensor-iso-map m p* (*c* ·$_m$ *x*) =
c ·$_m$ *operator-tensor-iso-map m p x*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-add*:
assumes *m-pos*: *0* < *m*
and *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
and *y*: *y* ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
shows *operator-tensor-iso-inv m p* (*x* + *y*) =
operator-tensor-iso-inv m p x + *operator-tensor-iso-inv m p y*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-zero*:
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *operator-tensor-iso-inv m p* (*0*$_m$ (*m*∗*p*) (*m*∗*p*)) = *0*$_m$ (*m*∗*m*) (*p*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-inv-smult*:
  assumes *m-pos*: $0 < m$
    and *p-pos*: $0 < p$
    and *x*: $x \in$ *carrier-mat* $(m*p)$ $(m*p)$
  shows *operator-tensor-iso-inv m p* $(c \cdot_m x) =$
      $c \cdot_m$ *operator-tensor-iso-inv m p x*
⟨*proof*⟩

proposition *operator-tensor-space-isomorphism*:
  assumes *m-pos*: $0 < m$
    and *p-pos*: $0 < p$
  shows *is-isomorphism-between*
        (*operator-tensor-coordinate-space m p*)
        (*bipartite-operator-space m p*)
        (*operator-tensor-iso-map m p*)
⟨*proof*⟩

corollary *operator-tensor-space-isomorphic*:
  assumes $0 < m$ **and** $0 < p$
  shows *is-isomorphic-to*
        (*operator-tensor-coordinate-space m p*)
        (*bipartite-operator-space m p*)
  ⟨*proof*⟩

32.2 Partial traces

Definition (notes). Identify $L(\mathcal{H}_A \otimes \mathcal{H}_B) = L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ and define $\mathrm{tr}_B(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(T_l) R_l \in L(\mathcal{H}_A)$ and $\mathrm{tr}_A(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(R_l) T_l \in L(\mathcal{H}_B)$; also written $X^A = \mathrm{tr}_B(X)$, $X^B = \mathrm{tr}_A(X)$. Well defined by the universal property, since $(R, T) \mapsto \mathrm{tr}(T)R$ is bilinear.

The formal definitions use the block formulas of Day 16: an $mp \times mp$ matrix is indexed by pairs (i, k), $i < m$, $k < p$, with flat index $ip + k$ ($\mathcal{H}_A$ is the major factor, matching $(A \otimes B)_{(r,c)} = A_{(r \div p,\, c \div p)} B_{(r \bmod p,\, c \bmod p)}$), and $(\mathrm{tr}_B\, X)_{ij} = \sum_{k<p} X_{ip+k,\, jp+k}$, $(\mathrm{tr}_A\, X)_{ij} = \sum_{k<m} X_{kp+i,\, kp+j}$. The defining property on elementary tensors is proved below.

record *'lab qspace* =
  qdim :: *nat*
  qlabel :: *'lab*

record *'lab bipartite-space* =
  left-space :: *'lab qspace*
  right-space :: *'lab qspace*

definition *partial-trace-B* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  partial-trace-B m p x ≡
    Matrix.mat m m (λ(*i*, *j*). ∑ *k*<*p*. *x* $$ (*i* ∗ *p* + *k*, *j* ∗ *p* + *k*))

definition *partial-trace-A* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  partial-trace-A m p x ≡
    Matrix.mat p p (λ(*i*, *j*). ∑ *k*<*m*. *x* $$ (*k* ∗ *p* + *i*, *k* ∗ *p* + *j*))

definition *partial-trace-on* ::
  'lab qspace ⇒ *'lab bipartite-space* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  partial-trace-on s ab x ≡
    (*if s* = *right-space ab then*
      partial-trace-B (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*
     else if s = *left-space ab then*
      partial-trace-A (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*

else 0_m *0 0*)

notation *partial-trace-on* (*tr*$_{-}^{-}$ - [*1000*, *1000*, *1000*] *1000*)

lemma *partial-trace-on-right*:
fixes S :: *'lab qspace* **and** AB :: *'lab bipartite-space*
assumes S = *right-space AB*
shows tr_S^{AB} x =
partial-trace-B (*qdim* (*left-space AB*)) (*qdim* (*right-space AB*)) x
⟨*proof*⟩

lemma *partial-trace-on-left*:
fixes S :: *'lab qspace* **and** AB :: *'lab bipartite-space*
assumes S = *left-space AB*
and *left-space AB* ≠ *right-space AB*
shows tr_S^{AB} x =
partial-trace-A (*qdim* (*left-space AB*)) (*qdim* (*right-space AB*)) x
⟨*proof*⟩

lemma *dim-row-partial-trace-B* [*simp*]: *dim-row* (*partial-trace-B m p x*) = *m*
⟨*proof*⟩

lemma *dim-col-partial-trace-B* [*simp*]: *dim-col* (*partial-trace-B m p x*) = *m*
⟨*proof*⟩

lemma *partial-trace-B-carrier* [*simp*]:
partial-trace-B m p x ∈ *carrier-mat m m*
⟨*proof*⟩

lemma *dim-row-partial-trace-A* [*simp*]: *dim-row* (*partial-trace-A m p x*) = *p*
⟨*proof*⟩

lemma *dim-col-partial-trace-A* [*simp*]: *dim-col* (*partial-trace-A m p x*) = *p*
⟨*proof*⟩

lemma *partial-trace-A-carrier* [*simp*]:
partial-trace-A m p x ∈ *carrier-mat p p*
⟨*proof*⟩

lemma *partial-trace-B-tensor*:
fixes R T :: *complex Matrix.mat*
assumes *Rsq*: R ∈ *carrier-mat m m*
and *Tsq*: T ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *partial-trace-B m p* (R ⊗ T) = *Tr T* ·$_m$ R
⟨*proof*⟩

lemma *partial-trace-A-tensor*:
fixes R T :: *complex Matrix.mat*
assumes *Rsq*: R ∈ *carrier-mat m m*
and *Tsq*: T ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *partial-trace-A m p* (R ⊗ T) = *Tr R* ·$_m$ T
⟨*proof*⟩

32.3 Partial traces and measurements

Writing $\rho = \sum_l R_l \otimes T_l$,

$$p_\alpha(\rho) = \mathrm{tr}((M_\alpha^* M_\alpha \otimes I)\rho) = \sum_l \mathrm{tr}(M_\alpha^* M_\alpha R_l)\,\mathrm{tr}(T_l) = \mathrm{tr}\big(M_\alpha^* M_\alpha\ \mathrm{tr}_B(\rho)\big) = p_\alpha(\rho^A)\,:$$

lab A only sees ρ^A, and symmetrically lab B only sees ρ^B. Check: $\mathrm{tr}(\rho^A) = \mathrm{tr}\,\rho = 1$, and $\rho \geq 0$ implies $\rho^A \geq 0$.

end
theory *Day15-Partial-Trace-A*
imports *Day16-Density-Postulates*
begin

33 Day 15: the partial trace over A (D15-6)

Reordering the basis: for a bijection σ of $\{0, \ldots, N-1\}$ the matrix $(x_{\sigma(r)\sigma(s)})$ has the trace of X and is Hermitian, resp. positive, with X.

definition *perm-conj* :: *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
perm-conj N σ X = *Matrix.mat N N* (λ(*r*, *s*). *X* $$ (*σ r*, *σ s*))

lemma *perm-conj-carrier* [*simp*]: *perm-conj N σ X* ∈ *carrier-mat N N*
⟨*proof*⟩

lemma *perm-conj-dims* [*simp*]: *dim-row* (*perm-conj N σ X*) = *N dim-col* (*perm-conj N σ X*) = *N*
⟨*proof*⟩

lemma *perm-conj-trace*:
assumes *σ*: *bij-betw σ* {..<*N*} {..<*N*} **and** *X*: *X* ∈ *carrier-mat N N*
shows *Tr* (*perm-conj N σ X*) = *Tr X*
⟨*proof*⟩

lemma *perm-conj-hermitian*:
assumes *σ*: ⋀*r*. *r* < *N* ⟹ *σ r* < *N* **and** *X*: *X* ∈ *carrier-mat N N* **and** *h*: *hermitian X*
shows *hermitian* (*perm-conj N σ X*)
⟨*proof*⟩

lemma *perm-conj-psd*:
assumes *σ*: *bij-betw σ* {..<*N*} {..<*N*} **and** *X*: *X* ∈ *carrier-mat N N* **and** *ps*: *psd X*
shows *psd* (*perm-conj N σ X*)
⟨*proof*⟩

Exchanging the factors: with $\sigma(im + k) = kp + i$ $(i < p,\ k < m)$, tracing out A is tracing out B after the exchange.

definition *swap-index* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* **where**
swap-index m p r = (*r mod m*) ∗ *p* + *r div m*

lemma *swap-index-less*:
assumes *r* < *m* ∗ *p* **shows** *swap-index m p r* < *m* ∗ *p*
⟨*proof*⟩

lemma *swap-index-bij*: *bij-betw* (*swap-index m p*) {..<*m* ∗ *p*} {..<*m* ∗ *p*}
⟨*proof*⟩

lemma *partial-trace-A-as-B*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)

shows *partial-trace-A m p X = partial-trace-B p m (perm-conj (m * p) (swap-index m p) X)*
⟨*proof*⟩

theorem *partial-trace-A-trace*:
assumes *X*: *X* ∈ *carrier-mat* (*m* * *p*) (*m* * *p*) **and** *m*: *0* < *m*
shows *Tr* (*partial-trace-A m p X*) = *Tr X*
⟨*proof*⟩

theorem *partial-trace-A-hermitian*:
assumes *X*: *X* ∈ *carrier-mat* (*m* * *p*) (*m* * *p*) **and** *h*: *hermitian X*
shows *hermitian* (*partial-trace-A m p X*)
⟨*proof*⟩

theorem *partial-trace-A-is-density*:
assumes *dm*: *density-matrix* (*m* * *p*) *X* **and** *m*: *0* < *m*
shows *density-matrix p* (*partial-trace-A m p X*)
⟨*proof*⟩

end

theory *Day16-Density-Postulates*
imports *Day15-Partial-Traces*
begin

34 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices

With matrix units E_{ij} of $L(\mathcal{H}_B)$, every $X \in L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is uniquely $X = \sum_{i,j} X_{ij} \otimes E_{ij}$ with $X_{ij} \in L(\mathcal{H}_A)$, and $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{tr}\, X_{ij})_{ij}$; symmetrically for $X = \sum E_{kl} \otimes Y_{kl}$. Choosing a basis $f_1, \ldots, f_p$ of $\mathcal{H}_B$ identifies $\mathcal{H}_A \otimes \mathcal{H}_B \cong \mathcal{H}_A^{(p)} = \mathcal{H}_A \oplus \cdots \oplus \mathcal{H}_A$ and $L(\mathcal{H}_A \otimes \mathcal{H}_B) \cong M_p(L(\mathcal{H}_A))$, block matrices (X_{ij}) acting by $(X_{ij})(h_j)_j = (\sum_j X_{ij} h_j)_i$; in block notation $\mathrm{tr}_B((X_{ij})) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A((X_{ij})) = (\mathrm{tr}\, X_{ij})$.

Reformulated postulates. $1'$: the state of a system is a density matrix $\rho \in L(\mathcal{H}_A)$. $2'$: closed evolution is $\rho \mapsto U\rho U^*$. $3'$: a measurement system $\{M_\alpha\}$ with $\sum M_\alpha^* M_\alpha = I$ yields outcome α with probability $\mathrm{tr}(M_\alpha^* M_\alpha \rho)$. $4'$: systems with densities ρ_1, ρ_2 compose to $\rho_1 \otimes \rho_2$ on $\mathcal{H}_A \otimes \mathcal{H}_B$.

lemma *partial-trace-B-trace*:
fixes *X* :: *complex Matrix.mat*
assumes *Xsq*: *X* ∈ *carrier-mat* (*m* * *p*) (*m* * *p*)
and *p-pos*: *0* < *p*
shows *Tr* (*partial-trace-B m p X*) = *Tr X*
⟨*proof*⟩

lemma *partial-trace-B-hermitian*:
fixes *X* :: *complex Matrix.mat*
assumes *Xsq*: *X* ∈ *carrier-mat* (*m* * *p*) (*m* * *p*)
and *herm*: *hermitian X*
shows *hermitian* (*partial-trace-B m p X*)
⟨*proof*⟩

lemma *partial-trace-B-is-density*:
fixes *X* :: *complex Matrix.mat*
assumes *dm*: *density-matrix* (*m* * *p*) *X*
and *p-pos*: *0* < *p*

shows *density-matrix m* (*partial-trace-B m p X*)
⟨*proof*⟩

Key identity: measurements in lab A only see $\rho^A = \mathrm{tr}_B\,\rho$.

theorem *partial-trace-measurement*:
fixes *meas* :: *complex Matrix.mat* **and** ϱ :: *complex Matrix.mat*
assumes *M-sq*: *meas* ∈ *carrier-mat m m*
and *dm*: *density-matrix* (*m* ∗ *p*) ϱ
and *p-pos*: *0* < *p*
shows *Tr* (((*meas*† ∗ *meas*) ⨂ *1*$_m$ *p*) ∗ ϱ) =
Tr (*meas*† ∗ *meas* ∗ *partial-trace-B m p* ϱ)
⟨*proof*⟩

end
theory *Day16-Block-Form*
imports *Day25-Arveson-Correspondence Day06-Tensor-Coordinates*
begin

35 Day 16: $H_A \otimes H_B \cong H_A^{(p)}$ and $L(H_A \otimes H_B) \cong M_p(L(H_A))$ (D16-2)

The notes identify $H_A \otimes H_B$ with p copies of H_A through $\sum_i h_i \otimes f_i \leftrightarrow (h_1, \ldots, h_p)$, with $\|v\|^2 = \sum_i \|h_i\|^2$, and an operator X with its block matrix (X_{ij}) over $L(H_A)$, acting by $(h_j) \mapsto (\sum_j X_{ij} h_j)$, so that $X = \sum_{i,j} X_{ij} \otimes E_{ij}$, $\mathrm{tr}_B(X) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A(X) = (\mathrm{Tr}\, X_{ij})$. Here $f_i = e_i$, and indices are A-major as everywhere in the development.

With $H_A = \mathbb{C}^m$, $H_B = \mathbb{C}^p$ and A-major indices $ap + i$, the part of v along e_i is $h_i = (v_{ap+i})_a \in H_A$, the i-th column of *vec-to-mat m p v*: for $m = 2$, $p = 3$, $h_0 = (v_0, v_3)$, $h_1 = (v_1, v_4)$ and $h_2 = (v_2, v_5)$.

Likewise an operator X on $\mathbb{C}^{mp}$ has the blocks $X_{ij} = (X_{ap+i,\,bp+j})_{a,b} \in L(H_A)$: the rows of X with second index i and its columns with second index j. For $m = p = 2$ the indices $0, 2$ have second index 0 and $1, 3$ have second index 1, so

$$X = \begin{pmatrix} x_{00} & x_{01} & x_{02} & x_{03} \\ x_{10} & x_{11} & x_{12} & x_{13} \\ x_{20} & x_{21} & x_{22} & x_{23} \\ x_{30} & x_{31} & x_{32} & x_{33} \end{pmatrix}, \qquad X_{00} = \begin{pmatrix} x_{00} & x_{02} \\ x_{20} & x_{22} \end{pmatrix}, \quad X_{01} = \begin{pmatrix} x_{01} & x_{03} \\ x_{21} & x_{23} \end{pmatrix}.$$

The blocks interleave because the second index runs fastest. X_{ij} carries the part of v along e_j to the part of Xv along e_i.

definition *block* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
block m p X i j = *Matrix.mat m m* (λ(*a*, *b*). *X* $$ (*a* ∗ *p* + *i*, *b* ∗ *p* + *j*))

lemma *block-carrier* [*simp*]: *block m p X i j* ∈ *carrier-mat m m*
⟨*proof*⟩

lemma *block-dims* [*simp*]: *dim-row* (*block m p X i j*) = *m dim-col* (*block m p X i j*) = *m*
⟨*proof*⟩

lemma *sum-mp-reindex*:
fixes *f* :: *nat* ⇒ *complex*
shows (∑ *r*<*m* ∗ *p*. *f r*) = (∑ *b*<*m*. ∑ *j*<*p*. *f* (*b* ∗ *p* + *j*))
⟨*proof*⟩

$H_A \otimes H_B \cong H_A^{(p)}$ isometrically: $\langle v|w\rangle = \sum_i \langle h_i|k_i\rangle$.

theorem *vec-to-mat-cols-inner-prod*:
assumes *v*: *dim-vec v* = *m* ∗ *p* **and** *w*: *dim-vec w* = *m* ∗ *p*
shows *Quantum.inner-prod v w* = (∑ *i*<*p*. *Quantum.inner-prod* (*Matrix.col* (*vec-to-mat m p v*) *i*) (*Matrix.col* (*vec-to-mat m p w*) *i*))
⟨*proof*⟩

$v = \sum_i h_i \otimes e_i$.

theorem *vec-to-mat-cols-expansion*:
assumes *v*: *dim-vec v* = *m* ∗ *p*
shows *v* = *tensor-combination* (*m* ∗ *p*) (*cols* (*vec-to-mat m p v*)) (*map* (*unit-vec p*) [*0*..<*p*])
⟨*proof*⟩

The block action: the i-th coordinate of Xv is $\sum_j X_{ij} h_j$.

theorem *block-action*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *v*: *dim-vec v* = *m* ∗ *p* **and** *i*: *i* < *p*
shows *Matrix.col* (*vec-to-mat m p* (*X* ∗*v v*)) *i* = *Matrix.vec m* (λ*a*. ∑ *j*<*p*. (*block m p X i j* ∗*v Matrix.col* (*vec-to-mat m p v*) *j*) $ *a*)
⟨*proof*⟩

$L(H_A \otimes H_B) \cong M_p(L(H_A))$: $X = \sum_{i,j} X_{ij} \otimes E_{ij}$, and the blocks are unique.

lemma *double-delta*:
fixes *c* :: *complex* **and** *a b p q* :: *nat*
assumes *a* < *p b* < *q*
shows (∑ *i*<*p*. ∑ *j*<*q*. *if i* = *a* ∧ *j* = *b then c else 0*) = *c*
⟨*proof*⟩

lemma *mat-unit-tensor-index*:
assumes *A*: *A* ∈ *carrier-mat m m* **and** *r*: *r* < *m* ∗ *p* **and** *s*: *s* < *m* ∗ *p* **and** *i*: *i* < *p* **and** *j*: *j* < *p*
shows (*A* ⨂ *E*⟦*i*,*j* ; *p*,*p*⟧) $$ (*r*, *s*) = (*if r mod p* = *i* ∧ *s mod p* = *j then A* $$ (*r div p*, *s div p*) *else 0*)
⟨*proof*⟩

theorem *tensor-block-decomposition*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
shows *X* = *msum* (*m* ∗ *p*) (*m* ∗ *p*) (λ(*i*, *j*). *block m p X i j* ⨂ *E*⟦*i*,*j* ; *p*,*p*⟧) (*List.product* [*0*..<*p*] [*0*..<*p*])
⟨*proof*⟩

theorem *tensor-block-decomposition-unique*:
assumes *Y*: ⋀*i j*. *Y i j* ∈ *carrier-mat m m*
and *X*: *X* = *msum* (*m* ∗ *p*) (*m* ∗ *p*) (λ(*i*, *j*). *Y i j* ⨂ *E*⟦*i*,*j* ; *p*,*p*⟧) (*List.product* [*0*..<*p*] [*0*..<*p*])
and *i*: *i* < *p* **and** *j*: *j* < *p*
shows *Y i j* = *block m p X i j*
⟨*proof*⟩

The partial traces in block form (the definitions, restated): $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{Tr}\, X_{ij})_{ij}$.

lemma *partial-trace-B-blocks*:
partial-trace-B m p X = *msum m m* (λ*i*. *block m p X i i*) [*0*..<*p*]
⟨*proof*⟩

lemma *partial-trace-A-blocks*:
partial-trace-A m p X = *Matrix.mat p p* (λ(*i*, *j*). *Tr* (*block m p X i j*))
⟨*proof*⟩

end

theory *Day17-Quantum-Noise*
imports *Day16-Density-Postulates*
begin

36 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps

36.1 Measurement maps

Measuring ψ with $\{M_\alpha\}$ and keeping the outcome as an ensemble gives the density matrix $\sum_\alpha p_\alpha(\psi)|M_\alpha\psi/\|M_\alpha\psi\|\rangle\langle M_\alpha\psi/\|M_\alpha\psi\|| = \sum_\alpha M_\alpha(\psi\psi^*)M_\alpha^*$; for an ensemble with density ρ_1 one obtains $M(\rho_1) = \sum_\alpha M_\alpha\rho_1 M_\alpha^*$: a linear map on $L(\mathcal{H}_A)$ sending density matrices to density matrices.

Classical noise flips a bit with some probability; probability vectors evolve by stochastic matrices (non-negative entries, columns summing to one; doubly stochastic if rows do too). Quantum noise: with an environment $\mathcal{H}_E$ in state $|\phi\rangle$, the system $\rho\otimes|\phi\rangle\langle\phi|$ evolves by a unitary $U = (U_{ij})$ on $\mathcal{H}_A \otimes \mathcal{H}_E$ and lab A sees

$$\mathrm{tr}_E\big(U(\rho\otimes|\phi\rangle\langle\phi|)U^*\big) = \sum_i W_i\rho W_i^*, \qquad W_i = \sum_k U_{ik}\alpha_k, \qquad \sum_i W_i^*W_i = I.$$

Measurement by an intruder and environmental noise are maps of the same form $\rho \mapsto \sum_\alpha X_\alpha\rho X_\alpha^*$ with $\sum X_\alpha^*X_\alpha = I$. The environment computation is *environment-model* in *Day17-Environment-Model*. Third way, axiomatic: a quantum event is a linear $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$.

Proposition (notes). Φ maps density matrices to density matrices iff Φ is positive ($\rho \geq 0 \Rightarrow \Phi(\rho) \geq 0$) and trace preserving ($\mathrm{tr}\,\Phi(X) = \mathrm{tr}\,X$); then $\Phi \otimes I_{L(\mathcal{H}_B)}$ also maps density matrices to density matrices for every $\mathcal{H}_B$.

Definition (notes). Φ is *completely positive* if $\Phi\otimes I_{L(\mathcal{H}_B)}$ is positive for every finite-dimensional $\mathcal{H}_B$.

Theorem (notes) (Choi–Kraus)**.** $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$ is completely positive and trace preserving iff $\Phi(X) = \sum_i E_i X E_i^*$ for matrices E_i with $\sum_i E_i^*E_i = I$.

The proposition is *density-preserving-iff-positive-tp* in *Day18-Positive-Maps*; complete positivity is defined below and the Choi–Kraus theorem is *choi-kraus-cptp* on Day 19.

definition *meas-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
meas-map Ms ϱ ≡
foldr (λ*M acc*. *M* ∗ *ϱ* ∗ (*M*†) + *acc*) *Ms* (0*m* (*dim-row ϱ*) (*dim-col ϱ*))

lemma *meas-map-Nil* [*simp*]:
meas-map [] *ϱ* = 0*m* (*dim-row ϱ*) (*dim-col ϱ*)
⟨*proof*⟩

lemma *meas-map-Cons* [*simp*]:
meas-map (*M* # *Ms*) *ϱ* = *M* ∗ *ϱ* ∗ (*M*†) + *meas-map Ms ϱ*
⟨*proof*⟩

lemma *meas-map-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *meas-map Ms ϱ* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-psd*:
assumes *Ms-c*: $\forall\, M \in$ *set Ms*. $M \in$ *carrier-mat n n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
and *ϱ-psd*: *psd* ϱ
shows *psd* (*meas-map Ms* ϱ)
⟨*proof*⟩

lemma *foldr-MdagM-carrier*:
assumes *Ms-c*: $\forall\, M \in$ *set Ms*. $M \in$ *carrier-mat n n*
shows *foldr* $(\lambda M\ acc.\ (M^\dagger) * M + acc)$ *Ms* $(0_m\ n\ n) \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-trace-aux*:
assumes *Ms-c*: $\forall\, M \in$ *set Ms*. $M \in$ *carrier-mat n n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *Tr* (*meas-map Ms* ϱ) =
Tr (*foldr* $(\lambda M\ acc.\ (M^\dagger) * M + acc)$ *Ms* $(0_m\ n\ n) * \varrho$)
⟨*proof*⟩

lemma *meas-map-density*:
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*meas-map Ms* ϱ)
⟨*proof*⟩

36.2 Kraus maps, positive maps, complete positivity

A *Kraus map* $\rho \mapsto \sum_i W_i \rho W_i^\dagger$ is *kraus-map-dim d Ws*; the list *Ws* is a list of *Kraus operators* (*kraus-operators n d Ws*) when $\sum_i W_i^\dagger W_i = I_n$. Positive, trace-preserving and unital maps are defined on the carrier M_n. The amplification $\Phi^{(p)} = \Phi \otimes \mathrm{id}_p$ acts on block matrices by $(X_{ab}) \mapsto (\Phi(X_{ab}))$ (*amplification n d p* Φ), and Φ is p-positive when $\Phi^{(p)}$ is positive, completely positive when it is p-positive for all p. The main result of the subsection: Kraus maps are completely positive.

definition *kraus-map-dim* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map-dim d Ws $\varrho \equiv$
foldr $(\lambda W\ acc.\ W * \varrho * (W^\dagger) + acc)$ *Ws* $(0_m\ d\ d)$

definition *kraus-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map Ws $\varrho \equiv$ *kraus-map-dim* (*dim-row* ϱ) *Ws* ϱ

definition *kraus-operators* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
kraus-operators n d Ws ≡
$(\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*$) \wedge$
foldr $(\lambda W\ acc.\ (W^\dagger) * W + acc)$ *Ws* $(0_m\ n\ n) = 1_m\ n$

lemma *kraus-map-dim-Nil* [*simp*]:
kraus-map-dim d [] $\varrho = 0_m\ d\ d$
⟨*proof*⟩

lemma *kraus-map-dim-Cons* [*simp*]:
kraus-map-dim d (*W # Ws*) ϱ =
$W * \varrho * (W^\dagger)$ + *kraus-map-dim d Ws* ϱ
⟨*proof*⟩

lemma *kraus-map-dim-carrier*:
assumes *Ws-c*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*

and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *kraus-map-dim d Ws ϱ* $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-map-dim-psd*:
assumes *Ws-c*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*kraus-map-dim d Ws ϱ*)
⟨*proof*⟩

definition *positive-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
positive-map n d Φ ≡
$(\forall\, A.\ A \in$ *carrier-mat n n* $\longrightarrow\ \Phi\ A \in$ *carrier-mat d d*$) \wedge$
$(\forall\, P.\ P \in$ *carrier-mat n n* $\longrightarrow$ *psd P* $\longrightarrow$ *psd* (Φ *P*))

lemma *kraus-map-dim-positive-map*:
assumes *Ws*: *kraus-operators n d Ws*
shows *positive-map n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

definition *trace-preserving* :: *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
trace-preserving n Φ ≡
$\forall\, X.\ X \in$ *carrier-mat n n* $\longrightarrow$ *Tr* (Φ *X*) = *Tr X*

definition *unital-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
unital-map n d Φ ≡ Φ (1_m *n*) = 1_m *d*

definition *block-of-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
block-of-mat n x a b ≡
Matrix.mat n n (λ(*r*,*c*). *x* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*))

lemma *block-of-mat-carrier* [*simp*]:
block-of-mat n X a b $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *block-of-mat-index*:
assumes $r < n$ **and** $c < n$
shows *block-of-mat n X a b* $$ (*r*,*c*) = *X* $$ (*a*∗*n* + *r*, *b*∗*n* + *c*)
⟨*proof*⟩

lemma *block-of-mat-single*:
assumes *X*: *X* $\in$ *carrier-mat n n*
shows *block-of-mat n X 0 0* = *X*
⟨*proof*⟩

definition *amplification* :: *nat* ⇒ *nat* ⇒ *nat* ⇒
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
amplification n d p Φ *x* ≡
Matrix.mat (*p*∗*d*) (*p*∗*d*)
(λ(*i*,*j*). Φ (*block-of-mat n x* (*i div d*) (*j div d*)) $$
(*i mod d*, *j mod d*))

lemma *amplification-carrier* [*simp*]:
amplification n d p Φ *X* $\in$ *carrier-mat* (*p*∗*d*) (*p*∗*d*)
⟨*proof*⟩

lemma *amplification-index*:

assumes $i < p*d$ **and** $j < p*d$
shows *amplification n d p* Φ *X* $$ (*i*,*j*) =
Φ (*block-of-mat n X* (*i div d*) (*j div d*)) $$
(*i mod d*, *j mod d*)
⟨*proof*⟩

lemma *amplification-one*:
assumes *X*: $X \in$ *carrier-mat n n*
and *range*: Φ $X \in$ *carrier-mat d d*
shows *amplification n d 1* Φ X = Φ X
⟨*proof*⟩

lemma *tensor-id-kraus-carrier*:
assumes *W*: $W \in$ *carrier-mat d n*
shows (*1*$_m$ *p* ⨂ W) $\in$ *carrier-mat* (*p*∗*d*) (*p*∗*n*)
⟨*proof*⟩

lemma *tensor-id-kraus-list-carrier*:
assumes *Ws*: ∀ *W* ∈ *set Ws*. $W \in$ *carrier-mat d n*
shows ∀ *L* ∈ *set* (*map* (λ*W*. *1*$_m$ *p* ⨂ W) *Ws*).
$L \in$ *carrier-mat* (*p*∗*d*) (*p*∗*n*)
⟨*proof*⟩

lemma *tensor-id-kraus-index*:
assumes *W*: $W \in$ *carrier-mat d n*
and *r*: $r < p*d$
and *c*: $c < p*n$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows (*1*$_m$ *p* ⨂ W) $$ (*r*,*c*) =
(*if r div d* = *c div n then W* $$ (*r mod d*, *c mod n*) *else 0*)
⟨*proof*⟩

lemma *sum-block-delta*:
fixes *f* :: *nat* ⇒ *complex*
assumes *a*: $a < p$
and *npos*: $0 < n$
shows (∑ *t* < *p*∗*n*. *if a* = *t div n then f* (*t mod n*) *else 0*) =
(∑ *r* < *n*. *f r*)
⟨*proof*⟩

lemma *sum-block-delta-index*:
fixes *h* :: *nat* ⇒ *complex*
assumes *a*: $a < p$
and *npos*: $0 < n$
shows (∑ *t* < *p*∗*n*. *if a* = *t div n then h t else 0*) =
(∑ *r* < *n*. *h* (*a*∗*n* + *r*))
⟨*proof*⟩

lemma *tensor-id-kraus-left-mult-index*:
assumes *W*: $W \in$ *carrier-mat d n*
and *x*: $x \in$ *carrier-mat* (*p*∗*n*) *q*
and *i*: $i < p*d$
and *s*: $s < q$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows ((*1*$_m$ *p* ⨂ W) ∗ x) $$ (*i*,*s*) =
(∑ *t* < *n*. *W* $$ (*i mod d*, *t*) ∗ *x* $$ ((*i div d*) ∗ *n* + *t*, *s*))

⟨*proof*⟩

lemma *tensor-id-kraus-right-mult-index*:
assumes *W*: $W \in$ *carrier-mat d n*
and *y*: $y \in$ *carrier-mat q* $(p*n)$
and *i*: $i < q$
and *j*: $j < p*d$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $(y * (1_m\ p \bigotimes (W^\dagger)))$ \$\$ $(i,j) =$
$(\sum s < n.\ y$ \$\$ $(i, (j\ div\ d) * n + s) * (W^\dagger)$ \$\$ $(s, j\ mod\ d))$
⟨*proof*⟩

lemma *kraus-term-index*:
assumes *W*: $W \in$ *carrier-mat d n*
and *B*: $B \in$ *carrier-mat n n*
and *i*: $i < d$
and *j*: $j < d$
shows $(W * B * (W^\dagger))$ \$\$ $(i,j) =$
$(\sum s < n. (\sum t < n.\ W$ \$\$ $(i,t) * B$ \$\$ $(t,s)) * (W^\dagger)$ \$\$ $(s,j))$
⟨*proof*⟩

lemma *kraus-amplification-single*:
assumes *W*: $W \in$ *carrier-mat d n*
and *x*: $x \in$ *carrier-mat* $(p*n)$ $(p*n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows *amplification n d p* $(\lambda\varrho.\ W * \varrho * (W^\dagger))\ x =$
$((1_m\ p \bigotimes W) * x) * (1_m\ p \bigotimes (W^\dagger))$
⟨*proof*⟩

lemma *amplification-zero-map*:
assumes *dpos*: $0 < d$
shows *amplification n d p* $(\lambda\varrho.\ 0_m\ d\ d)\ x = 0_m\ (p*d)\ (p*d)$
⟨*proof*⟩

lemma *amplification-add*:
assumes *dpos*: $0 < d$
and *F*: $\bigwedge a\ b.\ F$ *(block-of-mat n x a b)* $\in$ *carrier-mat d d*
and *G*: $\bigwedge a\ b.\ G$ *(block-of-mat n x a b)* $\in$ *carrier-mat d d*
shows *amplification n d p* $(\lambda\varrho.\ F\ \varrho + G\ \varrho)\ x =$
amplification n d p F x + amplification n d p G x
⟨*proof*⟩

lemma *amplification-kraus-map-dim*:
assumes *Ws*: $\forall\ W \in$ *set Ws. W* $\in$ *carrier-mat d n*
and *x*: $x \in$ *carrier-mat* $(p*n)$ $(p*n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows *amplification n d p* (*kraus-map-dim d Ws*) $x =$
kraus-map-dim $(p*d)$ (*map* $(\lambda W.\ 1_m\ p \bigotimes W)$ *Ws*) x
⟨*proof*⟩

definition *p-positive* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
p-positive n d p $\Phi \equiv$
$\forall X$ :: *complex Matrix.mat*. $X \in$ *carrier-mat* $(p*n)$ $(p*n) \longrightarrow$ *psd* $X \longrightarrow$
psd (*amplification n d p* Φ X)

definition *completely-positive* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
completely-positive n d Φ ≡ ∀ *p* > *0*. *p-positive n d p* Φ

theorem *kraus-is-cp*:
assumes *Ws*: *kraus-operators n d Ws*
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

end
theory *Day17-Environment-Model*
imports *Day22-Minimal-Kraus*
begin

37 Day 17: the environment model of noise

Day 17, the environment model of noise: a state ρ of lab A meets an environment in the state $|\varphi\rangle$, the pair evolves by a unitary U on $\mathbb{C}^m \otimes \mathbb{C}^p$, and lab A sees $\mathrm{tr}_E(U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*) = \sum_i W_i \rho W_i^*$ with $\sum_i W_i^* W_i = I$. The index of $\mathbb{C}^m \otimes \mathbb{C}^p$ is $a\,p + i$ (lab index a, environment index i), and W_i is the matrix with entries $\sum_k U_{(a,i),(c,k)} \varphi_k$, i.e. the notes' $W_i = \sum_k U_{ik} \alpha_k$ for the blocks U_{ik} of U.

definition *env-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
env-kraus m p U φ *i* = *Matrix.mat m m* (λ(*a*,*c*). ∑ *k*<*p*. *U* $$ (*a* ∗ *p* + *i*, *c* ∗ *p* + *k*) ∗ φ $ *k*)

lemma *env-kraus-carrier* [*simp*]: *env-kraus m p U* φ *i* ∈ *carrier-mat m m*
⟨*proof*⟩

lemma *env-kraus-dims* [*simp*]: *dim-row* (*env-kraus m p U* φ *i*) = *m dim-col* (*env-kraus m p U* φ *i*) = *m*
⟨*proof*⟩

lemma *env-kraus-index*: *a* < *m* ⟹ *c* < *m* ⟹ *env-kraus m p U* φ *i* $$ (*a*,*c*) = (∑ *k*<*p*. *U* $$ (*a* ∗ *p* + *i*, *c* ∗ *p* + *k*) ∗ φ $ *k*)
⟨*proof*⟩

lemma *sum-split-mp*:
fixes *f* :: *nat* ⇒ *'a*::*comm-monoid-add*
shows (∑ *x* < *m* ∗ *p*. *f x*) = (∑ *c*<*m*. ∑ *k*<*p*. *f* (*c* ∗ *p* + *k*))
⟨*proof*⟩

lemma *tensor-outer-index*:
fixes ϱ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes ϱ: ϱ ∈ *carrier-mat m m* **and** φ: *dim-vec* φ = *p*
and *c*: *c* < *m* **and** *d*: *d* < *m* **and** *k*: *k* < *p* **and** *l*: *l* < *p*
shows *Tensor.tensor-mat* ϱ (φ ⊘ φ) $$ (*c* ∗ *p* + *k*, *d* ∗ *p* + *l*) = ϱ $$ (*c*,*d*) ∗ (φ $ *k* ∗ *cnj* (φ $ *l*))
⟨*proof*⟩

The entries of $U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*$.

lemma *env-entry*:
fixes *U* ϱ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *U*: *U* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** ϱ: ϱ ∈ *carrier-mat m m* **and** φ: *dim-vec* φ = *p*
and *x*: *x* < *m* ∗ *p* **and** *y*: *y* < *m* ∗ *p*

shows $(U * \mathit{Tensor.tensor\text{-}mat}\ \varrho\ (\varphi \oslash \varphi) * (U^\dagger))$ \$\$ (x,y)
$= (\sum d{<}m.\ \sum c{<}m.\ (\sum k{<}p.\ U$ \$\$ $(x,\ c * p + k) * \varphi$ \$ $k) * \varrho$ \$\$ (c,d)
$* \mathit{cnj}\ (\sum l{<}p.\ U$ \$\$ $(y,\ d * p + l) * \varphi$ \$ $l))$
⟨*proof*⟩

theorem *environment-model*:
fixes $U\ \varrho$ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *U*: $U \in \mathit{carrier\text{-}mat}\ (m * p)\ (m * p)$ **and** *Uu*: *unitary U*
and φ: *dim-vec* $\varphi = p$ **and** $\varphi 1$: $\langle\varphi|\varphi\rangle = 1$ **and** ϱ: $\varrho \in \mathit{carrier\text{-}mat}\ m\ m$
defines $Ws \equiv \mathit{map}\ (\mathit{env\text{-}kraus}\ m\ p\ U\ \varphi)\ [0..{<}p]$
shows $\mathit{partial\text{-}trace\text{-}B}\ m\ p\ (U * \mathit{Tensor.tensor\text{-}mat}\ \varrho\ (\varphi \oslash \varphi) * (U^\dagger)) = \mathit{kraus\text{-}map\text{-}dim}\ m\ Ws\ \varrho$
and $\mathit{kraus\text{-}operators}\ m\ m\ Ws$
⟨*proof*⟩

end

theory *Day18-CP-Maps*
imports *Day17-Quantum-Noise*
begin

38 Day 18 (7 October 2011): the theory of completely positive maps

Proposition (notes). Every $X \in M_n$ is $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_i \geq 0$: $\mathrm{span}(M_n^+) = M_n$.

(Write $X = H + iK$ with H, K Hermitian and split each spectral decomposition into its non-negative and negative eigenvalues.)

Definition (notes). A linear $\Phi : M_n \to M_d$ is *positive* if $P \geq 0$ implies $\Phi(P) \geq 0$; it is *p-positive* if $\Phi^{(p)} : M_p(M_n) \to M_p(M_d)$, $(X_{ij}) \mapsto (\Phi(X_{ij}))$, is positive, and *completely positive* if it is p-positive for every p.

Proposition (notes). A linear $\Phi : M_n \to M_d$ maps density matrices to density matrices iff it is positive and trace preserving.

Example (notes): the transpose $\Phi(A) = A^t$ on M_2 is positive but not 2-positive, since $\Phi^{(2)}$ sends the positive matrix $(E_{ij})_{i,j}$ to a matrix of determinant -1. The decomposition into positives is formalised below; the proposition is *density-preserving-iff-positive-tp* and the example *transpose-not-2-positive*, both in *Day18-Positive-Maps*.

lemma *diag-mat-of-real-hermitian*:
$((\mathit{diag\text{-}mat\text{-}of\text{-}real}\ D)^\dagger) = \mathit{diag\text{-}mat\text{-}of\text{-}real}\ D$
⟨*proof*⟩

lemma *hermitian-decomp-psd*:
fixes H :: *complex Matrix.mat*
assumes *sq*: $H \in \mathit{carrier\text{-}mat}\ n\ n$
and *herm*: *hermitian H*
shows $\exists\, P1\ P2$ :: *complex Matrix.mat*.
$P1 \in \mathit{carrier\text{-}mat}\ n\ n \wedge P2 \in \mathit{carrier\text{-}mat}\ n\ n \wedge$
$\mathit{psd}\ P1 \wedge \mathit{psd}\ P2 \wedge$
$H = P1 - P2$
⟨*proof*⟩

lemma *matrix-decomp-psd*:
fixes X :: *complex Matrix.mat*
assumes *sq*: $X \in$ *carrier-mat n n*
shows $\exists$ *P1 P2 P3 P4* :: *complex Matrix.mat*.
$(\forall\, i \in \{0..3\}.\ [P1,P2,P3,P4]\ !\ i \in$ *carrier-mat n n* $\wedge$ *psd* $([P1,P2,P3,P4]\ !\ i)) \wedge$
$X = (P1 - P2) + \mathrm{i} \cdot_m (P3 - P4)$
⟨*proof*⟩

end
theory *Day18-Positive-Maps*
imports *QC-Prelims*
begin

39 Days 17–18: densities to densities iff positive and trace-preserving (D17-4, D18-2)

theorem *density-preserving-iff-positive-tp*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *npos*: $0 < n$
shows $(\forall \varrho.$ *density-matrix n* $\varrho \longrightarrow$ *density-matrix d* $(\Phi\ \varrho)) \longleftrightarrow$
(*positive-map n d* $\Phi \wedge$ *trace-preserving n* Φ)
⟨*proof*⟩

40 Day 18: the transpose is positive but not 2-positive (D18-4)

lemma *transpose-positive-map*: *positive-map n n* (*transpose-mat* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*)
⟨*proof*⟩

The notes' witness: $X = (E_{ij}) \in M_2(M_2)$ is positive (it is $|w\rangle\langle w|$ for $w = e_0 \otimes e_0 + e_1 \otimes e_1$) while $\Phi^{(2)}(X)$ has the eigenvector $(0, 1, -1, 0)$ with eigenvalue -1 (the notes compute $\det \Phi^{(2)}(X) = -1$ instead).

lemma *choi-seed-2-outer*: *choi-seed 2* $=$ *outer-sum 4* [*Matrix.vec 4* ($\lambda i.$ *if* $i = 0 \vee i = 3$ *then 1 else 0*)]
⟨*proof*⟩

lemma *choi-seed-2-psd*: *psd* (*choi-seed 2*)
⟨*proof*⟩

lemma *transpose-not-2-positive*:
$\neg$ *p-positive 2 2 2* (*transpose-mat* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*)
⟨*proof*⟩

corollary *transpose-not-completely-positive*:
$\neg$ *completely-positive 2 2* (*transpose-mat* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*)
⟨*proof*⟩

end

theory *Day19-Choi-Theorem*
imports *Day18-CP-Maps*

begin

41 Day 19 (10 October 2011): Choi's theorem

Theorem (notes) (Choi, 1975)**.** For linear $\Phi : M_n \to M_d$ the following are equivalent: (1) Φ is completely positive; (2) Φ is n-positive; (3) $P_\Phi = (\Phi(E_{ij}))_{i,j} \geq 0$ in $M_n(M_d)$; (4) there are $d \times n$ matrices $B_1, \ldots, B_K$ with $\Phi(X) = \sum_k B_k X B_k^*$.

Notes: (1) $\sum_k B_k X B_k^*$ is a *Choi–Kraus representation*; (2) P_Φ is the *Choi–Jamiołkowski matrix*; (3) $CP(M_n, M_d) \leftrightarrow M_n(M_d)^+$ via $\Phi \leftrightarrow P_\Phi$; (4) Φ is trace preserving iff $\mathrm{tr}(\Phi(E_{ij})) = \delta_{ij}$; (5) Φ is unital iff $\sum_i \Phi(E_{ii}) = I_d$.
Formally P_Φ is *choi-matrix n d* Φ (an $nd \times nd$ matrix built from the blocks $\Phi(E_{ij})$); *choi-theorem* is (1)⇔(3), *kraus-rep-of-psd-choi* is (3)⇒(4), and *kraus-is-cp* is (4)⇒(1). The step (2)⇒(3) is the observation that $P_\Phi = \Phi^{(n)}((E_{ij}))$ with $(E_{ij}) \geq 0$ (*choi-matrix-as-amplification*). Linearity of Φ is assumed on the carrier M_n (additivity and homogeneity hypotheses). The trace-preserving case is *choi-kraus-cptp*; notes (4) and (5) in Choi-matrix form are *choi-trace-preserving-iff* and *choi-unital-iff* in theory Day19_Choi_TP_Unital.

definition *choi-matrix* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* **where**
choi-matrix n d Φ ≡
Matrix.mat (*n*∗*d*) (*n*∗*d*)
(λ(*r*,*c*). Φ (*mat-unit* (*r div d*) (*c div d*) *n n*) $$ (*r mod d*, *c mod d*))

notation *choi-matrix* (P_Φ)

definition *choi-seed* :: *nat* ⇒ *complex Matrix.mat* **where**
choi-seed n ≡
Matrix.mat (*n*∗*n*) (*n*∗*n*)
(λ(*r*,*c*). *mat-unit* (*r div n*) (*c div n*) *n n* $$ (*r mod n*, *c mod n*))

lemma *choi-seed-carrier* [*simp*]:
choi-seed n ∈ *carrier-mat* (*n*∗*n*) (*n*∗*n*)
⟨*proof*⟩

lemma *choi-seed-block*:
assumes *npos*: *n* > *0*
and *a*: *a* < *n*
and *b*: *b* < *n*
shows *block-of-mat n* (*choi-seed n*) *a b* = *mat-unit a b n n*
⟨*proof*⟩

lemma *choi-matrix-as-amplification*:
assumes *npos*: *n* > *0*
and *dpos*: *d* > *0*
shows *choi-matrix n d* Φ = *amplification n d n* Φ (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-cp*:
assumes *npos*: *n* > *0*
and *dpos*: *d* > *0*
and *cp*: *completely-positive n d* Φ
shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

lemma *kraus-cp-no-tp*:

assumes *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

41.1 From a positive Choi matrix to a Kraus representation

The notes' proof of (3)⇒(4): write $P_\Phi = \sum_k v_k v_k^*$ with $v_k \in \mathbb{C}^{nd}$, cut each v_k into n blocks $w_1^k, \ldots, w_n^k \in \mathbb{C}^d$, and let $B_k = [w_1^k | \cdots | w_n^k]$; then $B_k E_{ij} B_k^* = w_i^k (w_j^k)^*$ and $\Phi(X) = \sum_{ij} x_{ij} \Phi(E_{ij}) = \sum_k B_k X B_k^*$ by linearity.

Complete positivity from a Kraus list without the trace-preservation constraint: the argument is the same as ⟦*kraus-operators ?n ?d ?Ws*; *0 < ?n*; *0 < ?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*), which only uses the carrier hypothesis.

Linearity of the Kraus-sum map in its matrix argument.

lemma *kraus-map-dim-add*:
assumes *A-c*: $A \in carrier\text{-}mat\ n\ n$ **and** *B-c*: $B \in carrier\text{-}mat\ n\ n$
and *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
shows *kraus-map-dim d Ws* $(A + B) =$
kraus-map-dim d Ws A + *kraus-map-dim d Ws B*
⟨*proof*⟩

lemma *kraus-map-dim-smult*:
assumes *A-c*: $A \in carrier\text{-}mat\ n\ n$
and *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
shows *kraus-map-dim d Ws* $(c \cdot_m A) = c \cdot_m$ *kraus-map-dim d Ws A*
⟨*proof*⟩

lemma *kraus-map-dim-zero* [*simp*]:
assumes *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
shows *kraus-map-dim d Ws* $(0_m\ n\ n) = 0_m\ d\ d$
⟨*proof*⟩

Enumeration of $[0..<n] \times [0..<n]$ as a list of pairs.

definition *mat-unit-pairs* :: *nat* ⇒ (*nat* × *nat*) *list* **where**
mat-unit-pairs n ≡
concat (*map* (λ*a*. *map* (*Pair a*) [*0..<n*]) [*0..<n*])

lemma *mat-unit-pairs-set*:
set (*mat-unit-pairs n*) = {*p*. *fst p* < *n* ∧ *snd p* < *n*}
⟨*proof*⟩

lemma *mat-unit-pairs-distinct*: *distinct* (*mat-unit-pairs n*)
⟨*proof*⟩

Generic pair-indexed linear combination of f (E_{ab}) contributions, weighted by the entries of A.

primrec *pair-comb* ::
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒
nat ⇒ *nat* ⇒ (*nat* × *nat*) *list* ⇒ *complex Matrix.mat* **where**
pair-comb f A n d [] = $0_m\ d\ d$ |
pair-comb f A n d (*p* # *ps*) =
A $$ *p* $\cdot_m$ *f* (*mat-unit* (*fst p*) (*snd p*) *n n*) + *pair-comb f A n d ps*

lemma *pair-comb-carrier*:
assumes *f-car*: $\bigwedge a\ b.\ a < n \Longrightarrow b < n \Longrightarrow f\ (mat\text{-}unit\ a\ b\ n\ n) \in carrier\text{-}mat\ d\ d$

and *ps-valid*: $\forall\, p \in set\ ps.\ fst\ p < n \wedge snd\ p < n$
shows *pair-comb f A n d ps* $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *pair-comb-entry*:
assumes *f-car*: $\bigwedge a\ b.\ a < n \Longrightarrow b < n \Longrightarrow f\ (mat\text{-}unit\ a\ b\ n\ n) \in carrier\text{-}mat\ d\ d$
and *ps-valid*: $\forall\, p \in set\ ps.\ fst\ p < n \wedge snd\ p < n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *pair-comb f A n d ps* $\$\$\ (r,c) =$
$(\sum p \leftarrow ps.\ A\ \$\$\ p * f\ (mat\text{-}unit\ (fst\ p)\ (snd\ p)\ n\ n)\ \$\$\ (r,c))$
⟨*proof*⟩

For the identity-like choice $f = id$, the pair combination reconstructs A when *ps* enumerates all basis positions.

lemma *pair-comb-id-reconstructs*:
assumes *A-car*: $A \in carrier\text{-}mat\ n\ n$
shows *pair-comb* $(\lambda X.\ X)$ *A n n* (*mat-unit-pairs n*) $= A$
⟨*proof*⟩

For a linear map $f : M_n \to M_d$, applying f to A equals the pair combination of f evaluated on the basis.

lemma *linear-eq-pair-comb*:
fixes *f* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow Y \in carrier\text{-}mat\ n\ n \Longrightarrow$
$f\ (X + Y) = f\ X + f\ Y$
and *f-smult*: $\bigwedge c\ X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow f\ (c \cdot_m X) = c \cdot_m f\ X$
and *f-car*: $\bigwedge X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow f\ X \in carrier\text{-}mat\ d\ d$
and *A-car*: $A \in carrier\text{-}mat\ n\ n$
and *npos*: $n > 0$ **and** *dpos*: $d > 0$
shows $f\ A =$ *pair-comb f A n d* (*mat-unit-pairs n*)
⟨*proof*⟩

Conjugation $W\ E_{ab}\ W^\dagger$ has entry $W_{ia} \cdot^\dagger W_{jb}$.

lemma *conjugation-mat-unit-entry*:
assumes *Wc*: $W \in carrier\text{-}mat\ d\ n$
and *a-lt*: $a < n$ **and** *b-lt*: $b < n$
and *i-lt*: $i < d$ **and** *j-lt*: $j < d$
shows $(W * mat\text{-}unit\ a\ b\ n\ n * (W^\dagger))\ \$\$\ (i,\ j) =$
$W\ \$\$\ (i,\ a) * cnj\ (W\ \$\$\ (j,\ b))$
⟨*proof*⟩

Entry-wise formula for *kraus-map-dim* as a sum over the list.

lemma *kraus-map-dim-entry-sum-list*:
assumes *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *rho-c*: $\varrho \in carrier\text{-}mat\ n\ n$
and *i-lt*: $i < d$ **and** *j-lt*: $j < d$
shows *kraus-map-dim d Ws* ϱ $\$\$\ (i,\ j) =$
sum-list (*map* $(\lambda W.\ (W * \varrho * (W^\dagger))\ \$\$\ (i,\ j))\ Ws$)
⟨*proof*⟩

Kraus representation extracted from a psd Choi matrix: reshape each column of a psd factorization of P_Φ into a Kraus operator, and the resulting list agrees with Φ on all of M_n.

lemma *kraus-rep-of-psd-choi*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n \Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$

and *smult*: $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d$
and *npos*: $n > 0$
and *dpos*: $d > 0$
and *psdP*: $psd\ (choi\text{-}matrix\ n\ d\ \Phi)$
shows $\exists\ Ws :: complex\ Matrix.mat\ list.$
$length\ Ws = n * d \wedge$
$(\forall\ W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n) \wedge$
$(\forall A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A = kraus\text{-}map\text{-}dim\ d\ Ws\ A)$
⟨*proof*⟩

theorem *choi-theorem*:
fixes $\Phi :: complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$
assumes *linear*: $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n \Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d$
and *npos*: $n > 0$
and *dpos*: $d > 0$
shows $completely\text{-}positive\ n\ d\ \Phi \longleftrightarrow$
$psd\ (choi\text{-}matrix\ n\ d\ \Phi)$
⟨*proof*⟩

Accumulated $\sum_i\ {W_i}^\dagger\ W_i$ over a Kraus list.

definition *sum-wdag-w* :: $nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat$ **where**
$sum\text{-}wdag\text{-}w\ n\ Ws \equiv$
$foldr\ (\lambda W\ acc.\ (W^\dagger) * W + acc)\ Ws\ (0_m\ n\ n)$

lemma *sum-wdag-w-Nil* [*simp*]: $sum\text{-}wdag\text{-}w\ n\ [] = 0_m\ n\ n$
⟨*proof*⟩

lemma *sum-wdag-w-Cons*:
$sum\text{-}wdag\text{-}w\ n\ (W\ \#\ Ws) = (W^\dagger) * W + sum\text{-}wdag\text{-}w\ n\ Ws$
⟨*proof*⟩

lemma *sum-wdag-w-carrier*:
assumes $\forall\ W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
shows $sum\text{-}wdag\text{-}w\ n\ Ws \in carrier\text{-}mat\ n\ n$
⟨*proof*⟩

lemma *kraus-operators-altdef*:
$kraus\text{-}operators\ n\ d\ Ws \longleftrightarrow$
$(\forall\ W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n) \wedge sum\text{-}wdag\text{-}w\ n\ Ws = 1_m\ n$
⟨*proof*⟩

lemma *Tr-kraus-map-dim-as-wdagw*:
fixes $\varrho :: complex\ Matrix.mat$
assumes *Ws-c*: $\forall\ W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *X-c*: $\varrho \in carrier\text{-}mat\ n\ n$
shows $Tr\ (kraus\text{-}map\text{-}dim\ d\ Ws\ \varrho) = Tr\ (sum\text{-}wdag\text{-}w\ n\ Ws * \varrho)$
⟨*proof*⟩

The $\sum_i\ {W_i}^\dagger\ W_i$ matrix entry equals a sum-list of $cnj(W_{ika}) \cdot W_{ikb}$ contributions (needed to recover 1_n from trace preservation).

lemma *sum-wdag-w-entry*:
assumes *Ws-c*: $\forall\ W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *a-lt*: $a < n$ **and** *b-lt*: $b < n$
shows $sum\text{-}wdag\text{-}w\ n\ Ws\ \$\$\ (a,\ b) =$

sum-list (*map* (λ*W*. ((*W*†) ∗ *W*) $$ (*a*, *b*)) *Ws*)
⟨*proof*⟩

Pairing with E_{ab} extracts the (b,a) entry under the trace.

lemma *Tr-mult-mat-unit*:
assumes *M-c*: *M* ∈ *carrier-mat n n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
shows *Tr* (*M* ∗ *mat-unit a b n n*) = *M* $$ (*b*, *a*)
⟨*proof*⟩

theorem *choi-kraus-cptp*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹
Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·*m A*) = *c* ·*m* Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
and *npos*: *n* > *0*
and *dpos*: *d* > *0*
shows (*completely-positive n d* Φ ∧ *trace-preserving n* Φ) ⟷
(∃ *Ws* :: *complex Matrix.mat list*.
kraus-operators n d Ws ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Ws X*))
⟨*proof*⟩

end
theory *Day19-Choi-Positivity*
imports
Day27-Entangled-Examples
Day23-ChoiExtremeUCP
begin

42 Day 19: Choi's theorem with (2) isolated (D19-1)

lemma *choi-seed-psd*: *psd* (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-n-positive*:
assumes *n*: *0* < *n* **and** *d*: *0* < *d* **and** *pp*: *p-positive n d n* Φ
shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

theorem *choi-four-conditions*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·*m A*) = *c* ·*m* Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
and *n*: *0* < *n* **and** *d*: *0* < *d*
shows (*completely-positive n d* Φ ⟷ *p-positive n d n* Φ)
∧ (*p-positive n d n* Φ ⟷ *psd* (*choi-matrix n d* Φ))
∧ (*psd* (*choi-matrix n d* Φ) ⟷
(∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧ (∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* = *kraus-map-dim d Ws A*)))
⟨*proof*⟩

end
theory *Day19-Choi-TP-Unital*
imports *Day25-Arveson-Correspondence*

begin

43 Day 19: trace preservation and unitality in Choi-matrix form (D19-3)

Day 19, notes (4) and (5) after Choi's theorem: write $P_\Phi = (R_{ij})$ with $R_{ij} = \Phi(E_{ij}) \in M_d$. Then Φ is trace preserving iff $\mathrm{Tr}\, R_{ij} = \delta_{ij}$, and unital iff $\sum_i R_{ii} = I_d$; with *choi-theorem* this characterises CPTP and UCP maps by their Choi matrix. Linearity on M_n is *lin-on n d* (*carrier-mat n n*) Φ.

lemma *choi-block*:
assumes *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d* **and** *i*: $i < n$ **and** *j*: $j < n$
shows *block-of-mat d* (*choi-matrix n d* Φ) *i j* $= \Phi\ (E[\![i,j\ ;\ n,n]\!])$
⟨*proof*⟩

lemma *msum-cong*: $(\bigwedge x.\ x \in$ *set xs* $\Longrightarrow$ *g x* $=$ *h x*$) \Longrightarrow$ *msum r c g xs* $=$ *msum r c h xs*
⟨*proof*⟩

lemma *Tr-msum*:
assumes $\forall x \in$ *set xs*. *g x* $\in$ *carrier-mat d d*
shows *Tr* (*msum d d g xs*) $= (\sum x \leftarrow xs.\ Tr\ (g\ x))$
⟨*proof*⟩

lemma *unit-expansion*:
assumes *A*: $A \in$ *carrier-mat n n*
shows $A =$ *msum n n* $(\lambda(i,j).\ A\ \$\$\ (i,j) \cdot_m E[\![i,j\ ;\ n,n]\!])$ (*List.product* $[0..<n]$ $[0..<n]$)
⟨*proof*⟩

lemma *id-expansion*: $1_m\ n =$ *msum n n* $(\lambda i.\ E[\![i,i\ ;\ n,n]\!])$ $[0..<n]$
⟨*proof*⟩

lemma *lin-expansion*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *A*: $A \in$ *carrier-mat n n*
shows $\Phi\ A =$ *msum d d* $(\lambda(i,j).\ A\ \$\$\ (i,j) \cdot_m \Phi\ (E[\![i,j\ ;\ n,n]\!]))$ (*List.product* $[0..<n]$ $[0..<n]$)
⟨*proof*⟩

D19-3, note (4): trace preservation in Choi-matrix form.

theorem *choi-trace-preserving-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows *trace-preserving n* $\Phi \longleftrightarrow$
$(\forall i < n.\ \forall j < n.\ Tr\ (block\text{-}of\text{-}mat\ d\ (choi\text{-}matrix\ n\ d\ \Phi)\ i\ j) = (if\ i = j\ then\ 1\ else\ 0))$
⟨*proof*⟩

D19-3, note (5): unitality in Choi-matrix form, $\sum_i R_{ii} = I_d$.

theorem *choi-unital-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows *unital-map n d* $\Phi \longleftrightarrow$ *msum d d* $(\lambda i.\ block\text{-}of\text{-}mat\ d\ (choi\text{-}matrix\ n\ d\ \Phi)\ i\ i)$ $[0..<n] = 1_m\ d$
⟨*proof*⟩

lemma *lin-on-choi-hyps*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
⟨*proof*⟩

Notes (4) and (5) with Choi's theorem: CPTP iff $P_\Phi \geq 0$ and $\mathrm{Tr}\, R_{ij} = \delta_{ij}$; UCP iff $P_\Phi \geq 0$ and $\sum_i R_{ii} = I_d$.

corollary *choi-cptp-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *n*: *n* > *0* **and** *d*: *d* > *0*
shows *completely-positive n d* Φ ∧ *trace-preserving n* Φ ⟷
psd (*choi-matrix n d* Φ) ∧
(∀ *i* < *n*. ∀ *j* < *n*. *Tr* (*block-of-mat d* (*choi-matrix n d* Φ) *i j*) = (*if i* = *j then 1 else 0*))
⟨*proof*⟩

corollary *choi-ucp-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *n*: *n* > *0* **and** *d*: *d* > *0*
shows *completely-positive n d* Φ ∧ *unital-map n d* Φ ⟷
psd (*choi-matrix n d* Φ) ∧ *msum d d* (λ*i*. *block-of-mat d* (*choi-matrix n d* Φ) *i i*) [*0*..<*n*] = *1*$_m$ *d*
⟨*proof*⟩

end

theory *Day20-Positive-Functionals*
imports *Day19-Choi-Theorem*
begin

44 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank

Proposition (notes). For linear $\phi : M_n \to \mathbb{C}$: ϕ positive $\Leftrightarrow$ ϕ completely positive $\Leftrightarrow$ $(\phi(E_{ij})) \in M_n^+$.

Proposition (notes). Let $\Phi : M_n \to M_d$ with Hilbert–Schmidt adjoint $\Phi^* : M_d \to M_n$ ($\langle Y|X\rangle = \mathrm{tr}(Y^*X)$). (1) If $\Phi(X) = \sum_i A_i X A_i^*$ then $\Phi^*(Y) = \sum_i A_i^* Y A_i$. (2) Φ is completely positive iff Φ^* is. (3) Φ is unital and completely positive iff Φ^* is completely positive and trace preserving.

Definition (notes). The *Choi rank* of a completely positive Φ is $\mathrm{cr}(\Phi) = \min\{L : \Phi(X) = \sum_{l=1}^{L} B_l X B_l^*\}$.

Theorem (notes) (Choi)**.** $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$.

The functional proposition, the adjoint proposition (for Kraus maps) and the definition of the Choi rank are formalised below; the rank theorem is *choi-rank-eq-rank* in *Day20-Choi-Rank* (its lemma $\mathrm{rank}((E_{ij})) = 1$ is not stated separately), and the bound $\mathrm{cr}(\Phi) \leq nd$ is below.

44.1 The adjoint of a Kraus map

For $\Phi(X) = \sum_i A_i X A_i^\dagger$ with $A_i \in M_{d,n}$, the adjoint is $\Phi^*(Y) = \sum_i A_i^\dagger Y A_i$ (*kraus-adjoint-dim n As*; the output dimension n is kept explicit for the empty list).

definition *kraus-adjoint-dim* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-adjoint-dim n As y ≡
foldr (λ*A acc*. (*A*†) ∗ *y* ∗ *A* + *acc*) *As* (*0*$_m$ *n n*)

lemma *kraus-adjoint-dim-Nil* [*simp*]:
fixes *Y* :: *complex Matrix.mat*
shows *kraus-adjoint-dim n* [] *Y* = *0*$_m$ *n n*

⟨*proof*⟩

lemma *kraus-adjoint-dim-Cons* [*simp*]:
fixes Y :: *complex Matrix.mat*
shows *kraus-adjoint-dim n* $(A \# As)$ $Y =$
$(A^\dagger) * Y * A +$ *kraus-adjoint-dim n As Y*
⟨*proof*⟩

lemma *kraus-adjoint-dim-carrier*:
fixes Y :: *complex Matrix.mat*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows *kraus-adjoint-dim n As Y* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-adjoint-dim-as-kraus-map-dim*:
fixes Y :: *complex Matrix.mat*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows *kraus-adjoint-dim n As Y* $=$
kraus-map-dim n (*map* $(\lambda A.\ (A^\dagger))$ *As*) Y
⟨*proof*⟩

The adjoint identity $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$, by induction on the Kraus list and cyclicity of the trace.

lemma *hs-inner-prod-right-zero*:
$\langle\langle\ y \mid 0_m\ m\ n\ \rangle\rangle = 0$
⟨*proof*⟩

lemma *hs-inner-prod-left-zero*:
fixes X :: *complex Matrix.mat*
assumes *X-c*: $X \in$ *carrier-mat m n*
shows $\langle\langle\ 0_m\ m\ n \mid X\ \rangle\rangle = 0$
⟨*proof*⟩

lemma *hs-inner-prod-kraus-term*:
fixes $A\ X\ Y$ :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat d n*
and *X-c*: $X \in$ *carrier-mat n n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows $\langle\langle\ Y \mid A * X * (A^\dagger)\ \rangle\rangle = \langle\langle\ (A^\dagger) * Y * A \mid X\ \rangle\rangle$
⟨*proof*⟩

theorem *kraus-adjoint-correct*:
fixes *As* :: *complex Matrix.mat list*
and X :: *complex Matrix.mat*
and Y :: *complex Matrix.mat*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *X-c*: $X \in$ *carrier-mat n n*
and *Y-c*: $Y \in$ *carrier-mat d d*
shows $\langle\langle\ Y \mid$ *kraus-map-dim d As X* $\rangle\rangle$
$= \langle\langle$ *kraus-adjoint-dim n As Y* $\mid X\ \rangle\rangle$
⟨*proof*⟩

44.2 Complete positivity and the UCP/CPTP duality

Φ^* is the Kraus map of the adjoint operators, hence completely positive, and Φ is unital iff Φ^* is trace preserving: $\sum_i A_i A_i^* = I$ in both cases, using that $\mathrm{tr}(BY) = \mathrm{tr}(Y)$ for all Y forces $B = I$.

theorem *adjoint-cp-from-kraus*:
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive d n* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

lemma *kraus-map-dim-one-eq-sum-wdag-w-dagger*:
fixes *As* :: *complex Matrix.mat list*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
shows *kraus-map-dim d As* (1_m *n*)
= *sum-wdag-w d* (*map* (λA. ($A^\dagger$)) *As*)
⟨*proof*⟩

theorem *cp-unital-iff-adjoint-tp*:
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *unital-map n d* (*kraus-map-dim d As*)
$\longleftrightarrow$ *trace-preserving d* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

44.3 Positive functionals are completely positive

The case $d = 1$ of Choi's theorem: for $P = (P_{ij}) \in M_r(M_n)^+$ and $v = (\alpha_i) \in \mathbb{C}^r$, $\langle v|(\phi(P_{ij}))v\rangle = \phi(\sum_{ij} \overline{\alpha_i}\alpha_j P_{ij}) = \phi(Y^*PY) \geq 0$ with Y the block column $(\alpha_i I)_i$.

lemma *phi-entry-double-sum*:
fixes φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y$. $X \in$ *carrier-mat n n* $\Longrightarrow$ $Y \in$ *carrier-mat n n*
$\Longrightarrow \varphi\ (X + Y) = \varphi\ X + \varphi\ Y$
and *f-smult*: $\bigwedge c\ X$. $X \in$ *carrier-mat n n* $\Longrightarrow \varphi\ (c \cdot_m X) = c \cdot_m \varphi\ X$
and *f-car*: $\bigwedge X$. $X \in$ *carrier-mat n n* $\Longrightarrow \varphi\ X \in$ *carrier-mat d d*
and *M-c*: $M \in$ *carrier-mat n n*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\varphi\ M$ \$\$ $(r, c) =$
$(\sum i{<}n.\ \sum j{<}n.\ M$ \$\$ $(i, j) * \varphi$ (*mat-unit i j n n*) \$\$ $(r, c))$
⟨*proof*⟩

theorem *positive-scalar-is-cp*:
fixes φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B$. $A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n*
$\Longrightarrow \varphi\ (A + B) = \varphi\ A + \varphi\ B$
and *smult*: $\bigwedge c\ A$. $A \in$ *carrier-mat n n* $\Longrightarrow \varphi\ (c \cdot_m A) = c \cdot_m \varphi\ A$
and *range*: $\forall A$. $A \in$ *carrier-mat n n* $\longrightarrow \varphi\ A \in$ *carrier-mat 1 1*
and *pos*: *positive-map n 1* φ
and *npos*: $0 < n$
shows *completely-positive n 1* φ
⟨*proof*⟩

44.4 Choi rank

choi-rank n d Φ is the least length of a Kraus list representing Φ on the carrier. The theorem $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$ is *choi-rank-eq-rank* in *Day20-Choi-Rank*; the bound $\mathrm{cr}(\Phi) \le nd$ follows from the Kraus-rank bound of Day 21.

definition *choi-rank* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *nat* **where**
  choi-rank n d Φ ≡
    Inf { *length As* | *As* :: *complex Matrix.mat list*.
        (∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*)
        ∧ (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d As X*) }

end
theory *Day20-Choi-Rank*
  imports *Day34-Knill-Laflamme Day22-Kraus-Independence*
begin

45 Day 20: Choi's rank theorem $\mathrm{cr}(\Phi) = \mathrm{rank}\, P_\Phi$ (D20-4)

Rank is JNF's *vec-space.rank*, the dimension of the column space.

abbreviation *mat-rank* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* **where**
  mat-rank m A ≡ *vec-space.rank m A*

45.1 $\mathrm{rank}(\sum_{l<L} w_l w_l^*) \le L$

lemma *rank-outer-prod-le-1*:
  assumes *vd*: *dim-vec v* = *m*
  shows *mat-rank m* (*v* ⊘ *v*) ≤ *1*
⟨*proof*⟩

lemma *rank-foldr-outer-le*:
  assumes *vsd*: ∀ *v* ∈ *set vs*. *dim-vec v* = *m*
  shows *mat-rank m* (*foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (*0*$_m$ *m m*)) ≤ *length vs*
  ⟨*proof*⟩

lemma *rank-outer-sum-le*:
  assumes *vsd*: ∀ *v* ∈ *set vs*. *dim-vec v* = *m*
  shows *mat-rank m* (*outer-sum m vs*) ≤ *length vs*
  ⟨*proof*⟩

45.2 Independent vectors inside the column space bound the rank from below

lemma *mult-mat-vec-in-span*:
  assumes *Ac*: *A* ∈ *carrier-mat m k* **and** *xc*: *x* ∈ *carrier-vec k*
  shows *A* ∗$_v$ *x* ∈ *module.span class-ring* (*module-vec TYPE*(*complex*) *m*) (*set* (*cols A*))
⟨*proof*⟩

lemma *rank-ge-indep-in-colspace*:
  assumes *Ac*: *A* ∈ *carrier-mat m k* **and** *li*: *lin-indep-vecs m ws*
    and *insp*: ⋀*w*. *w* ∈ *set ws* ⟹ *w* ∈ *module.span class-ring* (*module-vec TYPE*(*complex*) *m*) (*set* (*cols A*))
  shows *length ws* ≤ *mat-rank m A*
⟨*proof*⟩

45.3 The theorem

lemma *choi-rank-attained*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\exists$ *As* :: *complex Matrix.mat list. length As = choi-rank n d* $\Phi \wedge$
$(\forall A \in$ *set As. A* $\in$ *carrier-mat d n*$) \wedge$
$(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow \Phi\ X =$ *kraus-map-dim d As X*$)$
⟨*proof*⟩

theorem *choi-rank-eq-rank*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows *choi-rank n d* Φ = *mat-rank* $(n{*}d)$ *(choi-matrix n d* Φ*)*
⟨*proof*⟩

end

theory *Day21-Kraus-Rank*
imports *Day20-Positive-Functionals*
begin

46 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations

Corollary (notes). A completely positive $\Phi : M_n \to M_d$ has a Kraus representation $\Phi(X) = \sum_{l=1}^{K} A_l X A_l^*$ with $K \le nd$.

The notes then record a conjecture (every CP map is an average of n UCP maps of Choi rank $\le d$) and develop the uniqueness theory:

Lemma (notes). If $T : \mathbb{C}^r \to \mathbb{C}^m$ is one-to-one then T^* is onto.

Proposition (notes). If $v_1, \dots, v_r \in \mathbb{C}^k$ are linearly independent then so are $\{v_i v_j^*\}_{i,j} \subseteq M_k$.

Lemma (notes). If $0 \le ww^* \le P$ then $w \in \mathrm{range}(P)$; hence for $P = \sum_l w_l w_l^* \ge 0$, $\mathrm{range}(P) = \mathrm{span}\{w_l\}$.

Theorem (notes). If $P = \sum_{j=1}^{r} v_j v_j^* = \sum_{l=1}^{m} w_l w_l^*$ with $\operatorname{rank} P = r$, there is $U = (\alpha_{ij})_{m \times r}$ with $w_i = \sum_j \alpha_{ij} v_j$ and $U^*U = I_r$.

Theorem (notes) (Choi 2)**.** If Φ is completely positive with $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{j=1}^{r} V_j X V_j^* = \sum_{l=1}^{m} W_l X W_l^*$, then $W_i = \sum_j \alpha_{ij} V_j$ for a unique $U = (\alpha_{ij})$ with $U^*U = I_r$.

Only the Kraus rank bound is formalised here. The kernel form of the lemma ($\ker A \subseteq \ker B$ when $0 \le B \le A$) is proved in theory Day23 and used there to prove Choi's Theorem 3 without the unitary-freedom theorem; the remaining statements are audit items D21-3, D21-5, D21-6.

corollary *kraus-rank-bound*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
$\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows $\exists$ *As* :: *complex Matrix.mat list*.
length As $\leq n * d \wedge$
$(\forall A \in$ *set As*. $A \in$ *carrier-mat d n*$) \wedge$
$(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow \Phi\ X =$ *kraus-map-dim d As X*$)$
⟨*proof*⟩

Dimension bookkeeping for $T^\dagger$ (the onto statement is not formalised).

lemma *adjoint-image-in-dom*:
fixes *T* :: *complex Matrix.mat*
assumes *sq*: *T* $\in$ *carrier-mat m r*
shows $\forall w$ :: *complex Matrix.vec*. *dim-vec w* $= m$
$\longrightarrow$ *dim-vec* $(T^\dagger *_v w) = r$
⟨*proof*⟩

$\mathrm{cr}(\Phi) \leq nd$: the defining set of the Choi rank is non-empty.

corollary *choi-rank-le*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
$\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *choi-rank n d* $\Phi \leq n * d$
⟨*proof*⟩

end
theory *Day21-Unitary-Freedom*
imports *QC-Prelims*
begin

47 Day 21: unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6)

47.1 The Gram matrix of an independent list is invertible; dual vectors

lemma *gram-det-nonzero*:
assumes *li*: *lin-indep-vecs n vs*
shows *det* $((\textit{mat-of-cols } n\ \textit{vs})^\dagger * \textit{mat-of-cols } n\ \textit{vs}) \neq 0$
⟨*proof*⟩

lemma *gram-inverse*:
assumes *li*: *lin-indep-vecs n vs*
obtains *B* **where** *B* $\in$ *carrier-mat* (*length vs*) (*length vs*)
and $((\textit{mat-of-cols } n\ \textit{vs})^\dagger * \textit{mat-of-cols } n\ \textit{vs}) * B = 1_m\ (\textit{length vs})$
and $B * ((\textit{mat-of-cols } n\ \textit{vs})^\dagger * \textit{mat-of-cols } n\ \textit{vs}) = 1_m\ (\textit{length vs})$
and $B^\dagger = B$

⟨*proof*⟩

Dual vectors (the notes' "T^* is onto"): for independent $v_1, \ldots, v_k$ there are $\varphi_1, \ldots, \varphi_k$ with $\langle \varphi_i | v_j \rangle = \delta_{ij}$.

lemma *dual-vectors-exist*:
  assumes *li*: *lin-indep-vecs n vs*
  shows $\exists\, \varphi s ::$ *complex Matrix.vec list*. *length* φs = *length vs* $\wedge$
    $(\forall\, \varphi \in$ *set* φs. *dim-vec* φ = $n) \wedge$
    $(\forall\, i <$ *length vs*. $\forall\, j <$ *length vs*. $\langle \varphi s\, !\, i \mid vs\, !\, j \rangle$ = (*if* $i = j$ *then 1 else 0*))
⟨*proof*⟩

47.2 D21-3: independent vectors give independent rank-one matrices

definition *outer-prods* :: *complex Matrix.vec list* ⇒ *complex Matrix.mat list* **where**
  outer-prods vs = *map* (λ*k*. *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)) [*0*..<*length vs* ∗ *length vs*]

lemma *length-outer-prods* [*simp*]: *length* (*outer-prods vs*) = *length vs* ∗ *length vs*
  ⟨*proof*⟩

lemma *outer-prods-nth*:
  assumes $k <$ *length vs* ∗ *length vs*
  shows *outer-prods vs* ! k = *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)
  ⟨*proof*⟩

lemma *inner-prod-cnj-delta*:
  assumes *dim-vec* $\varphi = n$ **and** *dim-vec* $v = n$ **and** $\langle \varphi \mid v \rangle$ = (*if P then 1 else 0*)
  shows $\langle v \mid \varphi \rangle$ = (*if P then 1 else 0*)
  ⟨*proof*⟩

proposition *outer-prods-lin-indep*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *mat-lin-indep-complex n* (*outer-prods vs*)
⟨*proof*⟩

47.3 Isometric mixing of columns preserves VV^*

lemma *dagger-transpose-mult*:
  assumes *Uc*: $U \in$ *carrier-mat m r* **and** *UU*: $(U^\dagger) * U = 1_m\ r$
  shows *transpose-mat U* ∗ ((*transpose-mat U*)†) = $1_m\ r$
⟨*proof*⟩

lemma *isometry-mix-outer*:
  assumes *Vc*: $V \in$ *carrier-mat n r* **and** *Uc*: $U \in$ *carrier-mat m r* **and** *UU*: $(U^\dagger) * U = 1_m\ r$
  shows (V ∗ *transpose-mat U*) ∗ ((*V* ∗ *transpose-mat U*)†) = $V * (V^\dagger)$
⟨*proof*⟩

lemma *dagger-mult-zero-imp-zero*:
  fixes $Y ::$ *complex Matrix.mat*
  assumes *Yc*: $Y \in$ *carrier-mat p q* **and** *YY*: $(Y^\dagger) * Y = 0_m\ q\ q$
  shows $Y = 0_m\ p\ q$
⟨*proof*⟩

47.4 D21-5: two rank-one decompositions are related by an isometry

If $P = \sum_{j<r} v_j v_j^* = \sum_{l<m} w_l w_l^*$ with $v_1, \ldots, v_r$ linearly independent (the notes assume rank $P = r$, which is equivalent), then $w_l = \sum_j \alpha_{lj} v_j$ with $U = (\alpha_{lj})$ an $m \times r$ isometry, $U^*U = I_r$. Matrix form: $W = VU^t$.

theorem *rank-one-decomp-isometry*:
assumes *li*: *lin-indep-vecs n vs*
and *wsd*: $\forall\, w \in$ *set ws*. *dim-vec w* $= n$
and *same*: *outer-sum n vs* $=$ *outer-sum n ws*
shows $\exists\, U ::$ *complex Matrix.mat*. $U \in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length vs*) $\wedge$
mat-of-cols n ws $=$ *mat-of-cols n vs* $*$ *transpose-mat U*
⟨*proof*⟩

corollary *rank-one-decomp-isometry-cols*:
assumes *li*: *lin-indep-vecs n vs*
and *wsd*: $\forall\, w \in$ *set ws*. *dim-vec w* $= n$
and *same*: *outer-sum n vs* $=$ *outer-sum n ws*
shows $\exists\, U ::$ *complex Matrix.mat*. $U \in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length vs*) $\wedge$
$(\forall\, l <$ *length ws*. *ws* ! $l =$ *mat-of-cols n vs* $*_v$ *Matrix.row U l*)
⟨*proof*⟩

47.5 D21-6 (Choi 2): unitary freedom of Kraus representations

lemma *vec-of-kraus-mat-of-cols-index*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n* **and** *dpos*: $0 < d$
and *i*: $i < d$ **and** *a*: $a < n$ **and** *k*: $k <$ *length Vs*
shows *mat-of-cols* $(n{*}d)$ (*map* (*vec-of-kraus d n*) *Vs*) $\$\$$ $(a{*}d + i,\ k) =$ *Vs* ! *k* $\$\$$ $(i,\ a)$
⟨*proof*⟩

lemma *lin-indep-complex-mat-vec-of-kraus*:
assumes *li*: *lin-indep-complex-mat d n Vs* **and** *dpos*: $0 < d$
shows *lin-indep-vecs* $(n{*}d)$ (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

Choi's Theorem 2: if $\Phi(X) = \sum_{j<r} V_j X V_j^* = \sum_{l<m} W_l X W_l^*$ with $V_1, \dots, V_r$ linearly independent (the notes: $r = \mathrm{cr}(\Phi)$), then $W_l = \sum_j \alpha_{lj} V_j$ with $U = (\alpha_{lj})$ an isometry, $U^*U = I_r$.

theorem *choi-2-unitary-freedom*:
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and *Ws-c*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*
and *li*: *lin-indep-complex-mat d n Vs*
and *same*: $\forall\, X$. $X \in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim d Vs X* $=$ *kraus-map-dim d Ws X*
shows $\exists\, U ::$ *complex Matrix.mat*. $U \in$ *carrier-mat* (*length Ws*) (*length Vs*) $\wedge$
$(U^\dagger) * U = 1_m$ (*length Vs*) $\wedge$
$(\forall\, l <$ *length Ws*. *Ws* ! $l =$
foldr $(\lambda j\ acc.\ U\ \$\$\ (l,j) \cdot_m (Vs\ !\ j) + acc)\ [0..{<}length\ Vs]\ (0_m\ d\ n)$)
⟨*proof*⟩

end
theory *Day21-Ranges*
imports *Day02-Projection-Basis Day23-ChoiExtremeUCP*
begin

48 Day 21: ranges of positive matrices (D21-4)

The notes write $R(P)$ for the range of P (here *mat-range k P*) and use two facts: if $0 \le ww^* \le P$ then $w \in R(P)$, and $R(\sum_l w_l w_l^*) = \mathrm{span}\{w_l\}$ (*col-span*). Both rest on one lemma: for Hermitian P, every vector orthogonal to $\ker P$ lies in $R(P)$; it is proved from the spectral decomposition $P = UDU^*$ by inverting the nonzero eigenvalues. The audit's earlier formal counterpart was the kernel form *loewner-ker-subset*.

definition *mat-range* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec set* **where**
mat-range k P = {*P* ∗*v h* | *h*. *dim-vec h* = *k*}

lemma *diag-mat-of-real-mult-vec*:
assumes *length D* = *k dim-vec v* = *k*
shows *diag-mat-of-real D* ∗*v v* = *Matrix.vec k* (λ*i*. *complex-of-real* (*D* ! *i*) ∗ *v* $ *i*)
⟨*proof*⟩

For Hermitian P, a vector orthogonal to $\ker P$ lies in the range of P.

lemma *hermitian-perp-ker-in-range*:
assumes *P*: *P* ∈ *carrier-mat k k* **and** *h*: *hermitian P* **and** *w*: *dim-vec w* = *k*
and *perp*: ⋀*x*. *dim-vec x* = *k* ⟹ *P* ∗*v x* = 0*v k* ⟹ *Quantum.inner-prod x w* = 0
shows *w* ∈ *mat-range k P*
⟨*proof*⟩

The notes: if $0 \le ww^* \le P$ then $w \in R(P)$.

theorem *dominated-rank-one-in-range*:
assumes *P*: *P* ∈ *carrier-mat k k* **and** *Pp*: *psd P* **and** *w*: *dim-vec w* = *k*
and *dom*: *psd* (*P* − *outer-sum k* [*w*])
shows *w* ∈ *mat-range k P*
⟨*proof*⟩

The notes: if $P = \sum_l w_l w_l^*$ then $R(P) = \mathrm{span}\{w_1, \ldots, w_m\}$.

theorem *outer-sum-range*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *k*
shows *mat-range k* (*outer-sum k vs*) = *col-span k vs*
⟨*proof*⟩

48.1 Day 4 revisited: positive definiteness and spanning (D4-8)

The notes: $P = \sum_i |v_i\rangle\langle v_i| > 0$ iff $\mathrm{span}\{v_i\} = \mathbb{C}^n$.

theorem *pd-iff-span-all*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
shows *pd* (*outer-sum n vs*) ⟷ *col-span n vs* = *carrier-vec n*
⟨*proof*⟩

end
theory *Day21-Average-Conjecture*
imports *Day32-Phase-Flip-Code*
begin

49 Day 21: the conjecture on averages of unital maps (D21-2)

The notes ask whether every unital completely positive $\Phi : M_n \to M_d$ is the average of n unital completely positive maps of Choi rank at most d (*D21-2*). This is the strong Ruskai–Audenaert conjecture for the dual channel, open in general. Proved here: Fourier mixing settles every map whose Kraus operators split into n groups with vanishing cross sums (*fourier-split*); this covers measurement maps (*ra-measurement*), and Horn's lemma (*horn-lemma*, the case $d = 1$) covers maps with diagonal values (*ra-cq*). The notes' form (i) is false as written (*notes-form-i-false*).

lemma *conj-entry*:
assumes *W*: *W* ∈ *carrier-mat d n* **and** *X*: *X* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows (*W* ∗ *X* ∗ (*W*†)) $$ (*a*, *b*) = (∑ *p*<*n*. ∑ *q*<*n*. *W* $$ (*a*, *p*) ∗ *X* $$ (*p*, *q*) ∗ *cnj* (*W* $$ (*b*, *q*)))
⟨*proof*⟩

lemma *sum-list-concat-upt*:

sum-list (*map f* (*concat* (*map* (λ*m*. *map* (*g m*) [*0*..<*d*]) [*0*..<*n*]))) = (∑ *m*<*n*. ∑ *j*<*d*. *f* (*g m j*))
⟨*proof*⟩

Characters of $\mathbb{Z}_n$.

definition *om* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex* **where**
om n l m = *cis* (*2* ∗ *pi* ∗ *real l* ∗ *real m* / *real n*)

lemma *om-norm* [*simp*]: *om n l m* ∗ *cnj* (*om n l m*) = *1*
⟨*proof*⟩

lemma *om-orth*:
assumes *m*: *m* < *n* **and** *m'*: *m'* < *n*
shows (∑ *l*<*n*. *om n l m* ∗ *cnj* (*om n l m'*)) = (*if m* = *m' then of-nat n else 0*)
⟨*proof*⟩

lemma *om-mix*:
(∑ *l*<*n*. (∑ *m*<*n*. *om n l m* ∗ *x m*) ∗ *cnj* (∑ *m*<*n*. *om n l m* ∗ *y m*)) = *of-nat n* ∗ (∑ *m*<*n*. *x m* ∗ *cnj* (*y m*))
⟨*proof*⟩

lemma *kraus-concat-entry*:
assumes *g*: ⋀*m j*. *m* < *k* ⟹ *j* < *d* ⟹ *g m j* ∈ *carrier-mat d n* **and** *X*: *X* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows *kraus-map-dim d* (*concat* (*map* (λ*m*. *map* (*g m*) [*0*..<*d*]) [*0*..<*k*])) *X* $$ (*a*, *b*) = (∑ *m*<*k*. ∑ *j*<*d*. (*g m j* ∗ *X* ∗ ((*g m j*)†)) $$ (*a*, *b*))
⟨*proof*⟩

lemma *kraus-concat-carrier*:
assumes *g*: ⋀*m j*. *m* < *k* ⟹ *j* < *d* ⟹ *g m j* ∈ *carrier-mat d n* **and** *X*: *X* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*concat* (*map* (λ*m*. *map* (*g m*) [*0*..<*d*]) [*0*..<*k*])) *X* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *prod-dagger-entry*:
assumes *A*: *A* ∈ *carrier-mat d n* **and** *B*: *B* ∈ *carrier-mat d n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows (*A* ∗ (*B*†)) $$ (*a*, *b*) = (∑ *p*<*n*. *A* $$ (*a*, *p*) ∗ *cnj* (*B* $$ (*b*, *p*)))
⟨*proof*⟩

D21-2 in the notes' main form: a unital $\Phi : M_n \to M_d$ is the average of n unital completely positive maps of Choi rank at most d.

definition *ra-split* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
ra-split n d Φ ⟷ (∃ *Bs*. *length Bs* = *n* ∧ (∀ *l*<*n*. *length* (*Bs* ! *l*) ≤ *d* ∧ (∀ *B* ∈ *set* (*Bs* ! *l*). *B* ∈ *carrier-mat d n*)
∧ *kraus-map-dim d* (*Bs* ! *l*) (*1*$_m$ *n*) = *1*$_m$ *d*) ∧ (∀ *X* ∈ *carrier-mat n n*. Φ *X* = (*1* / *of-nat n*) ·$_m$ *kraus-map-dim d* (*concat Bs*) *X*))

The conjecture itself, for maps given by Kraus operators (every completely positive map is one). It is the strong Ruskai–Audenaert conjecture for the dual channel $M_d \to M_n$, open in general; the cases below are the ones settled in the literature by direct constructions.

definition *D21-2* :: *bool* **where**
D21-2 ⟷ (∀ *n d As*. *0* < *n* ⟶ (∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*) ⟶ *kraus-map-dim d As* (*1*$_m$ *n*) = *1*$_m$ *d*
⟶ *ra-split n d* (*kraus-map-dim d As*))

definition *fmix* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat* ⇒ *complex Matrix.mat*) ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
fmix n d A l j = *Matrix.mat d n* (λ(*a*, *p*). ∑ *m*<*n*. *om n l m* ∗ *A m j* $$ (*a*, *p*))

lemma *fmix-carrier* [*simp*]: *fmix n d A l j* ∈ *carrier-mat d n* ⟨*proof*⟩

lemma *fmix-index*: *a* < *d* ⟹ *p* < *n* ⟹ *fmix n d A l j* $$ (*a*, *p*) = (∑ *m*<*n*. *om n l m* ∗ *A m j* $$ (*a*, *p*))
⟨*proof*⟩

If the Kraus operators fall into n groups of d with $\sum_j A_{mj} A^*_{m'j} = 0$ for $m \neq m'$, Fourier mixing $B_{lj} = \sum_m \omega^{lm} A_{mj}$ gives the decomposition.

theorem *fourier-split*:
assumes *n*: *0* < *n* **and** *A*: ⋀*m j*. *m* < *n* ⟹ *j* < *d* ⟹ *A m j* ∈ *carrier-mat d n*
and *u*: *kraus-map-dim d* (*concat* (*map* (λ*m*. *map* (*A m*) [*0*..<*d*]) [*0*..<*n*])) (*1*$_m$ *n*) = *1*$_m$ *d*
and *orth*: ⋀*m m*′ *a b*. *m* < *n* ⟹ *m*′ < *n* ⟹ *m* ≠ *m*′ ⟹ *a* < *d* ⟹ *b* < *d* ⟹ (∑ *j*<*d*. (*A m j* ∗ ((*A m*′ *j*)†)) $$ (*a*, *b*)) = *0*
shows *ra-split n d* (*kraus-map-dim d* (*concat* (*map* (λ*m*. *map* (*A m*) [*0*..<*d*]) [*0*..<*n*])))
⟨*proof*⟩

49.1 Positive matrices as sums of d rank-one terms

lemma *udu-entry*:
assumes *U*: *U* ∈ *carrier-mat d d* **and** *D*: *length D* = *d* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows (*U* ∗ *diag-mat-of-real D* ∗ (*U*†)) $$ (*a*, *b*) = (∑ *k*<*d*. *U* $$ (*a*, *k*) ∗ *complex-of-real* (*D* ! *k*) ∗ *cnj* (*U* $$ (*b*, *k*)))
⟨*proof*⟩

lemma *psd-eig*:
assumes *Q*: *Q* ∈ *carrier-mat d d psd Q*
shows ∃ *U D*. *U* ∈ *carrier-mat d d* ∧ *length D* = *d* ∧ (∀ *k*<*d*. *0* ≤ *D* ! *k*) ∧ (*U*†) ∗ *U* = *1*$_m$ *d*
∧ (∀ *a*<*d*. ∀ *b*<*d*. *Q* $$ (*a*, *b*) = (∑ *k*<*d*. *U* $$ (*a*, *k*) ∗ *complex-of-real* (*D* ! *k*) ∗ *cnj* (*U* $$ (*b*, *k*))))
⟨*proof*⟩

lemma *psd-vecs*:
assumes *Q*: *Q* ∈ *carrier-mat d d psd Q*
shows ∃ *q*. ∀ *a*<*d*. ∀ *b*<*d*. *Q* $$ (*a*, *b*) = (∑ *k*<*d*. *q k a* ∗ *cnj* (*q k b*))
⟨*proof*⟩

lemma *ra-split-cong*: (⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = Ψ *X*) ⟹ *ra-split n d* Ψ ⟹ *ra-split n d* Φ
⟨*proof*⟩

lemma *delta-sum2*:
fixes *n i* :: *nat*
assumes *i*: *i* < *n*
shows (∑ *p*<*n*. ∑ *p*′<*n*. (*if p* = *i then x else 0*) ∗ *f p p*′ ∗ *cnj* (*if p*′ = *i then y else 0*)) = *x* ∗ *f i i* ∗ *cnj* (*y* :: *complex*)
⟨*proof*⟩

49.2 Measurement maps

$\Phi(X) = \sum_i X_{ii} Q_i$ with $Q_i \geq 0$, $\sum_i Q_i = I$ (dual to a channel with classical output): D21-2 holds.

definition *povm-map* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
povm-map n d Q X = *Matrix.mat d d* (λ(*a*, *b*). ∑ *i*<*n*. *X* $$ (*i*, *i*) ∗ *Q i* $$ (*a*, *b*))

theorem *ra-measurement*:
assumes *n*: *0* < *n* **and** *Q*: ⋀*i*. *i* < *n* ⟹ *Q i* ∈ *carrier-mat d d* ∧ *psd* (*Q i*)
and *QI*: ⋀*a b*. *a* < *d* ⟹ *b* < *d* ⟹ (∑ *i*<*n*. *Q i* $$ (*a*, *b*)) = *1*$_m$ *d* $$ (*a*, *b*)

shows *ra-split n d* (*povm-map n d Q*)
⟨*proof*⟩

49.3 The case $d = 1$: Horn's lemma

Every density matrix on $\mathbb{C}^n$ is the average of n pure states (Fourier mixing of an eigenbasis).

theorem *horn-lemma*:
assumes ϱ: *density-matrix n* ϱ **and** *n*: *0* < *n*
shows $\exists\,\varphi.\ (\forall\, l{<}n.\ \textit{dim-vec}\ (\varphi\ l) = n \wedge \|\varphi\ l\| = 1)$
$\wedge\ (\forall\, a{<}n.\ \forall\, b{<}n.\ \varrho\ \$\$\ (a,\ b) = (1\ /\ \textit{of-nat}\ n) * (\sum l{<}n.\ \varphi\ l\ \$\ a * \textit{cnj}\ (\varphi\ l\ \$\ b)))$
⟨*proof*⟩

lemma *delta-pair*:
fixes *c c′ d* :: *nat*
assumes *c* < *d*
shows $(\sum a{<}d.\ \textit{if}\ c = a \wedge c' = a\ \textit{then}\ f\ a\ \textit{else}\ 0) = (\textit{if}\ c = c'\ \textit{then}\ f\ c\ \textit{else}\ (0{::}\textit{complex}))$
⟨*proof*⟩

49.4 Maps with diagonal values

$\Phi(X) = \mathrm{diag}(\mathrm{tr}(R_1 X), \ldots, \mathrm{tr}(R_d X))$ with density matrices R_a (dual to a channel with classical input): D21-2 holds, by Horn's lemma for each R_a.

definition *cq-map* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
cq-map n d Rh X = *Matrix.mat d d* (λ(*a*, *b*). *if a* = *b then* ($\sum p{<}n.\ \sum q{<}n.$ *Rh a* $\$\$$ (*q*, *p*) * *X* $\$\$$ (*p*, *q*)) *else 0*)

theorem *ra-cq*:
assumes *n*: *0* < *n* **and** *Rh*: $\bigwedge a.\ a < d \Longrightarrow$ *density-matrix n* (*Rh a*)
shows *ra-split n d* (*cq-map n d Rh*)
⟨*proof*⟩

49.5 The notes' form (i) is false as written

Form (i) asks for n trace-preserving maps $M_n \to M_d$ of Choi rank at most d. For $n = 2$, $d = 1$ there is no trace-preserving map $M_2 \to \mathbb{C}$ of Choi rank at most one, while the trace is trace preserving. (The equivalent dual form has the dimensions exchanged: maps $M_d \to M_n$.)

lemma *no-tp-rank-one*: *length Ws* ≤ *1* ⟹ ¬ *kraus-operators 2 1 Ws*
⟨*proof*⟩

theorem *notes-form-i-false*:
¬ (∀ *Ws*. *kraus-operators 2 1 Ws* ⟶ (∃ *Bs*. *length Bs* = *2* ∧ (∀ *l*<*2*. *length* (*Bs* ! *l*) ≤ *1* ∧ *kraus-operators 2 1* (*Bs* ! *l*))
∧ (∀ *X* ∈ *carrier-mat 2 2*. *kraus-map-dim 1 Ws X* = (*1* / *2*) ·$_m$ *kraus-map-dim 1* (*concat Bs*) *X*)))
⟨*proof*⟩

end
theory *Day21-Average-Two*
imports *Day21-Average-Conjecture Quantum-Channels*
begin

50 Day 21: the conjecture D21-2 for $n = 2$

Every unital completely positive $\Phi : M_2 \to M_d$ is the average of two unital maps of Choi rank at most d (*ra-two*; Bacon, Smolin and Wolf). The off-diagonal block of the Choi matrix is $R_1 K R_2^*$

with $R_i R_i^*$ the diagonal blocks and K a contraction (*wide-factor*, *bessel-rows*), and a contraction is the average of two unitaries (*contraction-avg*, through a singular value decomposition, *svd-square*).

50.1 Completing an orthonormal family to an orthonormal basis

lemma *onb-complete*:
fixes $d :: nat$ **and** $J :: nat\ set$ **and** $w :: nat \Rightarrow nat \Rightarrow complex$
assumes J: $J \subseteq \{..<d\}$
and *on*: $\bigwedge k\ k'.\ k \in J \Longrightarrow k' \in J \Longrightarrow (\sum a{<}d.\ cnj\ (w\ k\ a) * w\ k'\ a) = (if\ k = k'\ then\ 1\ else\ 0)$
shows $\exists W.\ W \in carrier\text{-}mat\ d\ d \wedge (W^\dagger) * W = 1_m\ d \wedge (\forall k{\in}J.\ \forall a{<}d.\ W\ \$\$\ (a,\ k) = w\ k\ a)$
⟨*proof*⟩

50.2 Singular value decomposition

lemma *unitary-sq-right*: $V \in carrier\text{-}mat\ d\ d \Longrightarrow (V^\dagger) * V = 1_m\ d \Longrightarrow V * (V^\dagger) = 1_m\ d$
⟨*proof*⟩

lemma *svd-square*:
assumes *Km*: $Km \in carrier\text{-}mat\ d\ d$
shows $\exists W\ V.\ W \in carrier\text{-}mat\ d\ d \wedge (W^\dagger) * W = 1_m\ d \wedge V \in carrier\text{-}mat\ d\ d \wedge (V^\dagger) * V = 1_m\ d$
$\wedge\ (\forall a{<}d.\ \forall b{<}d.\ Km\ \$\$\ (a,\ b) = (\sum k{<}d.\ W\ \$\$\ (a,\ k) * complex\text{-}of\text{-}real\ (sqrt\ (\sum c{<}d.\ (cmod\ ((Km * V)\ \$\$\ (c,\ k)))^2)) * cnj\ (V\ \$\$\ (b,\ k))))$
⟨*proof*⟩

50.3 A contraction is the average of two unitaries

definition *phase-diag* :: $nat \Rightarrow (nat \Rightarrow real) \Rightarrow complex\ Matrix.mat$ **where**
$phase\text{-}diag\ d\ \vartheta = Matrix.mat\ d\ d\ (\lambda(i,\ j).\ if\ i = j\ then\ cis\ (\vartheta\ i)\ else\ 0)$

lemma *phase-diag-carrier* [*simp*]: $phase\text{-}diag\ d\ \vartheta \in carrier\text{-}mat\ d\ d$ ⟨*proof*⟩

lemma *phase-diag-unitary*: $(phase\text{-}diag\ d\ \vartheta)^\dagger * phase\text{-}diag\ d\ \vartheta = 1_m\ d$
⟨*proof*⟩

lemma *wdv-entry*:
assumes W: $W \in carrier\text{-}mat\ d\ d$ **and** V: $V \in carrier\text{-}mat\ d\ d$ **and** a: $a < d$ **and** b: $b < d$
shows $(W * phase\text{-}diag\ d\ \vartheta * (V^\dagger))\ \$\$\ (a,\ b) = (\sum k{<}d.\ W\ \$\$\ (a,\ k) * cis\ (\vartheta\ k) * cnj\ (V\ \$\$\ (b,\ k)))$
⟨*proof*⟩

lemma *unitary-triple*:
assumes W: $W \in carrier\text{-}mat\ d\ d\ (W^\dagger) * W = 1_m\ d$ **and** V: $V \in carrier\text{-}mat\ d\ d\ (V^\dagger) * V = 1_m\ d$
and D: $D \in carrier\text{-}mat\ d\ d\ (D^\dagger) * D = 1_m\ d$
shows $(W * D * (V^\dagger))^\dagger * (W * D * (V^\dagger)) = 1_m\ d$
⟨*proof*⟩

theorem *contraction-avg*:
assumes *Km*: $Km \in carrier\text{-}mat\ d\ d$ **and** *ct*: $\bigwedge x.\ dim\text{-}vec\ x = d \Longrightarrow \|Km *_v x\| \leq \|x\|$
shows $\exists U1\ U2.\ U1 \in carrier\text{-}mat\ d\ d \wedge U2 \in carrier\text{-}mat\ d\ d \wedge (U1^\dagger) * U1 = 1_m\ d \wedge (U2^\dagger) * U2 = 1_m\ d$
$\wedge\ (\forall a{<}d.\ \forall b{<}d.\ Km\ \$\$\ (a,\ b) = (U1\ \$\$\ (a,\ b) + U2\ \$\$\ (a,\ b))\ /\ 2)$
⟨*proof*⟩

50.4 Bessel's inequality for rows that are orthonormal or zero

lemma *cmod-sq-sum*: $complex\text{-}of\text{-}real\ (\sum k{\in}A.\ (cmod\ (f\ k))^2) = (\sum k{\in}A.\ f\ k * cnj\ (f\ k))$

⟨*proof*⟩

lemma *bessel-rows*:
fixes $s :: nat \Rightarrow nat \Rightarrow complex$ **and** $y :: nat \Rightarrow complex$ **and** $d\ K :: nat$
assumes *on*: $\bigwedge j\ j'.\ j < d \Longrightarrow j' < d \Longrightarrow (\sum k{<}K.\ s\ j\ k * cnj\ (s\ j'\ k)) = (if\ j = j' \wedge j \in J\ then\ 1\ else\ 0)$
shows $(\sum j{<}d.\ (cmod\ (\sum k{<}K.\ s\ j\ k * y\ k))^2) \leq (\sum k{<}K.\ (cmod\ (y\ k))^2)$
⟨*proof*⟩

50.5 Factoring a wide matrix

$M = RS$ with $RR^* = MM^*$ and the rows of S orthonormal or zero.

lemma *wide-factor*:
fixes $m :: nat \Rightarrow nat \Rightarrow complex$ **and** $d\ Kn :: nat$
shows $\exists R\ S\ J.\ R \in carrier\text{-}mat\ d\ d$
$\wedge\ (\forall k{<}Kn.\ \forall a{<}d.\ m\ k\ a = (\sum j{<}d.\ R\ \$\$\ (a,\ j) * S\ j\ k))$
$\wedge\ (\forall a{<}d.\ \forall b{<}d.\ (\sum j{<}d.\ R\ \$\$\ (a,\ j) * cnj\ (R\ \$\$\ (b,\ j))) = (\sum k{<}Kn.\ m\ k\ a * cnj\ (m\ k\ b)))$
$\wedge\ (\forall j{<}d.\ \forall j'{<}d.\ (\sum k{<}Kn.\ S\ j\ k * cnj\ (S\ j'\ k)) = (if\ j = j' \wedge j \in J\ then\ 1\ else\ 0))$
⟨*proof*⟩

50.6 The case $n = 2$

definition *gk* :: *complex Matrix.mat list* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex* **where**
gk Fs p q i i' = $(\sum k{<}length\ Fs.\ Fs\ !\ k\ \$\$\ (i,\ p) * cnj\ (Fs\ !\ k\ \$\$\ (i',\ q)))$

lemma *kraus2-entry*:
assumes *F*: $\forall A \in set\ Fs.\ A \in carrier\text{-}mat\ d\ 2$ **and** *X*: $X \in carrier\text{-}mat\ 2\ 2$ **and** *i*: $i < d$ **and** *i'*: $i' < d$
shows *kraus-map-dim d Fs X* $\$\$\ (i,\ i') = (\sum p{<}2.\ \sum q{<}2.\ X\ \$\$\ (p,\ q) * gk\ Fs\ p\ q\ i\ i')$
⟨*proof*⟩

lemma *gk-map*:
gk (*map f* $[0..{<}d]$) *p q i i'* = $(\sum j{<}d.\ f\ j\ \$\$\ (i,\ p) * cnj\ (f\ j\ \$\$\ (i',\ q)))$
⟨*proof*⟩

lemma *mult-entry3*:
assumes *A*: $A \in carrier\text{-}mat\ d\ d$ **and** *B*: $B \in carrier\text{-}mat\ d\ d$ **and** *C*: $C \in carrier\text{-}mat\ d\ d$ **and** *i*: $i < d$ **and** *i'*: $i' < d$
shows $(A * B * (C^\dagger))\ \$\$\ (i,\ i') = (\sum j{<}d.\ \sum m{<}d.\ A\ \$\$\ (i,\ j) * B\ \$\$\ (j,\ m) * cnj\ (C\ \$\$\ (i',\ m)))$
⟨*proof*⟩

Every unital $\Phi : M_2 \to M_d$ is the average of two unital maps of Choi rank at most d (Bacon–Smolin–Wolf): the off-diagonal Choi block is $R_1 K R_2^*$ with K a contraction, and $K = \frac{1}{2}(U_1 + U_2)$.

theorem *ra-two*:
assumes *As*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ d\ 2$ **and** *u*: *kraus-map-dim d As* $(1_m\ 2) = 1_m\ d$
shows *ra-split 2 d* (*kraus-map-dim d As*)
⟨*proof*⟩

end

theory *Day22-Convex-Sets*
imports *Day21-Kraus-Rank*
begin

51 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP

Proposition (notes). If Φ is completely positive, $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$, then $V_1, \ldots, V_r$ are linearly independent.

(Not formalised: on Day 23 linear independence of the Kraus operators is a hypothesis.)

Definition (notes). $C \subseteq V$ is *convex* if $tv_1 + (1-t)v_2 \in C$ for $v_1, v_2 \in C$, $0 \le t \le 1$; $v \in C$ is *extreme* if $v = \frac{1}{2}(v_1 + v_2)$ with $v_1, v_2 \in C$ forces $v = v_1 = v_2$. The *convex hull* of E is the set of convex combinations of points of E; a map between convex sets is *affine* if it preserves convex combinations.

Theorem (notes) (Krein–Milman)**.** A compact convex subset of a finite-dimensional real vector space is the convex hull of its extreme points.

Examples: $UCP(M_n, M_d)$, the unital completely positive maps, is convex ($P_{t\Phi_1+(1-t)\Phi_2} = tP_{\Phi_1} + (1-t)P_{\Phi_2} \ge 0$) and compact (closed; bounded because $0 \le \Phi(E_{ii}) \le I$ gives $|r_{ij}| \le 1$ for the entries of P_Φ); $CPTP(M_d, M_n)$ is convex and compact, being the image of $UCP(M_n, M_d)$ under the affine bijection $\Gamma(\Phi) = \Phi^*$.

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is an extreme point of $UCP(M_n, M_d)$ iff $\{V_i^* V_j : 1 \le i, j \le r\}$ is linearly independent in M_d.

Corollary (notes). If Φ is extreme then $\mathrm{cr}(\Phi) \le d$. If $\Phi : M_d \to M_n$ is CPTP with $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$, $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent.

Convexity of UCP and CPTP is proved below; compactness and the affine bijection are not formalised; the corollary $\mathrm{cr}(\Phi) \le d$ is *extreme-CPTP-choi-rank-le* in *Day22-Kraus-Independence*, and Choi's Theorem 3 is proved in theory Day23 in its CPTP form. Krein–Milman is available in HOL-Analysis (*Krein-Milman-Minkowski*).

51.1 Convex sets and extreme points

```
definition convex-set-mat :: complex Matrix.mat set ⇒ bool where
  convex-set-mat C ≡
    ∀ A ∈ C. ∀ B ∈ C. ∀ t :: real. 0 ≤ t ∧ t ≤ 1 ⟶
      complex-of-real t ·m A
      + (1 − complex-of-real t) ·m B ∈ C

definition extreme-point-mat ::
  complex Matrix.mat set ⇒ complex Matrix.mat ⇒ bool where
  extreme-point-mat C v ≡
    v ∈ C ∧
    ¬ (∃ v1 ∈ C. ∃ v2 ∈ C. v1 ≠ v ∧ v ≠ v2 ∧
         v = (1/2 :: complex) ·m v1 + (1/2 :: complex) ·m v2)

definition affine-map-mat ::
  (complex Matrix.mat ⇒ complex Matrix.mat) ⇒ bool where
  affine-map-mat tmap ≡
    ∀ A B. ∀ t :: real. 0 ≤ t ∧ t ≤ 1 ⟶
      tmap (complex-of-real t ·m A
            + (1 − complex-of-real t) ·m B) =
      complex-of-real t ·m tmap A
      + (1 − complex-of-real t) ·m tmap B
```

51.2 The convex sets UCP and CPTP

definition *UCP* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
  UCP n d ≡ {Φ. *completely-positive n d* Φ ∧ *unital-map n d* Φ}

definition *CPTP* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
  CPTP n d ≡ {Φ. *completely-positive n d* Φ ∧ *trace-preserving n* Φ}

lemma *amplification-smult*:
  assumes *dpos*: *0* < *d*
    and *F-block*: ⋀*a b*. *F* (*block-of-mat n x a b*) ∈ *carrier-mat d d*
  shows *amplification n d p* (λϱ. *c* ·$_m$ *F* ϱ) *x*
      = *c* ·$_m$ *amplification n d p F x*
⟨*proof*⟩

UCP is convex. The range hypotheses are explicit because *completely-positive* does not constrain the values of Φ_i off the carrier.

lemma *UCP-convex*:
  fixes *t* :: *real*
  assumes *t-nn*: *0* ≤ *t*
    and *t-le*: *t* ≤ *1*
    and Φ*1-in*: Φ*1* ∈ *UCP n d*
    and Φ*2-in*: Φ*2* ∈ *UCP n d*
    and *r1*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*1 X* ∈ *carrier-mat d d*
    and *r2*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*2 X* ∈ *carrier-mat d d*
    and *dpos*: *0* < *d*
  shows (λ*X*. *complex-of-real t* ·$_m$ Φ*1 X*
          + (*1* − *complex-of-real t*) ·$_m$ Φ*2 X*) ∈ *UCP n d*
⟨*proof*⟩

CPTP is convex.

lemma *CPTP-convex*:
  fixes *t* :: *real*
  assumes *t-nn*: *0* ≤ *t*
    and *t-le*: *t* ≤ *1*
    and Φ*1-in*: Φ*1* ∈ *CPTP n d*
    and Φ*2-in*: Φ*2* ∈ *CPTP n d*
    and *r1*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*1 X* ∈ *carrier-mat d d*
    and *r2*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*2 X* ∈ *carrier-mat d d*
    and *dpos*: *0* < *d*
  shows (λ*X*. *complex-of-real t* ·$_m$ Φ*1 X*
          + (*1* − *complex-of-real t*) ·$_m$ Φ*2 X*) ∈ *CPTP n d*
⟨*proof*⟩

end
theory *Day22-Kraus-Independence*
  imports *QC-Prelims Jordan-Normal-Form.DL-Rank*
begin

52 Day 22: linear independence bounds the number of Kraus operators (D22-6)

A linearly independent list of vectors in $\mathbb{C}^n$ has at most n members. The bridge is to JNF's *vec-space*: kernel-triviality of *mat-of-cols* is JNF's *lin-indpt* (via *vec-space.lin-depE*), and *li-le-dim* bounds the cardinality.

lemma *lin-indep-vecs-distinct*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *distinct vs*
⟨*proof*⟩

lemma *lin-indep-vecs-length-le*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *length vs* ≤ *n*
⟨*proof*⟩

Flattening a matrix into a vector carries *mat-lin-indep-complex* to *lin-indep-vecs*, so a linearly independent family in M_n has at most n^2 members.

definition *flatten-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* **where**
  flatten-mat n B = *Matrix.vec* (*n*∗*n*) (λ*k*. *B* $$ (*k div n*, *k mod n*))

lemma *flatten-mat-dim* [*simp*]: *dim-vec* (*flatten-mat n B*) = *n*∗*n*
  ⟨*proof*⟩

lemma *flatten-mat-index*:
  assumes *i* < *n* **and** *j* < *n*
  shows *flatten-mat n B* $ (*i*∗*n* + *j*) = *B* $$ (*i*,*j*)
⟨*proof*⟩

lemma *mat-lin-indep-complex-flatten*:
  assumes *li*: *mat-lin-indep-complex n Bs*
  shows *lin-indep-vecs* (*n*∗*n*) (*map* (*flatten-mat n*) *Bs*)
⟨*proof*⟩

lemma *mat-lin-indep-complex-length-le*:
  assumes *mat-lin-indep-complex n Bs*
  shows *length Bs* ≤ *n*∗*n*
  ⟨*proof*⟩

Corollary to Choi's Theorem 3 (D22-6): an extreme CPTP map $M_n \to M_d$ with a minimal Kraus representation $(V_i)_{i<r}$ has $r^2 \le n^2$, i.e. $\mathrm{cr}(\Phi) \le n$.

corollary *extreme-CPTP-kraus-length-le*:
  assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
    and Φ-*in*: Φ ∈ *CPTP n d*
    and *rep*: *cptp-kraus-rep n d* Φ *Vs*
    and *minimal*: *lin-indep-complex-mat d n Vs*
    and *ext*: *extreme-CPTP-carrier n d* Φ
  shows *length Vs* ≤ *n*
⟨*proof*⟩

corollary *extreme-CPTP-choi-rank-le*:
  assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
    and Φ-*in*: Φ ∈ *CPTP n d*
    and *rep*: *cptp-kraus-rep n d* Φ *Vs*
    and *minimal*: *lin-indep-complex-mat d n Vs*
    and *ext*: *extreme-CPTP-carrier n d* Φ

shows *choi-rank n d* $\Phi \leq n$
⟨*proof*⟩

end
theory *Day22-Minimal-Kraus*
imports *Day32-Recovery-Proposition*
begin

53 Day 22: a minimal Kraus representation is linearly independent (D22-1)

53.1 Mixing a Kraus list by an isometry does not change the map

definition *kraus-mix* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
kraus-mix d n U Vs =
map (λ*l*. *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)) [*0*..<*dim-row U*]

lemma *kraus-mix-length* [*simp*]: *length* (*kraus-mix d n U Vs*) = *dim-row U*
⟨*proof*⟩

lemma *kraus-mix-nth*:
assumes *l* < *dim-row U*
shows *kraus-mix d n U Vs* ! *l* = *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)
⟨*proof*⟩

lemma *kraus-mix-carrier*:
assumes ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows ∀ *W* ∈ *set* (*kraus-mix d n U Vs*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

lemma *kraus-mix-entry*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *l*: *l* < *dim-row U* **and** *i*: *i* < *d* **and** *a*: *a* < *n*
shows *kraus-mix d n U Vs* ! *l* $$ (*i*,*a*) = (∑ *j* = *0*..<*length Vs*. *U* $$ (*l*,*j*) ∗ *Vs* ! *j* $$ (*i*,*a*))
⟨*proof*⟩

lemma *kraus-mix-mat-of-cols*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *dpos*: *0* < *d*
shows *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) (*kraus-mix d n U Vs*))
= *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*) ∗ *transpose-mat U*
⟨*proof*⟩

lemma *kraus-mix-same-map*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *UU*: (*U*†) ∗ *U* = *1*$_m$ (*length Vs*)
and *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*kraus-mix d n U Vs*) *M* = *kraus-map-dim d Vs M*
⟨*proof*⟩

53.2 A unitary with a prescribed first row

lemma *mat-mult-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r s B* ∈ *carrier-mat s t k* < *r j* < *t*
shows (*A* ∗ *B*) $$ (*k*,*j*) = (∑ *l*<*s*. *A* $$ (*k*,*l*) ∗ *B* $$ (*l*,*j*))

⟨*proof*⟩

lemma *transpose-unitary-isometry*:
assumes *Mc*: $M \in$ *carrier-mat r r* **and** *MM*: $M * (M^\dagger) =$ *1*$_m$ *r*
shows $((\textit{transpose-mat } M)^\dagger) *$ *transpose-mat M* = *1*$_m$ *r*
⟨*proof*⟩

lemma *isometry-with-first-row*:
assumes *ud*: *dim-vec u* = *r* **and** *un*: $\|u\| = 1$
shows $\exists$ *U* :: *complex Matrix.mat*. *U* $\in$ *carrier-mat r r* $\wedge$ $(U^\dagger) * U =$ *1*$_m$ *r* $\wedge$ $(\forall j < r.$ *U* $$ (*0*,*j*) = *u* $ *j*)
⟨*proof*⟩

53.3 D22-1

lemma *kraus-map-dim-zero-head*:
assumes *Ws-c*: $\forall$ *W* $\in$ *set Ws*. *W* $\in$ *carrier-mat d n* **and** *Xc*: *M* $\in$ *carrier-mat n n*
shows *kraus-map-dim d* (*0*$_m$ *d n* # *Ws*) *M* = *kraus-map-dim d Ws M*
⟨*proof*⟩

If $\Phi(X) = \sum_{j<r} V_j X V_j^*$ with $r = \mathrm{cr}(\Phi)$, then $V_0, \ldots, V_{r-1}$ are linearly independent: a dependency $\sum_j c_j V_j = 0$ would allow a unitary change of representation whose first Kraus operator is 0, giving a representation with $r - 1$ terms.

theorem *minimal-kraus-rep-lin-indep*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: $\forall$ *V* $\in$ *set Vs*. *V* $\in$ *carrier-mat d n*
and *minimal*: *choi-rank n d* (*kraus-map-dim d Vs*) = *length Vs*
shows *lin-indep-complex-mat d n Vs*
⟨*proof*⟩

end
theory *Day22-Adjoint-Maps*
imports *Day19-Choi-TP-Unital Day19-Choi-Positivity Day26-CP-Factorization*
begin

54 Day 22: the adjoint map, an affine bijection from UCP onto CPTP (D22-5)

The notes: $\mathrm{UCP}(M_n, M_d)$ and $\mathrm{CPTP}(M_d, M_n)$ are convex, and $\Gamma(\Phi) = \Phi^*$ is an affine bijection between them with affine inverse, so the two convex sets are affinely isomorphic. The adjoint was available only for Kraus maps (*kraus-adjoint-dim*); here it is defined for every linear Φ. Maps are total functions on matrices, so Γ is a bijection up to agreement on M_n, resp. M_d: Γ sends linear UCP maps to linear CPTP maps and back, and $\Gamma(\Gamma(\Phi)) = \Phi$ on M_n. The set *UCP* does not record linearity, which the notes' maps have, so it is assumed.

For linear $\Phi : M_n \to M_d$, $\Phi^* : M_d \to M_n$ is determined by $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$; taking $X = E_{ij}$ gives $\Phi^*(Y)_{ij} = \langle \Phi(E_{ij})|Y\rangle$.

definition *map-adjoint* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
map-adjoint n d Φ *Y* = *Matrix.mat n n* (λ(*i*, *j*). ⟨⟨ Φ (*E*⟦*i*,*j* ; *n*,*n*⟧) | *Y* ⟩⟩)

lemma *map-adjoint-carrier* [*simp*]: *map-adjoint n d* Φ *Y* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *map-adjoint-dims* [*simp*]: *dim-row* (*map-adjoint n d* Φ *Y*) = *n dim-col* (*map-adjoint n d* Φ *Y*) = *n*
⟨*proof*⟩

lemma *lin-entry*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X* ∈ *carrier-mat n n* **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows Φ *X* $$ (*a*, *b*) = (∑ *i*<*n*. ∑ *j*<*n*. *X* $$ (*i*, *j*) ∗ Φ (*E*⟦*i*,*j* ; *n*,*n*⟧) $$ (*a*, *b*))
⟨*proof*⟩

theorem *map-adjoint-correct*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X* ∈ *carrier-mat n n* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *Y* | Φ *X* ⟩⟩ = ⟨⟨ *map-adjoint n d* Φ *Y* | *X* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-mat-unit*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *i*: *i* < *n* **and** *j*: *j* < *n*
shows ⟨⟨ *A* | *E*⟦*i*,*j* ; *n*,*n*⟧ ⟩⟩ = *cnj* (*A* $$ (*i*, *j*))
⟨*proof*⟩

lemma *hs-inner-prod-ext*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *B*: *B* ∈ *carrier-mat n n*
and *eq*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ ⟨⟨ *A* | *X* ⟩⟩ = ⟨⟨ *B* | *X* ⟩⟩
shows *A* = *B*
⟨*proof*⟩

lemma *map-adjoint-lin*: *lin-on d n* (*carrier-mat d d*) (*map-adjoint n d* Φ)
⟨*proof*⟩

lemma *map-adjoint-cong*:
assumes ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = Ψ *X*
shows *map-adjoint n d* Φ = *map-adjoint n d* Ψ
⟨*proof*⟩

For a Kraus map the adjoint is *kraus-adjoint-dim*.

lemma *map-adjoint-kraus*:
assumes *As*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows *map-adjoint n d* (*kraus-map-dim d As*) *Y* = *kraus-adjoint-dim n As Y*
⟨*proof*⟩

$\Phi^{**} = \Phi$.

theorem *map-adjoint-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X* ∈ *carrier-mat n n*
shows *map-adjoint d n* (*map-adjoint n d* Φ) *X* = Φ *X*
⟨*proof*⟩

lemma *cp-kraus-form*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *cp*: *completely-positive n d* Φ **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows ∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧ (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Ws X*)
⟨*proof*⟩

lemma *trace-preserving-cong*:
assumes ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = Ψ *X*
shows *trace-preserving n* Φ ⟷ *trace-preserving n* Ψ
⟨*proof*⟩

$\Gamma(\Phi) = \Phi^*$ maps the unital CP maps $M_n \to M_d$ to the CPTP maps $M_d \to M_n$ and back.

theorem *adjoint-ucp-imp-cptp*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *U*: Φ ∈ *UCP n d* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *map-adjoint n d* Φ ∈ *CPTP d n*
⟨*proof*⟩

theorem *adjoint-cptp-imp-ucp*:
assumes *L*: *lin-on d n* (*carrier-mat d d*) Ψ **and** *C*: Ψ ∈ *CPTP d n* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *map-adjoint d n* Ψ ∈ *UCP n d*
⟨*proof*⟩

Γ is affine (for real weights), and $\Gamma^{-1} = \Gamma$ by *map-adjoint-adjoint*.

lemma *hs-inner-prod-left-real-comb*:
fixes *t s* :: *real*
assumes *A*: *A* ∈ *carrier-mat d d* **and** *B*: *B* ∈ *carrier-mat d d* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *complex-of-real t* ·$_m$ *A* + *complex-of-real s* ·$_m$ *B* | *Y* ⟩⟩
= *complex-of-real t* ∗ ⟨⟨ *A* | *Y* ⟩⟩ + *complex-of-real s* ∗ ⟨⟨ *B* | *Y* ⟩⟩
⟨*proof*⟩

theorem *map-adjoint-affine*:
fixes *t* :: *real*
assumes *L1*: *lin-on n d* (*carrier-mat n n*) *Φ1* **and** *L2*: *lin-on n d* (*carrier-mat n n*) *Φ2* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows *map-adjoint n d* (λ*X*. *complex-of-real t* ·$_m$ *Φ1 X* + *complex-of-real* (*1* − *t*) ·$_m$ *Φ2 X*) *Y*
= *complex-of-real t* ·$_m$ *map-adjoint n d Φ1 Y* + *complex-of-real* (*1* − *t*) ·$_m$ *map-adjoint n d Φ2 Y*
⟨*proof*⟩

end
theory *Day22-UCP-Compact*
imports *Day16-Block-Form Day19-Choi-TP-Unital Day27-Positive-Map-Criterion*
begin

55 Day 22: UCP(M_n, M_d) is compact (D22-4)

The notes: UCP(M_n, M_d) is convex and compact, closed because the conditions are closed and bounded because $\Phi(I) = I$ forces $0 \le \Phi(E_{ii}) \le I$. The space $L(M_n, M_d)$ carries no topology in this development; the Choi matrix is a linear bijection onto M_{nd}, so the statement is made in Choi coordinates: the Choi matrices of linear UCP maps are exactly the positive P whose diagonal blocks sum to I_d (*ucp-choi-image*, with the inverse Choi map *choi-inv*), and that set is compact (*compact-ucp-choi-set*): it is closed, and it lies in d times the compact set of unit-trace positive matrices.

In Choi coordinates a linear map $\Phi : M_n \to M_d$ is UCP iff $P_\Phi \ge 0$ and $\sum_i R_{ii} = I_d$ (*choi-ucp-iff*); the set of such P is compact. Dimensions are cardinalities of finite types, $n = |'n|$, $d = |'d|$, so that the typed matrices of *Separable-Cone* (a Euclidean space) can be used.

definition *ucp-choi-set* :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat set* **where**
ucp-choi-set = {*M*. *psd-lmat M* ∧ (∀ *a* < *CARD*(*'d*). ∀ *b* < *CARD*(*'d*).
(∑ *i*<*CARD*(*'n*). *unwrap-lm M* $$ (*i* ∗ *CARD*(*'d*) + *a*, *i* ∗ *CARD*(*'d*) + *b*)) = (*if a* = *b then 1 else 0*))}

lemma *closed-block-sum-eq*:
assumes *a*: *a* < *CARD*(*'d* :: *finite*) **and** *b*: *b* < *CARD*(*'d*)
shows *closed* {*M* :: (*complex*, *'n* :: *finite* × *'d*, *'n* × *'d*) *lmat*.
(∑ *i*<*CARD*(*'n*). *unwrap-lm M* $$ (*i* ∗ *CARD*(*'d*) + *a*, *i* ∗ *CARD*(*'d*) + *b*)) = *c*}
⟨*proof*⟩

lemma *closed-ucp-choi-set*: *closed* (*ucp-choi-set* :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat set*)
⟨*proof*⟩

lemma *ucp-choi-set-trace*:
assumes $M \in$ (*ucp-choi-set* :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat set*)
shows *Tr-lmat M* = *of-nat CARD*('*d*)
⟨*proof*⟩

theorem *compact-ucp-choi-set*: *compact* (*ucp-choi-set* :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat set*)
⟨*proof*⟩

The inverse of the Choi map: $\Phi_P(X) = \sum_{i,j} x_{ij} P_{ij}$ with P_{ij} the (i,j) block.

definition *choi-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat*
where
choi-inv n d P X = *Matrix.mat d d* (λ(*a*, *b*). ∑*i*<*n*. ∑*j*<*n*. *X* $$ (*i*, *j*) ∗ *P* $$ (*i* ∗ *d* + *a*, *j* ∗ *d* + *b*))

lemma *choi-inv-lin*: *lin-on n d* (*carrier-mat n n*) (*choi-inv n d P*)
⟨*proof*⟩

lemma *choi-matrix-choi-inv*:
assumes *P*: *P* ∈ *carrier-mat* (*n* ∗ *d*) (*n* ∗ *d*)
shows *choi-matrix n d* (*choi-inv n d P*) = *P*
⟨*proof*⟩

lemma *block-sum-entry*:
assumes *P*: *P* ∈ *carrier-mat* (*n* ∗ *d*) (*n* ∗ *d*) **and** *a*: *a* < *d* **and** *b*: *b* < *d*
shows *msum d d* (λ*i*. *block-of-mat d P i i*) [*0*..<*n*] $$ (*a*, *b*) = (∑*i*<*n*. *P* $$ (*i* ∗ *d* + *a*, *i* ∗ *d* + *b*))
⟨*proof*⟩

UCP in Choi coordinates is exactly *ucp-choi-set*, so it is compact.

theorem *ucp-choi-image*:
(λΦ. *wrap-lm* (*choi-matrix CARD*('*n*) *CARD*('*d*) Φ) :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat*)
‘ {Φ. *lin-on CARD*('*n*) *CARD*('*d*) (*carrier-mat CARD*('*n*) *CARD*('*n*)) Φ ∧ Φ ∈ *UCP CARD*('*n*) *CARD*('*d*)}
= *ucp-choi-set*
⟨*proof*⟩

corollary *compact-ucp*:
compact ((λΦ. *wrap-lm* (*choi-matrix CARD*('*n*) *CARD*('*d*) Φ) :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat*)
‘ {Φ. *lin-on CARD*('*n*) *CARD*('*d*) (*carrier-mat CARD*('*n*) *CARD*('*n*)) Φ ∧ Φ ∈ *UCP CARD*('*n*) *CARD*('*d*)})
⟨*proof*⟩

end

Part III

Lectures 23–28: extreme points, operator systems, Arveson's theorem, entanglement

theory *Day23-ChoiExtremeUCP*
imports
Day22-Convex-Sets
begin

56 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP

Day 22 stated and Day 23 proves:

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $UCP(M_n, M_d)$ if and only if $\{V_i^* V_j : 1 \le i, j \le r\} \subseteq M_d$ is linearly independent.

Corollary (notes). If $\Phi : M_d \to M_n$ is completely positive and trace preserving, $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent (apply the theorem to Φ^*).

The notes' proof. ($\Rightarrow$) If $\sum_{ij} \lambda_{ij} V_i^* V_j = 0$ it suffices to treat Hermitian $\mu = (\mu_{ij})$ with $-I \le \mu \le I$: writing $I \pm \mu = (\alpha_{ij})^*(\alpha_{ij})$ and $W_i = \sum_j \alpha_{ij} V_j$ shows that $\psi_\pm(X) = \sum_i V_i^* X V_i \pm \sum_{ij} \mu_{ij} V_i^* X V_j$ are UCP with $\Phi = \frac{1}{2}(\psi_+ + \psi_-)$, so extremality gives $\psi_+ = \Phi$, and Choi 2 (unitary freedom) forces $I + \mu = U^*U = I$, i.e. $\mu = 0$. ($\Leftarrow$) If $\Phi = \frac{1}{2}(\psi_1 + \psi_2)$ with $\psi_k(X) = \sum_p W_p^* X W_p$, Choi 2 gives $W_p/\sqrt{2} = \sum_j \alpha_{pj} V_j$ with $U^*U = I_r$; unitality of ψ_1 and linear independence of the $V_i^* V_j$ force $U_1^* U_1 = \frac{1}{2} I_r$, whence $\psi_1 = \Phi = \psi_2$. Examples: $X \mapsto U^* X U$ is extreme; $\frac{1}{2}(U_1^* X U_1 + U_2^* X U_2)$, $X \mapsto \mathrm{tr}(X) I_d$ and $X \mapsto \mathrm{diag}(X)$ are not.
What is formalised here is the corollary, i.e. the CPTP form, for a fixed Kraus list *Vs* of linearly independent operators (the consequence of $r = \mathrm{cr}(\Phi)$ proved on Day 22): extremality (in the carrier sense *extreme-CPTP-carrier*) holds if and only if the products $V_j^\dagger V_i$ are linearly independent over $\mathbb{C}$ (*choi-theorem-3-cptp-iff*). The direction ($\Rightarrow$) follows the notes' perturbation argument (*kraus-quad-map*, *small-eps-psd*); the direction ($\Leftarrow$) replaces Choi 2 by a CP Radon–Nikodym theorem proved below from the Moore–Penrose pseudoinverse of the Choi matrix. Unitary freedom itself is *choi-2-unitary-freedom* in *Day21-Unitary-Freedom*, proved afterwards and independently of this theory; the examples are in theory Day23_Extremality_Examples (the UCP form is not formalised).

56.1 Extreme UCP maps

An *extreme UCP map* is one that cannot be written as a proper convex combination of two distinct UCP maps. Following Day-22's *extreme-point-mat*, we use the half-half mid-point formulation; this is equivalent to the standard definition for convex sets.

definition *extreme-UCP* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
extreme-UCP n d Φ ≡
Φ ∈ *UCP n d* ∧
¬ (∃ Φ₁ ∈ *UCP n d*. ∃ Φ₂ ∈ *UCP n d*.
Φ₁ ≠ Φ ∧ Φ ≠ Φ₂ ∧
(∀ *X*. *X* ∈ *carrier-mat n n*
⟶ Φ *X* = (*1/2* :: *complex*) ·$_m$ Φ₁ *X*
+ (*1/2* :: *complex*) ·$_m$ Φ₂ *X*))

lemma *extreme-UCP-member*:

assumes *extreme-UCP n d* Φ
shows $\Phi \in$ *UCP n d*
⟨*proof*⟩

Trivial unfolding: an extreme UCP map satisfies no non-trivial mid-point decomposition.

lemma *extreme-UCP-no-split*:
assumes *ext*: *extreme-UCP n d* Φ
and Φ*1-in*: $\Phi_1 \in$ *UCP n d*
and Φ*2-in*: $\Phi_2 \in$ *UCP n d*
and Φ*1-neq*: $\Phi_1 \neq \Phi$
and Φ*2-neq*: $\Phi \neq \Phi_2$
shows $\exists X \in$ *carrier-mat n n*.
$\Phi\ X \neq (1/2 :: complex) \cdot_m \Phi_1\ X + (1/2 :: complex) \cdot_m \Phi_2\ X$
⟨*proof*⟩

56.2 UCP-Kraus representation

A *UCP Kraus representation* for a map $\Phi : M_n \to M_d$ is a list *Vs* of $d \times n$ matrices such that

- each $V \in Vs$ is $d \times n$;
- the unitality identity $\sum_i\ V_i\ {V_i}^\dagger = I_d$ holds;
- $\Phi\ X = \sum_i\ V_i\ X\ {V_i}^\dagger$ for every $X \in M_n$.

Note that the standard *kraus-operators* predicate of Day 17 uses the *trace-preserving* identity $\sum\ {V_i}^\dagger\ V_i = I_n$; the *unital* form $\sum\ V_i\ {V_i}^\dagger = I_d$ is the Heisenberg-dual condition appropriate for *UCP*.

definition *ucp-kraus-rep* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat list* ⇒ *bool*
where
ucp-kraus-rep n d Φ *Vs* ≡
(∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*) ∧
foldr (λ*V acc*. *V* ∗ (*V*†) + *acc*) *Vs* (0_m *d d*) = 1_m *d* ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Vs X*)

The *products set* of a Kraus list *Vs* is the list of $n \times n$ matrices ${V_j}^\dagger\ V_i$ for all pairs of indices (i,j).

definition *kraus-products* ::
complex Matrix.mat list ⇒ *complex Matrix.mat list*
where
kraus-products Vs ≡
concat (*map* (λ*i*. *map* (λ*j*. (*Vs*!*j*)† ∗ *Vs*!*i*) [*0*..<*length Vs*]) [*0*..<*length Vs*])

Real linear independence for a list of complex matrices. A list *Bs* is independent over $\mathbb{R}$ iff the only $\mathbb{R}$-coefficients making $\sum_k\ c_k\ B_k = 0$ are all zero. We use the real-coefficient form because Choi's perturbation argument constructs real combinations of hermitian generators.

definition *mat-lin-indep-real* ::
nat ⇒ *complex Matrix.mat list* ⇒ *bool*
where
mat-lin-indep-real n Bs ≡
(∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n n*) ∧
(∀ *c* :: *nat* ⇒ *real*.

foldr (λk *acc. complex-of-real* (c k) $\cdot_m$ ($Bs!k$) + *acc*)
[0..<*length Bs*] (0_m n n)
= 0_m n n
$\longrightarrow$ ($\forall\, k <$ *length Bs*. c $k = 0$))

Complex linear independence: the classical Choi statement uses **complex**-coefficient linear independence. This is strictly stronger than *mat-lin-indep-real*.

definition *mat-lin-indep-complex* ::
nat $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *bool*
where
mat-lin-indep-complex n Bs $\equiv$
($\forall\, B \in$ *set Bs*. $B \in$ *carrier-mat n n*) $\wedge$
($\forall\, c ::$ *nat* $\Rightarrow$ *complex*.
foldr (λk *acc*. c k $\cdot_m$ ($Bs!k$) + *acc*)
[0..<*length Bs*] (0_m n n)
= 0_m n n
$\longrightarrow$ ($\forall\, k <$ *length Bs*. c $k = 0$))

Complex linear independence implies real.

lemma *mat-lin-indep-complex-imp-real*:
assumes *mat-lin-indep-complex n Bs*
shows *mat-lin-indep-real n Bs*
⟨*proof*⟩

56.3 Unital condition from the Kraus list

If *Ws* is a list of $d \times n$ matrices, then evaluating the Kraus map at the identity 1_m n gives exactly the sum $\sum$ W $W^\dagger$. Combined with unitality, this gives the $\sum$ W $W^\dagger = 1_m$ d identity that distinguishes UCP from plain CP.

lemma *kraus-map-dim-one*:
assumes $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*
shows *kraus-map-dim d Ws* (1_m n)
= *foldr* (λW *acc*. $W * (W^\dagger)$ + *acc*) *Ws* (0_m d d)
⟨*proof*⟩

56.4 UCP-Kraus existence

Every linear UCP map admits a UCP Kraus representation of length n d. Linearity, scalar-multiplication and range hypotheses match those of *kraus-rep-of-psd-choi*; the unital condition $\sum$ V $V^\dagger = 1_m$ d is supplied by Φ (1_m n) = 1_m d + *kraus-map-dim-one*.

lemma *ucp-has-kraus-rep*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B$. $A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$
Φ (A + B) = Φ A + Φ B
and *smult*: $\bigwedge c\ A$. $A \in$ *carrier-mat n n* $\Longrightarrow$ Φ ($c \cdot_m A$) = $c \cdot_m \Phi$ A
and *range*: $\forall\, A$. $A \in$ *carrier-mat n n* $\longrightarrow$ Φ $A \in$ *carrier-mat d d*
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
and *ucp*: $\Phi \in$ *UCP n d*
shows $\exists$ *Vs* :: *complex Matrix.mat list*.
ucp-kraus-rep n d Φ *Vs* $\wedge$ *length Vs* = $n * d$
⟨*proof*⟩

56.5 Sanity facts about *kraus-products*

lemma *kraus-products-carrier*:
assumes $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*

shows $\forall B \in set\ (kraus\text{-}products\ Vs).\ B \in carrier\text{-}mat\ n\ n$
⟨*proof*⟩

56.6 Quadratic Kraus map (perturbation primitive)

For a Kraus list Vs of $d \times n$ matrices and a coefficient matrix $T \in M_k$ (where $k = length\ Vs$), the *quadratic Kraus map* is $(kraus\text{-}quad\text{-}map\ d\ Vs\ T)\ X = \sum_{\{i,j\}}\ T_{\{ij\}}\ V_i\ X\ V_j{}^\dagger$. When $T = I_k$ this reduces to the ordinary Kraus map $kraus\text{-}map\text{-}dim\ d\ Vs\ X = \sum\ V_i\ X\ V_i{}^\dagger$; for PSD T the result is again a Kraus map (Lemma A below). The perturbations $T_\pm = I \pm \varepsilon\ A$ for small ε and hermitian A will provide the *splittings* witnessing non-extremity.

definition *kraus-quad-row* ::
  $nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat \Rightarrow$
    $complex\ Matrix.mat \Rightarrow nat \Rightarrow complex\ Matrix.mat$
where
  $kraus\text{-}quad\text{-}row\ d\ Vs\ Tm\ rho\ i \equiv$
    $foldr\ (\lambda j\ acc.\ Tm\ \$\$\ (i,\ j)\ \cdot_m\ (Vs!i * rho * ((Vs!j)^\dagger))\ +\ acc)$
      $[0..<length\ Vs]\ (0_m\ d\ d)$

definition *kraus-quad-map* ::
  $nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat \Rightarrow$
    $complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$
where
  $kraus\text{-}quad\text{-}map\ d\ Vs\ Tm\ rho \equiv$
    $foldr\ (\lambda i\ acc.\ kraus\text{-}quad\text{-}row\ d\ Vs\ Tm\ rho\ i\ +\ acc)$
      $[0..<length\ Vs]\ (0_m\ d\ d)$

Generic carrier-preservation lemma for accumulating folds over $[0..<N]$: if each per-index term lies in M_d, so does the fold.

lemma *foldr-upt-add-carrier*:
  fixes $f :: nat \Rightarrow complex\ Matrix.mat$
  assumes $\forall k < N.\ f\ k \in carrier\text{-}mat\ d\ d$
  shows $foldr\ (\lambda k\ acc.\ f\ k\ +\ acc)\ [0..<N]\ (0_m\ d\ d) \in carrier\text{-}mat\ d\ d$
  ⟨*proof*⟩

lemma *kraus-quad-row-carrier*:
  assumes *Vs-c*: $\forall V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
    and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
    and *i-lt*: $i < length\ Vs$
  shows $kraus\text{-}quad\text{-}row\ d\ Vs\ Tm\ rho\ i \in carrier\text{-}mat\ d\ d$
⟨*proof*⟩

lemma *kraus-quad-map-carrier*:
  assumes *Vs-c*: $\forall V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
    and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
  shows $kraus\text{-}quad\text{-}map\ d\ Vs\ Tm\ rho \in carrier\text{-}mat\ d\ d$
⟨*proof*⟩

Per-entry index formula for the accumulating fold (with arbitrary base).

lemma *foldr-upt-add-acc-index*:
  fixes $f :: nat \Rightarrow complex\ Matrix.mat$
  assumes *carr*: $\forall k < N.\ f\ k \in carrier\text{-}mat\ d\ d$
    and *z-c*: $z \in carrier\text{-}mat\ d\ d$
    and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
  shows $foldr\ (\lambda k\ acc.\ f\ k\ +\ acc)\ [0..<N]\ z\ \$\$\ (r,\ c)$
      $= (\sum k = 0..<N.\ f\ k\ \$\$\ (r,\ c)) + z\ \$\$\ (r,\ c)$
  ⟨*proof*⟩

lemma *foldr-upt-add-index*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *carr*: $\forall k < N.\ f\ k \in carrier\text{-}mat\ d\ d$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $foldr\ (\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ (0_m\ d\ d)\ \$\$\ (r,\ c)$
$= (\sum k = 0..<N.\ f\ k\ \$\$\ (r,\ c))$
⟨*proof*⟩

Per-entry formula for *kraus-quad-row*.

lemma *kraus-quad-row-index*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *i-lt*: $i < length\ Vs$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $kraus\text{-}quad\text{-}row\ d\ Vs\ Tm\ rho\ i\ \$\$\ (r,\ c)$
$= (\sum j = 0..<length\ Vs.$
$Tm\ \$\$\ (i,\ j) * (Vs!i * rho * ((Vs!j)^\dagger))\ \$\$\ (r,\ c))$
⟨*proof*⟩

Per-entry formula for *kraus-quad-map*: double sum over indices.

lemma *kraus-quad-map-index*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ Tm\ rho\ \$\$\ (r,\ c)$
$= (\sum i = 0..<length\ Vs.$
$\sum j = 0..<length\ Vs.$
$Tm\ \$\$\ (i,\ j) * (Vs!i * rho * ((Vs!j)^\dagger))\ \$\$\ (r,\ c))$
⟨*proof*⟩

Carrier-preservation in a different dimension pair.

lemma *foldr-add-carrier-dn*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ d\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ d\ n) \in carrier\text{-}mat\ d\ n$
⟨*proof*⟩

Linear combination of Kraus operators with complex coefficients. *kraus-combo d n Vs u* is $\sum_i u_i\ V_i$, a $d \times n$ matrix.

definition *kraus-combo* ::
$nat \Rightarrow nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.vec \Rightarrow$
$complex\ Matrix.mat$
where
$kraus\text{-}combo\ d\ n\ Vs\ u \equiv$
$foldr\ (\lambda i\ acc.\ u\ \$\ i \cdot_m (Vs!i) + acc)\ [0..<length\ Vs]\ (0_m\ d\ n)$

lemma *kraus-combo-carrier*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows $kraus\text{-}combo\ d\ n\ Vs\ u \in carrier\text{-}mat\ d\ n$
⟨*proof*⟩

Per-entry formula for *kraus-combo*: foldr index lemma in d×n.

lemma *foldr-upt-add-acc-index-dn*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *carr*: $\forall k < N.\ f\ k \in carrier\text{-}mat\ d\ n$
and *z-c*: $z \in carrier\text{-}mat\ d\ n$

and *r-lt*: $r < d$ **and** *c-lt*: $c < n$
shows *foldr* $(\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ z$ \$\$ $(r,\ c)$
$= (\sum k = 0..<N.\ f\ k$ \$\$ $(r,\ c)) + z$ \$\$ $(r,\ c)$
⟨*proof*⟩

lemma *kraus-combo-index*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < n$
shows *kraus-combo d n Vs u* \$\$ $(r,\ c)$
$= (\sum i = 0..<length\ Vs.\ u$ \$ $i * (Vs!i)$ \$\$ $(r,\ c))$
⟨*proof*⟩

A direct per-entry formula for matrix products.

lemma *mat-mult-index-sum*:
assumes *A-c*: $A \in carrier\text{-}mat\ p\ q$
and *B-c*: $B \in carrier\text{-}mat\ q\ s$
and *r-lt*: $r < p$ **and** *c-lt*: $c < s$
shows $(A * B)$ \$\$ $(r,\ c) = (\sum k = 0..<q.\ A$ \$\$ $(r,\ k) * B$ \$\$ $(k,\ c))$
⟨*proof*⟩

Per-entry formula for $W * rho$, where $W = kraus\text{-}combo\ d\ n\ Vs\ u$.

lemma *kraus-combo-times-rho-index*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < d$ **and** *k-lt*: $k < n$
shows $(kraus\text{-}combo\ d\ n\ Vs\ u * rho)$ \$\$ $(r,\ k)$
$= (\sum i = 0..<length\ Vs.\ u$ \$ $i * (Vs!i * rho)$ \$\$ $(r,\ k))$
⟨*proof*⟩

The full quadratic per-entry formula: $(W\ rho\ W^\dagger)$ \$\$ $(r,c) = \sum_{\{i,j\}} u_i\ |u_j|\ (V_i\ rho\ {V_j}^\dagger)$ \$\$ (r,c).

lemma *kraus-combo-quad-index*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $(kraus\text{-}combo\ d\ n\ Vs\ u * rho * ((kraus\text{-}combo\ d\ n\ Vs\ u)^\dagger))$ \$\$ $(r,\ c)$
$= (\sum i = 0..<length\ Vs.\ \sum j = 0..<length\ Vs.$
u \$ $i * cnj\ (u$ \$ $j) * (Vs!i * rho * ((Vs!j)^\dagger))$ \$\$ $(r,\ c))$
⟨*proof*⟩

Linearity of *kraus-quad-map* in the coefficient matrix (needed for the rank-1 decomposition below).

lemma *kraus-quad-map-add-Tm*:
assumes *Vs-c*: $\forall\ V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *T1-c*: $T1 \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *T2-c*: $T2 \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
shows *kraus-quad-map d Vs (T1 + T2) rho*
$=$ *kraus-quad-map d Vs T1 rho + kraus-quad-map d Vs T2 rho*
⟨*proof*⟩

Carrier of *outer-sum k vs*.

lemma *outer-sum-carrier*:
outer-sum k vs $\in$ *carrier-mat k k*
⟨*proof*⟩

Cons-recursion for *outer-sum*: adds a rank-1 piece.

lemma *outer-sum-Cons-split*:
assumes $\forall\, v \in set\ (u \#\ us).\ dim\text{-}vec\ v = k$
shows *outer-sum k* $(u \#\ us)$
$= outer\text{-}sum\ k\ [u] + outer\text{-}sum\ k\ us$
⟨*proof*⟩

Rank-1 identity: *kraus-quad-map d Vs* (*outer-sum k* $[u]$) *rho* $= W\ rho\ W^\dagger$ where $W =$ *kraus-combo d n Vs u*.

lemma *kraus-quad-map-outer-singleton*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *u-dim*: $dim\text{-}vec\ u = length\ Vs$
shows *kraus-quad-map d Vs* (*outer-sum* (*length Vs*) $[u]$) *rho*
$= kraus\text{-}combo\ d\ n\ Vs\ u * rho * ((kraus\text{-}combo\ d\ n\ Vs\ u)^\dagger)$
⟨*proof*⟩

Vanishing of the quadratic map when the coefficient matrix is zero.

lemma *kraus-quad-map-zero-Tm*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
shows *kraus-quad-map d Vs* $(0_m\ (length\ Vs)\ (length\ Vs))\ rho = 0_m\ d\ d$
⟨*proof*⟩

Sum-decomposition: a PSD-style coefficient matrix *outer-sum k us* gives *kraus-quad-map* as a kraus_map_dim of the combos.

lemma *kraus-quad-map-outer-sum-eq-kraus-map-dim*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *us-dim*: $\forall\, v \in set\ us.\ dim\text{-}vec\ v = length\ Vs$
shows *kraus-quad-map d Vs* (*outer-sum* (*length Vs*) *us*) *rho*
$= kraus\text{-}map\text{-}dim\ d\ (map\ (kraus\text{-}combo\ d\ n\ Vs)\ us)\ rho$
⟨*proof*⟩

56.7 Lemma A: PSD coefficient matrix gives a CP perturbation

Since *amplification* only evaluates Φ on $n \times n$ blocks, two maps that agree on *carrier-mat n n* yield the same amplification, hence the same notion of complete positivity.

lemma *amplification-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Xm* :: *complex Matrix.mat*
assumes *eq*: $\bigwedge r.\ r \in carrier\text{-}mat\ n\ n \Longrightarrow Phi\ r = Psi\ r$
shows *amplification n d p Phi Xm* = *amplification n d p Psi Xm*
⟨*proof*⟩

lemma *completely-positive-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes $\bigwedge r.\ r \in carrier\text{-}mat\ n\ n \Longrightarrow Phi\ r = Psi\ r$
shows *completely-positive n d Phi* ⟷ *completely-positive n d Psi*
⟨*proof*⟩

Lemma A: if T is PSD then *kraus-quad-map d Vs T* is completely positive.

Proof sketch. Use $[\![?P \in carrier\text{-}mat\ ?n\ ?n;\ ?P \geq_M 0]\!] \Longrightarrow \exists\, vs.\ length\ vs = ?n \wedge (\forall\, v{\in}set\ vs.\ dim\text{-}vec\ v = ?n) \wedge outer\text{-}sum\ ?n\ vs = ?P$ to write $T =$ *outer-sum k us*. Then by $[\![\forall\, V{\in}set\ ?Vs.\ V \in carrier\text{-}mat\ ?d\ ?n;\ ?rho \in carrier\text{-}mat\ ?n\ ?n;\ \forall\, v{\in}set\ ?us.\ dim\text{-}vec\ v = length\ ?Vs]\!] \Longrightarrow kraus\text{-}quad\text{-}map\ ?d\ ?Vs\ (outer\text{-}sum\ (length\ ?Vs)\ ?us)\ ?rho = kraus\text{-}map\text{-}dim\ ?d\ (map\ (kraus\text{-}combo\ ?d\ ?n\ ?Vs)\ ?us)\ ?rho$, *kraus-quad-map d Vs T* agrees on *carrier-mat n n* with

kraus-map-dim d Ws, where *Ws* := *map* (*kraus-combo d n Vs*) *us*. Apply ⟦∀ *W*∈*set ?Ws*. *W* ∈ *carrier-mat ?d ?n*; *0* < *?n*; *0* < *?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*).

lemma *kraus-perturbation-cp*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *Tm-c*: *Tm* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
    and *Tm-psd*: *psd Tm*
    and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
  shows *completely-positive n d* (*kraus-quad-map d Vs Tm*)
⟨*proof*⟩

56.8 Small-ε PSD perturbation of the identity

For any hermitian $A : k \times k$, the matrix $1_m\ k + \delta \cdot_m A$ is PSD whenever $|\delta|$ is small enough. The bound uses the entry-wise sum $S := \sum_{\{i,j\}} |A_{\{ij\}}|$, taking $\varepsilon := 1/(1 + S)$; this avoids any operator-norm computation and just uses Cauchy-Schwarz on entries.

Bilinearity of the inner product in the right slot.

lemma *inner-prod-add-right*:
  fixes *u v w* :: *complex Matrix.vec*
  assumes *ud*: *dim-vec u* = *k* **and** *vd*: *dim-vec v* = *k* **and** *wd*: *dim-vec w* = *k*
  shows ⟨*u* | *v* + *w*⟩ = ⟨*u* | *v*⟩ + ⟨*u* | *w*⟩
⟨*proof*⟩

lemma *inner-prod-smult-right*:
  fixes *u v* :: *complex Matrix.vec*
  assumes *ud*: *dim-vec u* = *k* **and** *vd*: *dim-vec v* = *k*
  shows ⟨*u* | *c* ·$_v$ *v*⟩ = *c* ∗ ⟨*u* | *v*⟩
⟨*proof*⟩

The hermitian sum.

Entry-wise bound on the quadratic form of a hermitian matrix.

lemma *quad-form-bound-by-entry-sum*:
  fixes *A* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
  assumes *A-c*: *A* ∈ *carrier-mat k k* **and** *x-d*: *dim-vec x* = *k*
  shows *cmod* (⟨*x* | *A* ∗$_v$ *x*⟩)
        ≤ (∑ *i* = *0*..<*k*. ∑ *j* = *0*..<*k*. *cmod* (*A* $$ (*i*,*j*))) ∗ *Re* ⟨*x* | *x*⟩
⟨*proof*⟩

The main lemma: small ε keeps the perturbation PSD.

lemma *small-eps-psd*:
  fixes *A* :: *complex Matrix.mat*
  assumes *A-c*: *A* ∈ *carrier-mat k k* **and** *herm*: *hermitian A*
  shows ∃ *ε* > *0*. ∀ *d* :: *real*. |*d*| ≤ *ε* ⟶
                psd (*1*$_m$ *k* + *complex-of-real d* ·$_m$ *A*)
⟨*proof*⟩

56.9 Symmetric / anti-hermitian split of the coefficient matrix

The *dual* linear combination *kraus-pair-sum n Vs A* = $\sum_{\{i,j\}}$ *A*$_{\{ij\}}$ *V*$_j{}^\dagger$ *V*$_i$ is an $n \times n$ matrix. The dependence assumption ¬ *mat-lin-indep-real n* (*kraus-products Vs*) says that some real-coefficient A makes this sum vanish.

definition *kraus-pair-row* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
    nat ⇒ *complex Matrix.mat*

where
kraus-pair-row n Vs Amat i ≡
foldr (λ*j acc*. *Amat* $$ (*i*, *j*) ·$_m$ ((*Vs*!*j*)† ∗ *Vs*!*i*) + *acc*)
[*0*..<*length Vs*] (*0*$_m$ *n n*)

definition *kraus-pair-sum* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
complex Matrix.mat
where
kraus-pair-sum n Vs Amat ≡
foldr (λ*i acc*. *kraus-pair-row n Vs Amat i* + *acc*)
[*0*..<*length Vs*] (*0*$_m$ *n n*)

lemma *kraus-pair-row-carrier*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *i-lt*: *i* < *length Vs*
shows *kraus-pair-row n Vs Amat i* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-pair-sum-carrier*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows *kraus-pair-sum n Vs Amat* ∈ *carrier-mat n n*
⟨*proof*⟩

Per-entry index for the dual sum.

lemma *kraus-pair-row-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *i-lt*: *i* < *length Vs*
and *r-lt*: *r* < *n* **and** *c-lt*: *c* < *n*
shows *kraus-pair-row n Vs Amat i* $$ (*r*, *c*)
= (∑*j* = *0*..<*length Vs*.
Amat $$ (*i*, *j*) ∗ ((*Vs*!*j*)† ∗ *Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

lemma *kraus-pair-sum-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *r-lt*: *r* < *n* **and** *c-lt*: *c* < *n*
shows *kraus-pair-sum n Vs Amat* $$ (*r*, *c*)
= (∑*i* = *0*..<*length Vs*. ∑*j* = *0*..<*length Vs*.
Amat $$ (*i*, *j*) ∗ ((*Vs*!*j*)† ∗ *Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

Linearity in the coefficient matrix.

lemma *kraus-pair-sum-add-A*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *A1-c*: *A1* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *A2-c*: *A2* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
shows *kraus-pair-sum n Vs* (*A1* + *A2*)
= *kraus-pair-sum n Vs A1* + *kraus-pair-sum n Vs A2*
⟨*proof*⟩

lemma *kraus-pair-sum-smult-A*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Amat-c*: *Amat* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
shows *kraus-pair-sum n Vs* (*c* ·$_m$ *Amat*)
= *c* ·$_m$ *kraus-pair-sum n Vs Amat*
⟨*proof*⟩

Dagger identity: taking the dagger of *kraus-pair-sum n Vs A* swaps and conjugates the coefficient indices.

lemma *kraus-pair-sum-dagger*:
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *Amat-c*: *Amat* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
  shows (*kraus-pair-sum n Vs Amat*)†
      = *kraus-pair-sum n Vs*
        (*Matrix.mat* (*length Vs*) (*length Vs*)
          (λ(*i*,*j*). *cnj* (*Amat* $$ (*j*, *i*))))
⟨*proof*⟩

Existence of a hermitian witness: if *A-orig* (with real entries) gives *kraus-pair-sum n Vs A-orig* = *0* and *A-orig* $\neq$ *0*, then either the symmetric or the anti-hermitian part yields a non-zero hermitian matrix with the same property.

lemma *sym-or-antisym-witness*:
  fixes *A-orig* :: *complex Matrix.mat*
  assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
    and *A-c*: *A-orig* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
    and *A-real*: $\bigwedge i\ j$. *i* < *length Vs* $\Longrightarrow$ *j* < *length Vs* $\Longrightarrow$
          cnj (*A-orig* $$ (*i*, *j*)) = *A-orig* $$ (*i*, *j*)
    and *A-zero*: *kraus-pair-sum n Vs A-orig* = 0_m *n n*
    and *A-nontriv*: *A-orig* $\neq$ 0_m (*length Vs*) (*length Vs*)
  shows $\exists\, H$. *H* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
      $\wedge$ *hermitian H*
      $\wedge$ *H* $\neq$ 0_m (*length Vs*) (*length Vs*)
      $\wedge$ *kraus-pair-sum n Vs H* = 0_m *n n*
⟨*proof*⟩

56.10 Bridge: ¬ *mat-lin-indep-real* gives a real *A-orig* witness

Generic concat-nth for uniform-length rows.

lemma *concat-uniform-length*:
  assumes $\forall\, xs \in$ *set L*. *length xs* = *m*
  shows *length* (*concat L*) = *length L* $*$ *m*
  ⟨*proof*⟩

lemma *concat-uniform-nth*:
  assumes *uniform*: $\forall\, xs \in$ *set L*. *length xs* = *m*
    and *i-lt*: *i* < *length L*
    and *j-lt*: *j* < *m*
  shows *concat L* ! (*i* $*$ *m* + *j*) = *L* ! *i* ! *j*
  ⟨*proof*⟩

kraus-products Vs has length k^2 with *kraus-products Vs* ! (*i*$*$*k* + *j*) = (V_j)† V_i.

lemma *length-kraus-products*:
  length (*kraus-products Vs*) = *length Vs* $*$ *length Vs*
⟨*proof*⟩

lemma *kraus-products-nth*:
  assumes *i-lt*: *i* < *length Vs* **and** *j-lt*: *j* < *length Vs*
  shows *kraus-products Vs* ! (*i* $*$ *length Vs* + *j*) = (*Vs*!*j*)† $*$ *Vs*!*i*
⟨*proof*⟩

Per-entry of the foldr that mat_lin_indep_real builds.

lemma *foldr-smult-nth-index*:
  fixes *c* :: *nat* $\Rightarrow$ *real* **and** *Bs* :: *complex Matrix.mat list*

assumes *Bs-c*: $\forall\, B \in \mathit{set}\ \mathit{Bs}.\ B \in \mathit{carrier\text{-}mat}\ n\ n$
and *r-lt*: $r < n$ **and** *c-lt*: $c1 < n$
shows $\mathit{foldr}\ (\lambda k\ \mathit{acc}.\ \mathit{complex\text{-}of\text{-}real}\ (c\ k) \cdot_m (\mathit{Bs}\ !\ k) + \mathit{acc})$
$[0..<\mathit{length}\ \mathit{Bs}]\ (0_m\ n\ n)\ \$\$\ (r,\ c1)$
$= (\sum k = 0..<\mathit{length}\ \mathit{Bs}.$
$\mathit{complex\text{-}of\text{-}real}\ (c\ k) * (\mathit{Bs}\ !\ k)\ \$\$\ (r,\ c1))$
⟨*proof*⟩

Reindex $\sum_{\{}\kappa = 0..<n{\cdot}k\}$ via $\kappa = i{*}k + j$.

lemma *sum-split-prod-atLeastLessThan*:
fixes $f :: \mathit{nat} \Rightarrow {'a}{::}\mathit{comm\text{-}monoid\text{-}add}$
shows $(\sum \kappa = 0..<N * k.\ f\ \kappa) = (\sum i = 0..<N.\ \sum j = 0..<k.\ f\ (i * k + j))$
⟨*proof*⟩

The main bridge: a non-trivial real dependence yields a real, non-zero matrix *A-orig* for which *kraus-pair-sum* vanishes.

lemma *mat-lin-indep-real-neg-witness*:
assumes *Vs-c*: $\forall\, V \in \mathit{set}\ \mathit{Vs}.\ V \in \mathit{carrier\text{-}mat}\ d\ n$
and *npos*: $0 < n$
and *dep*: $\neg\ \mathit{mat\text{-}lin\text{-}indep\text{-}real}\ n\ (\mathit{kraus\text{-}products}\ \mathit{Vs})$
shows $\exists\, \mathit{A\text{-}orig} :: \mathit{complex\ Matrix.mat}.$
$\mathit{A\text{-}orig} \in \mathit{carrier\text{-}mat}\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
$\wedge\ (\forall\, i\ j.\ i < \mathit{length}\ \mathit{Vs} \longrightarrow j < \mathit{length}\ \mathit{Vs} \longrightarrow$
$\mathit{cnj}\ (\mathit{A\text{-}orig}\ \$\$\ (i,\ j)) = \mathit{A\text{-}orig}\ \$\$\ (i,\ j))$
$\wedge\ \mathit{A\text{-}orig} \neq 0_m\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
$\wedge\ \mathit{kraus\text{-}pair\text{-}sum}\ n\ \mathit{Vs}\ \mathit{A\text{-}orig} = 0_m\ n\ n$
⟨*proof*⟩

The same with complex coefficients: a non-trivial complex relation among the products $V_j^\dagger V_i$ gives a non-zero complex coefficient matrix with vanishing pair sum, and then (closure of the family under adjoints) a non-zero *Hermitian* one.

lemma *mat-lin-indep-complex-neg-witness*:
assumes *Vs-c*: $\forall\, V \in \mathit{set}\ \mathit{Vs}.\ V \in \mathit{carrier\text{-}mat}\ d\ n$
and *npos*: $0 < n$
and *dep*: $\neg\ \mathit{mat\text{-}lin\text{-}indep\text{-}complex}\ n\ (\mathit{kraus\text{-}products}\ \mathit{Vs})$
shows $\exists\, \mathit{A\text{-}orig} :: \mathit{complex\ Matrix.mat}.$
$\mathit{A\text{-}orig} \in \mathit{carrier\text{-}mat}\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
$\wedge\ \mathit{A\text{-}orig} \neq 0_m\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
$\wedge\ \mathit{kraus\text{-}pair\text{-}sum}\ n\ \mathit{Vs}\ \mathit{A\text{-}orig} = 0_m\ n\ n$
⟨*proof*⟩

lemma *herm-witness-of-complex*:
fixes *A-orig* :: *complex Matrix.mat*
assumes *Vs-c*: $\forall\, V \in \mathit{set}\ \mathit{Vs}.\ V \in \mathit{carrier\text{-}mat}\ d\ n$
and *A-c*: $\mathit{A\text{-}orig} \in \mathit{carrier\text{-}mat}\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
and *A-zero*: $\mathit{kraus\text{-}pair\text{-}sum}\ n\ \mathit{Vs}\ \mathit{A\text{-}orig} = 0_m\ n\ n$
and *A-nontriv*: $\mathit{A\text{-}orig} \neq 0_m\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
shows $\exists\, H.\ H \in \mathit{carrier\text{-}mat}\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
$\wedge\ \mathit{hermitian}\ H$
$\wedge\ H \neq 0_m\ (\mathit{length}\ \mathit{Vs})\ (\mathit{length}\ \mathit{Vs})$
$\wedge\ \mathit{kraus\text{-}pair\text{-}sum}\ n\ \mathit{Vs}\ H = 0_m\ n\ n$
⟨*proof*⟩

56.11 Auxiliary lemmas for the assembly

Scalar-multiplication linearity of *kraus-quad-map* in T.

lemma *kraus-quad-map-smult-Tm*:

assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *Tm-c*: $Tm \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ (c \cdot_m Tm)\ rho$
$= c \cdot_m kraus\text{-}quad\text{-}map\ d\ Vs\ Tm\ rho$
⟨*proof*⟩

The identity-coefficient case: *kraus-quad-map* with $1_m\ k$ reduces to the ordinary *kraus-map-dim*.

lemma *kraus-quad-map-one-eq-kraus-map-dim*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ (1_m\ (length\ Vs))\ rho = kraus\text{-}map\text{-}dim\ d\ Vs\ rho$
⟨*proof*⟩

The pair-sum at the identity coefficient is exactly *sum-wdag-w*.

lemma *kraus-pair-sum-one-eq-sum-wdag-w*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows $kraus\text{-}pair\text{-}sum\ n\ Vs\ (1_m\ (length\ Vs)) = sum\text{-}wdag\text{-}w\ n\ Vs$
⟨*proof*⟩

56.12 Minimality and the distinctness lemma (Lemma B)

Linear independence of a complex-matrix list (over $\mathbb{C}$). This is the *minimality* assumption needed to make the reverse direction of Choi's Theorem 3 go through: without it, perturbations of a non-minimal Kraus list can produce the same map.

definition *lin-indep-complex-mat* ::
$nat \Rightarrow nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow bool$
where
lin-indep-complex-mat d n Ws ≡
$(\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n) \wedge$
$(\forall\, c :: nat \Rightarrow complex.$
$foldr\ (\lambda k\ acc.\ c\ k \cdot_m (Ws!k) + acc)\ [0..<length\ Ws]\ (0_m\ d\ n)$
$= 0_m\ d\ n$
$\longrightarrow (\forall\, k < length\ Ws.\ c\ k = 0))$

Lemma B: under minimality of *Vs*, a non-zero coefficient matrix gives a non-vanishing quadratic Kraus map for some ϱ.

The witness $\varrho := mat\text{-}unit\ p\ q\ n\ n$ (a single 1 at (p,q)) makes $(V_i\ \varrho\ {V_j}^\dagger)$ \$\$ $(r,c) = V_i$ \$\$ $(r,p)\ |V_j$ \$\$ $(c,q)|$, so universal vanishing of *kraus-quad-map* on all (p,q,r,c) combined with linear independence of *Vs* (applied twice, separated by conjugation) forces $A = 0$.

A convenient form of minimality: if the per-entry sum vanishes for all (r,p), then all coefficients are zero.

lemma *lin-indep-zero-combo*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *minimal*: *lin-indep-complex-mat d n Vs*
and *coeff-kills*: $\bigwedge r\ p.\ r < d \Longrightarrow p < n \Longrightarrow$
$(\sum i = 0..<length\ Vs.\ c\ i * (Vs!i)$ \$\$ $(r,\ p)) = 0$
shows $\forall\, i < length\ Vs.\ c\ i = 0$
⟨*proof*⟩

The per-entry computation for the rank-1 $\varrho = mat\text{-}unit\ p\ q\ n\ n$.

lemma *kraus-quad-map-mat-unit-index*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *i-lt*: $i < length\ Vs$ **and** *j-lt*: $j < length\ Vs$

and *p-lt*: $p < n$ **and** *q-lt*: $q < n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $(Vs!i * mat\text{-}unit\ p\ q\ n\ n * ((Vs!j)^\dagger))\ \$\$\ (r,\ c)$
$= (Vs!i)\ \$\$\ (r,\ p) * cnj\ ((Vs!j)\ \$\$\ (c,\ q))$
⟨*proof*⟩

Lemma B. Under minimality, a non-zero coefficient matrix gives a non-vanishing quadratic Kraus map for some rank-1 ϱ.

lemma *kraus-quad-map-nonzero-witness*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *minimal*: *lin-indep-complex-mat d n Vs*
and *A-c*: $A \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *A-nz*: $A \neq 0_m\ (length\ Vs)\ (length\ Vs)$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\exists\, rho \in carrier\text{-}mat\ n\ n.\ kraus\text{-}quad\text{-}map\ d\ Vs\ A\ rho \neq 0_m\ d\ d$
⟨*proof*⟩

Trace-preservation lemma for *kraus-quad-map*.

The proof shows *Tr* (*kraus-quad-map d Vs T ϱ*) = *Tr* (*kraus-pair-sum n Vs T* ∗ *ϱ*) by per-entry sum manipulation and one application of ⟦*dim-col ?A* = *dim-row ?B*; *dim-row ?A* = *dim-col ?B*⟧ ⟹ *Tr* (*?A* ∗ *?B*) = *Tr* (*?B* ∗ *?A*) to flip $V_i\ \varrho\ {V_j}^\dagger$ into ${V_j}^\dagger\ V_i\ \varrho$; the TP conclusion follows when the pair-sum equals $1_m\ n$.

lemma *Tr-kraus-quad-map-eq*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *Tmat-c*: $Tmat \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
shows *Tr* (*kraus-quad-map d Vs Tmat rho*)
$=$ *Tr* (*kraus-pair-sum n Vs Tmat* ∗ *rho*)
⟨*proof*⟩

lemma *kraus-quad-map-is-TP*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *Tmat-c*: $Tmat \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *Tmat-pair-sum*: *kraus-pair-sum n Vs Tmat* = $1_m\ n$
and *npos*: $0 < n$
shows *trace-preserving n* (*kraus-quad-map d Vs Tmat*)
⟨*proof*⟩

56.13 CPTP-Kraus representation and extreme CPTP maps

A *cptp-kraus-rep n d* Φ *Vs* says *Vs* is a Kraus operator list (trace-preserving) and Φ equals the corresponding *kraus-map-dim* on every $X \in carrier\text{-}mat\ n\ n$.

definition *cptp-kraus-rep* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒
complex Matrix.mat list ⇒ *bool*
where
cptp-kraus-rep n d Φ *Vs* ≡
kraus-operators n d Vs ∧
(∀ *X*. $X \in carrier\text{-}mat\ n\ n \longrightarrow$ Φ $X = kraus\text{-}map\text{-}dim\ d\ Vs\ X$)

Extreme points of $CPTP(n,d)$: analog of *extreme-UCP*.

definition *extreme-CPTP* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
extreme-CPTP n d Φ ≡
Φ ∈ *CPTP n d* ∧

$\neg$ ($\exists \Phi_1 \in$ *CPTP n d*. $\exists \Phi_2 \in$ *CPTP n d*.
$\Phi_1 \neq \Phi \wedge \Phi \neq \Phi_2 \wedge$
($\forall X$. $X \in$ *carrier-mat n n*
$\longrightarrow \Phi\ X = (1/2 :: complex) \cdot_m \Phi_1\ X$
$+ (1/2 :: complex) \cdot_m \Phi_2\ X$))

56.14 Vectorisation of Kraus operators

The *vectorisation* of a $d \times n$ matrix V is the length-$n{*}d$ vector with r-th entry $V(r\ mod\ d,\ r\ div\ d)$. This is the column-stacking isomorphism $M_{\{d \times n\}} \cong \mathbb{C}^{\{nd\}}$ used in the Choi/Stinespring picture.

definition *vec-of-kraus* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.vec*
where
vec-of-kraus d n V = *Matrix.vec* (*n*d*) (λr. *V* $$ (*r mod d*, *r div d*))

lemma *vec-of-kraus-dim* [*simp*]: *dim-vec* (*vec-of-kraus d n V*) = *n*d*
⟨*proof*⟩

lemma *vec-of-kraus-index*:
assumes *r-lt*: *r* < *n*d*
shows *vec-of-kraus d n V* $ *r* = *V* $$ (*r mod d*, *r div d*)
⟨*proof*⟩

For $i < d$ and $a < n$, indexing the vectorisation at the *combined* index $a{*}d + i$ recovers the matrix entry $V(i,\ a)$.

lemma *vec-of-kraus-combined-index*:
assumes *V-c*: *V* $\in$ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *i-lt*: *i* < *d* **and** *dpos*: *0* < *d*
shows *vec-of-kraus d n V* $ (*a*d* + *i*) = *V* $$ (*i*, *a*)
⟨*proof*⟩

56.15 Choi matrix of a Kraus map

Key structural identity: the Choi matrix of a Kraus map $\Phi(X) = \sum_i\ V_i\ X\ {V_i}^\dagger$ equals the outer sum $\sum_i\ v_i\ {v_i}^\dagger$ where $v_i =$ *vec-of-kraus*(V_i). In particular, its range is the linear span of the vectorisations of *Vs*.

lemma *kraus-map-dim-mat-unit-entry*:
assumes *Vs-c*: $\forall$ *V* $\in$ *set Vs*. *V* $\in$ *carrier-mat d n*
and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
and *i-lt*: *i* < *d* **and** *j-lt*: *j* < *d*
shows (*kraus-map-dim d Vs* (*mat-unit a b n n*)) $$ (*i*, *j*)
= ($\sum$ *k*<*length Vs*. *Vs*!*k* $$ (*i*, *a*) * *cnj* (*Vs*!*k* $$ (*j*, *b*)))
⟨*proof*⟩

lemma *choi-matrix-kraus-eq*:
assumes *Vs-c*: $\forall$ *V* $\in$ *set Vs*. *V* $\in$ *carrier-mat d n*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (*kraus-map-dim d Vs*)
= *outer-sum* (*n*d*) (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

56.16 Choi matrix is linear in the map

The Choi matrix is a linear function of the underlying map (evaluated on the basis *mat-unit*'s). In particular, $P_{\{\Phi - \Psi\}} = P_\Phi - P_\Psi$.

lemma *choi-matrix-carrier* [*simp*]:
choi-matrix n d Φ ∈ *carrier-mat* (*n*∗*d*) (*n*∗*d*)
⟨*proof*⟩

lemma *choi-matrix-diff*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *Phi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* ∈ *carrier-mat d d*
and *Psi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (λ*X*. *Phi X* − *Psi X*)
= *choi-matrix n d Phi* − *choi-matrix n d Psi*
⟨*proof*⟩

The Choi matrix agrees on maps that agree on the carrier.

lemma *choi-matrix-cong*:
assumes *agree*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *Psi X*
shows *choi-matrix n d Phi* = *choi-matrix n d Psi*
⟨*proof*⟩

56.17 The Choi matrix of a *kraus-quad-map*

The Choi matrix of $X \mapsto \sum_{\{ij\}} T_{\{ij\}}\ V_i\ X\ {V_j}^\dagger$ equals $V\ T\ V^\dagger$ where V = *mat-of-cols* (*n*∗*d*) (*map vec-of-kraus Vs*).

This is the bridge from the classical dominated-PSD parametrisation back to a Kraus quadratic form.

definition *kraus-vec-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat*
where *kraus-vec-mat d n Vs* = *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*)

lemma *kraus-vec-mat-carrier* [*simp*]:
kraus-vec-mat d n Vs ∈ *carrier-mat* (*n*∗*d*) (*length Vs*)
⟨*proof*⟩

lemma *kraus-vec-mat-index*:
assumes *r-lt*: *r* < *n*∗*d* **and** *k-lt*: *k* < *length Vs*
shows *kraus-vec-mat d n Vs* $$ (*r*, *k*) = *vec-of-kraus d n* (*Vs*!*k*) $ *r*
⟨*proof*⟩

lemma *kraus-vec-mat-dagger-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *k-lt*: *k* < *length Vs* **and** *c-lt*: *c* < *n*∗*d*
shows (*kraus-vec-mat d n Vs*)† $$ (*k*, *c*) = *cnj* (*vec-of-kraus d n* (*Vs*!*k*) $ *c*)
⟨*proof*⟩

The Choi matrix of *kraus-quad-map d Vs T* is the sandwich $V\ T\ V^\dagger$, where V = *kraus-vec-mat d n Vs*.

lemma *choi-matrix-kraus-quad-eq*:
fixes *Tmat* :: *complex Matrix.mat*
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *T-c*: *Tmat* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (*kraus-quad-map d Vs Tmat*)
= (*kraus-vec-mat d n Vs*) ∗ *Tmat* ∗ ((*kraus-vec-mat d n Vs*)†)
⟨*proof*⟩

56.18 CP Radon–Nikodym — the dominated-PSD kernel

Dominated PSD parametrisation. This is the genuinely classical fact behind CP Radon–Nikodym: any PSD matrix B dominated by the outer sum $\sum\ v_i\ {v_i}^\dagger$ can be sandwiched as $V\ T\ V^\dagger$ for some PSD T, where $V = \textit{mat-of-cols}\ k\ \textit{vs}$.

Geometric content: $\textit{range}(B) \subseteq \textit{range}(\textit{outer-sum}\ k\ \textit{vs})$ (equivalent to B being Loewner-below outer_sum k vs after PSD), and that range is spanned by the columns of V. Pulling B back through that span gives T.

Proved below as *dominated-outer-sum-param* through the Moore–Penrose pseudoinverse of the outer sum (no axioms).

Loewner–kernel inclusion. If $A \geq B \geq 0$ in Loewner order, then every vector annihilated by A is also annihilated by B. Used to derive the range inclusion $\textit{range}(B) \subseteq \textit{range}(A)$.

Proof structure:

1. $\langle x \mid A\ x\rangle = 0$ from $A\ x = 0$.
2. Linearity of the inner product: $\langle x \mid A\ x\rangle = \langle x \mid B\ x\rangle + \langle x \mid (A-B)\ x\rangle$.
3. Both summands have non-negative real part (PSD); the sum is 0, so each is 0.
4. Hermiticity of B makes $\langle x \mid B\ x\rangle$ real, so it equals 0.
5. Decompose $B = N\ N^\dagger$ via $[\![?P \in \textit{carrier-mat}\ ?n\ ?n;\ ?P \geq_M 0]\!] \Longrightarrow \exists\, \textit{vs}.\ \textit{length}\ \textit{vs} = ?n \wedge (\forall\, v{\in}\textit{set}\ \textit{vs}.\ \textit{dim-vec}\ v = ?n) \wedge \textit{outer-sum}\ ?n\ \textit{vs} = ?P$, then $\langle x \mid B\ x\rangle = \langle N^\dagger\ x \mid N^\dagger\ x\rangle = 0$ forces $N^\dagger\ x = 0$, hence $B\ x = N\ (N^\dagger\ x) = 0$.

lemma *loewner-ker-subset*:
  fixes $A\ B$ :: *complex Matrix.mat* **and** x :: *complex Matrix.vec*
  assumes *A-c*: $A \in \textit{carrier-mat}\ k\ k$
    and *B-c*: $B \in \textit{carrier-mat}\ k\ k$
    and *B-psd*: $\textit{psd}\ B$
    and *diff-psd*: $\textit{psd}\ (A - B)$
    and *x-dim*: $\textit{dim-vec}\ x = k$
    and *Ax-zero*: $A *_v x = 0_v\ k$
  shows $B *_v x = 0_v\ k$
⟨*proof*⟩

PSD congruence. If P is PSD and M is any matrix of compatible shape, then $M^\dagger\ P\ M$ is PSD. This is the standard "sandwich" fact, generalising $?M \in \textit{carrier-mat}\ ?n\ ?m \Longrightarrow ?M^\dagger * ?M \geq_M 0$ (the case $P = 1_m$).

lemma *psd-conjugate*:
  fixes P :: *complex Matrix.mat* **and** M :: *complex Matrix.mat*
  assumes *P-c*: $P \in \textit{carrier-mat}\ k\ k$
    and *P-psd*: $\textit{psd}\ P$
    and *M-c*: $M \in \textit{carrier-mat}\ k\ n$
  shows $\textit{psd}\ (M^\dagger * P * M)$
⟨*proof*⟩

56.18.1 Diagonal pseudo-inverse list and its identity

Pseudo-inverse of a real list (interpreted as a diagonal). Zero entries are kept at zero; non-zero entries are reciprocated. This is the diagonal Moore–Penrose pseudoinverse.

definition *pinv-list* :: *real list* ⇒ *real list* **where**
  $\textit{pinv-list}\ D = \textit{map}\ (\lambda d.\ \textit{if}\ d = 0\ \textit{then}\ 0\ \textit{else}\ 1/d)\ D$

lemma *pinv-list-length* [*simp*]: *length* (*pinv-list D*) = *length D*
⟨*proof*⟩

lemma *pinv-list-index*:
assumes *i* < *length D*
shows *pinv-list D* ! *i* = (*if D* ! *i* = *0 then 0 else 1* / (*D* ! *i*))
⟨*proof*⟩

lemma *pinv-list-nonneg*:
assumes ⋀*i*. *i* < *length D* ⟹ *0* ≤ *D* ! *i*
shows ⋀*i*. *i* < *length D* ⟹ *0* ≤ *pinv-list D* ! *i*
⟨*proof*⟩

The central identity: $D \cdot D^{+} \cdot D = D$ entrywise on the diagonal. For $D_i = 0$ the left-hand side is $0 \cdot 0 \cdot 0 = 0 = D_i$; for $D_i \neq 0$ it is $D_i \cdot (1/D_i) \cdot D_i = D_i$. This is the core diagonal pseudoinverse identity that lifts to operator level through unitary conjugation.

lemma *diag-pinv-diag-eq*:
fixes *D* :: *real list*
shows *diag-mat-of-real D*
∗ *diag-mat-of-real* (*pinv-list D*)
∗ *diag-mat-of-real D*
= *diag-mat-of-real D*
⟨*proof*⟩

Pseudoinverse identity at operator level. Given a hermitian spectral decomposition $A = U\ D\ U^{\dagger}$, define the Moore–Penrose pseudoinverse via $A^{+} = U\ D^{+}\ U^{\dagger}$. Then $A\ A^{+}\ A = A$. This is the unitary-conjugation lift of *diag-mat-of-real ?D* ∗ *diag-mat-of-real* (*pinv-list ?D*) ∗ *diag-mat-of-real ?D* = *diag-mat-of-real ?D*.

lemma *A-Aplus-A-eq*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
assumes *Uc*: *U* ∈ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* = *k*
shows (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real* (*pinv-list D*) ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
= *U* ∗ *diag-mat-of-real D* ∗ (*U*†)
⟨*proof*⟩

Two square matrices agreeing on every vector are equal.

lemma *mat-eq-of-mult-vec-eq*:
fixes *M N* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat k k*
and *N-c*: *N* ∈ *carrier-mat k k*
and *eq*: ⋀*x*. *x* ∈ *carrier-vec k* ⟹ *M* ∗*v x* = *N* ∗*v x*
shows *M* = *N*
⟨*proof*⟩

Loewner-implied pseudoinverse identity, proved at operator level: with $Q = A^{+}\ A$ one has $A\ Q = A$, so $x - Q\ x \in \ker A \subseteq \ker B$, whence $B = B\ Q$; taking adjoints gives $B = A\ A^{+}\ B$.

lemma *A-Aplus-B-eq-B*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list* **and** *B* :: *complex Matrix.mat*
assumes *Uc*: *U* ∈ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* = *k*

and *Dnn*: $\bigwedge i.\ i < k \Longrightarrow 0 \leq D\ !\ i$
and *B-c*: $B \in$ *carrier-mat k k*
and *B-psd*: *psd B*
and *dom-psd*: *psd* $((U * \textit{diag-mat-of-real}\ D * (U^\dagger)) - B)$
shows $(U * \textit{diag-mat-of-real}\ D * (U^\dagger))$
$* (U * \textit{diag-mat-of-real}\ (\textit{pinv-list}\ D) * (U^\dagger)) * B = B$
⟨*proof*⟩

Spectral decomposition of a psd matrix with non-negative diagonal.

Dominated psd matrices are parametrised through the dominating frame: if $0 \leq B \leq V\ V^\dagger$ then $B = V\ T\ V^\dagger$ with $T = V^\dagger\ A^+\ B\ A^+\ V \geq 0$, $A = V\ V^\dagger$, A^+ the Moore–Penrose pseudoinverse.

lemma *dominated-outer-sum-param*:
fixes *vs* :: *complex Matrix.vec list*
and *B* :: *complex Matrix.mat*
assumes *vs-dim*: $\forall\, v \in$ *set vs. dim-vec v* $= k$
and *k-pos*: $0 < k$
and *B-c*: $B \in$ *carrier-mat k k*
and *B-psd*: *psd B*
and *B-dom*: *psd* (*outer-sum k vs* $- B$)
shows $\exists\, T.\ T \in$ *carrier-mat* (*length vs*) (*length vs*) $\wedge$ *psd T* $\wedge$
$B =$ (*mat-of-cols k vs*) $*\ T *$ ((*mat-of-cols k vs*)†)
⟨*proof*⟩

56.19 Linearity of *kraus-quad-map* in its matrix argument

Per-entry computation shows *kraus-quad-map d Vs Tm* is linear in its third (matrix) argument: $\varrho \mapsto \sum_{\{ij\}}\ T_{\{ij\}}\ V_i\ \varrho\ {V_j}^\dagger$.

lemma *kraus-quad-map-add-rho*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs. V* $\in$ *carrier-mat d n*
and *rho1-c*: *rho1* $\in$ *carrier-mat n n*
and *rho2-c*: *rho2* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs Tm* (*rho1 + rho2*)
$=$ *kraus-quad-map d Vs Tm rho1 + kraus-quad-map d Vs Tm rho2*
⟨*proof*⟩

lemma *kraus-quad-map-smult-rho*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs. V* $\in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs Tm* ($c \cdot_m$ *rho*)
$= c \cdot_m$ *kraus-quad-map d Vs Tm rho*
⟨*proof*⟩

56.20 Choi matrix determines a linear CP map on carrier

If two maps that are linear on carrier have the same Choi matrix and the same target carrier, they agree pointwise on carrier.

lemma *map-eq-from-choi-when-linear*:
fixes *Phi Psi* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *Phi-add*: $\bigwedge X\ Y.\ X \in$ *carrier-mat n n* $\Longrightarrow\ Y \in$ *carrier-mat n n*
$\Longrightarrow$ *Phi* $(X + Y) =$ *Phi X + Phi Y*
and *Phi-smult*: $\bigwedge c\ X.\ X \in$ *carrier-mat n n* $\Longrightarrow$ *Phi* $(c \cdot_m X) = c \cdot_m$ *Phi X*
and *Phi-car*: $\bigwedge X.\ X \in$ *carrier-mat n n* $\Longrightarrow$ *Phi X* $\in$ *carrier-mat d d*
and *Psi-add*: $\bigwedge X\ Y.\ X \in$ *carrier-mat n n* $\Longrightarrow\ Y \in$ *carrier-mat n n*
$\Longrightarrow$ *Psi* $(X + Y) =$ *Psi X + Psi Y*
and *Psi-smult*: $\bigwedge c\ X.\ X \in$ *carrier-mat n n* $\Longrightarrow$ *Psi* $(c \cdot_m X) = c \cdot_m$ *Psi X*

and *Psi-car*: $\bigwedge X.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow \textit{Psi}\ X \in \textit{carrier-mat}\ d\ d$
and *choi-eq*: $\textit{choi-matrix}\ n\ d\ \textit{Phi} = \textit{choi-matrix}\ n\ d\ \textit{Psi}$
and *npos*: $n > 0$ **and** *dpos*: $d > 0$
and *A-c*: $A \in \textit{carrier-mat}\ n\ n$
shows $\textit{Phi}\ A = \textit{Psi}\ A$
⟨*proof*⟩

56.21 CP Radon–Nikodym

CP Radon–Nikodym (Choi level). At the Choi-matrix level, the parametrisation falls out from $[\![\forall\, v{\in}\textit{set}\ ?vs.\ \textit{dim-vec}\ v = ?k;\ 0 < ?k;\ ?B \in \textit{carrier-mat}\ ?k\ ?k;\ ?B \geq_M 0;\ \textit{outer-sum}\ ?k\ ?vs - ?B \geq_M 0]\!] \Longrightarrow \exists\, T.\ T \in \textit{carrier-mat}\ (\textit{length}\ ?vs)\ (\textit{length}\ ?vs) \wedge T \geq_M 0 \wedge ?B = \textit{mat-of-cols}\ ?k\ ?vs * T * \textit{mat-of-cols}\ ?k\ ?vs^\dagger$ composed with the structural identities $[\![\forall\, V{\in}\textit{set}\ ?Vs.\ V \in \textit{carrier-mat}\ ?d\ ?n;\ 0 < ?n;\ 0 < ?d]\!] \Longrightarrow P_\Phi\ ?n\ ?d\ (\textit{kraus-map-dim}\ ?d\ ?Vs) = \textit{outer-sum}\ (?n * ?d)\ (\textit{map}\ (\textit{vec-of-kraus}\ ?d\ ?n)\ ?Vs)$ and $[\![\forall\, V{\in}\textit{set}\ ?Vs.\ V \in \textit{carrier-mat}\ ?d\ ?n;\ ?Tmat \in \textit{carrier-mat}\ (\textit{length}\ ?Vs)\ (\textit{length}\ ?Vs);\ 0 < ?n;\ 0 < ?d]\!] \Longrightarrow P_\Phi\ ?n\ ?d\ (\textit{kraus-quad-map}\ ?d\ ?Vs\ ?Tmat) = \textit{kraus-vec-mat}\ ?d\ ?n\ ?Vs * ?Tmat * \textit{kraus-vec-mat}\ ?d\ ?n\ ?Vs^\dagger$. This proves what is needed at the matrix level; passing to the $\forall\, X$-quantified form requires linearity of Ψ, which is the project's *completely-positive* does *not* include (see the lemma after).

lemma *cp-radon-nikodym-choi*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *Vs-c*: $\forall\, V \in \textit{set}\ \textit{Vs}.\ V \in \textit{carrier-mat}\ d\ n$
and *Phi-kraus*: $\bigwedge X.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow \textit{Phi}\ X = \textit{kraus-map-dim}\ d\ \textit{Vs}\ X$
and *Psi-cp*: *completely-positive n d Psi*
and *Psi-range*: $\bigwedge X.\ X \in \textit{carrier-mat}\ n\ n \Longrightarrow \textit{Psi}\ X \in \textit{carrier-mat}\ d\ d$
and *diff-cp*: *completely-positive n d* $(\lambda X.\ \textit{Phi}\ X - \textit{Psi}\ X)$
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows $\exists\, T.\ T \in \textit{carrier-mat}\ (\textit{length}\ \textit{Vs})\ (\textit{length}\ \textit{Vs}) \wedge \textit{psd}\ T \wedge$
$\textit{choi-matrix}\ n\ d\ \textit{Psi} = \textit{choi-matrix}\ n\ d\ (\textit{kraus-quad-map}\ d\ \textit{Vs}\ T)$
⟨*proof*⟩

CP Radon–Nikodym (operator level). With Ψ linear on *carrier-mat n n*, the Choi-level parametrisation gives the pointwise equality $\Psi(X) = \sum\ T_{\{ij\}}\ V_i\ X\ V_j{}^\dagger$.

Linearity of Ψ is supplied as an explicit hypothesis: the project's *completely-positive* predicate does *not* imply linearity (it constrains only positivity under amplification). This corresponds to the classical fact that "CP map" standardly includes linearity; we make it explicit so the consumer can supply it.

Operator-level CP Radon–Nikodym, with linearity assumption. This version is fully discharged via $[\![\forall\, V{\in}\textit{set}\ ?Vs.\ V \in \textit{carrier-mat}\ ?d\ ?n;\ \bigwedge X.\ X \in \textit{carrier-mat}\ ?n\ ?n \Longrightarrow ?Phi\ X = \textit{kraus-map-dim}\ ?d\ ?Vs\ X;\ \textit{completely-positive}\ ?n\ ?d\ ?Psi;\ \bigwedge X.\ X \in \textit{carrier-mat}\ ?n\ ?n \Longrightarrow ?Psi\ X \in \textit{carrier-mat}\ ?d\ ?d;\ \textit{completely-positive}\ ?n\ ?d\ (\lambda X.\ ?Phi\ X - ?Psi\ X);\ 0 < ?n;\ 0 < ?d]\!] \Longrightarrow \exists\, T.\ T \in \textit{carrier-mat}\ (\textit{length}\ ?Vs)\ (\textit{length}\ ?Vs) \wedge T \geq_M 0 \wedge P_\Phi\ ?n\ ?d\ ?Psi = P_\Phi\ ?n\ ?d\ (\textit{kraus-quad-map}\ ?d\ ?Vs\ T) + [\![\bigwedge X\ Y.\ [\![X \in \textit{carrier-mat}\ ?n\ ?n;\ Y \in \textit{carrier-mat}\ ?n\ ?n]\!] \Longrightarrow ?Phi\ (X + Y) = ?Phi\ X + ?Phi\ Y;\ \bigwedge c\ X.\ X \in \textit{carrier-mat}\ ?n\ ?n \Longrightarrow ?Phi\ (c \cdot_m X) = c \cdot_m ?Phi\ X;\ \bigwedge X.\ X \in \textit{carrier-mat}\ ?n\ ?n \Longrightarrow ?Phi\ X \in \textit{carrier-mat}\ ?d\ ?d;\ \bigwedge X\ Y.\ [\![X \in \textit{carrier-mat}\ ?n\ ?n;\ Y \in \textit{carrier-mat}\ ?n\ ?n]\!] \Longrightarrow ?Psi\ (X + Y) = ?Psi\ X + ?Psi\ Y;\ \bigwedge c\ X.\ X \in \textit{carrier-mat}\ ?n\ ?n \Longrightarrow ?Psi\ (c \cdot_m X) = c \cdot_m ?Psi\ X;\ \bigwedge X.\ X \in \textit{carrier-mat}\ ?n\ ?n \Longrightarrow ?Psi\ X \in \textit{carrier-mat}\ ?d\ ?d;\ P_\Phi\ ?n\ ?d\ ?Phi = P_\Phi\ ?n\ ?d\ ?Psi;\ 0 < ?n;\ 0 < ?d;\ ?A \in \textit{carrier-mat}\ ?n\ ?n]\!] \Longrightarrow ?Phi\ ?A = ?Psi\ ?A$, which rest on $[\![\forall\, v{\in}\textit{set}\ ?vs.\ \textit{dim-vec}\ v = ?k;\ 0 < ?k;\ ?B \in \textit{carrier-mat}\ ?k\ ?k;\ ?B \geq_M 0;\ \textit{outer-sum}\ ?k\ ?vs - ?B \geq_M 0]\!] \Longrightarrow \exists\, T.\ T \in \textit{carrier-mat}\ (\textit{length}\ ?vs)\ (\textit{length}\ ?vs) \wedge T \geq_M 0 \wedge ?B = \textit{mat-of-cols}\ ?k\ ?vs * T * \textit{mat-of-cols}\ ?k\ ?vs^\dagger$. It requires

Psi to be linear on *carrier-mat n n*, which is not implied by the project's *completely-positive* predicate.

lemma *cp-radon-nikodym-linear*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Phi-kraus*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *kraus-map-dim d Vs X*
and *Psi-cp*: *completely-positive n d Psi*
and *Psi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
and *Psi-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
⟹ *Psi* (*X* + *Y*) = *Psi X* + *Psi Y*
and *Psi-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *Psi X*
and *diff-cp*: *completely-positive n d* (λ*X*. *Phi X* − *Psi X*)
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows ∃ *T*. *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*) ∧ *psd T* ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *Psi X* = *kraus-quad-map d Vs T X*)
⟨*proof*⟩

The use site *choi-theorem-3-cptp-forward-complex* obtains linearity of its Ψ through the strengthened *cptp-pair-mid* predicate (which carries *lin-on-carrier* for both midpoint witnesses) and dispatches to ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Phi X* = *kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi X* ∈ *carrier-mat ?d ?d*; ⋀*X Y*. ⟦*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*⟧ ⟹ *?Psi* (*X* + *Y*) = *?Psi X* + *?Psi Y*; ⋀*c X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *?Psi X*; *completely-positive ?n ?d* (λ*X*. *?Phi X* − *?Psi X*); *0* < *?n*; *0* < *?d*⟧ ⟹ ∃ *T*. *T* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *T* ≥$_M$ *0* ∧ (∀ *X*. *X* ∈ *carrier-mat ?n ?n* ⟶ *?Psi X* = *kraus-quad-map ?d ?Vs T X*). No operator-level *cp-radon-nikodym* without linearity is needed any more.

56.22 Choi's Theorem 3 — reverse direction

Reverse direction of Choi's Theorem 3 (CPTP form). Under minimality of the Kraus list *Vs*, the non-linear-independence of the products {$V_j^\dagger$ V_i} forces Φ to fail to be an extreme point of *CPTP n d*.

The witness uses the explicit perturbation $\Phi_\pm(\varrho) := \sum (I_{\{ij\}} \pm \varepsilon\ H_{\{ij\}})\ V_i\ \varrho\ V_j^\dagger$ for a hermitian *H* (from ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?A-orig* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); ⋀*i j*. ⟦*i* < *length ?Vs*; *j* < *length ?Vs*⟧ ⟹ *cnj* (*?A-orig* $$ (*i*, *j*)) = *?A-orig* $$ (*i*, *j*); *kraus-pair-sum ?n ?Vs ?A-orig* = *0*$_m$ *?n ?n*; *?A-orig* ≠ *0*$_m$ (*length ?Vs*) (*length ?Vs*)⟧ ⟹ ∃ *H*. *H* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *hermitian H* ∧ *H* ≠ *0*$_m$ (*length ?Vs*) (*length ?Vs*) ∧ *kraus-pair-sum ?n ?Vs H* = *0*$_m$ *?n ?n*) and a small ε (from ⟦*?A* ∈ *carrier-mat ?k ?k*; *hermitian ?A*⟧ ⟹ ∃ε>*0*. ∀ *d*. |*d*| ≤ ε ⟶ *1*$_m$ *?k* + *complex-of-real d* ·$_m$ *?A* ≥$_M$ *0*). Distinctness uses ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *lin-indep-complex-mat ?d ?n ?Vs*; *?A* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *?A* ≠ *0*$_m$ (*length ?Vs*) (*length ?Vs*); *0* < *?n*; *0* < *?d*⟧ ⟹ ∃ *rho*∈*carrier-mat ?n ?n*. *kraus-quad-map ?d ?Vs ?A rho* ≠ *0*$_m$ *?d ?d* (Lemma B).

theorem *choi-theorem-3-cptp-reverse*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP n d* Φ

⟨*proof*⟩

Caveat on *extreme-CPTP* **(forward shadow).** Because both *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*, the raw function inequality $\Phi_1 \neq \Phi$ in *extreme-CPTP* is trivially satisfiable. We therefore introduce a *carrier*-point non-extremity notion below.

To keep *extreme-CPTP-carrier*'s definition manageable for automated proofs, we factor it through a helper predicate *cptp-pair-mid*: the data of a CPTP midpoint pair witnessing non-extremity (range, linearity on carrier, off-equal at some carrier-point, and the midpoint identity). Unfolding only the outer definition keeps the goal small.

definition *lin-on-carrier* ::
  nat ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  lin-on-carrier n Φ ≡
    (∀ *X Y*. *X* ∈ *carrier-mat n n* ⟶ *Y* ∈ *carrier-mat n n*
        ⟶ Φ (*X* + *Y*) = Φ *X* + Φ *Y*) ∧
    (∀ *c X*. *X* ∈ *carrier-mat n n* ⟶ Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*)

lemma *lin-on-carrier-add*:
  assumes *lin-on-carrier n* Φ *X* ∈ *carrier-mat n n Y* ∈ *carrier-mat n n*
  shows Φ (*X* + *Y*) = Φ *X* + Φ *Y*
  ⟨*proof*⟩

lemma *lin-on-carrier-smult*:
  assumes *lin-on-carrier n* Φ *X* ∈ *carrier-mat n n*
  shows Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*
  ⟨*proof*⟩

lemma *lin-on-carrierI*:
  assumes ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
        ⟹ Φ (*X* + *Y*) = Φ *X* + Φ *Y*
    and ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*
  shows *lin-on-carrier n* Φ
  ⟨*proof*⟩

definition *cptp-pair-mid* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
    ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
    ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  cptp-pair-mid n d Φ Φ$_1$ Φ$_2$ ≡
    Φ$_1$ ∈ *CPTP n d* ∧ Φ$_2$ ∈ *CPTP n d* ∧
    (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_1$ *X* ∈ *carrier-mat d d*) ∧
    (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_2$ *X* ∈ *carrier-mat d d*) ∧
    lin-on-carrier n Φ$_1$ ∧ *lin-on-carrier n* Φ$_2$ ∧
    (∃ *X*$_1$ ∈ *carrier-mat n n*. Φ$_1$ *X*$_1$ ≠ Φ *X*$_1$) ∧
    (∃ *X*$_2$ ∈ *carrier-mat n n*. Φ *X*$_2$ ≠ Φ$_2$ *X*$_2$) ∧
    (∀ *X*. *X* ∈ *carrier-mat n n* ⟶
      Φ *X* = (*1/2* :: *complex*) ·$_m$ Φ$_1$ *X* + (*1/2* :: *complex*) ·$_m$ Φ$_2$ *X*)

definition *extreme-CPTP-carrier* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  extreme-CPTP-carrier n d Φ ≡
    Φ ∈ *CPTP n d* ∧ ¬ (∃ Φ$_1$ Φ$_2$. *cptp-pair-mid n d* Φ Φ$_1$ Φ$_2$)

The same construction as in ⟦*0 < ?n*; *0 < ?d*; *?*Φ ∈ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ

?Vs; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ ⟹ ¬ *extreme-CPTP ?n ?d ?*Φ yields the stronger fact that Φ fails the *extreme-CPTP-carrier* test: the perturbed maps *Phip*, *Phim* are CPTP, have range *carrier-mat d d*, and disagree with Φ at the carrier-point *rho*.

lemma *not-extreme-CPTP-carrier-of-herm-witness*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list* **and** *H* :: *complex Matrix.mat*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *H-c*: *H* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *H-herm*: *hermitian H*
and *H-nz*: *H* ≠ 0_m (*length Vs*) (*length Vs*)
and *H-pair*: *kraus-pair-sum n Vs H* = 0_m *n n*
shows ¬ *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

Immediate contrapositive: extreme-in-carrier-sense forces linear independence of the products.

theorem *choi-theorem-3-cptp-reverse-carrier*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

theorem *choi-theorem-3-cptp-reverse-carrier-complex*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-complex n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

corollary *choi-theorem-3-cptp-carrier-forward*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP-carrier n d* Φ
shows *mat-lin-indep-real n* (*kraus-products Vs*)
⟨*proof*⟩

56.23 Trace pairing duality and TP-converse

If *Tr* (*M* ∗ ϱ) = *Tr* (*N* ∗ ϱ) for every ϱ in carrier, then *M* = *N*. Used to invert ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?Tmat* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *kraus-pair-sum ?n ?Vs ?Tmat* = 1_m *?n*; *0* < *?n*⟧ ⟹ *trace-preserving ?n* (*kraus-quad-map ?d ?Vs ?Tmat*).

lemma *Tr-mat-unit-mult*:
fixes M :: *complex Matrix.mat*
assumes *M-c*: $M \in$ *carrier-mat n n*
and *r-lt*: $r < n$ **and** *s-lt*: $s < n$
shows *Tr* $(M *$ *mat-unit s r n n*$) = M$ \$\$ (r, s)
⟨*proof*⟩

lemma *mat-trace-pairing-unique*:
fixes $M\ N$ :: *complex Matrix.mat*
assumes *M-c*: $M \in$ *carrier-mat n n*
and *N-c*: $N \in$ *carrier-mat n n*
and *pair-eq*: $\bigwedge$*rho. rho* $\in$ *carrier-mat n n* $\Longrightarrow$ *Tr* $(M *$ *rho*$) =$ *Tr* $(N *$ *rho*$)$
shows $M = N$
⟨*proof*⟩

56.24 Choi's Theorem 3 — forward direction

Important caveat on *extreme-CPTP***.** The current definition uses raw function inequality $\Phi_1 \neq \Phi$, but both *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*. Hence for *any* CPTP Φ one can construct Φ_1 that agrees with Φ on the carrier but disagrees on (say) a *1×1* matrix; this Φ_1 is itself in CPTP, so the negation of *extreme-CPTP* is vacuously satisfied. Consequently *extreme-CPTP* **as currently defined is always false**, and Choi's Theorem 3 as stated (an iff with *mat-lin-indep-real*) does not hold.

The fix is to use a *carrier*-point notion of inequality (defined above as *extreme-CPTP-carrier*).

The forward direction targets the *carrier*-flavoured version: *mat-lin-indep-complex* $\Longrightarrow$ *extreme-CPTP-carrier*. Proof uses ⟦$\forall$ *V*$\in$*set ?Vs. V* $\in$ *carrier-mat ?d ?n*; $\bigwedge$*X. X* $\in$ *carrier-mat ?n ?n* $\Longrightarrow$ *?Phi X = kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; $\bigwedge$*X. X* $\in$ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi X* $\in$ *carrier-mat ?d ?d*; $\bigwedge$*X Y.* ⟦*X* $\in$ *carrier-mat ?n ?n*; *Y* $\in$ *carrier-mat ?n ?n*⟧ $\Longrightarrow$ *?Psi* $(X + Y) =$ *?Psi X + ?Psi Y*; $\bigwedge$*c X. X* $\in$ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi* $(c \cdot_m X) = c \cdot_m$ *?Psi X*; *completely-positive ?n ?d* $(\lambda$*X. ?Phi X − ?Psi X*$)$; *0 < ?n*; *0 < ?d*⟧ $\Longrightarrow \exists$ *T. T* $\in$ *carrier-mat* $($*length ?Vs*$)$ $($*length ?Vs*$) \wedge$ *T* $\geq_M$ *0* $\wedge$ $(\forall$*X. X* $\in$ *carrier-mat ?n ?n* $\longrightarrow$ *?Psi X = kraus-quad-map ?d ?Vs T X*$)$, obtaining linearity of Ψ from the *lin-on-carrier* conjuncts carried by *cptp-pair-mid*.

theorem *choi-theorem-3-cptp-forward-complex*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ*-in*: $\Phi \in$ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *lin-indep*: *mat-lin-indep-complex n* $($*kraus-products Vs*$)$
shows *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

56.25 Choi's Theorem 3 (statement)

NOTE on conventions. Choi (1975, Theorem 3) gives an extreme-point characterisation in terms of *linear independence of certain products of Kraus operators*. Which products appear depends on which of the two natural normalisations the underlying convex set uses:

- **CPTP** ($\sum$ $V_i{}^\dagger$ $V_i = I_n$): products are $\{V_j{}^\dagger$ $V_i\}$ in M_n. The perturbation $\Phi_\varepsilon(X) = \sum(\delta_{\{ij\}} + \varepsilon\ a_{\{ij\}})$ V_i X $V_j{}^\dagger$ stays trace-preserving iff $\sum$ $a_{\{ij\}}$ $V_j{}^\dagger$ $V_i = 0$.
- **UCP** ($\sum$ V_i $V_i{}^\dagger = I_d$): products are $\{V_i$ $V_j{}^\dagger\}$ in M_d. Same perturbation stays unital iff $\sum$ $a_{\{ij\}}$ V_i $V_j{}^\dagger = 0$.

The two are interchanged by the Hilbert–Schmidt adjoint $\Phi \mapsto \Phi^*$; this provides a UCP $\leftrightarrow$ CPTP bijection.

We state both versions, parametrised by a fixed Kraus representation; the "any/every" character follows from the unitary-freedom of Kraus reps (Day 22 / "Kraus uniqueness"), which is not yet formalised in this development.

Reverse direction (extreme $\Rightarrow$ lin. indep.): contrapositive. For the CPTP version: if $\sum a_{\{ij\}}\ {V_j}^\dagger\ V_i = 0$ non-trivially, define Φ_ε as above; for small hermitian a, both $\Phi_{\pm\varepsilon}$ are CPTP and distinct, giving a non-trivial midpoint of Φ.

Forward direction (lin. indep. $\Rightarrow$ extreme): requires Radon–Nikodym for CP maps (every CP $\Psi \leq_{\{cp\}} \lambda\Phi$ has $\Psi(X) = \sum\ T_{\{ij\}}\ V_i\ X\ {V_j}^\dagger$ for a PSD matrix T) which is itself a substantial development.

The CPTP form of Choi's Theorem 3 (carrier-flavoured). Φ is extreme in CPTP iff the products $\{{V_j}^\dagger\ V_i\}$ are linearly independent in M_n.

The project's *extreme-CPTP* uses a raw function inequality $\Phi_1 \neq \Phi$; since *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*, this is technically vacuous (every CPTP map admits a CPTP Φ_1 that agrees with it on carrier but disagrees off carrier). The theorem is therefore stated in terms of *extreme-CPTP-carrier*, the carrier-point variant.

The first biconditional below uses different lin-indep notions on each direction (the historical form); *choi-theorem-3-cptp-iff* states the genuine iff:

- *extreme-CPTP-carrier* $\Longrightarrow$ *mat-lin-indep-real* (contrapositive of ⟦*0 < ?n*; *0 < ?d*; *?Φ ∈ CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?Φ ?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ $\Longrightarrow$ ¬ *extreme-CPTP-carrier ?n ?d ?Φ*);
- *mat-lin-indep-complex* $\Longrightarrow$ *extreme-CPTP-carrier* (the strengthened Choi forward direction ⟦*0 < ?n*; *0 < ?d*; *?Φ ∈ CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?Φ ?Vs*; *mat-lin-indep-complex ?n* (*kraus-products ?Vs*)⟧ $\Longrightarrow$ *extreme-CPTP-carrier ?n ?d ?Φ*, modulo CP Radon–Nikodym).

  The complex-coefficient witness *herm-witness-of-complex* bridges the two notions, so the direction "extreme $\Rightarrow$ independent" also holds over $\mathbb{C}$.

theorem *choi-theorem-3-cptp*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
      and Φ-*in*: *Φ ∈ CPTP n d*
      and *rep*: *cptp-kraus-rep n d Φ Vs*
      and *minimal*: *lin-indep-complex-mat d n Vs*
  shows (*extreme-CPTP-carrier n d Φ*
          ⟶ *mat-lin-indep-real n* (*kraus-products Vs*))
      ∧ (*mat-lin-indep-complex n* (*kraus-products Vs*)
          ⟶ *extreme-CPTP-carrier n d Φ*)
⟨*proof*⟩

With the complex-coefficient witness, the two directions use the same notion of linear independence: Choi's Theorem 3 in the form of the notes' corollary.

theorem *choi-theorem-3-cptp-iff*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
      and Φ-*in*: *Φ ∈ CPTP n d*
      and *rep*: *cptp-kraus-rep n d Φ Vs*
      and *minimal*: *lin-indep-complex-mat d n Vs*

shows *extreme-CPTP-carrier n d* Φ ⟷ *mat-lin-indep-complex n* (*kraus-products Vs*)
⟨*proof*⟩

The UCP form of Choi's Theorem 3. In the UCP / Heisenberg convention, the relevant products live in M_d with the *opposite* order $\{V_i\ {V_j}^\dagger\}$.
The project's *kraus-products* uses the CPTP-side convention $\{{V_j}^\dagger\ V_i\}$, and there is no UCP-side carrier-extreme notion or analogue of ⟦*0 < ?n*; *0 < ?d*; *?*Φ ∈ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ *?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ ⟹ ¬ *extreme-CPTP-carrier ?n ?d ?*Φ in this development. A proper UCP form of Choi's theorem requires either:

- a parallel *kraus-products-ucp* $(V_i\ {V_j}^\dagger)$, an *extreme-UCP-carrier* predicate, and full UCP-side analogues of the carrier-flavoured lemmas; or
- a Hilbert–Schmidt duality between *extreme-UCP* and *extreme-CPTP* that transports the CPTP theorem above.

  The second route is taken in theory Day23_Choi3_UCP, which states the UCP form.

56.26 Sanity / easy facts about extreme UCP

If a UCP map has a non-trivial mid-point representation by two distinct UCP maps, it is not extreme *by definition*.

lemma *not-extreme-UCP-if-midpoint*:
assumes Φ₁ ∈ *UCP n d* Φ₂ ∈ *UCP n d*
and Φ₁ ≠ Φ **and** Φ ≠ Φ₂
and Φ ∈ *UCP n d*
and ⋀*X*. *X* ∈ *carrier-mat n n*
⟹ Φ *X* = (*1/2* :: *complex*) ·ₘ Φ₁ *X*
+ (*1/2* :: *complex*) ·ₘ Φ₂ *X*
shows ¬ *extreme-UCP n d* Φ
⟨*proof*⟩

The contrapositive: every extreme UCP map, when written as a mid-point of two UCP maps, must have those maps coincide with itself on every *X* ∈ *carrier-mat n n*.

lemma *extreme-UCP-midpoint-collapses*:
assumes *ext*: *extreme-UCP n d* Φ
and Φ*1-in*: Φ₁ ∈ *UCP n d*
and Φ*2-in*: Φ₂ ∈ *UCP n d*
and *mid*: ⋀*X*. *X* ∈ *carrier-mat n n*
⟹ Φ *X* = (*1/2* :: *complex*) ·ₘ Φ₁ *X*
+ (*1/2* :: *complex*) ·ₘ Φ₂ *X*
shows Φ₁ = Φ ∨ Φ = Φ₂
⟨*proof*⟩

This closes the Day-23 stub. The full content of Choi's Theorem 3 awaits a Kraus-list formalisation; the definitions collected above will plug into that proof once available.

end
theory *Day23-Choi3-UCP*
imports *Day22-Adjoint-Maps*
begin

57 Day 23: Choi's Theorem 3 for unital maps (D23-1)

The notes state Choi's Theorem 3 for UCP maps; *choi-theorem-3-cptp-iff* proves the CPTP form. The UCP form follows by the duality of Day 22: $\Gamma(\Phi) = \Phi^*$ maps midpoint decompositions of a UCP map to midpoint decompositions of its adjoint and back (*extreme-UCP-iff-adjoint*), and the Kraus operators of Φ^* are the adjoints W_i^*, which are linearly independent with the W_i. Extremality is the carrier-point notion used for CPTP maps.

The UCP-side midpoint relation and extremality, mirroring *cptp-pair-mid* and *extreme-CPTP-carrier*.

definition *ucp-pair-mid* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
ucp-pair-mid n d Φ Φ$_1$ Φ$_2$ ≡
Φ$_1$ ∈ *UCP n d* ∧ Φ$_2$ ∈ *UCP n d* ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_1$ *X* ∈ *carrier-mat d d*) ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_2$ *X* ∈ *carrier-mat d d*) ∧
lin-on-carrier n Φ$_1$ ∧ *lin-on-carrier n* Φ$_2$ ∧
(∃ *X*$_1$ ∈ *carrier-mat n n*. Φ$_1$ *X*$_1$ ≠ Φ *X*$_1$) ∧
(∃ *X*$_2$ ∈ *carrier-mat n n*. Φ *X*$_2$ ≠ Φ$_2$ *X*$_2$) ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = (*1/2* :: *complex*) ·$_m$ Φ$_1$ *X* + (*1/2* :: *complex*) ·$_m$ Φ$_2$ *X*)

definition *extreme-UCP-carrier* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
extreme-UCP-carrier n d Φ ≡ Φ ∈ *UCP n d* ∧ ¬ (∃ Φ$_1$ Φ$_2$. *ucp-pair-mid n d* Φ Φ$_1$ Φ$_2$)

lemma *lin-on-from-carrier*:
assumes ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* ∈ *carrier-mat d d lin-on-carrier n* Φ
shows *lin-on n d* (*carrier-mat n n*) Φ
⟨*proof*⟩

lemma *lin-on-carrier-from-lin-on*: *lin-on n d* (*carrier-mat n n*) Φ ⟹ *lin-on-carrier n* Φ
⟨*proof*⟩

lemma *map-adjoint-half*:
assumes *L1*: *lin-on n d* (*carrier-mat n n*) Φ*1* **and** *L2*: *lin-on n d* (*carrier-mat n n*) Φ*2* **and** *Y*: *Y* ∈ *carrier-mat d d*
shows *map-adjoint n d* (λ*X*. (*1/2* :: *complex*) ·$_m$ Φ*1 X* + (*1/2* :: *complex*) ·$_m$ Φ*2 X*) *Y*
= (*1/2* :: *complex*) ·$_m$ *map-adjoint n d* Φ*1 Y* + (*1/2* :: *complex*) ·$_m$ *map-adjoint n d* Φ*2 Y*
⟨*proof*⟩

Γ carries midpoint decompositions to midpoint decompositions, in both directions.

lemma *ucp-mid-imp-cptp-mid*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *mid*: *ucp-pair-mid n d* Φ Φ*1* Φ*2* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *cptp-pair-mid d n* (*map-adjoint n d* Φ) (*map-adjoint n d* Φ*1*) (*map-adjoint n d* Φ*2*)
⟨*proof*⟩

lemma *cptp-mid-imp-ucp-mid*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *mid*: *cptp-pair-mid d n* (*map-adjoint n d* Φ) Ψ*1* Ψ*2* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *ucp-pair-mid n d* Φ (*map-adjoint d n* Ψ*1*) (*map-adjoint d n* Ψ*2*)
⟨*proof*⟩

theorem *extreme-UCP-iff-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *U*: Φ ∈ *UCP n d* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *extreme-UCP-carrier n d* Φ ⟷ *extreme-CPTP-carrier d n* (*map-adjoint n d* Φ)

⟨*proof*⟩

Linear independence passes to the adjoints: $\sum_k c_k W_k^* = 0$ gives $\sum_k \bar{c}_k W_k = 0$.

lemma *lin-indep-complex-mat-dagger*:
assumes *li*: *lin-indep-complex-mat d n Ws*
shows *lin-indep-complex-mat n d* (*map* (λ*W*. *W*†) *Ws*)
⟨*proof*⟩

lemma *kraus-operators-dagger-of-unital*:
assumes *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *un*: *kraus-map-dim d Ws* (*1*$_m$ *n*) = *1*$_m$ *d*
shows *kraus-operators d n* (*map* (λ*W*. *W*†) *Ws*)
⟨*proof*⟩

Choi's Theorem 3: a unital CP map $\Phi(X) = \sum_i W_i X W_i^*$ with linearly independent W_i is extreme iff the products $W_j W_i^*$ are linearly independent (*kraus-products* of the adjoints, whose entries are $(W_j^*)^* W_i^* = W_j W_i^*$). The notes write $\Phi(X) = \sum_i V_i^* X V_i$, i.e. $V_i = W_i^*$, and the products $V_i^* V_j$.

theorem *choi-theorem-3-ucp*:
assumes *n*: *0* < *n* **and** *d*: *0* < *d* **and** *U*: Φ ∈ *UCP n d*
and *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *rep*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = *kraus-map-dim d Ws X*
and *li*: *lin-indep-complex-mat d n Ws*
shows *extreme-UCP-carrier n d* Φ ⟷ *mat-lin-indep-complex d* (*kraus-products* (*map* (λ*W*. *W*†) *Ws*))
⟨*proof*⟩

end
theory *Day23-Extremality-Examples*
imports *Day22-Kraus-Independence Quantum-Channels Day25-Arveson-Correspondence*
begin

58 Day 23: extreme and non-extreme maps (D23-4)

Day 23, examples after Choi's Theorem 3, in the CPTP form and with the Kraus convention $\Phi(X) = \sum_i W_i X W_i^*$ of the formalisation (the notes write $U^* X U$; replace U by U^*).

58.1 Linear dependence tests

lemma *foldr-smult-index*:
assumes *Bs*: ∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n m* **and** *r*: *r* < *n* **and** *s*: *s* < *m*
shows *foldr* (λ*k acc*. *c k* ·$_m$ (*Bs* ! *k*) + *acc*) [*0*..<*length Bs*] (*0*$_m$ *n m*) $$ (*r*,*s*)
= (∑ *k*<*length Bs*. *c k* ∗ *Bs* ! *k* $$ (*r*,*s*))
⟨*proof*⟩

lemma *not-indep-of-dup*:
assumes *a*: *a* < *length Bs* **and** *b*: *b* < *length Bs* **and** *ab*: *a* ≠ *b* **and** *eq*: *Bs* ! *a* = *Bs* ! *b*
shows ¬ *mat-lin-indep-complex n Bs*
⟨*proof*⟩

lemma *not-indep-of-zero*:
assumes *a*: *a* < *length Bs* **and** *z*: *Bs* ! *a* = *0*$_m$ *n n*
shows ¬ *mat-lin-indep-complex n Bs*
⟨*proof*⟩

lemma *single-lin-indep*:
assumes *W*: *W* ∈ *carrier-mat d n* **and** *r*: *r* < *d* **and** *s*: *s* < *n* **and** *nz*: *W* $$ (*r*,*s*) ≠ *0*

shows *lin-indep-complex-mat d n* [*W*]
⟨*proof*⟩

lemma *single-mat-lin-indep*:
assumes *B*: *B* ∈ *carrier-mat n n* **and** *r*: *r* < *n* **and** *s*: *s* < *n* **and** *nz*: *B* $$ (*r*,*s*) ≠ *0*
shows *mat-lin-indep-complex n* [*B*]
⟨*proof*⟩

lemma *nonzero-entry*:
fixes *A* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat m k* **and** *nz*: *A* ≠ *0*$_m$ *m k*
obtains *r s* **where** *r* < *m s* < *k A* $$ (*r*,*s*) ≠ *0*
⟨*proof*⟩

58.2 Example 1: unitary conjugation is extreme

theorem *unitary-conjugation-extreme*:
assumes *U*: *U* ∈ *carrier-mat n n* **and** *Uu*: *unitary U* **and** *n*: *0* < *n*
shows *extreme-CPTP-carrier n n* (*kraus-map-dim n* [*U*])
⟨*proof*⟩

58.3 Example 2: an average of two unitary conjugations

definition *avg2* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* **where**
avg2 U1 U2 = [(*1* / *csqrt 2*) ·$_m$ *U1*, (*1* / *csqrt 2*) ·$_m$ *U2*]

lemma *half-sq*: *cnj* (*1* / *csqrt 2*) * (*1* / *csqrt 2*) = *1*/*2*
⟨*proof*⟩

lemma *smult-prod*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k B* ∈ *carrier-mat k l*
shows (*a* ·$_m$ *A*) * (*b* ·$_m$ *B*) = (*a* * *b*) ·$_m$ (*A* * *B*)
⟨*proof*⟩

lemma *smult-triple*:
fixes *A X B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat m k X* ∈ *carrier-mat k k B* ∈ *carrier-mat k l*
shows (*a* ·$_m$ *A*) * *X* * (*b* ·$_m$ *B*) = (*a* * *b*) ·$_m$ (*A* * *X* * *B*)
⟨*proof*⟩

lemma *avg2-products*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
shows (*avg2 U1 U2* ! *0*)† * *avg2 U1 U2* ! *0* = (*1*/*2*) ·$_m$ *1*$_m$ *n*
and (*avg2 U1 U2* ! *1*)† * *avg2 U1 U2* ! *1* = (*1*/*2*) ·$_m$ *1*$_m$ *n*
⟨*proof*⟩

lemma *avg2-kraus*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
shows *kraus-operators n n* (*avg2 U1 U2*)
⟨*proof*⟩

lemma *avg2-map*:
assumes *U1*: *U1* ∈ *carrier-mat n n* **and** *U2*: *U2* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim n* (*avg2 U1 U2*) *M* = (*1*/*2*) ·$_m$ (*U1* * *M* * (*U1*†) + *U2* * *M* * (*U2*†))
⟨*proof*⟩

The notes' claim needs U_1, U_2 linearly independent: then the products $V_i^* V_j$ contain $\frac{1}{2}I$ twice and Φ is not extreme.

theorem *avg2-not-extreme*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
and *n*: *0* < *n* **and** *ind*: *lin-indep-complex-mat n n* (*avg2 U1 U2*)
shows ¬ *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*avg2 U1 U2*))
⟨*proof*⟩

Without that hypothesis the claim fails: for $U_2 = \lambda U_1$ with $|\lambda| = 1$ the map is $X \mapsto U_1 X U_1^*$, which is extreme.

theorem *avg2-proportional-extreme*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *lam*: *cmod z = 1* **and** *n*: *0* < *n*
shows *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*avg2 U1* (*z* ·$_m$ *U1*)))
⟨*proof*⟩

58.4 Example 4: the diagonal map

lemma *unit-mult-left-entry*:
assumes *Bm*: *Bm* ∈ *carrier-mat n m* **and** *r*: *r* < *p* **and** *k*: *k* < *m* **and** *j*: *j* < *n*
shows (*E*⟦*i*,*j* ; *p*,*n*⟧ ∗ *Bm*) $$ (*r*,*k*) = (*if r = i then Bm* $$ (*j*,*k*) *else 0*)
⟨*proof*⟩

lemma *unit-mult-right-entry*:
assumes *Cm*: *Cm* ∈ *carrier-mat p n* **and** *r*: *r* < *p* **and** *s*: *s* < *m* **and** *i*: *i* < *n*
shows (*Cm* ∗ *E*⟦*i*,*j* ; *n*,*m*⟧) $$ (*r*,*s*) = (*if s = j then Cm* $$ (*r*,*i*) *else 0*)
⟨*proof*⟩

definition *diag-kraus* :: *nat* ⇒ *complex Matrix.mat list* **where**
diag-kraus n = map (λ*i*. *E*⟦*i*,*i* ; *n*,*n*⟧) [*0*..<*n*]

lemma *diag-kraus-carrier*: ∀ *W* ∈ *set* (*diag-kraus n*). *W* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *diag-kraus-map*:
assumes *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim n* (*diag-kraus n*) *M = Matrix.mat n n* (λ(*r*,*s*). *if r = s then M* $$ (*r*,*r*) *else 0*)
⟨*proof*⟩

lemma *diag-kraus-operators*: *kraus-operators n n* (*diag-kraus n*)
⟨*proof*⟩

lemma *diag-kraus-lin-indep*: *lin-indep-complex-mat n n* (*diag-kraus n*)
⟨*proof*⟩

theorem *diag-not-extreme*:
assumes *n*: *2* ≤ *n*
shows ¬ *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*diag-kraus n*))
⟨*proof*⟩

58.5 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$

As written, $\Phi(X) = \mathrm{Tr}(X) I_d$ is neither unital ($\Phi(I_n) = n I_d$) nor trace preserving ($\mathrm{Tr}\,\Phi(X) = d\,\mathrm{Tr}\,X$) unless $n = 1$ resp. $d = 1$. The CPTP normalisation is $\Phi(X) = \mathrm{Tr}(X) I_d/d$, with the nd Kraus operators $E_{ab}/\sqrt{d}$; these are linearly independent, so $\mathrm{cr}(\Phi) = nd > n$ for $d \geq 2$ and Φ is not extreme (D22-6).

definition *tr-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* **where**
tr-kraus n d = map (λ(*a*,*b*). *complex-of-real* (*1 / sqrt* (*real d*)) ·$_m$ *E*⟦*a*,*b* ; *d*,*n*⟧) (*List.product* [*0*..<*d*] [*0*..<*n*])

lemma *tr-kraus-carrier*: $\forall\, W \in$ *set* (*tr-kraus n d*). $W \in$ *carrier-mat d n*
⟨*proof*⟩

lemma *tr-kraus-length*: *length* (*tr-kraus n d*) = $d * n$
⟨*proof*⟩

lemma *tr-scalar*:
assumes *d*: $0 < d$
shows *cnj* (*complex-of-real* (1 / *sqrt* (*real d*))) $*$ *complex-of-real* (1 / *sqrt* (*real d*)) = 1 / *of-nat d*
and *complex-of-real* (1 / *sqrt* (*real d*)) $*$ *cnj* (*complex-of-real* (1 / *sqrt* (*real d*))) = 1 / *of-nat d*
⟨*proof*⟩

lemma *tr-kraus-operators*:
assumes *d*: $0 < d$
shows *kraus-operators n d* (*tr-kraus n d*)
⟨*proof*⟩

lemma *tr-kraus-map*:
assumes *d*: $0 < d$ **and** *M*: $M \in$ *carrier-mat n n*
shows *kraus-map-dim d* (*tr-kraus n d*) *M* = (*Tr M* / *of-nat d*) $\cdot_m$ 1_m *d*
⟨*proof*⟩

lemma *tr-kraus-lin-indep*:
assumes *d*: $0 < d$ **and** *n*: $0 < n$
shows *lin-indep-complex-mat d n* (*tr-kraus n d*)
⟨*proof*⟩

theorem *tr-map-not-extreme*:
assumes *n*: $0 < n$ **and** *d*: $2 \le d$
shows ¬ *extreme-CPTP-carrier n d* (*kraus-map-dim d* (*tr-kraus n d*))
⟨*proof*⟩

end
theory *Day24-Operator-Systems*
imports *Day17-Environment-Model*
begin

59 Day 24 (24 October 2011): operator systems

Day 24: operator systems $S \subseteq M_n$ (subspaces containing I and closed under adjoints), the matrices $M_p(S)$ over them, and completely positive maps on S. An element of $M_p(M_n)$ is a $pn \times pn$ matrix whose (a, b) block is *block-of-mat n X a b*; the ampliation $\Phi^{(p)}$ is *amplification n d p* Φ (Day 19).

definition *operator-system* :: *nat* ⇒ *complex Matrix.mat set* ⇒ *bool* **where**
operator-system n OS ⟷ *OS* ⊆ *carrier-mat n n* ∧ 1_m *n* ∈ *OS* ∧ (∀ *A* ∈ *OS*. $A^\dagger \in$ *OS*)
∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. *A* + *B* ∈ *OS*) ∧ (∀ *A* ∈ *OS*. ∀ *c*. *c* $\cdot_m$ *A* ∈ *OS*)

definition *Mp* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ *complex Matrix.mat set* **where**
Mp p n OS = {*A* ∈ *carrier-mat* (*p* $*$ *n*) (*p* $*$ *n*). ∀ *a* < *p*. ∀ *b* < *p*. *block-of-mat n A a b* ∈ *OS*}

definition *lin-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
lin-on n d OS Φ ⟷ (∀ *A* ∈ *OS*. Φ *A* ∈ *carrier-mat d d*) ∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. Φ (*A* + *B*) = Φ *A* + Φ *B*)
∧ (∀ *A* ∈ *OS*. ∀ *c*. Φ (*c* $\cdot_m$ *A*) = *c* $\cdot_m$ Φ *A*)

definition *p-positive-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
p-positive-on n d p OS Φ ⟷ (∀ *A* ∈ *Mp p n OS*. *psd A* ⟶ *psd* (*amplification n d p* Φ *A*))

definition *cp-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
cp-on n d OS Φ ⟷ (∀ *p* > *0*. *p-positive-on n d p OS* Φ)

lemma *block-of-mat-dims* [*simp*]: *dim-row* (*block-of-mat n A a b*) = *n dim-col* (*block-of-mat n A a b*) = *n*
⟨*proof*⟩

lemma *operator-systemD*:
assumes *operator-system n OS*
shows *OS* ⊆ *carrier-mat n n* 1_m *n* ∈ *OS* ⋀*A*. *A* ∈ *OS* ⟹ $A^\dagger$ ∈ *OS*
⋀*A B*. *A* ∈ *OS* ⟹ *B* ∈ *OS* ⟹ *A* + *B* ∈ *OS* ⋀*A c*. *A* ∈ *OS* ⟹ *c* $\cdot_m$ *A* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-zero*:
assumes *OS*: *operator-system n OS*
shows 0_m *n n* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-diff*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS* **and** *B*: *B* ∈ *OS*
shows *A* − *B* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-full*: *operator-system n* (*carrier-mat n n*)
⟨*proof*⟩

lemma *Mp-full*: *Mp p n* (*carrier-mat n n*) = *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*)
⟨*proof*⟩

lemma *cp-on-full*: *cp-on n d* (*carrier-mat n n*) Φ ⟷ *completely-positive n d* Φ
⟨*proof*⟩

lemma *cp-on-mono*:
assumes *cp-on n d OS* Φ **and** *OS′* ⊆ *OS*
shows *cp-on n d OS′* Φ
⟨*proof*⟩

59.1 $M_p(OS)$ is an operator system

lemma *block-of-mat-add*:
assumes *A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) *B* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) *a* < *p b* < *p*
shows *block-of-mat n* (*A* + *B*) *a b* = *block-of-mat n A a b* + *block-of-mat n B a b*
⟨*proof*⟩

lemma *block-of-mat-smult*:
assumes *A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) *a* < *p b* < *p*
shows *block-of-mat n* (*k* $\cdot_m$ *A*) *a b* = *k* $\cdot_m$ *block-of-mat n A a b*
⟨*proof*⟩

lemma *block-of-mat-dagger*:
assumes *A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) *a* < *p b* < *p*
shows *block-of-mat n* ($A^\dagger$) *a b* = (*block-of-mat n A b a*)†
⟨*proof*⟩

lemma *block-of-mat-one*:
assumes $a < p$ $b < p$
shows *block-of-mat n* $(1_m\ (p * n))$ *a b* = (*if a* = *b then* 1_m *n else* 0_m *n n*)
⟨*proof*⟩

The note of Day 24: $M_p(S)$ is again an operator system.

lemma *Mp-operator-system*:
assumes *OS*: *operator-system n OS*
shows *operator-system* $(p * n)$ (*Mp p n OS*)
⟨*proof*⟩

59.2 Every element is a combination of four positives

lemma *herm-bound*:
assumes *Hmc*: *Hm* ∈ *carrier-mat n n* **and** *Hmh*: *hermitian Hm*
shows ∃ *c*::*real*. *0* ≤ *c* ∧ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* − *Hm*) ∧ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* + *Hm*)
⟨*proof*⟩

Proposition (Day 24): $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_k \in S^+$.

proposition *operator-system-four-positives*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS*
shows ∃ *P1 P2 P3 P4*. *P1* ∈ *OS* ∧ *P2* ∈ *OS* ∧ *P3* ∈ *OS* ∧ *P4* ∈ *OS* ∧ *psd P1* ∧ *psd P2* ∧ *psd P3* ∧ *psd P4*
∧ *A* = (*P1* − *P2*) + i $\cdot_m$ (*P3* − *P4*)
⟨*proof*⟩

end

theory *Day25-Hahn-Banach*
imports *Matrix-Typed*
begin

60 Day 25 (26 October 2011): the Hahn–Banach theorem

Definition (notes). Let W be a normed space. A linear functional $f : W \to \mathbb{C}$ is *bounded* if $|f(w)| \le C\|w\|$ for some constant C; the least such C is the norm $\|f\| = \sup\{|f(w)| : \|w\| \le 1\}$.

Theorem (notes) (Hahn–Banach)**.** Let W be a normed space, $V \subseteq W$ a subspace and $g : V \to \mathbb{C}$ a bounded linear functional. Then there is a bounded linear functional $f : W \to \mathbb{C}$ with $\|f\| = \|g\|$ and $f(v) = g(v)$ for all $v \in V$.

In the notes the theorem is used (Day 26) to extend the state S_Φ from $M_d(S)$ to $M_d(M_n)$ in the proof of Arveson's extension theorem, which is not formalised. Here it is proved for $W = (\mathbb{C}$-valued $m \times n$ matrices) with the Hilbert–Schmidt norm: HOL's Hahn–Banach gives a real-linear norm-preserving extension of $\Re g$, and the Bohnenblust–Sobczyk construction $f(x) = F(x) - iF(ix)$ recovers a complex-linear extension with the same bound.

60.1 Bounded complex-linear functionals on lmat

definition *clinear-functional-on* ::
(*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*) ⇒ *bool*
where

clinear-functional-on V g ≡
(∀ *x* ∈ *V*. ∀ *y* ∈ *V*. *g* (*x* + *y*) = *g x* + *g y*) ∧
(∀ *x* ∈ *V*. ∀ *c*. *g* (*cmult-lmat c x*) = *c* ∗ *g x*)

definition *bounded-clinear-functional-on* ::
(*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*) ⇒ *bool*
where
bounded-clinear-functional-on V g ≡
clinear-functional-on V g ∧
(∃ *C* ≥ *0*. ∀ *x* ∈ *V*. *cmod* (*g x*) ≤ *C* ∗ *hs-norm-lmat x*)

60.2 Real-part functional and linearform/continuous instances

definition *re-part-lmat* ::
((*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *complex*)
⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *real*)
where
re-part-lmat g x = *Re* (*g x*)

lemma *linearform-re-part-lmat*:
fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
assumes *V-sub*: *Subspace.subspace V UNIV*
and *g-clin*: *clinear-functional-on V g*
shows *linearform V* (*re-part-lmat g*)
⟨*proof*⟩

lemma *continuous-re-part-lmat*:
fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
assumes *V-sub*: *Subspace.subspace V UNIV*
and *g-bnd*: *bounded-clinear-functional-on V g*
shows *continuous V* (*re-part-lmat g*) *hs-norm-lmat*
⟨*proof*⟩

60.3 Application of HOL/Hahn_Banach (norm-preserving extension)

Apply HOL/Hahn_Banach to extend the real-linearform *re-part-lmat g* to all of UNIV preserving the function norm.

theorem *hahn-banach-re-part-lmat*:
fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
and *g* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *complex*
assumes *V-sub*: *Subspace.subspace V* (*UNIV* :: (*complex*, *'m*, *'n*) *lmat set*)
and *g-bnd*: *bounded-clinear-functional-on V g*
shows
∃ *F* :: (*complex*, *'m*, *'n*) *lmat* ⇒ *real*.
linearform UNIV F
∧ *continuous UNIV F hs-norm-lmat*
∧ (∀ *x* ∈ *V*. *F x* = *re-part-lmat g x*)
⟨*proof*⟩

60.4 Bohnenblust-Sobczyk: complex extension from real extension

Given a real-linear extension F of *re-part-lmat g* on all of UNIV, define the complex-linear extension $f\,x = F\,x - i \cdot F\,(i \cdot x)$.

definition *bs-complex-extension* ::

((*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*)
⇒ ((*complex*, *'m*, *'n*) *lmat* ⇒ *complex*)
where
bs-complex-extension F x =
complex-of-real (*F x*) − i ∗ *complex-of-real* (*F* (*cmult-lmat* i *x*))

lemma *bs-complex-extension-add*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* (*x* + *y*)
= *bs-complex-extension F x* + *bs-complex-extension F y*
⟨*proof*⟩

lemma *bs-complex-extension-real-smult*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* (*Vector-Space.prod a x*)
= *complex-of-real a* ∗ *bs-complex-extension F x*
⟨*proof*⟩

lemma *bs-complex-extension-i-smult*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* (*cmult-lmat* i *x*)
= i ∗ *bs-complex-extension F x*
⟨*proof*⟩

lemma *bs-complex-extension-csmult*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
shows *bs-complex-extension F* (*cmult-lmat c x*)
= *c* ∗ *bs-complex-extension F x*
⟨*proof*⟩

lemma *bs-complex-extension-clinear*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
shows *clinear-functional-on UNIV* (*bs-complex-extension F*)
⟨*proof*⟩

The exact Bohnenblust-Sobczyk bound: any C bound on F lifts verbatim to a bound on *bs-complex-extension F*. This is the standard $e\hat{}\{-i\vartheta\}$-rotation argument used to prove the complex Hahn-Banach theorem from the real one.

lemma *bs-complex-extension-tight-bound*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
and *F-C*: ⋀*x*. |*F x*| ≤ *C* ∗ *hs-norm-lmat x*
shows *cmod* (*bs-complex-extension F x*) ≤ *C* ∗ *hs-norm-lmat x*
⟨*proof*⟩

lemma *bs-complex-extension-bounded*:
fixes *F* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
assumes *F-lin*: *linearform UNIV F*
and *F-cont*: *continuous UNIV F hs-norm-lmat*
shows ∃ *C* ≥ *0*. ∀ *x*. *cmod* (*bs-complex-extension F x*) ≤ *C* ∗ *hs-norm-lmat x*
⟨*proof*⟩

lemma *bs-complex-extension-extends*:

fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
and g :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*
and F :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *real*
assumes *g-clin*: *clinear-functional-on V g*
and *V-clos-i*: $\forall\, x \in V.$ *cmult-lmat* i $x \in V$
and *F-ext*: $\forall\, x \in V.\ F\ x =$ *re-part-lmat g x*
shows $\forall\, x \in V.$ *bs-complex-extension F x* $=$ *g x*
⟨*proof*⟩

60.5 The matrix-space Hahn-Banach theorem

Combine all the steps: a bounded complex-linear functional on a complex-linear subspace V of (*complex*, $'m$, $'n$) *lmat* extends to a bounded complex-linear functional on all of (*complex*, $'m$, $'n$) *lmat* (without claiming norm preservation; for the application to Arveson Extension this is enough together with the bound chase).

theorem *matrix-hahn-banach-lmat*:
fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
and g :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*
assumes *V-csub*: *cmat-subspace-lmat V*
and *g-bnd*: *bounded-clinear-functional-on V g*
shows $\exists f$ :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*.
bounded-clinear-functional-on UNIV f
$\wedge\ (\forall\, x \in V.\ f\ x = g\ x)$
⟨*proof*⟩

The explicit-bound form of the matrix Hahn-Banach theorem: any C-bound on $|Re \circ g|$ over the subspace V lifts verbatim to a C-bound on *cmod* $\circ$ f over the whole space.

The proof unfolds HOL/Hahn_Banach's locale-local *fn-norm* to its *Sup*-based definition, bounds the supremum on the subspace side by C via *g-re-C*, transports the bound to the extension side via *norm-Hahn-Banach*'s norm-preservation conclusion, then uses *bs-complex-extension-tight-bound* to turn the bound on the real extension into a bound on the complex extension.

theorem *matrix-hahn-banach-lmat-with-bound*:
fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
and g :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*
and C :: *real*
assumes *V-csub*: *cmat-subspace-lmat V*
and *g-clin*: *clinear-functional-on V g*
and *g-re-C*: $\forall\, x \in V.\ |Re\ (g\ x)| \leq C * $ *hs-norm-lmat x*
and *C-nn*: $0 \leq C$
shows $\exists f$ :: (*complex*, $'m$, $'n$) *lmat* $\Rightarrow$ *complex*.
clinear-functional-on UNIV f
$\wedge\ (\forall\, x \in V.\ f\ x = g\ x)$
$\wedge\ (\forall\, x.$ *cmod* $(f\ x) \leq C * $ *hs-norm-lmat x*)
⟨*proof*⟩

end
theory *Day25-Arveson-Correspondence*
imports *Day24-Operator-Systems*
begin

61 Days 24–25: Arveson's correspondence between maps and functionals

Days 24–25: Arveson's correspondence between maps $\Phi : S \to M_d$ and functionals $S_\Phi : M_d(S) \to \mathbb{C}$, $S_\Phi((A_{ij})) = \frac{1}{d}\sum_{i,j} \Phi(A_{ij})_{ij}$, with inverse $f \mapsto \Phi_f$, $\Phi_f(M) = d\,(f(E_{ij} \otimes M))_{ij}$.

61.1 Finite sums of matrices

definition *msum* :: *nat* ⇒ *nat* ⇒ (*'a* ⇒ *complex Matrix.mat*) ⇒ *'a list* ⇒ *complex Matrix.mat* **where**
msum r c g xs = *foldr* (λ*x acc*. *g x* + *acc*) *xs* (*0*$_m$ *r c*)

lemma *msum-Nil* [*simp*]: *msum r c g* [] = *0*$_m$ *r c*
⟨*proof*⟩

lemma *msum-Cons* [*simp*]: *msum r c g* (*x* # *xs*) = *g x* + *msum r c g xs*
⟨*proof*⟩

lemma *msum-carrier*: ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r c* ⟹ *msum r c g xs* ∈ *carrier-mat r c*
⟨*proof*⟩

lemma *msum-index*:
assumes ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r c* **and** *i* < *r* **and** *j* < *c*
shows *msum r c g xs* $$ (*i*,*j*) = (∑ *x*←*xs*. *g x* $$ (*i*,*j*))
⟨*proof*⟩

lemma *msum-in-os*:
assumes *OS*: *operator-system n OS* **and** *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *OS*
shows *msum n n g xs* ∈ *OS*
⟨*proof*⟩

lemma *lin-on-zero*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows Φ (*0*$_m$ *n n*) = *0*$_m$ *d d*
⟨*proof*⟩

lemma *lin-on-msum*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *OS*
shows Φ (*msum n n g xs*) = *msum d d* (λ*x*. Φ (*g x*)) *xs*
⟨*proof*⟩

lemma *sum-list-product-upt*:
(∑ *x*←*List.product* [*0*..<*a*] [*0*..<*b*]. *h x*) = (∑ *i*<*a*. ∑ *j*<*b*. *h* (*i*,*j*))
⟨*proof*⟩

lemma *sum-delta2*:
fixes *a d b e* :: *nat*
assumes *a*: *a* < *d* **and** *b*: *b* < *e*
shows (∑ *i*<*d*. ∑ *j*<*e*. *if i* = *a* ∧ *j* = *b then v else 0*) = *v*
⟨*proof*⟩

61.2 $E_{ij} \otimes M$ and the block decomposition

definition *unit-block* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unit-block d n i j M = *Matrix.mat* (*d* ∗ *n*) (*d* ∗ *n*)
(λ(*r*,*c*). *if r div n* = *i* ∧ *c div n* = *j then M* $$ (*r mod n*, *c mod n*) *else 0*)

lemma *unit-block-carrier* [*simp*]: *unit-block d n i j M* ∈ *carrier-mat* (*d* ∗ *n*) (*d* ∗ *n*)
⟨*proof*⟩

lemma *unit-block-dims* [*simp*]: *dim-row* (*unit-block d n i j M*) = *d* ∗ *n dim-col* (*unit-block d n i j M*) = *d* ∗ *n*
⟨*proof*⟩

unit-block d n i j M is $E_{ij} \otimes M$.

lemma *unit-block-tensor*:

assumes M: $M \in$ *carrier-mat n n* **and** i: $i < d$ **and** j: $j < d$
shows *unit-block d n i j M* = *Tensor.tensor-mat* (*Matrix.mat d d* (λ(*a*,*b*). *if a* = *i* $\wedge$ *b* = *j then 1 else 0*)) *M*
⟨*proof*⟩

lemma *block-unit-block*:
assumes M: $M \in$ *carrier-mat n n* **and** a: $a < d$ **and** b: $b < d$
shows *block-of-mat n* (*unit-block d n i j M*) *a b* = (*if a* = *i* $\wedge$ *b* = *j then M else* 0_m *n n*)
⟨*proof*⟩

lemma *unit-block-Mp*:
assumes *OS*: *operator-system n OS* **and** M: $M \in$ *OS* **and** i: $i < d$ **and** j: $j < d$
shows *unit-block d n i j M* $\in$ *Mp d n OS*
⟨*proof*⟩

lemma *block-decomposition*:
assumes A: $A \in$ *carrier-mat* (*d* * *n*) (*d* * *n*)
shows *A* = *msum* (*d* * *n*) (*d* * *n*) (λ(*i*,*j*). *unit-block d n i j* (*block-of-mat n A i j*)) (*List.product* [*0*..<*d*] [*0*..<*d*])
⟨*proof*⟩

61.3 The correspondence

definition *arveson-functional* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex* **where**
arveson-functional n d Φ *A* = ($\sum$ *i*<*d*. $\sum$ *j*<*d*. Φ (*block-of-mat n A i j*) $$ (*i*,*j*)) / *of-nat d*

definition *arveson-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
arveson-map n d f M = *Matrix.mat d d* (λ(*i*,*j*). *of-nat d* * *f* (*unit-block d n i j M*))

definition *lin-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
lin-fun-on TS f ⟷ (∀ *A* ∈ *TS*. ∀ *B* ∈ *TS*. *f* (*A* + *B*) = *f A* + *f B*) ∧ (∀ *A* ∈ *TS*. ∀ *c*. *f* (*c* ·$_m$ *A*) = *c* * *f A*)

definition *positive-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
positive-fun-on TS f ⟷ (∀ *A* ∈ *TS*. *psd A* ⟶ *0* ≤ *f A*)

lemma *amplification-idx*:
assumes r: $r < q$ **and** s: $s < q$ **and** i: $i < d$ **and** j: $j < d$
shows *amplification n d q* Φ *A* $$ (*r* * *d* + *i*, *s* * *d* + *j*) = Φ (*block-of-mat n A r s*) $$ (*i*,*j*)
⟨*proof*⟩

lemma *arveson-map-index*: *i* < *d* ⟹ *j* < *d* ⟹ *arveson-map n d f M* $$ (*i*,*j*) = *of-nat d* * *f* (*unit-block d n i j M*)
⟨*proof*⟩

lemma *arveson-map-carrier* [*simp*]: *arveson-map n d f M* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *arveson-map-dims* [*simp*]: *dim-row* (*arveson-map n d f M*) = *d dim-col* (*arveson-map n d f M*) = *d*
⟨*proof*⟩

lemma *lin-fun-zero*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f*
shows *f* (0_m *N N*) = *0*
⟨*proof*⟩

lemma *lin-fun-msum*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f* **and** *g*: $\forall\, x \in$ *set xs*. *g x* $\in$ *TS*
shows *f* (*msum N N g xs*) = ($\sum x \leftarrow xs$. *f* (*g x*))
⟨*proof*⟩

$S_{\Phi_f} = f$ on $M_d(S)$.

theorem *arveson-functional-of-map*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on* (*Mp d n OS*) *f* **and** *A*: *A* $\in$ *Mp d n OS* **and** *d*: *0* < *d*
shows *arveson-functional n d* (*arveson-map n d f*) *A* = *f A*
⟨*proof*⟩

$\Phi_{S_\Phi} = \Phi$ on S.

theorem *arveson-map-of-functional*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *M*: *M* $\in$ *OS* **and** *d*: *0* < *d*
shows *arveson-map n d* (*arveson-functional n d* Φ) *M* = Φ *M*
⟨*proof*⟩

lemma *unit-block-add*:
assumes *M*: *M* $\in$ *carrier-mat n n* **and** *M'*: *M'* $\in$ *carrier-mat n n*
shows *unit-block d n i j* (*M* + *M'*) = *unit-block d n i j M* + *unit-block d n i j M'*
⟨*proof*⟩

lemma *unit-block-smult*:
assumes *M*: *M* $\in$ *carrier-mat n n*
shows *unit-block d n i j* (*c* $\cdot_m$ *M*) = *c* $\cdot_m$ *unit-block d n i j M*
⟨*proof*⟩

lemma *arveson-map-lin*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on* (*Mp d n OS*) *f*
shows *lin-on n d OS* (*arveson-map n d f*)
⟨*proof*⟩

lemma *arveson-functional-lin*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows *lin-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

lemma *quad-split*:
fixes *M* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *M*: *M* $\in$ *carrier-mat* (*q* * *d*) (*q* * *d*) **and** *w*: *dim-vec w* = *q* * *d*
shows ⟨*w* | *M* $*_v$ *w*⟩ = ($\sum r{<}q$. $\sum i{<}d$. $\sum s{<}q$. $\sum j{<}d$.
cnj (*w* $ (*r* * *d* + *i*)) * *M* $$ (*r* * *d* + *i*, *s* * *d* + *j*) * *w* $ (*s* * *d* + *j*))
⟨*proof*⟩

61.4 Theorem (Arveson): completely positive iff d-positive iff S_Φ positive

lemma *if-zero-mult*: (*if P then x else 0*) * (*y*::*complex*) = (*if P then x* * *y else 0*)
⟨*proof*⟩

lemma *mult-if-zero*: (*y*::*complex*) * (*if P then x else 0*) = (*if P then y* * *x else 0*)
⟨*proof*⟩

lemma *cnj-if-zero*: *cnj* (*if P then x else 0*) = (*if P then cnj x else 0*)
⟨*proof*⟩

lemma *nonneg-of-real-quad*:

fixes z :: *complex*
assumes $z \in \mathbb{R}$ **and** $0 \le Re\ z$ **and** d: $0 < d$
shows $0 \le z\ /\ of\text{-}nat\ d$
⟨*proof*⟩

lemma *diag-quad-amplification*:
fixes $d\ n$ :: *nat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **and** A :: *complex Matrix.mat*
defines $v \equiv Matrix.vec\ (d * d)\ (\lambda x.\ if\ x\ div\ d = x\ mod\ d\ then\ 1\ else\ (0::complex))$
shows $\langle v \mid amplification\ n\ d\ d\ \Phi\ A *_v v\rangle = (\sum i{<}d.\ \sum j{<}d.\ \Phi\ (block\text{-}of\text{-}mat\ n\ A\ i\ j)\ \$\$\ (i,j))$
⟨*proof*⟩

lemma *dpos-imp-functional-pos*:
assumes *OS*: *operator-system n OS* **and** d: $0 < d$ **and** *P*: *p-positive-on n d d OS* Φ
shows *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

lemma *conj-tensor-index*:
fixes w :: *complex Matrix.vec*
assumes s: $s < q$ **and** k: $k < d$ **and** b': $b' < n$ **and** b: $b < n$
defines $V \equiv Tensor.tensor\text{-}mat\ (Matrix.mat\ q\ d\ (\lambda(r,t).\ w\ \$\ (r * d + t)))\ (1_m\ n)$
shows $V\ \$\$\ (s * n + b',\ k * n + b) = (if\ b' = b\ then\ w\ \$\ (s * d + k)\ else\ 0)$
⟨*proof*⟩

lemma *conj-block*:
fixes A :: *complex Matrix.mat* **and** w :: *complex Matrix.vec*
assumes A: $A \in carrier\text{-}mat\ (q * n)\ (q * n)$ **and** i: $i < d$ **and** j: $j < d$ **and** a: $a < n$ **and** b: $b < n$
defines $V \equiv Tensor.tensor\text{-}mat\ (Matrix.mat\ q\ d\ (\lambda(r,t).\ w\ \$\ (r * d + t)))\ (1_m\ n)$
shows $((V^\dagger) * A * V)\ \$\$\ (i * n + a,\ j * n + b)$
$= (\sum r{<}q.\ \sum s{<}q.\ cnj\ (w\ \$\ (r * d + i)) * w\ \$\ (s * d + j) * A\ \$\$\ (r * n + a,\ s * n + b))$
⟨*proof*⟩

lemma *functional-pos-imp-cp*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** d: $0 < d$
and *Pos*: *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
shows *cp-on n d OS* Φ
⟨*proof*⟩

Theorem (Arveson, Day 24): for linear $\Phi : S \to M_d$, completely positive $\iff$ d-positive $\iff$ S_Φ positive.

theorem *arveson-cp-iff*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** d: $0 < d$
shows *cp-on n d OS* $\Phi \longleftrightarrow$ *p-positive-on n d d OS* Φ
and *p-positive-on n d d OS* $\Phi \longleftrightarrow$ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

Theorem (Arveson correspondence, Day 25): $\Phi \mapsto S_\Phi$ maps $CP(S, M_d)$ onto the positive functionals on $M_d(S)$, with inverse $f \mapsto \Phi_f$; it is affine and sends unital maps to states.

theorem *arveson-correspondence*:
assumes *OS*: *operator-system n OS* **and** d: $0 < d$
shows *lin-on n d OS* $\Phi \Longrightarrow$ *cp-on n d OS* $\Phi \longleftrightarrow$ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
and *lin-fun-on* (*Mp d n OS*) $f \Longrightarrow$ *cp-on n d OS* (*arveson-map n d f*) $\longleftrightarrow$ *positive-fun-on* (*Mp d n OS*) f
and *lin-on n d OS* $\Phi \Longrightarrow M \in OS \Longrightarrow$ *arveson-map n d* (*arveson-functional n d* Φ) $M = \Phi\ M$
and *lin-fun-on* (*Mp d n OS*) $f \Longrightarrow A \in Mp\ d\ n\ OS \Longrightarrow$ *arveson-functional n d* (*arveson-map n d f*) $A = f\ A$
⟨*proof*⟩

lemma *arveson-functional-affine*:
assumes *OS*: *operator-system n OS* **and** *L1*: *lin-on n d OS* Φ **and** *L2*: *lin-on n d OS* Ψ **and** *A*: *A* ∈ *Mp d n OS*
shows *arveson-functional n d* (λ*M*. *t* ·$_m$ Φ *M* + (*1* − *t*) ·$_m$ Ψ *M*) *A*
= *t* ∗ *arveson-functional n d* Φ *A* + (*1* − *t*) ∗ *arveson-functional n d* Ψ *A*
⟨*proof*⟩

lemma *arveson-functional-unital*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *u*: Φ (*1*$_m$ *n*) = *1*$_m$ *d* **and** *d*: *0* < *d*
shows *arveson-functional n d* Φ (*1*$_m$ (*d* ∗ *n*)) = *1*
⟨*proof*⟩

end
theory *Day25-Unital-Functional-Norm*
imports *Day25-Arveson-Correspondence*
HOL−Analysis.L2-Norm
begin

62 Day 25: a unital functional is positive iff it has norm one (D25-4)

Day 25 proposition (D25-4): for an operator system $S \subseteq M_n$ and a linear $f : S \to \mathbb{C}$ with $f(I) = 1$, f is positive iff $\|f\| = 1$. The operator norm is $\|A\| = \sup\{\|Av\| : \|v\| \le 1\}$ and, as in the notes, $\|f\|$ is defined for bounded f. The proof differs from the notes in one step: for $P \ge 0$ the bound $\|P - sI + itI\|^2 \le s^2 + t^2$ ($s = \|P\|/2$) comes from $P^2 \le \|P\|\,P$, which uses the square root of P, instead of from the spectrum of P. Cauchy–Schwarz is *cauchy-schwarz-ineq* of Isabelle_Marries_Dirac.

definition *op-norm* :: *nat* ⇒ *complex Matrix.mat* ⇒ *real* **where**
op-norm n A = *Sup* {‖*A* ∗$_v$ *v*‖ | *v*. *dim-vec v* = *n* ∧ ‖*v*‖ ≤ *1*}

definition *fun-norm* :: *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *real* **where**
fun-norm n TS f = *Sup* {*cmod* (*f A*) | *A*. *A* ∈ *TS* ∧ *op-norm n A* ≤ *1*}

62.1 Vectors

lemma *vnorm-nonneg*: ‖*v*‖ ≥ *0*
⟨*proof*⟩

lemma *cauchy-schwarz-norm*:
assumes *dim-vec v* = *dim-vec w*
shows *cmod* ⟨*v*|*w*⟩ ≤ ‖*v*‖ ∗ ‖*w*‖
⟨*proof*⟩

lemma *inner-add-smult-expand*:
assumes *dim-vec u* = *dim-vec x*
shows ⟨*u* + *z* ·$_v$ *x* | *u* + *z* ·$_v$ *x*⟩ = ⟨*u*|*u*⟩ + *z* ∗ ⟨*u*|*x*⟩ + *cnj z* ∗ ⟨*x*|*u*⟩ + *cnj z* ∗ *z* ∗ ⟨*x*|*x*⟩
⟨*proof*⟩

lemma *smult-mat-vec*: *A* ∈ *carrier-mat n m* ⟹ *dim-vec v* = *m* ⟹ (*c* ·$_m$ *A*) ∗$_v$ *v* = *c* ·$_v$ (*A* ∗$_v$ *v*)
⟨*proof*⟩

lemma *entry-le-vnorm*: *i* < *dim-vec v* ⟹ *cmod* (*v* $ *i*) ≤ ‖*v*‖
⟨*proof*⟩

lemma *vnorm-le-sum*: ‖*w*‖ ≤ (∑ *i*<*dim-vec w*. *cmod* (*w* $ *i*))

⟨*proof*⟩

lemma *vnorm-zero-iff*: *dim-vec* $v = n \Longrightarrow \|v\| = 0 \longleftrightarrow v = 0_v\ n$
⟨*proof*⟩

62.2 The operator norm

lemma *mult-vec-norm-bound*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$ **and** *v*: *dim-vec* $v = n$
shows $\|A *_v v\| \leq (\sum i{<}n.\ \sum k{<}n.\ cmod\ (A\ \$\$\ (i,k))) * \|v\|$
⟨*proof*⟩

lemma *op-norm-bdd*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$
shows *bdd-above* $\{\|A *_v v\| \mid v.\ dim\text{-}vec\ v = n \wedge \|v\| \leq 1\}$
⟨*proof*⟩

lemma *mult-zero-vec'*: $A \in$ *carrier-mat* $n\ n \Longrightarrow A *_v 0_v\ n = 0_v\ n$
⟨*proof*⟩

lemma *op-norm-ge*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$ **and** *v*: *dim-vec* $v = n$ $\|v\| \leq 1$
shows $\|A *_v v\| \leq$ *op-norm* $n\ A$
⟨*proof*⟩

lemma *op-norm-nonneg*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$
shows $0 \leq$ *op-norm* $n\ A$
⟨*proof*⟩

lemma *op-norm-mult*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$ **and** *v*: *dim-vec* $v = n$
shows $\|A *_v v\| \leq$ *op-norm* $n\ A * \|v\|$
⟨*proof*⟩

lemma *op-norm-le*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$ **and** *c*: $0 \leq c$ **and** *b*: $\bigwedge v.$ *dim-vec* $v = n \Longrightarrow \|A *_v v\| \leq c * \|v\|$
shows *op-norm* $n\ A \leq c$
⟨*proof*⟩

lemma *op-norm-one*:
assumes *n*: $0 < n$
shows *op-norm* $n\ (1_m\ n) = 1$
⟨*proof*⟩

lemma *op-norm-smult-le*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$
shows *op-norm* $n\ (c \cdot_m A) \leq cmod\ c *$ *op-norm* $n\ A$
⟨*proof*⟩

lemma *op-norm-zero-imp*:
assumes *A*: $A \in$ *carrier-mat* $n\ n$ **and** *z*: *op-norm* $n\ A = 0$
shows $A = 0_m\ n\ n$
⟨*proof*⟩

62.3 Unital functionals

context

fixes n :: *nat* **and** TS :: *complex Matrix.mat set* **and** f :: *complex Matrix.mat* $\Rightarrow$ *complex*
assumes *OS*: *operator-system n TS* **and** *L*: *lin-fun-on TS f* **and** *f1*: $f\ (1_m\ n) = 1$
begin

lemma *TSc*: $A \in TS \Longrightarrow A \in$ *carrier-mat n n* ⟨*proof*⟩

lemma *lin2*: $A \in TS \Longrightarrow B \in TS \Longrightarrow f\ (a \cdot_m A + b \cdot_m B) = a * f\ A + b * f\ B$
⟨*proof*⟩

lemma *comb-in*: $A \in TS \Longrightarrow B \in TS \Longrightarrow a \cdot_m A + b \cdot_m B \in TS$
⟨*proof*⟩

lemma *f-zero*: $f\ (0_m\ n\ n) = 0$
⟨*proof*⟩

A Hermitian $A \in S$ with $|\Re\langle x|Ax\rangle| \le c\|x\|^2$ satisfies $cI \pm A \ge 0$, so a positive f has $f(A)$ real with $|f(A)| \le c$.

lemma *herm-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in TS$ *hermitian A* **and** *c*: $0 \le c$
and *b*: $\bigwedge x.$ *dim-vec* $x = n \Longrightarrow |Re\ \langle x|A *_v x\rangle| \le c * \|x\|^2$
shows $Im\ (f\ A) = 0$ **and** $|Re\ (f\ A)| \le c$
⟨*proof*⟩

lemma *adjoint-inner*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *x*: *dim-vec* $x = n$
shows $\langle x|A^\dagger *_v x\rangle = cnj\ \langle x|A *_v x\rangle$
⟨*proof*⟩

($\Rightarrow$): a positive unital f has $|f(A)| \le \|A\|$.

lemma *positive-imp-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in TS$
shows *cmod* $(f\ A) \le$ *op-norm n A*
⟨*proof*⟩

($\Leftarrow$): if $|f(A)| \le \|A\|$ on S then f is positive.

lemma *sq-le-norm-quad*:
assumes *P*: $P \in$ *carrier-mat n n psd P* **and** *x*: *dim-vec* $x = n$
shows $\|P *_v x\|^2 \le$ *op-norm n P* $*$ $Re\ \langle x|P *_v x\rangle$
⟨*proof*⟩

lemma *bound-imp-positive*:
assumes *n*: $0 < n$ **and** *b*: $\forall A \in TS.$ *cmod* $(f\ A) \le$ *op-norm n A*
shows *positive-fun-on TS f*
⟨*proof*⟩

lemma *fun-norm-one-iff*:
assumes *n*: $0 < n$ **and** *bdd*: $\exists C.\ \forall A \in TS.$ *cmod* $(f\ A) \le C *$ *op-norm n A*
shows *fun-norm n TS f* $= 1 \longleftrightarrow (\forall A \in TS.$ *cmod* $(f\ A) \le$ *op-norm n A*$)$
⟨*proof*⟩

D25-4.

theorem *unital-positive-iff-norm-one*:
assumes *n*: $0 < n$ **and** *bdd*: $\exists C.\ \forall A \in TS.$ *cmod* $(f\ A) \le C *$ *op-norm n A*
shows *positive-fun-on TS f* $\longleftrightarrow$ *fun-norm n TS f* $= 1$
⟨*proof*⟩

end

end
theory *Day26-Arveson-Extension*
imports *Day25-Arveson-Correspondence Quantum-Computing-by-VP-3-10-Lmat.Separable-Cone*
begin

63 Day 26: Arveson's extension theorem

Day 26: Arveson's extension theorem. The notes extend the state S_Φ by Hahn–Banach (norm one) and use that a unital functional of norm one is positive. Here the positive functional S_Φ on the operator system $M_d(S)$ is extended to a positive functional on $M_d(M_n)$ directly, by separating the positive definite cone from the kernel of S_Φ (*separating-hyperplane-set-0* in the Euclidean space of typed matrices); the extension is $X \mapsto \mathrm{Tr}(DX)$ with $D \geq 0$. Dimensions are cardinalities of finite types, as in *Separable-Cone*.

63.1 Auxiliary facts

lemma *operator-system-herm-parts*:
assumes *OS*: *operator-system n OS* **and** *A*: $A \in OS$
shows $\exists\, Hp\ Kp.\ Hp \in OS \wedge Kp \in OS \wedge$ *hermitian Hp* $\wedge$ *hermitian Kp* $\wedge\ A = Hp + \mathrm{i} \cdot_m Kp$
⟨*proof*⟩

lemma *Tr-herm-mult-real*:
assumes *Wc*: $W \in$ *carrier-mat n n* **and** *Mc*: $M \in$ *carrier-mat n n* **and** *Wh*: *hermitian W* **and** *Mh*: *hermitian M*
shows *Tr* $(W * M) \in \mathbb{R}$
⟨*proof*⟩

lemma *Tr-mult-outer*:
fixes A :: *complex Matrix.mat* **and** v :: *complex Matrix.vec*
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec* $v = n$
shows *Tr* $(A * (v \oslash v)) = \langle v \mid A *_v v \rangle$
⟨*proof*⟩

lemma *Tr-psd-mult-nonneg*:
assumes *Dc*: $D \in$ *carrier-mat n n* **and** *Pc*: $P \in$ *carrier-mat n n* **and** *Dp*: *psd D* **and** *Pp*: *psd P*
shows $0 \leq$ *Tr* $(D * P)$
⟨*proof*⟩

lemma *herm-entry*:
assumes $M \in$ *carrier-mat n n hermitian M* $i < n\ j < n$
shows M \$\$ $(i,j) =$ *cnj* $(M$ \$\$ $(j,i))$
⟨*proof*⟩

lemma *herm-real-comb*:
assumes *Ac*: $A \in$ *carrier-mat n n* **and** *Bc*: $B \in$ *carrier-mat n n* **and** *Ah*: *hermitian A* **and** *Bh*: *hermitian B*
shows *hermitian* (*complex-of-real u* $\cdot_m$ A + *complex-of-real v* $\cdot_m$ B)
⟨*proof*⟩

lemma *lin-t-zero*:
fixes $b\ c$ :: *real*
assumes $\forall\, t.\ 0 \leq c + t * b$
shows $b = 0$
⟨*proof*⟩

lemma *eps-nonneg*:
fixes $x\ y$ :: *real*
assumes $\forall \varepsilon{>}0.\ 0 \leq x + \varepsilon * y$
shows $0 \leq x$
⟨*proof*⟩

lemma *psd-one-mat*: *psd* $(1_m\ n)$
⟨*proof*⟩

63.2 Matrices as points of the Euclidean space *lmat*

lemma *unwrap-wrap-lm*:
fixes M :: *complex Matrix.mat*
assumes $M \in$ *carrier-mat CARD('k::finite) CARD('k)*
shows *unwrap-lm* (*wrap-lm* M :: (*complex*, *'k*, *'k*) *lmat*) $= M$
⟨*proof*⟩

lemma *wrap-lm-add*:
fixes $M\ M'$ :: *complex Matrix.mat*
assumes M: $M \in$ *carrier-mat CARD('k::finite) CARD('k)* **and** M': $M' \in$ *carrier-mat CARD('k) CARD('k)*
shows (*wrap-lm* $(M + M')$ :: (*complex*, *'k*, *'k*) *lmat*) $=$ *wrap-lm* $M +$ *wrap-lm* M'
⟨*proof*⟩

lemma *wrap-lm-scale*:
fixes M :: *complex Matrix.mat*
assumes M: $M \in$ *carrier-mat CARD('k::finite) CARD('k)*
shows (*wrap-lm* (*complex-of-real* $r \cdot_m M$) :: (*complex*, *'k*, *'k*) *lmat*) $= r *_R$ *wrap-lm* M
⟨*proof*⟩

lemma *wrap-lm-zero*: (*wrap-lm* $(0_m$ *CARD('k::finite) CARD('k))* :: (*complex*, *'k*, *'k*) *lmat*) $= 0$
⟨*proof*⟩

63.3 Extension of positive functionals

A positive functional on an operator system $T \subseteq M_N$ is $X \mapsto \mathrm{Tr}(DX)$ on T for some $D \geq 0$; hence it extends to a positive functional on M_N.

theorem *positive-functional-extension*:
fixes *TS* :: *complex Matrix.mat set* **and** f :: *complex Matrix.mat* $\Rightarrow$ *complex*
assumes *TS*: *operator-system CARD('k::finite) TS* **and** f: *lin-fun-on TS f* **and** *fpos*: *positive-fun-on TS f*
shows $\exists D.\ D \in$ *carrier-mat CARD('k) CARD('k)* $\wedge$ *psd* $D \wedge (\forall A \in TS.\ f\ A = Tr\ (D * A))$
⟨*proof*⟩

63.4 Arveson's extension theorem

theorem *arveson-extension*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ **and** *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows $\exists \Psi.$ *lin-on CARD('n) CARD('d)* (*carrier-mat CARD('n) CARD('n)*) Ψ
$\wedge$ *completely-positive CARD('n) CARD('d)* $\Psi \wedge (\forall M \in OS.\ \Psi\ M = \Phi\ M)$
⟨*proof*⟩

Corollary: a completely positive map on an operator system has a Kraus form.

corollary *arveson-kraus*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*

assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ
and *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows ∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat CARD('d) CARD('n)*) ∧ (∀ *M* ∈ *OS*. Φ *M* = *kraus-map-dim CARD('d) Ws M*)
⟨*proof*⟩

end
theory *Day26-CP-Factorization*
imports *Day26-Arveson-Extension Day32-Recovery-Proposition*
begin

64 Day 26: factoring a CP map through a unital or a trace-preserving one (D26-1)

The i-th block embedding $B_i : \mathbb{C}^n \to \mathbb{C}^{kn}$. The B_i are isometries with orthogonal ranges, $\sum_i B_i B_i^* = I$, so a Kraus representation $\Phi(X) = \sum_i W_i X W_i^*$ gives $\Phi(X) = V^* \psi(X) V$ with $\psi(X) = \sum_i B_i X B_i^*$ unital and $V = \sum_i B_i W_i^*$.

definition *blk-emb* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
blk-emb k n i = *Matrix.mat* (*k* ∗ *n*) *n* (λ(*x*, *y*). *if x div n* = *i* ∧ *x mod n* = *y then 1 else 0*)

lemma *blk-emb-carrier* [*simp*]: *blk-emb k n i* ∈ *carrier-mat* (*k* ∗ *n*) *n*
⟨*proof*⟩

lemma *blk-emb-dims* [*simp*]: *dim-row* (*blk-emb k n i*) = *k* ∗ *n dim-col* (*blk-emb k n i*) = *n*
⟨*proof*⟩

lemma *blk-cond*:
fixes *x y i n* :: *nat*
assumes *y*: *y* < *n*
shows (*x div n* = *i* ∧ *x mod n* = *y*) ⟷ *x* = *i* ∗ *n* + *y*
⟨*proof*⟩

lemma *blk-emb-gram-entry*:
assumes *x*: *x* < *k* ∗ *n* **and** *y*: *y* < *k* ∗ *n*
shows (*blk-emb k n i* ∗ ((*blk-emb k n i*)†)) $$ (*x*, *y*) =
(*if x div n* = *i* ∧ *y div n* = *i* ∧ *x mod n* = *y mod n then 1 else 0*)
⟨*proof*⟩

lemma *blk-emb-gram*:
foldr (λ*B acc*. *B* ∗ (*B*†) + *acc*) (*map* (*blk-emb k n*) [*0*..<*k*]) (*0*$_m$ (*k* ∗ *n*) (*k* ∗ *n*)) = *1*$_m$ (*k* ∗ *n*)
⟨*proof*⟩

lemma *kraus-lin-on*:
assumes ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
shows *lin-on n d* (*carrier-mat n n*) (*kraus-map-dim d Ws*)
⟨*proof*⟩

lemma *kraus-compose-single*: *kraus-compose* [*A*] *Es* = *map* (λ*E*. *A* ∗ *E*) *Es*
⟨*proof*⟩

V has the blocks W_i^* stacked vertically, so $V^* B_i = W_i$.

definition *kraus-stack-dag* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
kraus-stack-dag n d Ws = *Matrix.mat* (*length Ws* ∗ *n*) *d* (λ(*x*, *c*). *cnj* (*Ws* ! (*x div n*) $$ (*c*, *x mod n*)))

lemma *kraus-stack-dag-blk*:

assumes *Wc*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n* **and** *i*: $i <$ *length Ws*
shows (*kraus-stack-dag n d Ws*)† $*$ *blk-emb* (*length Ws*) *n i* $=$ *Ws* ! *i*
⟨*proof*⟩

Proposition (Day 26): a CP map $\Phi : S \to M_d$ on an operator system with $\Phi(I) = P$ is $\Phi(X) = V^*\psi(X)V$ with ψ unital and completely positive (here even on all of M_n) and $V^*V = P$. The notes compress Φ to the range of P; here the Kraus form of *arveson-kraus* is used.

theorem *cp-unital-factor*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ
and *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows ∃ *r V* ψ. *0* $<$ *r* ∧ *V* ∈ *carrier-mat r CARD('d)* ∧ ψ ∈ *UCP CARD('n) r*
∧ *lin-on CARD('n) r* (*carrier-mat CARD('n) CARD('n)*) ψ
∧ (∀ *X* ∈ *OS*. Φ *X* $=$ (*V*†) $*$ ψ *X* $*$ *V*) ∧ (*V*†) $*$ *V* $=$ Φ (*1*$_m$ *CARD('n)*)
⟨*proof*⟩

V with the blocks W_i stacked vertically, so $B_i^*V = W_i$.

definition *kraus-stack* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* **where**
kraus-stack n d Ws $=$ *Matrix.mat* (*length Ws* $*$ *d*) *n* (λ(*x*, *c*). *Ws* ! (*x div d*) $$ (*x mod d*, *c*))

lemma *kraus-stack-blk*:
assumes *Wc*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n* **and** *i*: $i <$ *length Ws*
shows (*blk-emb* (*length Ws*) *d i*)† $*$ *kraus-stack n d Ws* $=$ *Ws* ! *i*
⟨*proof*⟩

Proposition (Day 26): a CP map $\Phi : M_n \to M_d$ is $\Phi(X) = \psi(VXV^*)$ with $\psi : M_r \to M_d$ completely positive and trace preserving. The notes dualise the previous proposition; here $\psi(Y) = \sum_i B_i^* Y B_i$ and $V = \sum_i B_i W_i$ directly.

theorem *cp-tp-factor*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *L*: *lin-on CARD('n::finite) CARD('d::finite)* (*carrier-mat CARD('n) CARD('n)*) Φ
and *CP*: *completely-positive CARD('n) CARD('d)* Φ
shows ∃ *r V* ψ. *0* $<$ *r* ∧ *V* ∈ *carrier-mat r CARD('n)* ∧ ψ ∈ *CPTP r CARD('d)*
∧ *lin-on r CARD('d)* (*carrier-mat r r*) ψ
∧ (∀ *X* ∈ *carrier-mat CARD('n) CARD('n)*. Φ *X* $=$ ψ (*V* $*$ *X* $*$ (*V*†)))
⟨*proof*⟩

end

theory *Day26-Separability*
imports *Quantum-Computing-by-VP-3-10-Lmat.Separable-Cone*
begin

65 Days 26–28 (28 October – 2 November 2011): separability and entanglement witnesses

Entanglement revisited. Ensembles $\{\psi_i, p_i\}$ on $\mathcal{H}_A$ and $\{\phi_j, q_j\}$ on $\mathcal{H}_B$ compose to the density matrix $\sum_{i,j} p_i q_j |\psi_i\rangle\langle\psi_i| \otimes |\phi_j\rangle\langle\phi_j|$.

Definition (notes) (Day 26)**.** A density matrix $P \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ is *separable* if $P = \sum_l p_l |\psi_l\rangle\langle\psi_l| \otimes |\phi_l\rangle\langle\phi_l|$ with unit vectors $\psi_l \in \mathcal{H}_A$, $\phi_l \in \mathcal{H}_B$, $p_l \geq 0$, $\sum p_l = 1$; otherwise P is *entangled*. More generally (Day 27) $Sep \subseteq M_r(M_n)$ is the set of matrices $\sum_l P_l \otimes Q_l$ with $P_l \geq 0$, $Q_l \geq 0$.

Are there entangled density matrices, and how can one tell? The notes answer with the theory of convex cones:

Theorem (notes) (Days 27–28)**.** Let V be a finite-dimensional real vector space, $K \subseteq V$ closed and convex, $y \notin K$. Then there is a linear $f : V \to \mathbb{R}$ and $\alpha \in \mathbb{R}$ with $f(y) < \alpha \leq f(K)$; if C is a closed cone and $w \notin C$ there is f with $f(w) < 0 \leq f(C)$.

The separable matrices form a closed cone C in the real vector space of Hermitian matrices; an entangled R is separated from C by a real functional, which complexifies to $\tilde{f}(H + iK) = f(H) + if(K)$, and $\tilde{f}$ corresponds to a Hermitian matrix W with $\mathrm{Tr}(WR) < 0 \leq \mathrm{Tr}(W\sigma)$ for all separable σ: an *entanglement witness*. The notes phrase the final criterion through positive maps Φ_f obtained from the Arveson correspondence; that form is *separable-iff-positive-maps* in *Day27-Positive-Map-Criterion*, derived from the witness form proved here.

The formal development uses the typed matrices *lmat* so that HOL-Analysis' separation theorem applies: *separable-cone-lmat* is the convex cone generated by tensor products of positive matrices and *Sprime* is the set of product density matrices.

65.1 The separable cone (Day 27)

The cone is convex, conic, contains 0, and consists of Hermitian matrices.

65.2 Separable density matrices (Day 26)

For a density matrix ρ (positive with $\mathrm{Tr}\,\rho = 1$), membership in the cone is the notes' definition: ρ is a convex combination of product density matrices $\sigma_i \otimes \tau_i$.

65.3 The separable cone is closed (Day 28)

The notes record that C is a closed cone (needed for the separation theorem); here the cone is closed in the Hilbert–Schmidt topology and the separable density matrices form a compact convex set.

65.4 Separation (Days 27–28)

The separation theorem applied to the cone: a non-separable ρ is separated from C by a real-linear functional (an inner product against some a), strictly for density matrices because the separable density matrices are compact.

65.5 Entanglement witnesses (Day 28)

An *entanglement witness* for ρ is a Hermitian W with $\mathrm{Tr}(W\sigma) \geq 0$ on every separable σ and $\mathrm{Tr}(W\rho) < 0$. (i) Every non-separable Hermitian ρ has a witness; (ii) density matrices have a witness with a strict margin; (iii) (Horodecki) ρ is separable iff every Hermitian W that is non-negative on the cone is non-negative on ρ.

This closes the entanglement chapter of the notes. The Horodecki–Peres criterion for two qubits is *Day28-Horodecki-Peres*, the positive-map formulation *Day27-Positive-Map-Criterion*; the universal-witness question (Day 28) is not formalised.

end
theory *Day27-Positive-Map-Criterion*
imports *Day26-Arveson-Extension Day34-Knill-Laflamme*
begin

66 Days 27–28: positive maps detect entanglement

Days 27–28: positive maps detect entanglement. For $R \in M_r(M_n)$ (block size n), $\Phi^{(r)}(R) \geq 0$ for every positive Φ when R is separable (D27-1); a functional f on $M_r(M_n)$ is nonnegative on the separable matrices iff Φ_f is positive (D27-3); and with the entanglement witnesses of *Separable-Cone* this gives the positive-map criterion (D28-3).

66.1 Tensor products with a matrix in the first factor

lemma *tensor-add-first*:
fixes *A B P* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat r r* **and** *B*: *B* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*A* + *B*) *P* = *Tensor.tensor-mat A P* + *Tensor.tensor-mat B P*
⟨*proof*⟩

lemma *tensor-zero-first*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*0*$_m$ *r r*) *P* = *0*$_m$ (*r* ∗ *n*) (*r* ∗ *n*)
⟨*proof*⟩

lemma *tensor-msum-first*:
fixes *P* :: *complex Matrix.mat*
assumes *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*msum r r g xs*) *P* = *msum* (*r* ∗ *n*) (*r* ∗ *n*) (λ*x*. *Tensor.tensor-mat* (*g x*) *P*) *xs*
⟨*proof*⟩

66.2 The quadratic form of Φ_f

definition *rank1* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
rank1 w = *Matrix.mat* (*dim-vec w*) (*dim-vec w*) (λ(*i*,*j*). *cnj* (*w* $ *i*) ∗ *w* $ *j*)

lemma *rank1-carrier* [*simp*]: *rank1 w* ∈ *carrier-mat* (*dim-vec w*) (*dim-vec w*)
⟨*proof*⟩

lemma *rank1-dims* [*simp*]: *dim-row* (*rank1 w*) = *dim-vec w dim-col* (*rank1 w*) = *dim-vec w*
⟨*proof*⟩

lemma *rank1-outer*: *rank1* (*conjugate u*) = *u* ⊘ *u*
⟨*proof*⟩

lemma *rank1-psd*: *psd* (*rank1 w*)
⟨*proof*⟩

lemma *quad-arveson-map*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex* **and** *w* :: *complex Matrix.vec*
assumes *f*: *lin-fun-on* (*carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)) *f* **and** *w*: *dim-vec w* = *r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows ⟨*w* | *arveson-map n r f P* ∗$_v$ *w*⟩ = *of-nat r* ∗ *f* (*Tensor.tensor-mat* (*rank1 w*) *P*)
⟨*proof*⟩

66.3 Proposition (Day 27): $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive

theorem *positive-map-iff-sep-nonneg*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex*

assumes f: *lin-fun-on* (*carrier-mat* $(r * n)$ $(r * n)$) f **and** r: $0 < r$ **and** n: $0 < n$
shows *positive-map* n r (*arveson-map* n r f)
$\longleftrightarrow$ $(\forall Q\ P.\ Q \in$ *carrier-mat* r $r \longrightarrow$ *psd* $Q \longrightarrow P \in$ *carrier-mat* n $n \longrightarrow$ *psd* P
$\longrightarrow 0 \leq f$ (*Tensor.tensor-mat* Q P))
⟨*proof*⟩

66.4 Proposition (Day 27): positive maps keep separable matrices positive

lemma *block-tensor*:
fixes Q P :: *complex Matrix.mat*
assumes Q: $Q \in$ *carrier-mat* r r **and** P: $P \in$ *carrier-mat* n n **and** a: $a < r$ **and** b: $b < r$
shows *block-of-mat* n (*Tensor.tensor-mat* Q P) a $b = Q$ $\$\$$ (a,b) $\cdot_m$ P
⟨*proof*⟩

lemma *amplification-tensor*:
fixes Q P :: *complex Matrix.mat* **and** Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes L: *lin-on* n d (*carrier-mat* n n) Φ **and** Q: $Q \in$ *carrier-mat* r r **and** P: $P \in$ *carrier-mat* n n
and d: $0 < d$ **and** n: $0 < n$
shows *amplification* n d r Φ (*Tensor.tensor-mat* Q P) = *Tensor.tensor-mat* Q (Φ P)
⟨*proof*⟩

lemma *amplification-lin*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes L: *lin-on* n d (*carrier-mat* n n) Φ **and** A: $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ **and** B: $B \in$ *carrier-mat* $(p * n)$ $(p * n)$
shows *amplification* n d p Φ $(A + B)$ = *amplification* n d p Φ A + *amplification* n d p Φ B
and *amplification* n d p Φ $(c \cdot_m A)$ = $c \cdot_m$ *amplification* n d p Φ A
⟨*proof*⟩

lemma *unwrap-lm-scaleR-mat*:
fixes R :: (*complex*, $'k$ :: *finite*, $'k$) *lmat*
shows *unwrap-lm* $(u *_R R)$ = *complex-of-real* $u \cdot_m$ *unwrap-lm* R
⟨*proof*⟩

theorem *positive-map-separable*:
fixes R :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat* **and** Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes L: *lin-on CARD*($'p$) d (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ **and** *pos*: *positive-map CARD*($'p$) d Φ
and d: $0 < d$ **and** R: $R \in$ *separable-cone-lmat*
shows *psd* (*amplification CARD*($'p$) d *CARD*($'m$) Φ (*unwrap-lm* R))
⟨*proof*⟩

66.5 Theorem (Day 28): the positive-map criterion

A Hermitian $R \in M_r(M_n)$ is separable iff $\Phi^{(r)}(R) \geq 0$ for every positive $\Phi : M_n \to M_r$.

theorem *separable-iff-positive-maps*:
fixes R :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
assumes *Rh*: *hermitian* (*unwrap-lm* R)
shows $R \in$ *separable-cone-lmat* $\longleftrightarrow$
$(\forall \Phi.$ *lin-on CARD*($'p$) *CARD*($'m$) (*carrier-mat CARD*($'p$) *CARD*($'p$)) $\Phi \longrightarrow$ *positive-map CARD*($'p$) *CARD*($'m$) Φ
$\longrightarrow$ *psd* (*amplification CARD*($'p$) *CARD*($'m$) *CARD*($'m$) Φ (*unwrap-lm* R)))
⟨*proof*⟩

end
theory *Day27-Entangled-Examples*
imports *Day18-Positive-Maps Day27-Positive-Map-Criterion*

begin

67 Day 27: entangled matrices detected by the transpose (D27-2)

For $f : \{0,\dots,r-1\} \to \{0,\dots,n-1\}$ the block matrix $(E_{f(a)f(b)})_{a,b} \in M_r(M_n)$. The notes' examples are $f = (i,j)$ with $r = 2$, giving $\begin{pmatrix} E_{ii} & E_{ij} \\ E_{ji} & E_{jj} \end{pmatrix}$, and $f = \mathrm{id}$ with $r = n$, giving (E_{ij}).

definition *unit-blocks* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.mat* **where**
unit-blocks r n f = *Matrix.mat* (*r* ∗ *n*) (*r* ∗ *n*)
(λ(*x*, *y*). *if x mod n* = *f* (*x div n*) ∧ *y mod n* = *f* (*y div n*) *then 1 else 0*)

lemma *unit-blocks-carrier* [*simp*]: *unit-blocks r n f* ∈ *carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)
⟨*proof*⟩

lemma *unit-blocks-block*:
assumes *a* < *r b* < *r f a* < *n f b* < *n*
shows *block-of-mat n* (*unit-blocks r n f*) *a b* = *mat-unit* (*f a*) (*f b*) *n n*
⟨*proof*⟩

definition *unit-blocks-vec* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.vec* **where**
unit-blocks-vec r n f = *Matrix.vec* (*r* ∗ *n*) (λ*x*. *if x mod n* = *f* (*x div n*) *then 1 else 0*)

lemma *unit-blocks-outer*: *unit-blocks r n f* = *unit-blocks-vec r n f* ⊘ *unit-blocks-vec r n f*
⟨*proof*⟩

lemma *unit-blocks-psd*: *psd* (*unit-blocks r n f*)
⟨*proof*⟩

lemma *sum-two-support*:
fixes *g* :: *nat* ⇒ *complex*
assumes *p* ≠ *q p* < *m q* < *m* ⋀*x*. *x* < *m* ⟹ *x* ≠ *p* ⟹ *x* ≠ *q* ⟹ *g x* = *0*
shows (∑ *x* = *0*..<*m*. *g x*) = *g p* + *g q*
⟨*proof*⟩

lemma *transpose-lin-on*: *lin-on n n* (*carrier-mat n n*) (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

The partial transpose of $(E_{f(a)f(b)})$ is $(E_{f(b)f(a)})$; its quadratic form at $e_0 \otimes e_{f(1)} - e_1 \otimes e_{f(0)}$ is -2.

lemma *unit-blocks-partial-transpose-not-psd*:
assumes *r*: *2* ≤ *r* **and** *f*: *f 0* < *n f 1* < *n f 0* ≠ *f 1*
shows ¬ *psd* (*amplification n n r* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*unit-blocks r n f*))
⟨*proof*⟩

theorem *unit-blocks-entangled*:
fixes *R* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *R*: *unwrap-lm R* = *unit-blocks CARD('m) CARD('p) f*
and *r*: *2* ≤ *CARD('m)* **and** *f*: *f 0* < *CARD('p) f 1* < *CARD('p) f 0* ≠ *f 1*
shows *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

The notes' first example: $\begin{pmatrix} E_{ii} & E_{ij} \\ E_{ji} & E_{jj} \end{pmatrix} \in M_2(M_n)$, $i \neq j$.

corollary *entangled-example-2*:
fixes R :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *m*: *CARD*($'m$) = *2* **and** *ij*: i < *CARD*($'p$) j < *CARD*($'p$) $i \neq j$
and *R*: *unwrap-lm R* = *unit-blocks 2 CARD*($'p$) (λ*a*. *if a* = *0 then i else j*)
shows *block-of-mat CARD*($'p$) (*unwrap-lm R*) *0 0* = *E*⟦*i*,*i* ; *CARD*($'p$),*CARD*($'p$)⟧
and *block-of-mat CARD*($'p$) (*unwrap-lm R*) *0 1* = *E*⟦*i*,*j* ; *CARD*($'p$),*CARD*($'p$)⟧
and *block-of-mat CARD*($'p$) (*unwrap-lm R*) *1 0* = *E*⟦*j*,*i* ; *CARD*($'p$),*CARD*($'p$)⟧
and *block-of-mat CARD*($'p$) (*unwrap-lm R*) *1 1* = *E*⟦*j*,*j* ; *CARD*($'p$),*CARD*($'p$)⟧
and *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

The second: $(E_{ij}) \in M_n(M_n)$, $n \geq 2$, which is *choi-seed*.

lemma *choi-seed-unit-blocks*: *choi-seed n* = *unit-blocks n n id*
⟨*proof*⟩

corollary *entangled-example-choi-seed*:
fixes R :: (*complex*, $'p$ :: *finite* × $'p$, $'p$ × $'p$) *lmat*
assumes *n*: *2* ≤ *CARD*($'p$) **and** *R*: *unwrap-lm R* = *choi-seed CARD*($'p$)
shows *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

end
theory *Day28-Horodecki-Peres*
imports *QC-Prelims*
begin

68 Day 28: the Horodecki–Peres criterion for two qubits (D28-5)

Entries of the partial transpose $\Phi^{(2)}(X)$ of a 4×4 matrix.

lemma *partial-transpose-2-index*:
fixes *M* :: *complex Matrix.mat*
assumes *i*: *i* < *4* **and** *j*: *j* < *4*
shows *amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) *M* $$ (*i*,*j*)
= *M* $$ ((*i div 2*) ∗ *2* + *j mod 2*, (*j div 2*) ∗ *2* + *i mod 2*)
⟨*proof*⟩

lemma *product-vec-iff-det-zero*:
fixes *v* :: *complex Matrix.vec*
assumes *vd*: *dim-vec v* = *4*
shows (∃ α β. *dim-vec* α = *2* ∧ *dim-vec* β = *2* ∧ *v* = *tensor-vec* α β)
⟷ *v* $ *0* ∗ *v* $ *3* − *v* $ *1* ∗ *v* $ *2* = *0*
⟨*proof*⟩

The notes' witness $w = (-\overline{v_2}, \overline{v_1})$ gives $\langle w|\Phi^{(2)}(|v\rangle\langle v|)w\rangle = -2|v_0v_3 - v_1v_2|^2$ ($= 2|\langle v_2|v_1\rangle|^2 - 2\|v_1\|^2\|v_2\|^2$ by Lagrange's identity).

lemma *partial-transpose-witness-value*:
fixes *v* :: *complex Matrix.vec*
assumes *vd*: *dim-vec v* = *4*
defines *w* ≡ *Matrix.vec 4* (λ*k*. *if k* = *0 then* − *cnj* (*v* $ *2*) *else if k* = *1 then* − *cnj* (*v* $ *3*)
else if k = *2 then cnj* (*v* $ *0*) *else cnj* (*v* $ *1*))
shows ⟨*w* | *amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*v* ⊘ *v*) ∗$_v$ *w*⟩
= − *2* ∗ *cnj* (*v* $ *0* ∗ *v* $ *3* − *v* $ *1* ∗ *v* $ *2*) ∗ (*v* $ *0* ∗ *v* $ *3* − *v* $ *1* ∗ *v* $ *2*)
⟨*proof*⟩

theorem *horodecki-peres-two-qubits*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec* $v = 4$
shows *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) ($v \oslash v$))
⟷ ($\exists \alpha\ \beta$. *dim-vec* $\alpha = 2 \wedge$ *dim-vec* $\beta = 2 \wedge v =$ *tensor-vec* $\alpha\ \beta$)
⟨*proof*⟩

corollary *horodecki-peres-entangled*:
fixes v :: *complex Matrix.vec*
assumes *vd*: *dim-vec* $v = 4$
shows *entangled 2 2* v ⟷
¬ *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) ($v \oslash v$))
⟨*proof*⟩

end

Part IV
Lectures 29–34: error correction

theory *Day29-Error-Correction*
imports *Day22-Convex-Sets*
begin

69 Day 29 (7 November 2011): classical error detection and correction

Days 23–28 are formalised in the theories Day23_ChoiExtremeUCP, Separable_Cone and Day24_Separability.

To transmit $v \in \mathbb{Z}_2^n$ through a channel that may flip bits (+1 in $\mathbb{Z}_2$), one encodes $v \mapsto \phi(v) \in \mathbb{Z}_2^{n+k}$, transmits, and decodes. Examples: repetition $v \mapsto (v, v)$ detects one error but corrects none and doubles the cost; the parity check $(a_1, \ldots, a_n) \mapsto (a_1, \ldots, a_n; a_1 + \cdots + a_n)$ detects one error cheaply; the Hamming $[7, 1, 3]$ code detects and corrects one error (theory Day29_Classical_Codes: the code as written detects one error but cannot correct it; the $[7, 4, 3]$ code corrects one).

Definition (notes). The *Hamming weight* $\|w\|$ of $w \in \mathbb{Z}_2^m$ is its number of non-zero entries; the weight of a code $S \subseteq \mathbb{Z}_2^m$ is $\min\{\|v - w\| : v \neq w \in S\}$, the least number of errors turning one code word into another; for a subspace code it is $\min\{\|v\| : 0 \neq v \in S\}$.

Decoding amounts to choosing a left inverse of ϕ (different left inverses correct differently); the best binary codes are cyclic codes, ideals in $\mathbb{Z}_2[x]/\langle x^m - 1\rangle$ (not formalised).

69.1 Binary strings, Hamming weight and distance

definition *hamming-weight* :: *nat list* ⇒ *nat* **where**
hamming-weight $v \equiv$ *length* (*filter* ($\lambda b.\ b \neq 0$) v)

definition *hamming-dist* :: *nat list* ⇒ *nat list* ⇒ *nat* **where**

hamming-dist v w ≡ *hamming-weight* (*map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*)

lemma *hamming-weight-Nil* [*simp*]: *hamming-weight* [] = *0*
⟨*proof*⟩

lemma *hamming-weight-Cons*:
hamming-weight (*a* # *v*)
= (*if a* ≠ *0 then Suc* (*hamming-weight v*) *else hamming-weight v*)
⟨*proof*⟩

lemma *hamming-dist-Nil* [*simp*]:
hamming-dist [] [] = *0*
⟨*proof*⟩

lemma *hamming-dist-Cons*:
hamming-dist (*a* # *v*) (*b* # *w*)
= (*if* (*a* + *b*) *mod 2* ≠ *0*
then Suc (*hamming-dist v w*)
else hamming-dist v w)
⟨*proof*⟩

lemma *hamming-dist-symm*:
assumes *length v* = *length w*
shows *hamming-dist v w* = *hamming-dist w v*
⟨*proof*⟩

lemma *hamming-dist-zero*:
assumes *lenw*: *length v* = *length w*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-w*: ∀ *x* ∈ *set w*. *x* < *2*
shows *hamming-dist v w* = *0* ⟷ *v* = *w*
⟨*proof*⟩

lemma *hamming-dist-self* [*simp*]:
hamming-dist v v = *0*
⟨*proof*⟩

69.2 Code distance

A code of distance at least $2t+1$ can correct t errors (recorded as the definition *corrects-errors*; the decoding theorem itself is not formalised).

definition *code-distance* :: *nat list set* ⇒ *nat* **where**
code-distance code ≡
if (∃ *v* ∈ *code*. ∃ *w* ∈ *code*. *v* ≠ *w*)
then Min {*hamming-dist v w* | *v w*. *v* ∈ *code* ∧ *w* ∈ *code* ∧ *v* ≠ *w*}
else 0

definition *corrects-errors* :: *nat list set* ⇒ *nat* ⇒ *bool* **where**
corrects-errors code t ≡ *code-distance code* ≥ *2*∗*t* + *1*

69.3 The parity-check and repetition codes

Parity check: append the sum of the bits; any single error changes the code word. Repetition doubles Hamming distances.

definition *parity-bit* :: *nat list* ⇒ *nat* **where**
parity-bit v ≡ *foldr* (λ*a b*. (*a* + *b*) *mod 2*) *v 0*

definition *parity-check-encode* :: *nat list* ⇒ *nat list* **where**
parity-check-encode v ≡ *v* @ [*parity-bit v*]

lemma *length-parity-check-encode* [*simp*]:
length (*parity-check-encode v*) = *length v* + *1*
⟨*proof*⟩

lemma *hamming-weight-pos-iff*:
0 < *hamming-weight v* ⟷ (∃ *x* ∈ *set v*. *x* ≠ *0*)
⟨*proof*⟩

lemma *hamming-dist-pos-iff*:
assumes *lenw*: *length v* = *length w*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-w*: ∀ *x* ∈ *set w*. *x* < *2*
shows *0* < *hamming-dist v w* ⟷ *v* ≠ *w*
⟨*proof*⟩

lemma *parity-bit-lt-2*: *parity-bit v* < *2*
⟨*proof*⟩

lemma *parity-check-encode-binary*:
assumes ∀ *x* ∈ *set v*. *x* < *2*
shows ∀ *x* ∈ *set* (*parity-check-encode v*). *x* < *2*
⟨*proof*⟩

lemma *parity-check-detects-one-error*:
assumes *lenE*: *length e* = *length v* + *1*
and *bits-v*: ∀ *x* ∈ *set v*. *x* < *2*
and *bits-e*: ∀ *x* ∈ *set e*. *x* < *2*
and *weight*: *hamming-weight e* = *1*
shows *parity-check-encode v*
≠ *map2* (λ*a b*. (*a* + *b*) *mod 2*) (*parity-check-encode v*) *e*
⟨*proof*⟩

definition *repetition-encode* :: *nat list* ⇒ *nat list* **where**
repetition-encode v ≡ *v* @ *v*

lemma *length-repetition-encode* [*simp*]:
length (*repetition-encode v*) = *2* ∗ *length v*
⟨*proof*⟩

lemma *hamming-dist-repetition-doubles*:
assumes *length v* = *length w*
shows *hamming-dist* (*repetition-encode v*) (*repetition-encode w*)
= *2* ∗ *hamming-dist v w*
⟨*proof*⟩

end
theory *Day29-Classical-Codes*
imports *Day29-Error-Correction*
begin

70 Days 29–30: classical codes over $\mathbb{Z}_2$

Days 29–30: classical codes over $\mathbb{Z}_2$. Words are lists over $\{0,1\}$; all finite checks are decided by *code-simp* (proof-producing evaluation, no oracle).

definition *bin-words* :: *nat* ⇒ *nat list set* **where**
bin-words n = *set* (*List.n-lists n* [*0*, *1*])

definition *xor-word* :: *nat list* ⇒ *nat list* ⇒ *nat list* **where**
xor-word v w = *map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*

70.1 The notes' "Hamming [7,1,3]" code does not correct one error

The notes encode $(a_1,\dots,a_7) \mapsto (a_1,\dots,a_7;\ a_1{+}a_2{+}a_3{+}a_4;\ a_1{+}a_2{+}a_5{+}a_6;\ a_1{+}a_3{+}a_5{+}a_7)$ and claim it detects and corrects one error. It detects one error, but two codewords are at distance 2: the word e_4 is at distance 1 from the codewords of 0 and of e_4, so no decoder corrects every single error.

definition *notes-hamming-encode* :: *nat list* ⇒ *nat list* **where**
notes-hamming-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*1* + *a*!*4* + *a*!*5*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*4* + *a*!*6*) *mod 2*]

definition *notes-hamming-code* :: *nat list set* **where**
notes-hamming-code = *notes-hamming-encode* ‘ *bin-words 7*

definition *flip-at* :: *nat* ⇒ *nat list* ⇒ *nat list* **where**
flip-at i w = *w*[*i* := (*w*!*i* + *1*) *mod 2*]

lemma *notes-hamming-code-iff*:
w ∈ *notes-hamming-code* ⟷ *take 7 w* ∈ *bin-words 7* ∧ *w* = *notes-hamming-encode* (*take 7 w*)
⟨*proof*⟩

lemma *bin-words-iff*: *w* ∈ *bin-words n* ⟷ *length w* = *n* ∧ *set w* ⊆ {*0*, *1*}
⟨*proof*⟩

lemma *bin-nth-lt2*: *set v* ⊆ {*0*,*1*} ⟹ *k* < *length v* ⟹ *v* ! *k* < (*2*::*nat*)
⟨*proof*⟩

lemma *xor-word-length* [*simp*]: *length* (*xor-word v w*) = *min* (*length v*) (*length w*)
⟨*proof*⟩

lemma *xor-word-nth*: *k* < *length v* ⟹ *k* < *length w* ⟹ *xor-word v w* ! *k* = (*v* ! *k* + *w* ! *k*) *mod 2*
⟨*proof*⟩

lemma *xor-word-cancel*:
assumes *length c* = *length v length c* = *length w set v* ⊆ {*0*,*1*} *set w* ⊆ {*0*,*1*}
shows *xor-word c v* = *xor-word c w* ⟷ *v* = *w*
⟨*proof*⟩

lemma *mod2-eq-iff*: (*p*::*nat*) *mod 2* = *q mod 2* ⟷ (*even p* ⟷ *even q*)
⟨*proof*⟩

The encoding is linear over $\mathbb{Z}_2$.

lemma *notes-hamming-encode-xor*:
assumes *length a* = *7 length b* = *7*
shows *notes-hamming-encode* (*xor-word a b*) = *xor-word* (*notes-hamming-encode a*) (*notes-hamming-encode b*)
⟨*proof*⟩

lemma *flip-at-xor*:
  assumes *set c* ⊆ {*0*,*1*} *i* < *length c*
  shows *flip-at i c* = *xor-word c* ((*replicate* (*length c*) *0*)[*i* := *1*])
⟨*proof*⟩

theorem *notes-hamming-detects-one-error*:
  ∀ *c* ∈ *notes-hamming-code*. ∀ *i* < *10*. *flip-at i c* ∉ *notes-hamming-code*
⟨*proof*⟩

theorem *notes-hamming-ambiguous-word*:
  defines *w* ≡ [*0*,*0*,*0*,*1*,*0*,*0*,*0*,*0*,*0*,*0*]
  shows *notes-hamming-encode* (*replicate 7 0*) ∈ *notes-hamming-code*
    and *notes-hamming-encode* [*0*,*0*,*0*,*1*,*0*,*0*,*0*] ∈ *notes-hamming-code*
    and *notes-hamming-encode* (*replicate 7 0*) ≠ *notes-hamming-encode* [*0*,*0*,*0*,*1*,*0*,*0*,*0*]
    and *hamming-dist* (*notes-hamming-encode* (*replicate 7 0*)) *w* = *1*
    and *hamming-dist* (*notes-hamming-encode* [*0*,*0*,*0*,*1*,*0*,*0*,*0*]) *w* = *1*
  ⟨*proof*⟩

theorem *notes-hamming-not-correcting*: ¬ *corrects-errors notes-hamming-code 1*
⟨*proof*⟩

70.2 The Hamming [7,4,3] code corrects one error

definition *hamming74-encode* :: *nat list* ⇒ *nat list* **where**
  hamming74-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*1* + *a*!*2* + *a*!*3*) *mod 2*]

definition *hamming74-code* :: *nat list set* **where**
  hamming74-code = *hamming74-encode* ‘ *bin-words 4*

lemma *ex1-in-set*: (∃!*c*. *c* ∈ *set xs* ∧ *P c*) ⟷ *card* (*set* (*filter P xs*)) = *1*
⟨*proof*⟩

theorem *hamming74-unique-decoding*:
  ∀ *w* ∈ *bin-words 7*. ∃!*c*. *c* ∈ *hamming74-code* ∧ *hamming-dist c w* ≤ *1*
  ⟨*proof*⟩

theorem *hamming74-corrects-one-error*: *corrects-errors hamming74-code 1*
⟨*proof*⟩

70.3 Probabilities of undetected and uncorrected errors (Day 30)

Each bit flips independently with probability p; an error pattern e of weight k on n bits has probability $p^k(1-p)^{n-k}$. An error is undetected when it turns the sent codeword into another codeword.

definition *word-prob* :: *real* ⇒ *nat* ⇒ *nat list* ⇒ *real* **where**
  word-prob p n e = *p* ^ *hamming-weight e* ∗ (*1* − *p*) ^ (*n* − *hamming-weight e*)

definition *err-prob* :: *real* ⇒ *nat* ⇒ *nat list set* ⇒ *real* **where**
  err-prob p n ES = (∑ *e* ∈ *ES*. *word-prob p n e*)

lemma *err-prob-list*: *distinct es* ⟹ *err-prob p n* (*set es*) = (∑ *e*←*es*. *word-prob p n e*)
  ⟨*proof*⟩

definition *rep-code* :: *nat list set* **where** *rep-code* = *repetition-encode* ‘ *bin-words 3*
definition *par-code* :: *nat list set* **where** *par-code* = *parity-check-encode* ‘ *bin-words 3*

lemma *rep-code-list*: *rep-code* = *set* (*map repetition-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

lemma *par-code-list*: *par-code* = *set* (*map parity-check-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

definition *undetected* :: *nat list set* ⇒ *nat* ⇒ *nat list* ⇒ *nat list set* **where**
undetected C n c = {*e* ∈ *bin-words n*. *e* ≠ *replicate n 0* ∧ *xor-word c e* ∈ *C*}

lemma *set-filter-bin*: {*e* ∈ *bin-words n*. *P e*} = *set* (*filter P* (*List.n-lists n* [*0*, *1*]))
⟨*proof*⟩

Repetition on three bits: $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$.

theorem *repetition-undetected-prob*:
assumes *c*: *c* ∈ *rep-code*
shows *err-prob p 6* (*undetected rep-code 6 c*) = *3* ∗ *p*^*2* ∗ (*1* − *p*)^*4* + *3* ∗ *p*^*4* ∗ (*1* − *p*)^*2* + *p*^*6*
⟨*proof*⟩

Parity on three bits: $\binom{4}{2}p^2(1-p)^2 + p^4$.

theorem *parity-undetected-prob*:
assumes *c*: *c* ∈ *par-code*
shows *err-prob p 4* (*undetected par-code 4 c*) = *6* ∗ *p*^*2* ∗ (*1* − *p*)^*2* + *p*^*4*
⟨*proof*⟩

The notes compare the two for $p < \frac{1}{2}$; the inequality in fact holds for all $0 < p < 1$.

theorem *repetition-better-than-parity*:
assumes *p0*: *0* < *p* **and** *p1*: *p* < *1*
shows *3* ∗ *p*^*2* ∗ (*1* − *p*)^*4* + *3* ∗ *p*^*4* ∗ (*1* − *p*)^*2* + *p*^*6* < *6* ∗ *p*^*2* ∗ (*1* − *p*)^*2* + (*p*::*real*)^*4*
⟨*proof*⟩

corollary *repetition-better-than-parity-notes*:
0 < *p* ⟹ *p* < *1/2* ⟹ *3* ∗ *p*^*2* ∗ (*1* − *p*)^*4* + *3* ∗ *p*^*4* ∗ (*1* − *p*)^*2* + *p*^*6* < *6* ∗ *p*^*2* ∗ (*1* − *p*)^*2* + (*p*::*real*)^*4*
⟨*proof*⟩

Majority rule $0 \mapsto 000$, $1 \mapsto 111$: decoding fails with probability $3p^2(1-p) + p^3$.

definition *maj-decode* :: *nat list* ⇒ *nat* **where**
maj-decode w = (*if 2* ≤ *sum-list w then 1 else 0*)

theorem *majority-uncorrected-prob*:
assumes *b*: *b* ∈ {*0*, *1*}
shows *err-prob p 3* {*e* ∈ *bin-words 3*. *maj-decode* (*xor-word* (*replicate 3 b*) *e*) ≠ *b*}
= *3* ∗ *p*^*2* ∗ (*1* − *p*) + *p*^*3*
⟨*proof*⟩

end
theory *Day29-Subspace-Codes*
imports *Day29-Classical-Codes*
begin

71 Day 29: Hamming distance and subspace codes (D29-4)

lemma *hamming-dist-xor*: *hamming-dist v w* = *hamming-weight* (*xor-word v w*)
⟨*proof*⟩

The triangle inequality for words of bits of equal length.

lemma *hamming-dist-triangle*:
assumes *length u = length v length v = length w*
and $\forall x \in set\ u.\ x < 2\ \forall x \in set\ v.\ x < 2\ \forall x \in set\ w.\ x < 2$
shows *hamming-dist u w* $\leq$ *hamming-dist u v + hamming-dist v w*
⟨*proof*⟩

For a subspace code (closed under addition mod 2) the distance is the least weight of a nonzero code word.

definition *zero-word* :: *nat* ⇒ *nat list* **where**
zero-word m = replicate m 0

lemma *xor-word-self*: *xor-word v v = zero-word* (*length v*)
⟨*proof*⟩

lemma *xor-word-zero*: $\forall x \in set\ v.\ x < 2 \Longrightarrow$ *xor-word v* (*zero-word* (*length v*)) = *v*
⟨*proof*⟩

lemma *xor-word-eq-zero-iff*:
assumes *length v = length w* $\forall x \in set\ v.\ x < 2\ \forall x \in set\ w.\ x < 2$
shows *xor-word v w = zero-word* (*length v*) $\longleftrightarrow$ *v = w*
⟨*proof*⟩

lemma *hamming-weight-zero-word* [*simp*]: *hamming-weight* (*zero-word m*) = *0*
⟨*proof*⟩

lemma *hamming-weight-eq-zero-iff*: $\forall x \in set\ v.\ x < 2 \Longrightarrow$ *hamming-weight v = 0* $\longleftrightarrow$ *v = zero-word* (*length v*)
⟨*proof*⟩

definition *subspace-code* :: *nat* ⇒ *nat list set* ⇒ *bool* **where**
subspace-code m S $\longleftrightarrow$ $(\forall v \in S.\ length\ v = m \wedge (\forall x \in set\ v.\ x < 2))$
$\wedge\ (\forall v \in S.\ \forall w \in S.\ xor\text{-}word\ v\ w \in S)$

lemma *xor-word-bits*: $\forall x \in set\ (xor\text{-}word\ v\ w).\ x < 2$
⟨*proof*⟩

theorem *subspace-code-distance*:
assumes *S*: *subspace-code m S* **and** *fin*: *finite S* **and** *two*: $\exists v \in S.\ \exists w \in S.\ v \neq w$
shows *code-distance S = Min* {*hamming-weight v* | *v*. $v \in S \wedge v \neq$ *zero-word m*}
⟨*proof*⟩

A code of distance at least $2t + 1$ corrects t errors: a received word is within distance t of at most one code word.

theorem *corrects-errors-unique-decoding*:
assumes *ce*: *corrects-errors S t* **and** *fin*: *finite S*
and *S*: $\forall v \in S.\ length\ v = m \wedge (\forall x \in set\ v.\ x < 2)$
and *r*: *length r = m* $\forall x \in set\ r.\ x < 2$
and *c*: *c* $\in$ *S hamming-dist r c* $\leq$ *t* **and** *c'*: *c'* $\in$ *S hamming-dist r c'* $\leq$ *t*
shows *c = c'*
⟨*proof*⟩

end

theory *Day30-Error-Probabilities*
imports *Day29-Error-Correction*
begin

72 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code

If a bit flips with probability p (independently), the repetition code on three bits leaves an error undetected with probability $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$, the parity code with probability $\binom{4}{2}p^2(1-p)^2 + p^4$ (the former is smaller for $p < \frac{1}{2}$), and the majority-rule code $0 \mapsto 000$, $1 \mapsto 111$ decodes wrongly with probability $3p^2(1-p)+p^3$ (theory Day29_Classical_Codes; the comparison holds for all $0 < p < 1$).

Quantum error correction: one cannot clone, but basis vectors can be copied and measurements can decode. With $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the allowed errors are qubit flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, ... The *three-qubit bit-flip code* encodes $a|0\rangle + b|1\rangle \mapsto a|000\rangle + b|111\rangle$ (two CNOTs). Decoding: (1) syndrome measurement with $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, $P_1 = |100\rangle\langle 100| + |011\rangle\langle 011|$, $P_2 = |010\rangle\langle 010| + |101\rangle\langle 101|$, $P_3 = |001\rangle\langle 001| + |110\rangle\langle 110|$, $\sum P_i^2 = I$, which leaves a state with at most one flip unchanged and reveals where the flip occurred; (2) recovery $R(Y) = P_0 Y P_0 + (X \otimes I \otimes I) P_1 Y P_1 (X \otimes I \otimes I) + \cdots$, a completely positive trace-preserving map with $R(|\psi_i\rangle\langle\psi_i|) = |\psi\rangle\langle\psi|$ after zero or one error. The code's recovery theorem is *bit-flip-code-recovery* in *Day30-BitFlip-Code* (syndrome projections, $\sum P_i = I$, R CPTP); the Pauli matrices and the bit-flip channel are below.

72.1 The Pauli matrices

The notes use two sign conventions for Y: $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ on Day 10 (the library's Y) and $\begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}$ on Day 32. *Pauli-Y* follows Day 32; both are Hermitian unitary involutions, and the Pauli identities below carry the corresponding signs. *Pauli-X* and *Pauli-Z* are the library's *Quantum.X* and *Quantum.Z*.

definition *Pauli-X* :: *complex Matrix.mat* **where**
Pauli-X ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if* (*i* = *0* ∧ *j* = *1*) ∨ (*i* = *1* ∧ *j* = *0*) *then 1 else 0*)

definition *Pauli-Y* :: *complex Matrix.mat* **where**
Pauli-Y ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i* = *0* ∧ *j* = *1 then* i
else if i = *1* ∧ *j* = *0 then* −i *else 0*)

definition *Pauli-Z* :: *complex Matrix.mat* **where**
Pauli-Z ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i* = *0* ∧ *j* = *0 then 1*
else if i = *1* ∧ *j* = *1 then* −*1 else 0*)

lemma *Pauli-X-carrier* [*simp*]: *Pauli-X* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Y-carrier* [*simp*]: *Pauli-Y* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Z-carrier* [*simp*]: *Pauli-Z* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-X-dagger*: *Pauli-X*† = *Pauli-X*
⟨*proof*⟩

lemma *Pauli-X-squared*: *Pauli-X* ∗ *Pauli-X* = *1*$_m$ *2*

⟨*proof*⟩

lemma *Pauli-X-unitary*: *unitary Pauli-X*
⟨*proof*⟩

lemma *Pauli-Y-dagger*: $Pauli\text{-}Y^\dagger = Pauli\text{-}Y$
⟨*proof*⟩

lemma *Pauli-Y-squared*: $Pauli\text{-}Y * Pauli\text{-}Y = 1_m\ 2$
⟨*proof*⟩

lemma *Pauli-Y-unitary*: *unitary Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Z-dagger*: $Pauli\text{-}Z^\dagger = Pauli\text{-}Z$
⟨*proof*⟩

lemma *Pauli-Z-squared*: $Pauli\text{-}Z * Pauli\text{-}Z = 1_m\ 2$
⟨*proof*⟩

lemma *Pauli-Z-unitary*: *unitary Pauli-Z*
⟨*proof*⟩

72.2 Pauli magic (Day 32)

Proposition (notes). I, X, Y, Z are orthogonal in M_2 and all have 2-norm $\sqrt{2}$.

proposition *pauli-orthogonal*:
shows $\langle\langle\ 1_m\ 2 \mid Pauli\text{-}X\ \rangle\rangle = 0$
and $\langle\langle\ 1_m\ 2 \mid Pauli\text{-}Y\ \rangle\rangle = 0$
and $\langle\langle\ 1_m\ 2 \mid Pauli\text{-}Z\ \rangle\rangle = 0$
and $\langle\langle\ Pauli\text{-}X \mid Pauli\text{-}Y\ \rangle\rangle = 0$
and $\langle\langle\ Pauli\text{-}X \mid Pauli\text{-}Z\ \rangle\rangle = 0$
and $\langle\langle\ Pauli\text{-}Y \mid Pauli\text{-}Z\ \rangle\rangle = 0$
⟨*proof*⟩

proposition *pauli-norm*:
shows $\langle\langle\ 1_m\ 2 \mid (1_m\ 2 :: complex\ Matrix.mat)\ \rangle\rangle = 2$
and $\langle\langle\ Pauli\text{-}X \mid Pauli\text{-}X\ \rangle\rangle = 2$
and $\langle\langle\ Pauli\text{-}Y \mid Pauli\text{-}Y\ \rangle\rangle = 2$
and $\langle\langle\ Pauli\text{-}Z \mid Pauli\text{-}Z\ \rangle\rangle = 2$
⟨*proof*⟩

72.3 The bit-flip channel (Day 31)

Doing nothing to ψ while a flip occurs with probability p produces the ensemble $\{1-p, \psi\}, \{p, X\psi\}$, i.e. the error map $E(Y) = (1-p)Y + pXYX^*$, which is completely positive and trace preserving.

definition *bit-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bit-flip-channel p ϱ ≡
complex-of-real (1 − *p*) ·$_m$ *ϱ*
+ *complex-of-real p* ·$_m$ (*Pauli-X* ∗ *ϱ* ∗ (*Pauli-X*†))

lemma *bit-flip-channel-carrier*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *bit-flip-channel p ϱ* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-trace-preserving*:
fixes p :: *real*
assumes ϱ-*c*: $\varrho \in$ *carrier-mat 2 2*
shows *Tr* (*bit-flip-channel p* ϱ) = *Tr* ϱ
⟨*proof*⟩

lemma *bit-flip-channel-psd*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$
and *p-le-1*: $p \leq 1$
and ϱ-*c*: $\varrho \in$ *carrier-mat 2 2*
and ϱ-*psd*: *psd* ϱ
shows *psd* (*bit-flip-channel p* ϱ)
⟨*proof*⟩

end
theory *Day30-BitFlip-Code*
imports *QC-Prelims Quantum-Channels*
begin

73 Day 30: the three-qubit bit-flip code and its recovery map (D30-3, D30-4)

The syndrome projections $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, …, as diagonal 0/1 matrices ($|abc\rangle \leftrightarrow e_{4a+2b+c}$), and the single flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, $I \otimes I \otimes X$ as permutation matrices.

definition *syndrome-proj* :: *nat set* ⇒ *complex Matrix.mat* **where**
syndrome-proj K = *Matrix.mat 8 8* (λ(*i*,*j*). *if i* = *j* ∧ *i* ∈ *K then 1 else 0*)

abbreviation P_0 ≡ *syndrome-proj* {*0*, *7*}
abbreviation P_1 ≡ *syndrome-proj* {*4*, *3*}
abbreviation P_2 ≡ *syndrome-proj* {*2*, *5*}
abbreviation P_3 ≡ *syndrome-proj* {*1*, *6*}

definition *flip-bit-mat* :: *nat* ⇒ *complex Matrix.mat* **where**
flip-bit-mat b = *Matrix.mat 8 8* (λ(*i*,*j*). *if j* = (*if i div 2^b mod 2* = *0 then i* + *2^b else i* − *2^b*) *then 1 else 0*)

abbreviation X_1 ≡ *flip-bit-mat 2*
abbreviation X_2 ≡ *flip-bit-mat 1*
abbreviation X_3 ≡ *flip-bit-mat 0*

lemma *syndrome-proj-carrier* [*simp*]: *syndrome-proj K* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *flip-bit-mat-carrier* [*simp*]: *flip-bit-mat b* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *syndrome-proj-dims* [*simp*]: *dim-row* (*syndrome-proj K*) = *8 dim-col* (*syndrome-proj K*) = *8*
⟨*proof*⟩

lemma *flip-bit-mat-dims* [*simp*]: *dim-row* (*flip-bit-mat b*) = *8 dim-col* (*flip-bit-mat b*) = *8*
⟨*proof*⟩

lemma *encode-mat-dims* [*simp*]: *dim-row encode-mat = 8 dim-col encode-mat = 2*
⟨*proof*⟩

lemma *flip-proj-carrier* [*simp*]: *flip-bit-mat b ∗ syndrome-proj K ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *b3-cases*: *b < (3::nat) ⟹ b = 0 ∨ b = 1 ∨ b = 2* ⟨*proof*⟩

Entry of an 8×8 product, kept as an abstract sum: *simp* unrolls $\sum k{<}8$ into eight terms and then splits every *if*, which does not terminate in practice.

lemma *mat-mult-entry-8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A ∈ carrier-mat 8 8 B ∈ carrier-mat 8 8 i < 8 j < 8*
shows *(A ∗ B) $$ (i,j) = (∑ k<8. A $$ (i,k) ∗ B $$ (k,j))*
⟨*proof*⟩

lemma *syndrome-proj-index*:
assumes *i < 8 j < 8*
shows *syndrome-proj K $$ (i,j) = (if i = j ∧ i ∈ K then 1 else 0)*
⟨*proof*⟩

lemma *syndrome-proj-two*:
assumes *a*: *a < 8* **and** *b*: *b < 8* **and** *ab*: *a ≠ b*
shows *syndrome-proj {a, b} = unit-vec 8 a ⊗ unit-vec 8 a + unit-vec 8 b ⊗ unit-vec 8 b*
⟨*proof*⟩

lemma *syndrome-proj-as-outer*:
P_0 = unit-vec 8 0 ⊗ unit-vec 8 0 + unit-vec 8 7 ⊗ unit-vec 8 7
P_1 = unit-vec 8 4 ⊗ unit-vec 8 4 + unit-vec 8 3 ⊗ unit-vec 8 3
P_2 = unit-vec 8 2 ⊗ unit-vec 8 2 + unit-vec 8 5 ⊗ unit-vec 8 5
P_3 = unit-vec 8 1 ⊗ unit-vec 8 1 + unit-vec 8 6 ⊗ unit-vec 8 6
⟨*proof*⟩

The flips are the notes' tensors.

lemma *tensor3-index*:
fixes *A B C* :: *complex Matrix.mat*
assumes *A*: *A ∈ carrier-mat 2 2* **and** *B*: *B ∈ carrier-mat 2 2* **and** *C*: *C ∈ carrier-mat 2 2*
and *i*: *i < 8* **and** *j*: *j < 8*
shows *Tensor.tensor-mat (Tensor.tensor-mat A B) C $$ (i,j)*
= A $$ (i div 4, j div 4) ∗ B $$ (i div 2 mod 2, j div 2 mod 2) ∗ C $$ (i mod 2, j mod 2)
⟨*proof*⟩

lemma *flips-as-tensors*:
X_1 = Tensor.tensor-mat (Tensor.tensor-mat Pauli-X (1_m 2)) (1_m 2)
X_2 = Tensor.tensor-mat (Tensor.tensor-mat (1_m 2) Pauli-X) (1_m 2)
X_3 = Tensor.tensor-mat (Tensor.tensor-mat (1_m 2) (1_m 2)) Pauli-X
⟨*proof*⟩

lemma *flip-bit-mat-mult-vec*:
assumes *b*: *b < 3* **and** *vd*: *dim-vec v = 8* **and** *i*: *i < 8*
shows *(flip-bit-mat b ∗$_v$ v) $ i = v $ (if i div 2^b mod 2 = 0 then i + 2^b else i − 2^b)*
⟨*proof*⟩

lemma *syndrome-proj-mult-vec*:
assumes *vd*: *dim-vec v = 8* **and** *i*: *i < 8*
shows *(syndrome-proj K ∗$_v$ v) $ i = (if i ∈ K then v $ i else 0)*
⟨*proof*⟩

lemma *syndrome-proj-hermitian*: $(\textit{syndrome-proj } K)^\dagger = \textit{syndrome-proj } K$
⟨*proof*⟩

lemma *syndrome-proj-idem*: $\textit{syndrome-proj } K * \textit{syndrome-proj } K = \textit{syndrome-proj } K$
⟨*proof*⟩

lemma *syndrome-proj-sum*:
$P_0 + P_1 + P_2 + P_3 = 1_m\ 8$
⟨*proof*⟩

The flip *flip-bit-mat b* permutes the basis by the involution *flip-tgt b*.

definition *flip-tgt* :: *nat* ⇒ *nat* ⇒ *nat* **where**
flip-tgt b i = (*if i div 2^b mod 2 = 0 then i + 2^b else i − 2^b*)

lemma *flip-tgt-lt*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **shows** *flip-tgt b i* < 8
⟨*proof*⟩

lemma *flip-tgt-invol*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **shows** *flip-tgt b* (*flip-tgt b i*) $= i$
⟨*proof*⟩

lemma *flip-bit-mat-index*:
assumes *i*: $i < 8$ **and** *j*: $j < 8$
shows *flip-bit-mat b* $$ (*i*,*j*) = (*if j = flip-tgt b i then 1 else 0*)
⟨*proof*⟩

lemma *flip-tgt-iff*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **and** *j*: $j < 8$
shows $i =$ *flip-tgt b j* $\longleftrightarrow$ $j =$ *flip-tgt b i*
⟨*proof*⟩

lemma *flip-bit-mat-hermitian*:
assumes *b*: $b < 3$
shows $(\textit{flip-bit-mat } b)^\dagger = \textit{flip-bit-mat } b$
⟨*proof*⟩

lemma *flip-bit-mat-squared*:
assumes *b*: $b < 3$
shows *flip-bit-mat b* $*$ *flip-bit-mat b* $= 1_m\ 8$
⟨*proof*⟩

Kraus operators of the recovery map $R(Y) = P_0 Y P_0 + \sum_k X_k P_k Y P_k X_k$.

definition *recovery-ops* :: *complex Matrix.mat list* **where**
recovery-ops = $[P_0, X_1 * P_1, X_2 * P_2, X_3 * P_3]$

definition *recovery-map* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
recovery-map = *kraus-map-dim 8 recovery-ops*

lemma *flip-proj-dag-mult*:
assumes *b*: $b < 3$
shows ((*flip-bit-mat b* $*$ *syndrome-proj K*)†) $*$ (*flip-bit-mat b* $*$ *syndrome-proj K*) = *syndrome-proj K*
⟨*proof*⟩

theorem *recovery-ops-kraus*: *kraus-operators 8 8 recovery-ops*
⟨*proof*⟩

corollary *recovery-map-CPTP*: *recovery-map* ∈ *CPTP 8 8*

⟨*proof*⟩

The encoded state $\psi = a|000\rangle + b|111\rangle$ and its single-flip corruptions.

lemma *encode-mat-mult-vec*:
assumes *xd*: *dim-vec x = 2* **and** *i*: *i < 8*
shows (*encode-mat* $*_v$ *x*) $ *i* = (*if i = 0 then x $ 0 else if i = 7 then x $ 1 else 0*)
⟨*proof*⟩

lemma *encode-mat-dim* [*simp*]: *dim-vec* (*encode-mat* $*_v$ *x*) = *8* ⟨*proof*⟩

lemma *recovery-kraus-term*:
fixes φ :: *complex Matrix.vec*
assumes *φd*: *dim-vec* φ = *8*
shows *recovery-map* ($\varphi \oslash \varphi$) =
(*P*$_0$ $*_v$ φ) $\oslash$ (*P*$_0$ $*_v$ φ) + ((*X*$_1$ $*_v$ (*P*$_1$ $*_v$ φ)) $\oslash$ (*X*$_1$ $*_v$ (*P*$_1$ $*_v$ φ))
+ ((*X*$_2$ $*_v$ (*P*$_2$ $*_v$ φ)) $\oslash$ (*X*$_2$ $*_v$ (*P*$_2$ $*_v$ φ))
+ ((*X*$_3$ $*_v$ (*P*$_3$ $*_v$ φ)) $\oslash$ (*X*$_3$ $*_v$ (*P*$_3$ $*_v$ φ)) + *0*$_m$ *8 8*)))
⟨*proof*⟩

theorem *bit-flip-code-recovery*:
fixes *x* :: *complex Matrix.vec*
assumes *xd*: *dim-vec x = 2*
defines $\psi \equiv$ *encode-mat* $*_v$ *x*
shows *recovery-map* ($\psi \oslash \psi$) = $\psi \oslash \psi$
and *recovery-map* ((*X*$_1$ $*_v$ ψ) $\oslash$ (*X*$_1$ $*_v$ ψ)) = $\psi \oslash \psi$
and *recovery-map* ((*X*$_2$ $*_v$ ψ) $\oslash$ (*X*$_2$ $*_v$ ψ)) = $\psi \oslash \psi$
and *recovery-map* ((*X*$_3$ $*_v$ ψ) $\oslash$ (*X*$_3$ $*_v$ ψ)) = $\psi \oslash \psi$
⟨*proof*⟩

end

theory *Day31-Bit-Flip-Channel*
imports *Day30-Error-Probabilities*
begin

74 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity

Encoding $\tilde{\psi} = a|000\rangle + b|111\rangle$ and flipping each qubit independently with probability p gives the error map $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_j (X_j) Y (X_j)^* + \cdots + p^3 (X \otimes X \otimes X) Y (X \otimes X \otimes X)^*$, and with the recovery R of Day 30,

$$R \circ E(Y) = \big((1-p)^3 + 3p(1-p)^2\big) Y + \big(3p^2(1-p) + p^3\big)(X \otimes X \otimes X) Y (X \otimes X \otimes X).$$

The code helps iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$ (theory Day31_Code_Fidelity, where the formula for $R \circ E$ holds for Y supported on the code space). Below: the encoding isometry.

definition *encode-mat* :: *complex Matrix.mat* **where**
encode-mat ≡ *Matrix.mat 8 2* (λ(*i*,*j*).
if i = 0 ∧ j = 0 then 1
else if i = 7 ∧ j = 1 then 1
else 0)

lemma *encode-mat-carrier* [*simp*]: *encode-mat* ∈ *carrier-mat 8 2*
⟨*proof*⟩

74.1 Fidelity

A state ψ perturbed to $\psi' = a\psi + b\psi^{\perp}$ is close to ψ when $|a|$ is close to 1, so the *fidelity* of states is $F = |\langle\psi'|\psi\rangle|$; for the density matrices $\rho = |\psi\rangle\langle\psi|$, $\rho' = |\psi'\rangle\langle\psi'|$ one has $\langle\rho|\rho'\rangle = \mathrm{Tr}(\rho\rho') = |\langle\psi'|\psi\rangle|^2$, so $F = \sqrt{\mathrm{Tr}(\rho\rho')}$. Error correction is compared by $F = \sqrt{\langle\psi|R \circ E(|\psi\rangle\langle\psi|)\psi\rangle}$, minimised or averaged over ψ: doing nothing gives $\min F = \sqrt{1-p}$, the bit-flip code gives $\min F = \sqrt{(1-p)^3 + 3p(1-p)^2}$ (theory Day31_Code_Fidelity). Formally *fidelity* ψ ϱ is $\sqrt{\Re\langle\psi|\rho\psi\rangle}$.

definition *fidelity* ::
  complex Matrix.vec ⇒ *complex Matrix.mat* ⇒ *real* **where**
  fidelity ψ ϱ ≡ *sqrt* (*Re* ($\langle\psi$ | ϱ $*_v$ $\psi\rangle$))

lemma *fidelity-def-eq*:
  fidelity ψ ϱ = *sqrt* (*Re* ($\langle\psi$ | ϱ $*_v$ $\psi\rangle$))
  ⟨*proof*⟩

lemma *fidelity-pure-states*:
  fixes ψ ψ' :: *complex Matrix.vec*
  assumes ψ-*qs*: *quantum-state n* ψ
    and ψ'-*qs*: *quantum-state n* ψ'
  shows *fidelity* ψ ((*mat-of-cols n* [ψ']) ∗ ((*mat-of-cols n* [ψ'])†))
      = *cmod* ($\langle\psi$ | $\psi'\rangle$)
⟨*proof*⟩

lemma *fidelity-self*:
  fixes ψ :: *complex Matrix.vec*
  assumes ψ-*qs*: *quantum-state n* ψ
  shows *fidelity* ψ ((*mat-of-cols n* [ψ]) ∗ ((*mat-of-cols n* [ψ])†)) = *1*
⟨*proof*⟩

lemma *fidelity-orthogonal*:
  fixes ψ ψ' :: *complex Matrix.vec*
  assumes ψ-*qs*: *quantum-state n* ψ
    and ψ'-*qs*: *quantum-state n* ψ'
    and *ortho*: $\langle\psi$ | $\psi'\rangle$ = *0*
  shows *fidelity* ψ ((*mat-of-cols n* [ψ']) ∗ ((*mat-of-cols n* [ψ'])†)) = *0*
  ⟨*proof*⟩

end
theory *Day31-Code-Fidelity*
  imports *Day30-BitFlip-Code*
begin

75 Day 31: the three-qubit code against independent bit flips

Day 31: the three-qubit code against independent bit flips. Each qubit flips with probability p, so $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_i X_i Y X_i + p^2(1-p)\sum_{i<j} X_i X_j Y X_j X_i + p^3\, X_1X_2X_3 Y X_3X_2X_1$. On an encoded state the recovery R leaves $((1-p)^3 + 3p(1-p)^2)|\tilde\psi\rangle\langle\tilde\psi| + (3p^2(1-p) + p^3)|X_1X_2X_3\tilde\psi\rangle\langle X_1X_2X_3\tilde\psi|$. The notes state this formula for a general Y; it holds for Y supported on the code space (for $Y = |100\rangle\langle 100|$ and $p = 0$ one gets $R(Y) = |000\rangle\langle 000| \neq Y$).

definition *three-bit-channel* :: *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  three-bit-channel p M =

complex-of-real ((1 − *p*)^2) ·$_m$ *M*
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_1 ∗ *M* ∗ (X_1†))
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_2 ∗ *M* ∗ (X_2†))
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_3 ∗ *M* ∗ (X_3†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_2 ∗ X_3) ∗ *M* ∗ ((X_2 ∗ X_3)†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_1 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_3)†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_1 ∗ X_2) ∗ *M* ∗ ((X_1 ∗ X_2)†))
+ *complex-of-real* (*p*^3) ·$_m$ ((X_1 ∗ X_2 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_2 ∗ X_3)†))

definition *swap2* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
swap2 x = *Matrix.vec 2* (λ*i*. *x* $ (1 − *i*))

lemma *swap2-dim* [*simp*]: *dim-vec* (*swap2 x*) = 2 ⟨*proof*⟩

lemma *flips-on-codeword*:
assumes *x*: *dim-vec x* = 2
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows (X_1 ∗ X_2 ∗ X_3) ∗$_v$ ψ = φ
and (X_2 ∗ X_3) ∗$_v$ ψ = X_1 ∗$_v$ φ
and (X_1 ∗ X_3) ∗$_v$ ψ = X_2 ∗$_v$ φ
and (X_1 ∗ X_2) ∗$_v$ ψ = X_3 ∗$_v$ φ
⟨*proof*⟩

75.1 Errors followed by recovery

lemma *recovery-ops-carrier'*: ∀ *W* ∈ *set recovery-ops*. *W* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *recovery-map-add*: *A* ∈ *carrier-mat 8 8* ⟹ *B* ∈ *carrier-mat 8 8* ⟹ *recovery-map* (*A* + *B*) = *recovery-map A* + *recovery-map B*
⟨*proof*⟩

lemma *recovery-map-smult*: *A* ∈ *carrier-mat 8 8* ⟹ *recovery-map* (*c* ·$_m$ *A*) = *c* ·$_m$ *recovery-map A*
⟨*proof*⟩

lemma *comb8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n n* **and** *B*: *B* ∈ *carrier-mat n n*
shows *complex-of-real a* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A*
+ *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real d* ·$_m$ *B*
= *complex-of-real* (*a* + 3 ∗ *b*) ·$_m$ *A* + *complex-of-real* (3 ∗ *c* + *d*) ·$_m$ *B*
⟨*proof*⟩

lemma *outer8*: *dim-vec v* = 8 ⟹ *v* ⊘ *v* ∈ *carrier-mat 8 8*
⟨*proof*⟩

D31-2: errors followed by recovery on an encoded state.

theorem *three-bit-code-recovery*:
assumes *x*: *dim-vec x* = 2
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows *recovery-map* (*three-bit-channel p* (ψ ⊘ ψ))
= *complex-of-real* ((1 − *p*)^3 + 3 ∗ *p* ∗ (1 − *p*)^2) ·$_m$ (ψ ⊘ ψ)
+ *complex-of-real* (3 ∗ *p*^2 ∗ (1 − *p*) + *p*^3) ·$_m$ (φ ⊘ φ)
⟨*proof*⟩

The code is better than doing nothing iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$.

theorem *three-bit-code-better-iff*:
assumes *p0*: $0 \le p$ **and** *p1*: $p < 1$
shows $(1 - p) \le (1 - p)\hat{}3 + 3 * p * (1 - p)\hat{}2 \longleftrightarrow p \le (1/2 :: real)$
⟨*proof*⟩

75.2 Minimum fidelities (D31-4)

lemma *quad-two-outer*:
fixes ψ φ :: *complex Matrix.vec*
assumes ψd: *dim-vec* $\psi = n$ **and** φd: *dim-vec* $\varphi = n$
shows $\langle \psi \mid (a \cdot_m (\psi \oslash \psi) + b \cdot_m (\varphi \oslash \varphi)) *_v \psi \rangle$
$= a * (\langle \psi | \psi \rangle * \langle \psi | \psi \rangle) + b * (\langle \varphi | \psi \rangle * \langle \psi | \varphi \rangle)$
⟨*proof*⟩

lemma *encode-inner*:
assumes *u*: *dim-vec u* = *2* **and** *v*: *dim-vec v* = *2*
shows ⟨*encode-mat* $*_v$ *u* | *encode-mat* $*_v$ *v*⟩ = ⟨*u*|*v*⟩
⟨*proof*⟩

theorem *code-min-fidelity*:
assumes *p0*: *0* ≤ *p* **and** *p1*: *p* ≤ *1*
defines *c1* ≡ (*1* − *p*)^*3* + *3* * *p* * (*1* − *p*)^*2*
shows ∀ *x*. *dim-vec x* = *2* ⟶ ⟨*x*|*x*⟩ = *1* ⟶
sqrt c1 ≤ *fidelity* (*encode-mat* $*_v$ *x*) (*recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *x*) ⊘ (*encode-mat* $*_v$ *x*))))
and *fidelity* (*encode-mat* $*_v$ *unit-vec 2 0*)
(*recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *unit-vec 2 0*) ⊘ (*encode-mat* $*_v$ *unit-vec 2 0*)))) = *sqrt c1*
⟨*proof*⟩

theorem *nothing-min-fidelity*:
assumes *p0*: *0* ≤ *p* **and** *p1*: *p* ≤ *1*
shows ∀ ψ. *dim-vec* ψ = *2* ⟶ ⟨ψ|ψ⟩ = *1* ⟶ *sqrt* (*1* − *p*) ≤ *fidelity* ψ (*bit-flip-channel p* (ψ ⊘ ψ))
and *fidelity* (*unit-vec 2 0*) (*bit-flip-channel p* (*unit-vec 2 0* ⊘ *unit-vec 2 0*)) = *sqrt* (*1* − *p*)
⟨*proof*⟩

end

theory *Day32-Phase-Flip-Code*
imports *Day31-Bit-Flip-Channel*
begin

76 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic

76.1 The phase-flip code

With $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$ and $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ one has $|+\rangle \perp |-\rangle$, $Z|+\rangle = |-\rangle$, $Z|-\rangle = |+\rangle$: Z acts on $\{|+\rangle, |-\rangle\}$ as the bit flip acts on $\{|0\rangle, |1\rangle\}$. Encoding $|0\rangle \mapsto |++\rangle$, $|1\rangle \mapsto |--\rangle$ gives the three-qubit *phase-flip code*, which detects and corrects phase flips exactly as the bit-flip code corrects bit flips; with $\beta = Y|+\rangle$ (orthogonal to $|+\rangle$, $Y\beta = |+\rangle$) the same works for Y.

The *Shor code* encodes $|0\rangle \mapsto |0_L\rangle = 2^{-3/2}(|000\rangle + |111\rangle)^{\otimes 3}$ and $|1\rangle \mapsto |1_L\rangle = 2^{-3/2}(|000\rangle - |111\rangle)^{\otimes 3}$ in $\mathbb{C}^{2^9}$.

Theorem (notes) (Shor)**.** There is a completely positive trace-preserving $R : M_{2^9} \to M_{2^9}$

such that for every 2×2 unitary U, every position j, $\tilde{U} = I \otimes \cdots \otimes U \otimes \cdots \otimes I$, and every $\psi = a|0_L\rangle + b|1_L\rangle$, $R(\tilde{U}|\psi\rangle\langle\psi|\tilde{U}^*) = |\psi\rangle\langle\psi|$.

Proposition (notes). If $V \subseteq \mathbb{C}^n$ is a subspace and $U_1, \ldots, U_t$ are unitaries with $U_i V \perp U_j V$ for $i \neq j$, then there is a CPTP R with $R(U_i|\psi\rangle\langle\psi|U_i^*) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ and all i (take $R(X) = \sum_i U_i^* P_i X P_i U_i$ with P_i the projection onto $U_i V$, $P_0 = I - \sum P_i$, $U_0 = I$).

The $3 \cdot 9 + 1$ *1-Pauli* operators $I \otimes \cdots \otimes U \otimes \cdots \otimes I$ (U a Pauli matrix) map $V = \mathrm{span}\{|0_L\rangle, |1_L\rangle\}$ to mutually orthogonal subspaces (up to the coincidences $Z_1 v = Z_2 v = Z_3 v$ etc.). The proposition is *recovery-single-error* in *Day32-Recovery-Proposition*, together with its linear-combination form *recovery-linear-combination* (Day 33); the Pauli expansion identity is *pauli-expansion-unitary* in *Day32-Pauli-Expansion*. The Shor code and Shor's theorem are *Day33-Shor-Code* (*shor-code-theorem*). As first stated, the orthogonality claim is false ($Z_1 \neq Z_2$ but $Z_1|0_L\rangle = Z_2|0_L\rangle$, *shor-notes-Z1-Z2*); Day 33 corrects it.

definition *ket-plus* :: *complex Matrix.vec* **where**
ket-plus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 + unit-vec 2 1*)

definition *ket-minus* :: *complex Matrix.vec* **where**
ket-minus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 − unit-vec 2 1*)

lemma *ket-plus-carrier* [*simp*]: *ket-plus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-minus-carrier* [*simp*]: *ket-minus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-plus-dim* [*simp*]: *dim-vec ket-plus = 2*
⟨*proof*⟩

lemma *ket-minus-dim* [*simp*]: *dim-vec ket-minus = 2*
⟨*proof*⟩

lemma *ket-plus-index-0*: *ket-plus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-index-1*: *ket-plus* $ *1 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-minus-index-0*: *ket-minus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-minus-index-1*: *ket-minus* $ *1 = − 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-unit*: ‖*ket-plus*‖ = *1*
⟨*proof*⟩

lemma *ket-minus-unit*: ‖*ket-minus*‖ = *1*
⟨*proof*⟩

lemma *ket-plus-minus-orthog*: ⟨*ket-plus* | *ket-minus*⟩ = *0*
⟨*proof*⟩

lemma *Pauli-Z-ket-plus*: *Pauli-Z* ∗$_v$ *ket-plus = ket-minus*
⟨*proof*⟩

lemma *Pauli-Z-ket-minus*: *Pauli-Z* $*_v$ *ket-minus* = *ket-plus*
⟨*proof*⟩

lemma *Pauli-X-ket-plus*: *Pauli-X* $*_v$ *ket-plus* = *ket-plus*
⟨*proof*⟩

lemma *Pauli-X-ket-minus*: *Pauli-X* $*_v$ *ket-minus* = − *ket-minus*
⟨*proof*⟩

The code words $|{+}{+}{+}\rangle$ and $|{-}{-}{-}\rangle$ of the three-qubit phase-flip code (an earlier version of this file called them *phase-code-zero*/*phase-code-one*; the nine-qubit Shor code words are not defined).

definition *phase-code-zero* :: *complex Matrix.vec* **where**
phase-code-zero ≡
let v = *unit-vec 2 0* + *unit-vec 2 1 in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$\cdot_v$ *tensor-vec* (*tensor-vec v v*) *v*

definition *phase-code-one* :: *complex Matrix.vec* **where**
phase-code-one ≡
let v = *unit-vec 2 0* − *unit-vec 2 1 in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$\cdot_v$ *tensor-vec* (*tensor-vec v v*) *v*

lemma *phase-code-zero-dim* [*simp*]: *dim-vec phase-code-zero* = *8*
⟨*proof*⟩

lemma *phase-code-one-dim* [*simp*]: *dim-vec phase-code-one* = *8*
⟨*proof*⟩

lemma *phase-code-zero-carrier* [*simp*]: *phase-code-zero* ∈ *carrier-vec 8*
⟨*proof*⟩

lemma *phase-code-one-carrier* [*simp*]: *phase-code-one* ∈ *carrier-vec 8*
⟨*proof*⟩

76.2 Projections

A *projection* on $\mathbb{C}^n$ is a Hermitian idempotent, the notion of Day 2; projections are the building blocks of syndrome measurements and recovery maps (Days 30–34). Day 2's theorem that such a P is the orthogonal projection onto its range follows the basic facts.

definition *projection* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
projection n P ≡
P ∈ *carrier-mat n n* ∧ *hermitian P* ∧ *P* ∗ *P* = *P*

lemma *projection-carrier*:
projection n P ⟹ *P* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *projection-hermitian*:
projection n P ⟹ *hermitian P*
⟨*proof*⟩

lemma *projection-idempotent*:
projection n P ⟹ *P* ∗ *P* = *P*
⟨*proof*⟩

lemma *projection-zero*: *projection n* $(0_m\ n\ n)$
⟨*proof*⟩

lemma *projection-one*: *projection n* $(1_m\ n)$
⟨*proof*⟩

lemma *projection-psd*:
assumes *projection n P*
shows *psd P*
⟨*proof*⟩

The theorem of Day 2: a Hermitian idempotent P is the orthogonal projection onto its range, i.e. $Pv = v$ for v in the range and $Pw = 0$ for w orthogonal to the range.

lemma *projection-fixes-range*:
fixes *P* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
assumes *P*: *projection n P* **and** *x*: *dim-vec x* = *n*
shows $P *_v (P *_v x) = P *_v x$
⟨*proof*⟩

lemma *projection-annihilates-orthogonal*:
fixes *P* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *P*: *projection n P* **and** *w*: *dim-vec w* = *n*
and *orth*: $\bigwedge x$ :: *complex Matrix.vec*. *dim-vec x* = *n* $\Longrightarrow \langle P *_v x \mid w\rangle = 0$
shows $P *_v w = 0_v\ n$
⟨*proof*⟩

lemma *projection-complement-idempotent*:
assumes *projection n P*
shows $(1_m\ n - P) * (1_m\ n - P) = 1_m\ n - P$
⟨*proof*⟩

76.3 More on the repetition code

lemma *hamming-weight-append*:
hamming-weight (*xs* @ *ys*) = *hamming-weight xs* + *hamming-weight ys*
⟨*proof*⟩

lemma *hamming-weight-repetition-encode*:
hamming-weight (*repetition-encode v*) = 2 ∗ *hamming-weight v*
⟨*proof*⟩

lemma *repetition-encode-binary*:
assumes ∀ *x* ∈ *set v*. *x* < 2
shows ∀ *x* ∈ *set* (*repetition-encode v*). *x* < 2
⟨*proof*⟩

76.4 Algebra of projections; towards Days 33 and 34

Day 33 constructs the Shor recovery map from the projections onto the 22 orthogonal subspaces $V, X_1V, \ldots, X_9V, Y_1V, \ldots, Y_9V, Z_1V, Z_4V, Z_7V$ and proves, using $U = a_0I + a_1X + a_2Y + a_3Z$ with $\sum |a_k|^2 = 1$ for unitary U (Pauli magic), that it corrects every single-qubit unitary error. Day 34 gives the general theory:

Proposition (notes). For $V \subseteq \mathbb{C}^n$ with projection P and CPTP maps E (errors) and R (recovery), $R \circ E(|\psi\rangle\langle\psi|) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ iff $R \circ E(PXP) = PXP$ for all X.

Theorem (notes) (1)**.** Let $E(X) = \sum_{i=1}^{r} E_i X E_i^*$ be CPTP. There is a CPTP R with $R \circ E(PXP) = PXP$ for all X iff $PE_i^* E_j P = \alpha_{ij} P$ for some scalars α_{ij}.

Theorem (notes) (2)**.** If R corrects E as in Theorem 1 and $G_i \in \mathrm{span}\{E_j\}$ with $\sum G_i^* G_i = I$, then R also corrects $G(X) = \sum_i G_i X G_i^*$.

Of the Knill–Laflamme results, the equivalence of Theorem 1's two readings (*recovery-states-iff-PXP*), the necessity of $PE_i^* E_j P = \alpha_{ij} P$ (*knill-laflamme-necessity*) and Theorem 2 (*knill-laflamme-closure*) are proved in *Day34-Knill-Laflamme*; the converse, the construction of R from the conditions, and the equivalence itself (*knill-laflamme*) are in *Day34-KL-Sufficiency*. The elementary algebra of orthogonal projections used by the proofs follows.

lemma *projection-complement*:
assumes *projection n P*
shows *projection n* (1_m *n* − *P*)
⟨*proof*⟩

Two projections are orthogonal if their products in both orders vanish.

definition *orthogonal-projections* ::
nat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
orthogonal-projections n P Q ≡
projection n P ∧ *projection n Q* ∧
P ∗ *Q* = 0_m *n n* ∧ *Q* ∗ *P* = 0_m *n n*

lemma *orthogonal-projections-carrier-left*:
orthogonal-projections n P Q ⟹ *P* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *orthogonal-projections-carrier-right*:
orthogonal-projections n P Q ⟹ *Q* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *orthogonal-projections-symm*:
orthogonal-projections n P Q ⟷ *orthogonal-projections n Q P*
⟨*proof*⟩

lemma *orthogonal-projections-sum*:
assumes *opq*: *orthogonal-projections n P Q*
shows *projection n* (*P* + *Q*)
⟨*proof*⟩

lemma *orthogonal-projections-decomp-id*:
assumes *orthogonal-projections n P Q*
shows *P* + *Q* + (1_m *n* − *P* − *Q*) = 1_m *n*
⟨*proof*⟩

end
theory *Day32-Pauli-Expansion*
imports *QC-Prelims Quantum-Channels*
begin

77 Day 32: the Pauli expansion identity for unitaries (D32-7)

lemma *cmod-parallelogram*:
fixes *a b* :: *complex*

shows $(cmod\ (a + b))^2 + (cmod\ (a - b))^2 = 2 * (cmod\ a)^2 + 2 * (cmod\ b)^2$
⟨*proof*⟩

proposition *pauli-expansion-unitary*:
fixes U :: *complex Matrix.mat* **and** *a0 a1 a2 a3* :: *complex*
assumes *Uu*: *unitary U*
and *U-eq*: $U = a0\ \cdot_m\ 1_m\ 2 + a1\ \cdot_m$ *Pauli-X* $+ a2\ \cdot_m$ *Pauli-Y* $+ a3\ \cdot_m$ *Pauli-Z*
shows $(cmod\ a0)^2 + (cmod\ a1)^2 + (cmod\ a2)^2 + (cmod\ a3)^2 = 1$
⟨*proof*⟩

end
theory *Day32-Y-Code*
imports *Day33-Shor-Code*
begin

78 Day 32: the code for Y flips

Day 32: the code for Y flips. With $\beta = Y|+\rangle$ one has $|+\rangle \perp \beta$, $Y|+\rangle = \beta$, $Y\beta = |+\rangle$, and the code $|0\rangle \mapsto |++\!+\rangle$, $|1\rangle \mapsto |\beta\beta\beta\rangle$ corrects one Y flip exactly as the bit-flip code corrects one X flip.

definition *ket-beta* :: *complex Matrix.vec* **where** *ket-beta* $= \mathrm{i}\ \cdot_v$ *ket-minus*

definition *ub* :: *bool* $\Rightarrow$ *complex Matrix.vec* **where** *ub s* = (*if s then ket-plus else ket-beta*)

lemma *ket-beta-dim* [*simp*]: *dim-vec ket-beta* = *2* ⟨*proof*⟩
lemma *ub-dim* [*simp*]: *dim-vec* (*ub s*) = *2* ⟨*proof*⟩

lemma *ket-beta-index*: *ket-beta* $ *0* $= \mathrm{i}$ / *sqrt 2 ket-beta* $ *1* $= -\ \mathrm{i}$ / *sqrt 2*
⟨*proof*⟩

lemma *Y-plus*: *Pauli-Y* $*_v$ *ket-plus* = *ket-beta*
⟨*proof*⟩

lemma *Y-beta*: *Pauli-Y* $*_v$ *ket-beta* = *ket-plus*
⟨*proof*⟩

lemma *Y-ub*: *Pauli-Y* $*_v$ *ub s* = *ub* ($\neg$ *s*)
⟨*proof*⟩

lemma *inner2*: *dim-vec u* = *2* $\Longrightarrow$ *dim-vec v* = *2* $\Longrightarrow \langle u|v\rangle =$ *cnj* (*u* $ *0*) * *v* $ *0* + *cnj* (*u* $ *1*) * *v* $ *1*
⟨*proof*⟩

lemma *ub-inner*: $\langle$*ub s* | *ub t*$\rangle$ = (*if s* = *t then 1 else 0*)
⟨*proof*⟩

theorem *ket-plus-beta-orth*: $\langle$*ket-plus* | *ket-beta*$\rangle$ = *0*
⟨*proof*⟩

78.1 Codes with orthogonal error images

For a code with orthonormal basis $v_1, \ldots, v_k$ (the columns of V, projection VV^*) and unitary errors whose images of the basis are mutually orthogonal, the recovery proposition gives one CPTP map correcting each error on every code state Vc.

lemma *proj-of-orthonormal*:
assumes *orth*: *orthonormal-list n vs*

defines $V \equiv$ *mat-of-cols n vs*
shows *projection n* $(V * (V^\dagger))$
⟨*proof*⟩

lemma *proj-fixes-code*:
assumes *orth*: *orthonormal-list n vs* **and** *c*: *dim-vec c* = *length vs*
defines $V \equiv$ *mat-of-cols n vs*
shows $(V * (V^\dagger)) *_v (V *_v c) = V *_v c$
⟨*proof*⟩

theorem *orthogonal-images-recovery*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: $\forall U \in$ *set Us*. $U \in$ *carrier-mat n n* $\wedge$ *unitary U*
and *img*: $\forall i <$ *length Us*. $\forall j <$ *length Us*. $i \neq j \longrightarrow$
$(\forall a <$ *length vs*. $\forall b <$ *length vs*. $\langle Us ! i *_v vs ! a \mid Us ! j *_v vs ! b\rangle = 0)$
defines $V \equiv$ *mat-of-cols n vs*
shows *recovery-setting n* $(V * (V^\dagger))$ *Us*
⟨*proof*⟩

corollary *orthogonal-images-correct*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: $\forall U \in$ *set Us*. $U \in$ *carrier-mat n n* $\wedge$ *unitary U*
and *img*: $\forall i <$ *length Us*. $\forall j <$ *length Us*. $i \neq j \longrightarrow$
$(\forall a <$ *length vs*. $\forall b <$ *length vs*. $\langle Us ! i *_v vs ! a \mid Us ! j *_v vs ! b\rangle = 0)$
shows $\exists$ *Rs. kraus-operators n n Rs* $\wedge$
$(\forall (c ::$ *complex Matrix.vec*$)$ *j. dim-vec c* = *length vs* $\longrightarrow$ *j* $<$ *length Us* $\longrightarrow$
kraus-map-dim n Rs $(Us ! j * ((mat\text{-}of\text{-}cols\ n\ vs *_v c) \oslash (mat\text{-}of\text{-}cols\ n\ vs *_v c)) * ((Us ! j)^\dagger))$
$= (mat\text{-}of\text{-}cols\ n\ vs *_v c) \oslash (mat\text{-}of\text{-}cols\ n\ vs *_v c))$
⟨*proof*⟩

78.2 The Y-flip code

definition *tv2* :: *bool* ⇒ *bool* ⇒ *bool* ⇒ *complex Matrix.vec* **where**
tv2 a b c = *tensor-vec* (*tensor-vec* (*ub a*) (*ub b*)) (*ub c*)

definition *y-errors* :: *complex Matrix.mat list* **where**
y-errors = $[1_m\ 8,$ *bop 0 Pauli-Y*, *bop 1 Pauli-Y*, *bop 2 Pauli-Y*$]$

definition *y-codewords* :: *complex Matrix.vec list* **where**
y-codewords = [*tv2 True True True*, *tv2 False False False*]

lemma *tv2-dim* [*simp*]: *dim-vec* (*tv2 a b c*) = 8
⟨*proof*⟩

lemma *tv2-inner*: $\langle$*tv2 a b c* | *tv2 a' b' c'*$\rangle$ = (*if* $a = a' \wedge b = b' \wedge c = c'$ *then 1 else 0*)
⟨*proof*⟩

lemma *y-actions*:
bop 0 Pauli-Y $*_v$ *tv2 a b c* = *tv2* $(\neg a)$ *b c*
bop 1 Pauli-Y $*_v$ *tv2 a b c* = *tv2 a* $(\neg b)$ *c*
bop 2 Pauli-Y $*_v$ *tv2 a b c* = *tv2 a b* $(\neg c)$
⟨*proof*⟩

lemma *y-codewords-orthonormal*: *orthonormal-list 8 y-codewords*
⟨*proof*⟩

lemma *y-errors-unitary*: $\forall U \in$ *set y-errors*. $U \in$ *carrier-mat 8 8* $\wedge$ *unitary U*

⟨*proof*⟩

lemma *y-images-orthogonal*:
∀ *i* < *length y-errors*. ∀ *j* < *length y-errors*. *i* ≠ *j* ⟶
(∀ *a* < *length y-codewords*. ∀ *b* < *length y-codewords*.
⟨*y-errors* ! *i* ∗$_v$ *y-codewords* ! *a* | *y-errors* ! *j* ∗$_v$ *y-codewords* ! *b*⟩ = *0*)
⟨*proof*⟩

D32-2: one CPTP map corrects every single Y flip on every state $a|++\rangle + b|\beta\beta\beta\rangle$.

theorem *y-code-corrects*:
∃ *Rs*. *kraus-operators 8 8 Rs* ∧
(∀ *c j*. *dim-vec c* = *2* ⟶ *j* < *4* ⟶
kraus-map-dim 8 Rs (*y-errors* ! *j* ∗ ((*mat-of-cols 8 y-codewords* ∗$_v$ *c*) ⊘ (*mat-of-cols 8 y-codewords* ∗$_v$ *c*)) ∗ ((*y-errors* ! *j*)†))
= (*mat-of-cols 8 y-codewords* ∗$_v$ *c*) ⊘ (*mat-of-cols 8 y-codewords* ∗$_v$ *c*))
⟨*proof*⟩

end
theory *Day32-Recovery-Proposition*
imports *Day21-Unitary-Freedom Quantum-Channels*
begin

79 Days 32–33: composition of Kraus maps and the general recovery proposition (D32-4, D33-2)

79.1 Algebra of *foldr*-sums of matrices

lemma *foldr-add-mat-mult-left*:
fixes *f* :: *'a* ⇒ *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m n* **and** *fc*: ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat n p*
shows *A* ∗ *foldr* (λ*x acc*. *f x* + *acc*) *xs* (*0*$_m$ *n p*) = *foldr* (λ*x acc*. *A* ∗ *f x* + *acc*) *xs* (*0*$_m$ *m p*)
⟨*proof*⟩

lemma *foldr-add-mat-mult-right*:
fixes *f* :: *'a* ⇒ *complex Matrix.mat*
assumes *Bc*: *B* ∈ *carrier-mat n p* **and** *fc*: ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat m n*
shows *foldr* (λ*x acc*. *f x* + *acc*) *xs* (*0*$_m$ *m n*) ∗ *B* = *foldr* (λ*x acc*. *f x* ∗ *B* + *acc*) *xs* (*0*$_m$ *m p*)
⟨*proof*⟩

lemma *foldr-add-mat-dagger*:
fixes *f* :: *'a* ⇒ *complex Matrix.mat*
assumes ∀ *x* ∈ *set xs*. *f x* ∈ *carrier-mat m n*
shows (*foldr* (λ*x acc*. *f x* + *acc*) *xs* (*0*$_m$ *m n*))† = *foldr* (λ*x acc*. (*f x*)† + *acc*) *xs* (*0*$_m$ *n m*)
⟨*proof*⟩

lemma *foldr-add-mat-cong*:
fixes *f g* :: *'a* ⇒ *complex Matrix.mat*
assumes ⋀*x*. *x* ∈ *set xs* ⟹ *f x* = *g x*
shows *foldr* (λ*x acc*. *f x* + *acc*) *xs z* = *foldr* (λ*x acc*. *g x* + *acc*) *xs z*
⟨*proof*⟩

lemma *foldr-add-mat-zero*:
foldr (λ*x acc*. (*0*$_m$ *m n* :: *complex Matrix.mat*) + *acc*) *xs* (*0*$_m$ *m n*) = *0*$_m$ *m n*
⟨*proof*⟩

lemma *foldr-add-mat-delta*:
fixes *f* :: *nat* ⇒ *complex Matrix.mat*

assumes *fc*: $\forall\, x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$ **and** *d*: *distinct xs* **and** *i*: $i \in set\ xs$
shows $foldr\ (\lambda x\ acc.\ (if\ x = i\ then\ f\ x\ else\ 0_m\ m\ n) + acc)\ xs\ (0_m\ m\ n) = f\ i$
⟨*proof*⟩

lemma *foldr-add-mat-smult-pull*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$ **and** $c :: 'a \Rightarrow complex$
assumes *fc*: $\forall\, x \in set\ xs.\ f\ x = c\ x \cdot_m M$ **and** *Mc*: $M \in carrier\text{-}mat\ m\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n) = sum\text{-}list\ (map\ c\ xs) \cdot_m M$
⟨*proof*⟩

lemma *foldr-list-as-index*:
fixes $g :: complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$
shows $foldr\ (\lambda A\ acc.\ g\ A + acc)\ As\ z = foldr\ (\lambda l\ acc.\ g\ (As\ !\ l) + acc)\ [0..<length\ As]\ z$
⟨*proof*⟩

79.2 Composition of Kraus maps

definition *kraus-compose* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
$kraus\text{-}compose\ As\ Es = concat\ (map\ (\lambda E.\ map\ (\lambda A.\ A * E)\ As)\ Es)$

lemma *kraus-compose-Nil* [*simp*]: $kraus\text{-}compose\ As\ [] = []$
⟨*proof*⟩

lemma *kraus-compose-Cons*: $kraus\text{-}compose\ As\ (E\ \#\ Es) = map\ (\lambda A.\ A * E)\ As\ @\ kraus\text{-}compose\ As\ Es$
⟨*proof*⟩

lemma *length-kraus-compose* [*simp*]: $length\ (kraus\text{-}compose\ As\ Es) = length\ Es * length\ As$
⟨*proof*⟩

lemma *kraus-compose-nth*:
assumes *i*: $i < length\ Es$ **and** *l*: $l < length\ As$
shows $kraus\text{-}compose\ As\ Es\ !\ (i * length\ As + l) = As\ !\ l * Es\ !\ i$
⟨*proof*⟩

lemma *kraus-compose-carrier*:
assumes $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** $\forall\, E \in set\ Es.\ E \in carrier\text{-}mat\ d\ n$
shows $\forall\, W \in set\ (kraus\text{-}compose\ As\ Es).\ W \in carrier\text{-}mat\ m\ n$
⟨*proof*⟩

lemma *kraus-map-dim-map-mult-right*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Ec*: $E \in carrier\text{-}mat\ d\ n$ **and** *Xc*: $M \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}map\text{-}dim\ m\ (map\ (\lambda A.\ A * E)\ As)\ M = kraus\text{-}map\text{-}dim\ m\ As\ (E * M * (E^\dagger))$
⟨*proof*⟩

lemma *kraus-map-dim-compose*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Es-c*: $\forall\, E \in set\ Es.\ E \in carrier\text{-}mat\ d\ n$
and *Xc*: $M \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}map\text{-}dim\ m\ (kraus\text{-}compose\ As\ Es)\ M = kraus\text{-}map\text{-}dim\ m\ As\ (kraus\text{-}map\text{-}dim\ d\ Es\ M)$
⟨*proof*⟩

lemma *sum-wdag-w-map-mult-right*:
assumes *As-c*: $\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Ec*: $E \in carrier\text{-}mat\ d\ n$
shows $sum\text{-}wdag\text{-}w\ n\ (map\ (\lambda A.\ A * E)\ As) = (E^\dagger) * sum\text{-}wdag\text{-}w\ d\ As * E$
⟨*proof*⟩

lemma *sum-wdag-w-compose*:
assumes *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ m\ d$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ d\ n$
and *As-tp*: $sum\text{-}wdag\text{-}w\ d\ As = 1_m\ d$
shows $sum\text{-}wdag\text{-}w\ n\ (kraus\text{-}compose\ As\ Es) = sum\text{-}wdag\text{-}w\ n\ Es$
⟨*proof*⟩

lemma *kraus-operators-compose*:
assumes *As*: *kraus-operators d m As* **and** *Es*: *kraus-operators n d Es*
shows *kraus-operators n m* (*kraus-compose As Es*)
⟨*proof*⟩

79.3 The general recovery proposition (D32-4) and its linear-combination form (D33-2)

79.4 Matrix sums acting on vectors

lemma *smult-zero-vec* [*simp*]: $(c :: complex) \cdot_v 0_v\ n = 0_v\ n$
⟨*proof*⟩

lemma *zero-mat-mult-vec*:
assumes *dim-vec* $w = n$ **shows** $(0_m\ m\ n :: complex\ Matrix.mat) *_v w = 0_v\ m$
⟨*proof*⟩

lemma *mult-mat-vec-zero-vec*:
assumes *Ac*: $A \in carrier\text{-}mat\ m\ n$ **shows** $A *_v (0_v\ n :: complex\ Matrix.vec) = 0_v\ m$
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-zero*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ n\ n$ **and** *wd*: *dim-vec* $w = n$
and *z*: $\bigwedge x.\ x \in set\ xs \Longrightarrow f\ x *_v w = 0_v\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ n) *_v w = 0_v\ n$
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-delta*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ n\ n$ **and** *wd*: *dim-vec* $w = n$
and *d*: *distinct xs* **and** *j*: $j \in set\ xs$
and *del*: $\bigwedge x.\ x \in set\ xs \Longrightarrow f\ x *_v w = (if\ x = j\ then\ w\ else\ 0_v\ n)$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ n) *_v w = w$
⟨*proof*⟩

lemma *vec-sum-delta*:
assumes *fin*: *finite Js* **and** *j*: $j \in Js$ **and** *wd*: $\bigwedge i.\ i \in Js \Longrightarrow dim\text{-}vec\ (w\ i) = n$
shows $vec\text{-}sum\ n\ Js\ (\lambda i.\ if\ i = j\ then\ w\ i\ else\ 0_v\ n) = w\ j$
⟨*proof*⟩

Let $V = \operatorname{ran} P$ for a projection P and let $U_0, \ldots, U_{t-1}$ be unitaries with $U_i V \perp U_j V$ for $i \neq j$, i.e. $P U_i^* U_j P = 0$. With $P_i = U_i P U_i^*$, $Q = I - \sum_i P_i$ and the Kraus operators $P U_i^*$ (the notes write $U_i^* P_i = P U_i^*$) the map $R(X) = QXQ + \sum_i P U_i^* X U_i P$ is CPTP and recovers every $\psi \in V$ from any $\varphi = \sum_i a_i U_i \psi$: $R(|\varphi\rangle\langle\varphi|) = (\sum_i |a_i|^2) |\psi\rangle\langle\psi|$.

definition *image-proj-sum* :: $nat \Rightarrow complex\ Matrix.mat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat$ **where**
$image\text{-}proj\text{-}sum\ n\ P\ Us = foldr\ (\lambda i\ acc.\ Us\ !\ i * P * ((Us\ !\ i)^\dagger) + acc)\ [0..<length\ Us]\ (0_m\ n\ n)$

definition *recovery-ops* :: $nat \Rightarrow complex\ Matrix.mat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat\ list$ **where**
$recovery\text{-}ops\ n\ P\ Us = (1_m\ n - image\text{-}proj\text{-}sum\ n\ P\ Us)\ \#\ map\ (\lambda i.\ P * ((Us\ !\ i)^\dagger))\ [0..<length\ Us]$

The hypotheses of the proposition, bundled: P a projection and $PU_i^* U_j P = \delta_{ij} P$. For unitaries U_i the case $i = j$ is automatic (*recovery-setting-unitaries*); the weaker form also covers the rescaled errors of the Knill–Laflamme theorem.

definition *recovery-setting* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
recovery-setting n P Us ⟷ *projection n P* ∧ (∀ *U* ∈ *set Us*. *U* ∈ *carrier-mat n n*)
∧ (∀ *i* < *length Us*. ∀ *j* < *length Us*. *P* ∗ (((*Us* ! *i*)†) ∗ *Us* ! *j*) ∗ *P* = (*if i* = *j then P else* 0_m *n n*))

lemma *recovery-settingD*:
assumes *recovery-setting n P Us*
shows *projection n P* ∀ *U* ∈ *set Us*. *U* ∈ *carrier-mat n n*
∀ *i* < *length Us*. ∀ *j* < *length Us*. *i* ≠ *j* ⟶ *P* ∗ (((*Us* ! *i*)†) ∗ *Us* ! *j*) ∗ *P* = 0_m *n n*
∀ *i* < *length Us*. *P* ∗ (((*Us* ! *i*)†) ∗ *Us* ! *i*) ∗ *P* = *P*
⟨*proof*⟩

lemma *recovery-setting-unitaries*:
assumes *P*: *projection n P* **and** *Us*: ∀ *U* ∈ *set Us*. *U* ∈ *carrier-mat n n* ∧ *unitary U*
and *orth*: ∀ *i* < *length Us*. ∀ *j* < *length Us*. *i* ≠ *j* ⟶ *P* ∗ (((*Us* ! *i*)†) ∗ *Us* ! *j*) ∗ *P* = 0_m *n n*
shows *recovery-setting n P Us*
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Pc*: *P* ∈ *carrier-mat n n* **and** *rec-Ph*: *P*† = *P* **and** *rec-PP*: *P* ∗ *P* = *P*
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Uc*: *i* < *length Us* ⟹ *Us* ! *i* ∈ *carrier-mat n n*
and *rec-PUUP*: *i* < *length Us* ⟹ *P* ∗ (((*Us* ! *i*)†) ∗ *Us* ! *i*) ∗ *P* = *P*
⟨*proof*⟩

lemma *rec-Pi-c*:
assumes *S*: *recovery-setting n P Us* **and** *i*: *i* < *length Us*
shows *Us* ! *i* ∗ *P* ∗ ((*Us* ! *i*)†) ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *rec-Pi-mult*:
assumes *S*: *recovery-setting n P Us* **and** *i*: *i* < *length Us* **and** *j*: *j* < *length Us*
shows (*Us* ! *i* ∗ *P* ∗ ((*Us* ! *i*)†)) ∗ (*Us* ! *j* ∗ *P* ∗ ((*Us* ! *j*)†))
= (*if i* = *j then Us* ! *i* ∗ *P* ∗ ((*Us* ! *i*)†) *else* 0_m *n n*)
⟨*proof*⟩

lemma *image-proj-sum-carrier*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *image-proj-sum-hermitian*:
assumes *S*: *recovery-setting n P Us*
shows (*image-proj-sum n P Us*)† = *image-proj-sum n P Us*
⟨*proof*⟩

lemma *image-proj-sum-idem*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* ∗ *image-proj-sum n P Us* = *image-proj-sum n P Us*
⟨*proof*⟩

lemma *image-proj-sum-projection*:
assumes *S*: *recovery-setting n P Us*
shows *projection n* (*image-proj-sum n P Us*)
⟨*proof*⟩

lemma *dagger-one-minus*:
assumes *Sc*: $M \in$ *carrier-mat n n* **and** *Sh*: $M^\dagger = M$
shows $(1_m\ n - M)^\dagger = 1_m\ n - M$
⟨*proof*⟩

lemma *recovery-ops-carrier*:
assumes *S*: *recovery-setting n P Us*
shows $\forall\ W \in$ *set* (*recovery-ops n P Us*). $W \in$ *carrier-mat n n*
⟨*proof*⟩

theorem *recovery-ops-kraus-operators*:
assumes *S*: *recovery-setting n P Us*
shows *kraus-operators n n* (*recovery-ops n P Us*)
⟨*proof*⟩

corollary *recovery-map-CPTP'*:
assumes *S*: *recovery-setting n P Us* **and** *npos*: $0 < n$
shows *kraus-map-dim n* (*recovery-ops n P Us*) $\in$ *CPTP n n*
⟨*proof*⟩

Action of the Kraus operators on $\varphi = \sum_i a_i U_i \psi$, $\psi \in V$.

lemma *rec-kraus-on-image*:
assumes *S*: *recovery-setting n P Us*
and ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$ **and** *i*: $i <$ *length Us* **and** *j*: $j <$ *length Us*
shows $(P * ((Us\ !\ i)^\dagger)) *_v (Us\ !\ j *_v \psi) =$ (*if* $i = j$ *then* ψ *else* $0_v\ n$)
⟨*proof*⟩

lemma *rec-Q-on-image*:
assumes *S*: *recovery-setting n P Us*
and ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$ **and** *j*: $j <$ *length Us*
shows $(1_m\ n -$ *image-proj-sum n P Us*$) *_v (Us\ !\ j *_v \psi) = 0_v\ n$
⟨*proof*⟩

theorem *recovery-linear-combination*:
fixes *a* :: *nat* ⇒ *complex*
assumes *S*: *recovery-setting n P Us* **and** ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$
defines $\varphi \equiv$ *vec-sum n* $\{i.\ i <$ *length Us*$\}$ $(\lambda i.\ a\ i \cdot_v (Us\ !\ i *_v \psi))$
shows *kraus-map-dim n* (*recovery-ops n P Us*) $(\varphi \oslash \varphi)$
$=$ *complex-of-real* $(\sum i <$ *length Us*. $(cmod\ (a\ i))^2) \cdot_m (\psi \oslash \psi)$
⟨*proof*⟩

corollary *recovery-single-error*:
assumes *S*: *recovery-setting n P Us*
and ψd: *dim-vec* $\psi = n$ **and** ψV: $P *_v \psi = \psi$ **and** *j*: $j <$ *length Us*
shows *kraus-map-dim n* (*recovery-ops n P Us*) $((Us\ !\ j *_v \psi) \oslash (Us\ !\ j *_v \psi)) = \psi \oslash \psi$
⟨*proof*⟩

end
theory *Day33-Shor-Code*
imports *Day32-Recovery-Proposition Day32-Pauli-Expansion Day30-BitFlip-Code*
begin

80 Days 32–33: the nine-qubit Shor code

Days 32–33: the nine-qubit Shor code. The 2^9-dimensional space is never handled entrywise: nine qubits are three blocks of three, every single-qubit operator acts inside one block, and all inner products factor over the blocks (*tensor-inner-prod*).

80.1 Tensor products of matrices

lemma *tensor-index-c*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r c B* ∈ *carrier-mat p q i* < *r* ∗ *p j* < *c* ∗ *q* 0 < *c* 0 < *q*
shows *Tensor.tensor-mat A B* $$ (*i*,*j*) = *A* $$ (*i div p*, *j div q*) ∗ *B* $$ (*i mod p*, *j mod q*)
⟨*proof*⟩

lemma *tensor-one-left*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: 0 < *m*
shows *Tensor.tensor-mat* (1$_m$ 1) *A* = *A*
⟨*proof*⟩

lemma *tensor-one-right*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: 0 < *m*
shows *Tensor.tensor-mat A* (1$_m$ 1) = *A*
⟨*proof*⟩

lemma *tensor-mat-add-right*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat A* (*B* + *C*) = *Tensor.tensor-mat A B* + *Tensor.tensor-mat A C*
⟨*proof*⟩

lemma *tensor-mat-add-left*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat r c* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat* (*A* + *B*) *C* = *Tensor.tensor-mat A C* + *Tensor.tensor-mat B C*
⟨*proof*⟩

lemma *tensor-mat-smult-right*:
fixes *A B* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat A* (*k* ·$_m$ *B*) = *k* ·$_m$ *Tensor.tensor-mat A B*
⟨*proof*⟩

lemma *tensor-mat-smult-left*:
fixes *A B* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat* (*k* ·$_m$ *A*) *B* = *k* ·$_m$ *Tensor.tensor-mat A B*
⟨*proof*⟩

80.2 Single-qubit operators on nine qubits, and on a block of three

$\tilde{U} = I \otimes \cdots \otimes I \otimes U \otimes I \otimes \cdots \otimes I$ with U in position i ($i = 0, \ldots, 8$).

definition *qubit-op* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
qubit-op i U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1$_m$ (2^*i*)) *U*) (1$_m$ (2^(8−*i*)))

definition *bop* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bop s U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ (*2*^*s*)) *U*) (*1*$_m$ (*2*^(*2*−*s*)))

definition *blk* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
blk b M = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ (*8*^*b*)) *M*) (*1*$_m$ (*8*^(*2*−*b*)))

lemma *qubit-op-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *i*: *i* < *9*
shows *qubit-op i U* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *qubit-op-blk*:
assumes *b*: *b* < *3* **and** *s*: *s* < *3*
shows *qubit-op* (*3* ∗ *b* + *s*) *U* = *blk b* (*bop s U*)
⟨*proof*⟩

lemma *qubit-op-blk-div*:
assumes *i*: *i* < *9*
shows *qubit-op i U* = *blk* (*i div 3*) (*bop* (*i mod 3*) *U*)
⟨*proof*⟩

lemma *bop-cases*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat 2 2*
shows *bop 0 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat U* (*1*$_m$ *2*)) (*1*$_m$ *2*)
and *bop 1 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) *U*) (*1*$_m$ *2*)
and *bop 2 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) (*1*$_m$ *2*)) *U*
⟨*proof*⟩

lemma *bop-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *s*: *s* < *3*
shows *bop s U* ∈ *carrier-mat 8 8*
⟨*proof*⟩

definition *tv3* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
tv3 x y z = *tensor-vec* (*tensor-vec x y*) *z*

lemma *tv3-dim*: *dim-vec x* = *8* ⟹ *dim-vec y* = *8* ⟹ *dim-vec z* = *8* ⟹ *dim-vec* (*tv3 x y z*) = *512*
⟨*proof*⟩

lemma *blk-tv3*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *x*: *dim-vec x* = *8* **and** *y*: *dim-vec y* = *8* **and** *z*: *dim-vec z* = *8*
shows *blk 0 M* ∗$_v$ *tv3 x y z* = *tv3* (*M* ∗$_v$ *x*) *y z*
and *blk 1 M* ∗$_v$ *tv3 x y z* = *tv3 x* (*M* ∗$_v$ *y*) *z*
and *blk 2 M* ∗$_v$ *tv3 x y z* = *tv3 x y* (*M* ∗$_v$ *z*)
⟨*proof*⟩

80.3 The block vectors $|m\rangle \pm |7-m\rangle$

In a block of three qubits, *gvec p 0* is $|000\rangle \pm |111\rangle$; a bit flip moves it to *gvec p m* with $m \in \{4, 2, 1\}$. These eight vectors are mutually orthogonal.

definition *pm* :: *bool* ⇒ *complex* **where** *pm p* = (*if p then 1 else* −*1*)

definition *gvec* :: *bool* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
gvec p m = *unit-vec 8 m* + *pm p* ·$_v$ *unit-vec 8* (*7* − *m*)

lemma *gvec-dim* [*simp*]: *dim-vec* (*gvec p m*) = *8*
⟨*proof*⟩

lemma *gvec-index*:
assumes *r*: *r* < *8* **and** *m*: *m* < *8*
shows *gvec p m* $ *r* = (*if r* = *m then 1 else 0*) + *pm p* ∗ (*if r* = *7* − *m then 1 else 0*)
⟨*proof*⟩

lemma *mult-gvec0*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *r*: *r* < *8*
shows (*M* ∗$_v$ *gvec p 0*) $ *r* = *M* $$ (*r*,*0*) + *pm p* ∗ *M* $$ (*r*,*7*)
⟨*proof*⟩

lemma *bop-entry-cases*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *r*: *r* < *8* **and** *k*: *k* < *8*
shows *bop 0 U* $$ (*r*,*k*) = *U* $$ (*r div 4*, *k div 4*) ∗ *1*$_m$ *2* $$ (*r div 2 mod 2*, *k div 2 mod 2*) ∗ *1*$_m$ *2* $$ (*r mod 2*, *k mod 2*)
and *bop 1 U* $$ (*r*,*k*) = *1*$_m$ *2* $$ (*r div 4*, *k div 4*) ∗ *U* $$ (*r div 2 mod 2*, *k div 2 mod 2*) ∗ *1*$_m$ *2* $$ (*r mod 2*, *k mod 2*)
and *bop 2 U* $$ (*r*,*k*) = *1*$_m$ *2* $$ (*r div 4*, *k div 4*) ∗ *1*$_m$ *2* $$ (*r div 2 mod 2*, *k div 2 mod 2*) ∗ *U* $$ (*r mod 2*, *k mod 2*)
⟨*proof*⟩

lemma *bop-X-gvec*:
assumes *s*: *s* < *3*
shows *bop s Pauli-X* ∗$_v$ *gvec p 0* = *gvec p* (*2*^(*2*−*s*))
⟨*proof*⟩

lemma *bop-Y-gvec*:
assumes *s*: *s* < *3*
shows *bop s Pauli-Y* ∗$_v$ *gvec p 0* = (−i) ·$_v$ *gvec* (¬ *p*) (*2*^(*2*−*s*))
⟨*proof*⟩

lemma *bop-Z-gvec*:
assumes *s*: *s* < *3*
shows *bop s Pauli-Z* ∗$_v$ *gvec p 0* = *gvec* (¬ *p*) *0*
⟨*proof*⟩

lemma *inner-two-units*:
assumes *a*: *a* < *n* **and** *b*: *b* < *n* **and** *c*: *c* < *n* **and** *d*: *d* < *n*
shows ⟨*unit-vec n a* + *α* ·$_v$ *unit-vec n b* | *unit-vec n c* + *β* ·$_v$ *unit-vec n d*⟩
= (*if a* = *c then 1 else 0*) + *β* ∗ (*if a* = *d then 1 else 0*)
+ *cnj α* ∗ (*if b* = *c then 1 else 0*) + *cnj α* ∗ *β* ∗ (*if b* = *d then 1 else 0*)
⟨*proof*⟩

lemma *gvec-inner*:
assumes *m*: *m* ∈ {*0*,*1*,*2*,*4*} **and** *m'*: *m'* ∈ {*0*,*1*,*2*,*4*}
shows ⟨*gvec p m* | *gvec p' m'*⟩ = (*if m* = *m'* ∧ *p* = *p' then 2 else 0*)
⟨*proof*⟩

80.4 Code words and the images of the errors

definition *gv* :: *bool* × *nat* ⇒ *complex Matrix.vec* **where** *gv x* = *gvec* (*fst x*) (*snd x*)

A label lists the block vectors of a product state, one per block.

definition *lab-vec* :: (*bool* × *nat*) *list* ⇒ *complex Matrix.vec* **where**
lab-vec l = *tv3* (*gv* (*l* ! *0*)) (*gv* (*l* ! *1*)) (*gv* (*l* ! *2*))

definition *valid-label* :: (*bool* × *nat*) *list* ⇒ *bool* **where**
valid-label l ⟷ *length l* = *3* ∧ (∀ *x* ∈ *set l*. *snd x* ∈ {*0*,*1*,*2*,*4*})

definition *sk* :: *complex* **where** *sk* = *complex-of-real* (*1* / *sqrt 8*)

definition *shor-cw* :: *bool* ⇒ *complex Matrix.vec* **where**
shor-cw p = *sk* ·$_v$ *lab-vec* (*replicate 3* (*p*, *0*))

The notes' code words $|0_L\rangle, |1_L\rangle$.

lemma *shor-cw-notes*:
shor-cw True = *complex-of-real* (*1* / *sqrt* (*2*^*3*)) ·$_v$
tensor-vec (*tensor-vec* (*unit-vec 8 0* + *unit-vec 8 7*) (*unit-vec 8 0* + *unit-vec 8 7*)) (*unit-vec 8 0* + *unit-vec 8 7*)
shor-cw False = *complex-of-real* (*1* / *sqrt* (*2*^*3*)) ·$_v$
tensor-vec (*tensor-vec* (*unit-vec 8 0* − *unit-vec 8 7*) (*unit-vec 8 0* − *unit-vec 8 7*)) (*unit-vec 8 0* − *unit-vec 8 7*)
⟨*proof*⟩

lemma *gv-dim* [*simp*]: *dim-vec* (*gv x*) = *8*
⟨*proof*⟩

lemma *lab-vec-dim* [*simp*]: *dim-vec* (*lab-vec l*) = *512*
⟨*proof*⟩

lemma *shor-cw-dim* [*simp*]: *dim-vec* (*shor-cw p*) = *512*
⟨*proof*⟩

lemma *tv3-smult*:
assumes *dim-vec x* = *8 dim-vec y* = *8 dim-vec z* = *8*
shows *tv3* (*k* ·$_v$ *x*) *y z* = *k* ·$_v$ *tv3 x y z tv3 x* (*k* ·$_v$ *y*) *z* = *k* ·$_v$ *tv3 x y z*
tv3 x y (*k* ·$_v$ *z*) = *k* ·$_v$ *tv3 x y z*
⟨*proof*⟩

lemma *qubit-op-tl*:
assumes *i*: *i* < *9* **and** *U*: *U* ∈ *carrier-mat 2 2*
and *act*: *bop* (*i mod 3*) *U* ∗$_v$ *gvec p 0* = *κ* ·$_v$ *gvec q m*
shows *qubit-op i U* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*)) = *κ* ·$_v$ *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (*q*, *m*)])
⟨*proof*⟩

lemma *qubit-op-X-tl*:
i < *9* ⟹ *qubit-op i Pauli-X* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*))
= *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))])
⟨*proof*⟩

lemma *qubit-op-Y-tl*:
i < *9* ⟹ *qubit-op i Pauli-Y* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*))
= (−i) ·$_v$ *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))])
⟨*proof*⟩

lemma *qubit-op-Z-tl*:
i < *9* ⟹ *qubit-op i Pauli-Z* ∗$_v$ *lab-vec* (*replicate 3* (*p*, *0*))
= *lab-vec* ((*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *0*)])
⟨*proof*⟩

lemma *lab-vec-inner*:
assumes *l*: *valid-label l* **and** *l'*: *valid-label l'*

shows ⟨*lab-vec l* | *lab-vec l′*⟩ = (*if l* = *l′ then 8 else 0*)
⟨*proof*⟩

80.5 The 22 correctable errors

I, $X_1, \ldots, X_9$, $Y_1, \ldots, Y_9$, Z_1, Z_4, Z_7 (positions counted from 0 here).

definition *shor-errors* :: *complex Matrix.mat list* **where**
shor-errors = 1_m *512* # *map* (λ*i*. *qubit-op i Pauli-X*) [*0*..<*9*] @ *map* (λ*i*. *qubit-op i Pauli-Y*) [*0*..<*9*]
@ *map* (λ*b*. *qubit-op* (*3* ∗ *b*) *Pauli-Z*) [*0*..<*3*]

definition *shor-labels* :: *bool* ⇒ (*bool* × *nat*) *list list* **where**
shor-labels p = *replicate 3* (*p*, *0*)
map (λ*i*. (*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))]) [*0*..<*9*]
@ *map* (λ*i*. (*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))]) [*0*..<*9*]
@ *map* (λ*b*. (*replicate 3* (*p*, *0*))[*b* := (¬ *p*, *0*)]) [*0*..<*3*]

definition *shor-kappa* :: *complex list* **where**
shor-kappa = *1* # *replicate 9 1* @ *replicate 9* (−i) @ *replicate 3 1*

lemma *shor-lengths* [*simp*]: *length shor-errors* = *22 length* (*shor-labels p*) = *22 length shor-kappa* = *22*
⟨*proof*⟩

lemma *shor-errors-cases*:
assumes *j*: *j* < *22*
obtains *j* = *0 shor-errors* ! *j* = 1_m *512 shor-labels p* ! *j* = *replicate 3* (*p*, *0*) *shor-kappa* ! *j* = *1*
| *i* **where** *i* < *9 j* = *Suc i shor-errors* ! *j* = *qubit-op i Pauli-X*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))] *shor-kappa* ! *j* = *1*
| *i* **where** *i* < *9 j* = *10* + *i shor-errors* ! *j* = *qubit-op i Pauli-Y*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))] *shor-kappa* ! *j* = −i
| *b* **where** *b* < *3 j* = *19* + *b shor-errors* ! *j* = *qubit-op* (*3* ∗ *b*) *Pauli-Z*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*b* := (¬ *p*, *0*)] *shor-kappa* ! *j* = *1*
⟨*proof*⟩

lemma *shor-errors-carrier*: *j* < *22* ⟹ *shor-errors* ! *j* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *shor-errors-action*:
assumes *j*: *j* < *22*
shows *shor-errors* ! *j* ∗$_v$ *shor-cw p* = (*shor-kappa* ! *j* ∗ *sk*) ·$_v$ *lab-vec* (*shor-labels p* ! *j*)
⟨*proof*⟩

lemma *shor-labels-valid*: *j* < *22* ⟹ *valid-label* (*shor-labels p* ! *j*)
⟨*proof*⟩

lemma *shor-labels-distinct*: *distinct* (*shor-labels True* @ *shor-labels False*)
⟨*proof*⟩

lemma *shor-labels-differ*:
assumes *i*: *i* < *22* **and** *j*: *j* < *22* **and** *ij*: *i* ≠ *j*
shows *shor-labels p* ! *i* ≠ *shor-labels p′* ! *j*
⟨*proof*⟩

lemma *shor-errors-orth*:
assumes *i*: *i* < *22* **and** *j*: *j* < *22* **and** *ij*: *i* ≠ *j*
shows ⟨*shor-errors* ! *i* ∗$_v$ *shor-cw p* | *shor-errors* ! *j* ∗$_v$ *shor-cw p′*⟩ = *0*
⟨*proof*⟩

lemma *sk-sq*: *cnj sk * sk = 1 / 8*
⟨*proof*⟩

lemma *shor-cw-inner*: ⟨*shor-cw p | shor-cw p'*⟩ = (*if p = p' then 1 else 0*)
⟨*proof*⟩

80.6 The code space and its projection

definition *shor-V* :: *complex Matrix.mat* **where** *shor-V = mat-of-cols 512* [*shor-cw True*, *shor-cw False*]

definition *shor-P* :: *complex Matrix.mat* **where** *shor-P = shor-V * (shor-V†)*

definition *shor-state* :: *complex ⇒ complex ⇒ complex Matrix.vec* **where**
shor-state α β = α ·_v shor-cw True + β ·_v shor-cw False

lemma *shor-state-dim* [*simp*]: *dim-vec (shor-state α β) = 512*
⟨*proof*⟩

lemma *shor-V-carrier*: *shor-V ∈ carrier-mat 512 2*
⟨*proof*⟩

lemma *shor-orthonormal*: *orthonormal-list 512* [*shor-cw True*, *shor-cw False*]
⟨*proof*⟩

lemma *shor-VdV*: *(shor-V†) * shor-V = 1_m 2*
⟨*proof*⟩

lemma *mat-vec-unit-col*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M ∈ carrier-mat n k* **and** *a*: *a < k*
shows *M *_v unit-vec k a = Matrix.col M a*
⟨*proof*⟩

lemma *shor-V-col*: *a < 2 ⟹ Matrix.col shor-V a = [shor-cw True, shor-cw False] ! a*
⟨*proof*⟩

lemma *shor-V-unit*: *shor-V *_v unit-vec 2 0 = shor-cw True shor-V *_v unit-vec 2 1 = shor-cw False*
⟨*proof*⟩

lemma *shor-P-projection*: *projection 512 shor-P*
⟨*proof*⟩

lemma *shor-P-cw*: *shor-P *_v shor-cw p = shor-cw p*
⟨*proof*⟩

lemma *shor-P-fixes*: *shor-P *_v shor-state α β = shor-state α β*
⟨*proof*⟩

80.7 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$

lemma *gram-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A ∈ carrier-mat n k* **and** *B*: *B ∈ carrier-mat n k* **and** *a*: *a < k* **and** *b*: *b < k*
shows *((A†) * B) $$ (a,b) = ⟨Matrix.col A a | Matrix.col B b⟩*
⟨*proof*⟩

lemma *sandwich-VVd*:

fixes V W :: *complex Matrix.mat*
assumes V: $V \in$ *carrier-mat n k* **and** W: $W \in$ *carrier-mat n n*
shows $(V * (V^\dagger)) * W * (V * (V^\dagger)) = V * ((V^\dagger) * W * V) * (V^\dagger)$
⟨*proof*⟩

lemma *qubit-op-unitary*: *unitary U* ⟹ *unitary* (*qubit-op i U*)
⟨*proof*⟩

lemma *shor-errors-unitary*: $U \in$ *set shor-errors* ⟹ $U \in$ *carrier-mat 512 512* ∧ *unitary U*
⟨*proof*⟩

theorem *shor-orthogonal-images*:
assumes *i*: $i < 22$ **and** *j*: $j < 22$ **and** *ij*: $i \neq j$
shows *shor-P* $*$ (((*shor-errors* ! *i*)†) $*$ *shor-errors* ! *j*) $*$ *shor-P* $= 0_m$ *512 512*
⟨*proof*⟩

lemma *shor-recovery-setting*: *recovery-setting 512 shor-P shor-errors*
⟨*proof*⟩

lemma *qubit-op-Z-cw*:
assumes *i*: $i < 9$
shows *qubit-op i Pauli-Z* $*_v$ *shor-cw p* = *qubit-op* (*3* $*$ (*i div 3*)) *Pauli-Z* $*_v$ *shor-cw p*
⟨*proof*⟩

theorem *shor-Z-coincide*:
assumes *i*: $i < 9$ **and** *j*: $j < 9$ **and** *ij*: *i div 3* = *j div 3*
shows *qubit-op i Pauli-Z* $*_v$ *shor-state* α β = *qubit-op j Pauli-Z* $*_v$ *shor-state* α β
⟨*proof*⟩

The Day 32 proposition as first stated (distinct 1-Paulis map V to orthogonal subspaces) is false: $Z_1 \neq Z_2$, yet $Z_1|0_L\rangle = Z_2|0_L\rangle \neq 0$. The notes correct it on Day 33.

lemma *shor-notes-Z1-Z2*:
qubit-op 0 Pauli-Z ≠ *qubit-op 1 Pauli-Z*
⟨*qubit-op 0 Pauli-Z* $*_v$ *shor-cw True* | *qubit-op 1 Pauli-Z* $*_v$ *shor-cw True*⟩ = *1*
⟨*proof*⟩

80.8 Shor's theorem

lemma *pauli-decomp*:
fixes U :: *complex Matrix.mat*
assumes U: $U \in$ *carrier-mat 2 2*
shows U = ((*U* $$ (*0*,*0*) + *U* $$ (*1*,*1*)) / *2*) $\cdot_m$ 1_m *2* + ((*U* $$ (*0*,*1*) + *U* $$ (*1*,*0*)) / *2*) $\cdot_m$ *Pauli-X* + (−i $*$ (*U* $$ (*0*,*1*) − *U* $$ (*1*,*0*)) / *2*) $\cdot_m$ *Pauli-Y* + ((*U* $$ (*0*,*0*) − *U* $$ (*1*,*1*)) / *2*) $\cdot_m$ *Pauli-Z*
(**is** U = *?R*)
⟨*proof*⟩

lemma *qubit-op-add*:
assumes A: $A \in$ *carrier-mat 2 2* **and** B: $B \in$ *carrier-mat 2 2*
shows *qubit-op i* (*A* + *B*) = *qubit-op i A* + *qubit-op i B*
⟨*proof*⟩

lemma *qubit-op-smult*:
assumes A: $A \in$ *carrier-mat 2 2*
shows *qubit-op i* (*k* $\cdot_m$ *A*) = *k* $\cdot_m$ *qubit-op i A*
⟨*proof*⟩

lemma *qubit-op-one*:

assumes *i*: $i < 9$
shows *qubit-op i* $(1_m\ 2) = 1_m\ 512$
⟨*proof*⟩

lemma *shor-errors-nth*:
assumes *i*: $i < 9$ **and** *b*: $b < 3$
shows *shor-errors* ! $0 = 1_m\ 512$ *shor-errors* ! *Suc i* = *qubit-op i Pauli-X*
shor-errors ! $(10 + i)$ = *qubit-op i Pauli-Y shor-errors* ! $(19 + b)$ = *qubit-op* $(3 * b)$ *Pauli-Z*
⟨*proof*⟩

Shor: one CPTP recovery map R corrects every single-qubit unitary error on every qubit, for every state $\psi = a|0_L\rangle + b|1_L\rangle$ of the code space.

theorem *shor-code-theorem*:
$\exists$ *Rs. kraus-operators 512 512 Rs* $\wedge$
$(\forall\ U\ i\ \alpha\ \beta.\ U \in$ *carrier-mat 2 2* $\longrightarrow$ *unitary U* $\longrightarrow$ $i < 9 \longrightarrow$
kraus-map-dim 512 Rs (*qubit-op i U* $*$ (*shor-state* $\alpha\ \beta \oslash$ *shor-state* $\alpha\ \beta$) $*$ ((*qubit-op i U*)†))
= *shor-state* $\alpha\ \beta \oslash$ *shor-state* $\alpha\ \beta$)
⟨*proof*⟩

end
theory *Day34-Knill-Laflamme*
imports *Day22-Minimal-Kraus Quantum-Channels*
begin

81 Day 34: the Knill–Laflamme conditions (D34-1, D34-2, D34-3)

81.1 Vectors in the range of a projection; rank-one decompositions inside V

lemma *projection-fixed-of-orth-kernel*:
assumes *P*: *projection n P* **and** *vd*: *dim-vec v* = *n*
and *orth*: $\bigwedge x.$ *dim-vec x* = *n* $\Longrightarrow$ *P* $*_v$ *x* = 0_v *n* $\Longrightarrow \langle v \mid x\rangle = 0$
shows *P* $*_v$ *v* = *v*
⟨*proof*⟩

lemma *outer-sum-Nil*: *outer-sum n* [] = 0_m *n n*
⟨*proof*⟩

lemma *outer-sum-singleton-eq*:
assumes *vd*: *dim-vec v* = *n*
shows *outer-sum n* [*v*] = *v* $\oslash$ *v*
⟨*proof*⟩

lemma *outer-sum-as-foldr*:
assumes $\forall\ v \in$ *set vs. dim-vec v* = *n*
shows *outer-sum n vs* = *foldr* $(\lambda v\ acc.\ v \oslash v + acc)$ *vs* $(0_m\ n\ n)$
⟨*proof*⟩

For $X \geq 0$ the matrix PXP is a sum of $|\psi_l\rangle\langle\psi_l|$ with $\psi_l \in V$ (the notes: "non-zero eigenvectors of PXP are all in V").

lemma *psd-PXP-decomp-in-range*:
assumes *P*: *projection n P* **and** *Xc*: *M* $\in$ *carrier-mat n n* **and** *Xpsd*: *psd M*
shows $\exists$ *vs* :: *complex Matrix.vec list*. $(\forall\ v \in$ *set vs. dim-vec v* = *n* $\wedge$ *P* $*_v$ *v* = *v*) $\wedge$
P $*$ *M* $*$ *P* = *foldr* $(\lambda v\ acc.\ v \oslash v + acc)$ *vs* $(0_m\ n\ n)$
⟨*proof*⟩

81.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$

definition *RE* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
RE n As Es M = *kraus-map-dim n As* (*kraus-map-dim n Es M*)

lemma *RE-carrier*: **assumes** *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n* **shows** *RE n As Es M* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *RE-add*: **assumes** *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n* **and** *N*: *N* ∈ *carrier-mat n n* **shows** *RE n As Es* (*M* + *N*) = *RE n As Es M* + *RE n As Es N*
⟨*proof*⟩

lemma *RE-smult*: **assumes** *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n* **shows** *RE n As Es* (*c* ·$_m$ *M*) = *c* ·$_m$ *RE n As Es M*
⟨*proof*⟩

lemma *RE-minus*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n*
and *Xc*: *M* ∈ *carrier-mat n n* **and** *Yc*: *N* ∈ *carrier-mat n n*
shows *RE n As Es* (*M* − *N*) = *RE n As Es M* − *RE n As Es N*
⟨*proof*⟩

lemma *RE-foldr*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
shows *RE n As Es* (*foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (0$_m$ *n n*)) = *foldr* (λ*v acc*. *RE n As Es* (*v* ⊘ *v*) + *acc*) *vs* (0$_m$ *n n*)
⟨*proof*⟩

D34-1: $\mathcal{R} \circ \mathbb{E}$ fixes $|\psi\rangle\langle\psi|$ for all $\psi \in V$ iff it fixes PXP for all X.

proposition *recovery-states-iff-PXP*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *P*: *projection n P*
shows (∀ *ψ*. *dim-vec ψ* = *n* ⟶ *P* ∗$_v$ *ψ* = *ψ* ⟶ *RE n As Es* (*ψ* ⊘ *ψ*) = *ψ* ⊘ *ψ*)
⟷ (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *RE n As Es* (*P* ∗ *X* ∗ *P*) = *P* ∗ *X* ∗ *P*)
⟨*proof*⟩

81.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$

lemma *lin-indep-complex-mat-single*:
assumes *Pc*: *P* ∈ *carrier-mat d n* **and** *Pnz*: *P* ≠ 0$_m$ *d n*
shows *lin-indep-complex-mat d n* [*P*]
⟨*proof*⟩

The core of the necessity proof: by Choi's Theorem 2 applied to the two Kraus representations $\{P\}$ and $\{A_l E_i P\}$ of $X \mapsto PXP$, every $A_l E_i P$ is a multiple of P.

lemma *kl-kraus-scalars*:
assumes *P*: *projection n P* **and** *Pnz*: *P* ≠ 0$_m$ *n n* **and** *npos*: *0* < *n*
and *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat n n* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n*
and *corr*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶
kraus-map-dim n As (*kraus-map-dim n Es* (*P* ∗ *X* ∗ *P*)) = *P* ∗ *X* ∗ *P*
shows ∃ *β* :: *nat* ⇒ *nat* ⇒ *complex*. ∀ *l* < *length As*. ∀ *i* < *length Es*.
As ! *l* ∗ *Es* ! *i* ∗ *P* = *β l i* ·$_m$ *P*
⟨*proof*⟩

theorem *knill-laflamme-necessity*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As* **and** *Es*: *kraus-operators n n Es*
and *corr*: $\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
shows $\exists \alpha :: nat \Rightarrow nat \Rightarrow complex.\ \forall i < length\ Es.\ \forall j < length\ Es.$
$P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
⟨*proof*⟩

81.4 Kraus lists consisting of multiples of one operator

lemma *kraus-map-dim-scalar-list*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Xc*: $M \in$ *carrier-mat n n*
shows *kraus-map-dim n* (*map* $(\lambda\gamma.\ \gamma \cdot_m P)$ *gs*) *M*
$=$ *sum-list* (*map* $(\lambda\gamma.$ *cnj* $\gamma * \gamma)$ *gs*) $\cdot_m (P * M * (P^\dagger))$
⟨*proof*⟩

lemma *sum-wdag-w-scalar-list*:
assumes *Pc*: $P \in$ *carrier-mat n n*
shows *sum-wdag-w n* (*map* $(\lambda\gamma.\ \gamma \cdot_m P)$ *gs*) $=$ *sum-list* (*map* $(\lambda\gamma.$ *cnj* $\gamma * \gamma)$ *gs*) $\cdot_m ((P^\dagger) * P)$
⟨*proof*⟩

lemma *smult-eq-self-imp-one*:
fixes *P* :: *complex Matrix.mat*
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $s \cdot_m P = P$
shows $s = 1$
⟨*proof*⟩

81.5 Theorem 2

If $\mathcal{R} \circ \mathbb{E}$ fixes PXP and $G_i = \sum_j g_{ij} E_j$ with $\sum_i G_i^* G_i = I$, then $\mathcal{R} \circ \mathbb{G}$ fixes PXP as well (same $\mathcal{R}$).

theorem *knill-laflamme-closure*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As*
and *Es-c*: $\forall E \in$ *set Es*. $E \in$ *carrier-mat n n*
and *corr*: $\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* $(P * X * P)) = P * X * P$
and *Gc*: $G \in$ *carrier-mat t* (*length Es*)
and *Gs-tp*: *sum-wdag-w n* (*kraus-mix n n G Es*) $= 1_m\ n$
shows $\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$
kraus-map-dim n As (*kraus-map-dim n* (*kraus-mix n n G Es*) $(P * X * P)) = P * X * P$
⟨*proof*⟩

end
theory *Day34-KL-Sufficiency*
imports *Day34-Knill-Laflamme*
begin

82 Day 34: sufficiency of the Knill–Laflamme conditions (D34-2)

D34-2, sufficiency: if $PE_i^* E_j P = \alpha_{ij} P$ then a recovery channel exists. Diagonalise $(\alpha_{ij}) = u\,\mathrm{diag}(d)\,u^*$; the errors $F_k = \sum_i u_{ik} E_i$ give the same channel and satisfy $PF_k^* F_l P = \delta_{kl} d_k P$, so the $F_k/\sqrt{d_k}$ with $d_k > 0$ satisfy the hypotheses of the recovery proposition (*recovery-setting*).

lemma *smult-cancel-nonzero*:
fixes P :: *complex Matrix.mat*
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $c \cdot_m P = d \cdot_m P$
shows $c = d$
⟨*proof*⟩

lemma *outer-prod-smult-vec*:
fixes v :: *complex Matrix.vec*
shows $(s \cdot_v v) \oslash (s \cdot_v v) = (s * cnj\ s) \cdot_m (v \oslash v)$
⟨*proof*⟩

lemma *kraus-map-dim-foldr-add*:
assumes *Ws-c*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ n\ n$ **and** *fc*: $\forall k \in set\ xs.\ f\ k \in carrier\text{-}mat\ n\ n$
shows *kraus-map-dim n Ws* (*foldr* ($\lambda k\ acc.\ f\ k + acc$) *xs* $(0_m\ n\ n)$)
$= foldr\ (\lambda k\ acc.\ kraus\text{-}map\text{-}dim\ n\ Ws\ (f\ k) + acc)\ xs\ (0_m\ n\ n)$
⟨*proof*⟩

lemma *kraus-map-dim-outer-as-foldr*:
assumes *Ws-c*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ n\ n$ **and** ψd: *dim-vec* $\psi = n$
shows *kraus-map-dim n Ws* $(\psi \oslash \psi)$
$= foldr\ (\lambda k\ acc.\ (Ws\ !\ k *_v \psi) \oslash (Ws\ !\ k *_v \psi) + acc)\ [0..<length\ Ws]\ (0_m\ n\ n)$
⟨*proof*⟩

lemma *foldr-smult-sandwich*:
fixes V :: *nat* ⇒ *complex Matrix.mat*
assumes *Mc*: $M \in carrier\text{-}mat\ n\ n$ **and** *Nc*: $N \in carrier\text{-}mat\ n\ n$ **and** *Pc*: $P \in carrier\text{-}mat\ n\ n$
and *Vc*: $\forall j < r.\ V\ j \in carrier\text{-}mat\ n\ n$ **and** *h*: $\forall j < r.\ M * V\ j * N = \beta\ j \cdot_m P$
shows $M * foldr\ (\lambda j\ acc.\ b\ j \cdot_m V\ j + acc)\ [0..<r]\ (0_m\ n\ n) * N = (\sum j{<}r.\ b\ j * \beta\ j) \cdot_m P$
⟨*proof*⟩

lemma *foldr-smult-dagger*:
fixes V :: *nat* ⇒ *complex Matrix.mat*
assumes *Vc*: $\forall j < r.\ V\ j \in carrier\text{-}mat\ n\ n$
shows $(foldr\ (\lambda j\ acc.\ b\ j \cdot_m V\ j + acc)\ [0..<r]\ (0_m\ n\ n))^\dagger$
$= foldr\ (\lambda j\ acc.\ cnj\ (b\ j) \cdot_m ((V\ j)^\dagger) + acc)\ [0..<r]\ (0_m\ n\ n)$
⟨*proof*⟩

The bracket $P\,(\sum_i a_i E_i)^*(\sum_j b_j E_j)\,P$ under the Knill–Laflamme conditions.

lemma *kl-mix-bracket*:
assumes *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** *Pc*: $P \in carrier\text{-}mat\ n\ n$
and *kl*: $\forall i < length\ Es.\ \forall j < length\ Es.\ P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
and *r*: $r = length\ Es$
shows $P * (((foldr\ (\lambda i\ acc.\ a\ i \cdot_m (Es\ !\ i) + acc)\ [0..<r]\ (0_m\ n\ n))^\dagger)$
$* foldr\ (\lambda j\ acc.\ b\ j \cdot_m (Es\ !\ j) + acc)\ [0..<r]\ (0_m\ n\ n)) * P$
$= (\sum j{<}r.\ b\ j * (\sum i{<}r.\ cnj\ (a\ i) * \alpha\ i\ j)) \cdot_m P$
⟨*proof*⟩

lemma *dagger-sandwich*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Ph*: $P^\dagger = P$ **and** *Ac*: $A \in carrier\text{-}mat\ n\ n$ **and** *Bc*: $B \in carrier\text{-}mat\ n\ n$
shows $(P * ((A^\dagger) * B) * P)^\dagger = P * ((B^\dagger) * A) * P$
⟨*proof*⟩

lemma *smult-bracket*:
assumes *Pc*: $P \in carrier\text{-}mat\ n\ n$ **and** *Ac*: $A \in carrier\text{-}mat\ n\ n$ **and** *Bc*: $B \in carrier\text{-}mat\ n\ n$
shows $P * (((x \cdot_m A)^\dagger) * (y \cdot_m B)) * P = (cnj\ x * y) \cdot_m (P * ((A^\dagger) * B) * P)$
⟨*proof*⟩

theorem *knill-laflamme-sufficiency*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *Es*: *kraus-operators n n Es*
and *kl*: $\forall\, i < \mathit{length}\ \mathit{Es}.\ \forall\, j < \mathit{length}\ \mathit{Es}.\ P * (((\mathit{Es}\ !\ i)^\dagger) * \mathit{Es}\ !\ j) * P = \alpha\ i\ j \cdot_m P$
shows $\exists\, \mathit{As}.\ \mathit{kraus\text{-}operators}\ n\ n\ \mathit{As} \wedge$
$(\forall\, X.\ X \in \mathit{carrier\text{-}mat}\ n\ n \longrightarrow \mathit{kraus\text{-}map\text{-}dim}\ n\ \mathit{As}\ (\mathit{kraus\text{-}map\text{-}dim}\ n\ \mathit{Es}\ (P * X * P)) = P * X * P)$
⟨*proof*⟩

D34-2 (Theorem 1): the code is correctable iff $PE_i^* E_j P = \alpha_{ij} P$.

theorem *knill-laflamme*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *Es*: *kraus-operators n n Es*
shows $(\exists\, \mathit{As}.\ \mathit{kraus\text{-}operators}\ n\ n\ \mathit{As} \wedge$
$(\forall\, X.\ X \in \mathit{carrier\text{-}mat}\ n\ n \longrightarrow \mathit{kraus\text{-}map\text{-}dim}\ n\ \mathit{As}\ (\mathit{kraus\text{-}map\text{-}dim}\ n\ \mathit{Es}\ (P * X * P)) = P * X * P))$
$\longleftrightarrow (\exists\, \alpha :: \mathit{nat} \Rightarrow \mathit{nat} \Rightarrow \mathit{complex}.\ \forall\, i < \mathit{length}\ \mathit{Es}.\ \forall\, j < \mathit{length}\ \mathit{Es}.$
$P * (((\mathit{Es}\ !\ i)^\dagger) * \mathit{Es}\ !\ j) * P = \alpha\ i\ j \cdot_m P)$
⟨*proof*⟩

end

Part V
Beyond the notes, and supporting theories

theory *Quantum-Channels*
imports *Day32-Phase-Flip-Code*
begin

83 Extensions beyond the notes

Nothing from here on is in Paulsen's notes. These are standard quantum-information facts proved with the machinery above: rank-one projections, the bit-flip, phase-flip, bit-phase-flip and unitary-conjugation channels and their membership in UCP and CPTP, composition of channels, Kraus lists, the maximally mixed state, and the Pauli product identities.

83.1 Rank-one projections

lemma *outer-sum-singleton-hermitian*:
fixes ψ :: *complex Matrix.vec*
shows $(\mathit{outer\text{-}sum}\ n\ [\psi])^\dagger = \mathit{outer\text{-}sum}\ n\ [\psi]$
⟨*proof*⟩

lemma *outer-sum-singleton-idempotent*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows $\mathit{outer\text{-}sum}\ n\ [\psi] * \mathit{outer\text{-}sum}\ n\ [\psi] = \mathit{outer\text{-}sum}\ n\ [\psi]$
⟨*proof*⟩

lemma *rank1-projection*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ

shows *projection n* (*outer-sum n* [ψ])
⟨*proof*⟩

lemma *rank1-projection-acts-identity*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *outer-sum n* [ψ] $*_v$ ψ = ψ
⟨*proof*⟩

lemma *rank1-projection-trace-one*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *Tr* (*outer-sum n* [ψ]) = *1*
⟨*proof*⟩

83.2 The bit-flip channel at $p = 0$ and $p = 1$

lemma *bit-flip-channel-zero*:
assumes ϱ ∈ *carrier-mat 2 2*
shows *bit-flip-channel 0* ϱ = ϱ
⟨*proof*⟩

lemma *bit-flip-channel-one*:
assumes ϱ ∈ *carrier-mat 2 2*
shows *bit-flip-channel 1* ϱ = *Pauli-X* ∗ ϱ ∗ (*Pauli-X*†)
⟨*proof*⟩

83.3 The phase-flip channel

definition *phase-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
phase-flip-channel p ϱ ≡
complex-of-real (*1* − *p*) $\cdot_m$ ϱ
+ *complex-of-real p* $\cdot_m$ (*Pauli-Z* ∗ ϱ ∗ (*Pauli-Z*†))

lemma *phase-flip-channel-carrier*:
assumes ϱ ∈ *carrier-mat 2 2*
shows *phase-flip-channel p* ϱ ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes ϱ-*c*: ϱ ∈ *carrier-mat 2 2*
shows *Tr* (*phase-flip-channel p* ϱ) = *Tr* ϱ
⟨*proof*⟩

lemma *phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: *0* ≤ *p*
and *p-le-1*: *p* ≤ *1*
and ϱ-*c*: ϱ ∈ *carrier-mat 2 2*
and ϱ-*psd*: *psd* ϱ
shows *psd* (*phase-flip-channel p* ϱ)
⟨*proof*⟩

83.4 Unitary-conjugation channels

For a unitary $U \in M_n$ and $0 \leq p \leq 1$, $\rho \mapsto (1-p)\rho + pU\rho U^\dagger$ is trace preserving and positive; the bit-flip and phase-flip channels are the instances $U = X$, $U = Z$.

definition *unitary-conj-channel* ::
complex Matrix.mat ⇒ *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unitary-conj-channel U p ϱ ≡
complex-of-real (*1* − *p*) ·$_m$ *ϱ*
+ *complex-of-real p* ·$_m$ (*U* ∗ *ϱ* ∗ (*U*†))

lemma *unitary-conj-channel-carrier*:
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *unitary-conj-channel U p ϱ* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *unitary-conj-channel-trace-preserving*:
fixes *p* :: *real*
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *U-unit*: *unitary U*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *Tr* (*unitary-conj-channel U p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *unitary-conj-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: *0* ≤ *p*
and *p-le-1*: *p* ≤ *1*
and *U-c*: *U* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*unitary-conj-channel U p ϱ*)
⟨*proof*⟩

lemma *bit-flip-eq-unitary-conj*:
bit-flip-channel p ϱ = *unitary-conj-channel Pauli-X p ϱ*
⟨*proof*⟩

lemma *phase-flip-eq-unitary-conj*:
phase-flip-channel p ϱ = *unitary-conj-channel Pauli-Z p ϱ*
⟨*proof*⟩

83.5 The bit-phase-flip channel and the identity channel

definition *bit-phase-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bit-phase-flip-channel p ϱ ≡ *unitary-conj-channel Pauli-Y p ϱ*

lemma *bit-phase-flip-channel-carrier*:
assumes *ϱ* ∈ *carrier-mat 2 2*
shows *bit-phase-flip-channel p ϱ* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
shows *Tr* (*bit-phase-flip-channel p ϱ*) = *Tr ϱ*

⟨*proof*⟩

lemma *bit-phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$
and *p-le-1*: $p \leq 1$
and *ϱ-c*: $\varrho \in$ *carrier-mat 2 2*
and *ϱ-psd*: *psd* ϱ
shows *psd* (*bit-phase-flip-channel p* ϱ)
⟨*proof*⟩

definition *identity-channel* ::
complex Matrix.mat ⇒ *complex Matrix.mat* **where**
identity-channel $\varrho \equiv \varrho$

lemma *identity-channel-carrier*:
$\varrho \in$ *carrier-mat n n* ⟹ *identity-channel* $\varrho \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *identity-channel-trace-preserving*:
$\varrho \in$ *carrier-mat n n* ⟹ *Tr* (*identity-channel* ϱ) = *Tr* ϱ
⟨*proof*⟩

lemma *identity-channel-psd*:
$\varrho \in$ *carrier-mat n n* ⟹ *psd* ϱ ⟹ *psd* (*identity-channel* ϱ)
⟨*proof*⟩

lemma *identity-channel-eq-unitary-conj-zero*:
assumes *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *identity-channel* ϱ = *unitary-conj-channel* (1_m *n*) *0* ϱ
⟨*proof*⟩

83.6 Composition of channels

lemma *compose-trace-preserving*:
assumes *F-tp*: *trace-preserving n F*
and *G-tp*: *trace-preserving n G*
and *F-range*: ∀ ϱ. $\varrho \in$ *carrier-mat n n* ⟶ *F* $\varrho \in$ *carrier-mat n n*
shows *trace-preserving n* (λϱ. *G* (*F* ϱ))
⟨*proof*⟩

lemma *identity-channel-compose-left*:
(λϱ. *identity-channel* (*F* ϱ)) = *F*
⟨*proof*⟩

lemma *identity-channel-compose-right*:
(λϱ. *F* (*identity-channel* ϱ)) = *F*
⟨*proof*⟩

lemma *compose-unital-map*:
assumes *F-un*: *unital-map n n F*
and *G-un*: *unital-map n n G*

shows *unital-map n n* ($\lambda\varrho$. *F* (*G* ϱ))
⟨*proof*⟩

lemma *identity-channel-unital*: *unital-map n n identity-channel*
⟨*proof*⟩

lemma *identity-channel-trace-preserving-full*:
trace-preserving n identity-channel
⟨*proof*⟩

lemma *identity-channel-completely-positive*:
assumes $0 < n$
shows *completely-positive n n identity-channel*
⟨*proof*⟩

83.7 Conjugation channels are completely positive

definition *conjugation-channel* ::
complex Matrix.mat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
conjugation-channel U $\varrho \equiv U * \varrho * (U^\dagger)$

lemma *conjugation-channel-eq-kraus*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and ϱ*-c*: $\varrho \in$ *carrier-mat n n*
shows *conjugation-channel U* ϱ = *kraus-map-dim d* [*U*] ϱ
⟨*proof*⟩

lemma *conjugation-channel-cp*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* (*kraus-map-dim d* [*U*])
⟨*proof*⟩

lemma *identity-channel-UCP*:
assumes $0 < n$
shows *identity-channel* ∈ *UCP n n*
⟨*proof*⟩

lemma *identity-channel-CPTP*:
assumes $0 < n$
shows *identity-channel* ∈ *CPTP n n*
⟨*proof*⟩

lemma *map-conj-cp*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *U-range*: $\forall \varrho.\ \varrho \in$ *carrier-mat n n*
$\longrightarrow U * \varrho * (U^\dagger) \in$ *carrier-mat d d*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* ($\lambda\varrho.\ U * \varrho * (U^\dagger)$)

⟨*proof*⟩

lemma *map-conj-unital*:
fixes U :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows $U * (1_m\ n) * (U^\dagger) = 1_m\ n$
⟨*proof*⟩

83.8 Unitary-conjugation channels are UCP and CPTP

lemma *map-conj-in-UCP*:
fixes U :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *npos*: $0 < n$
shows $(\lambda\varrho.\ U * \varrho * (U^\dagger)) \in UCP\ n\ n$
⟨*proof*⟩

lemma *unitary-conj-channel-in-UCP*:
fixes U :: *complex Matrix.mat* **and** p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-CPTP*:
fixes U :: *complex Matrix.mat* **and** p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *CPTP n n*
⟨*proof*⟩

83.9 The Pauli channels are UCP and CPTP

lemma *bit-flip-channel-eq*:
bit-flip-channel p = unitary-conj-channel Pauli-X p
⟨*proof*⟩

lemma *phase-flip-channel-eq*:
phase-flip-channel p = unitary-conj-channel Pauli-Z p
⟨*proof*⟩

lemma *bit-phase-flip-channel-eq*:
bit-phase-flip-channel p = unitary-conj-channel Pauli-Y p
⟨*proof*⟩

lemma *bit-flip-channel-UCP*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$

shows *bit-flip-channel p ∈ UCP 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p* **and** *p-le*: *p ≤ 1*
shows *bit-flip-channel p ∈ CPTP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p* **and** *p-le*: *p ≤ 1*
shows *phase-flip-channel p ∈ UCP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p* **and** *p-le*: *p ≤ 1*
shows *phase-flip-channel p ∈ CPTP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p* **and** *p-le*: *p ≤ 1*
shows *bit-phase-flip-channel p ∈ UCP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p* **and** *p-le*: *p ≤ 1*
shows *bit-phase-flip-channel p ∈ CPTP 2 2*
⟨*proof*⟩

83.10 Pauli matrices as Kraus operators

lemma *Pauli-X-kraus-operators*:
kraus-operators 2 2 [*Pauli-X*]
⟨*proof*⟩

lemma *Pauli-Y-kraus-operators*:
kraus-operators 2 2 [*Pauli-Y*]
⟨*proof*⟩

lemma *Pauli-Z-kraus-operators*:
kraus-operators 2 2 [*Pauli-Z*]
⟨*proof*⟩

83.11 Kraus maps of Kraus-operator lists are CPTP

lemma *kraus-is-trace-preserving*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
shows *trace-preserving n* (*kraus-map-dim d Ws*)
⟨*proof*⟩

lemma *kraus-is-cptp*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*

and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *kraus-map-dim d Ws* $\in$ *CPTP n d*
⟨*proof*⟩

lemma *Pauli-X-kraus-map-CPTP*:
kraus-map-dim 2 [*Pauli-X*] $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *Pauli-Y-kraus-map-CPTP*:
kraus-map-dim 2 [*Pauli-Y*] $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *Pauli-Z-kraus-map-CPTP*:
kraus-map-dim 2 [*Pauli-Z*] $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *identity-kraus-operators*:
kraus-operators n n [1_m *n*]
⟨*proof*⟩

lemma *identity-channel-eq-kraus-map-dim*:
fixes *n* :: *nat*
assumes *ϱ-c*: *ϱ* $\in$ *carrier-mat n n*
shows *identity-channel ϱ* = *kraus-map-dim n* [1_m *n*] *ϱ*
⟨*proof*⟩

83.12 Concatenation of Kraus lists

lemma *sum-wdag-w-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: $\forall$ *W* $\in$ *set Ws*. *W* $\in$ *carrier-mat d n*
and *Vs-c*: $\forall$ *V* $\in$ *set Vs*. *V* $\in$ *carrier-mat d n*
shows *sum-wdag-w n* (*Ws* @ *Vs*)
= *sum-wdag-w n Ws* + *sum-wdag-w n Vs*
⟨*proof*⟩

lemma *kraus-map-dim-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: $\forall$ *W* $\in$ *set Ws*. *W* $\in$ *carrier-mat d n*
and *Vs-c*: $\forall$ *V* $\in$ *set Vs*. *V* $\in$ *carrier-mat d n*
and *ϱ-c*: *ϱ* $\in$ *carrier-mat n n*
shows *kraus-map-dim d* (*Ws* @ *Vs*) *ϱ*
= *kraus-map-dim d Ws ϱ* + *kraus-map-dim d Vs ϱ*
⟨*proof*⟩

83.13 Fixed points of Pauli channels

lemma *Pauli-X-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* $\in$ *carrier-mat 2 2*
and *commute*: *Pauli-X* $*$ *M* = *M* $*$ *Pauli-X*

shows *Pauli-X* $*$ *M* $*$ (*Pauli-X*†) = *M*
⟨*proof*⟩

lemma *bit-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-X* $*$ *M* = *M* $*$ *Pauli-X*
shows *bit-flip-channel p M* = *M*
⟨*proof*⟩

lemma *Pauli-Z-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-Z* $*$ *M* = *M* $*$ *Pauli-Z*
shows *Pauli-Z* $*$ *M* $*$ (*Pauli-Z*†) = *M*
⟨*proof*⟩

lemma *phase-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: *M* ∈ *carrier-mat 2 2*
and *commute*: *Pauli-Z* $*$ *M* = *M* $*$ *Pauli-Z*
shows *phase-flip-channel p M* = *M*
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *unitary-conj-channel* (*1*$_m$ *n*) *p ϱ* = *ϱ*
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op-eq*:
fixes *p* :: *real*
shows (λ*ϱ*. *if ϱ* ∈ *carrier-mat n n*
then unitary-conj-channel (*1*$_m$ *n*) *p ϱ else identity-channel ϱ*)
= *identity-channel*
⟨*proof*⟩

83.14 The maximally mixed state

definition *maximally-mixed-state* :: *nat* ⇒ *complex Matrix.mat* **where**
maximally-mixed-state n ≡ *complex-of-real* (*1* / *real n*) ·$_m$ (*1*$_m$ *n*)

lemma *maximally-mixed-state-carrier* [*simp*]:
maximally-mixed-state n ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *Tr-one-mat*:
Tr (*1*$_m$ *n* :: *complex Matrix.mat*) = *of-nat n*
⟨*proof*⟩

lemma *maximally-mixed-state-trace*:
assumes *npos*: *0* < *n*
shows *Tr* (*maximally-mixed-state n*) = *1*
⟨*proof*⟩

lemma *maximally-mixed-state-psd*:
psd (*maximally-mixed-state n*)
⟨*proof*⟩

lemma *maximally-mixed-state-density-matrix*:
assumes *npos*: $0 < n$
shows *density-matrix n* (*maximally-mixed-state n*)
⟨*proof*⟩

83.15 Fixed points of unitary-conjugation channels

lemma *unitary-conj-commute*:
fixes M :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: $M \in$ *carrier-mat n n*
and *commute*: $U * M = M * U$
shows $U * M * (U^\dagger) = M$
⟨*proof*⟩

lemma *unitary-conj-channel-fixed-point-commute*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: $M \in$ *carrier-mat n n*
and *commute*: $U * M = M * U$
shows *unitary-conj-channel U p M* $= M$
⟨*proof*⟩

lemma *maximally-mixed-state-commutes*:
fixes U :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
shows $U *$ *maximally-mixed-state n* $=$ *maximally-mixed-state n* $* U$
⟨*proof*⟩

lemma *unitary-conj-channel-fixes-max-mixed*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows *unitary-conj-channel U p* (*maximally-mixed-state n*)
$=$ *maximally-mixed-state n*
⟨*proof*⟩

lemma *bit-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *bit-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

lemma *phase-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *phase-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *bit-phase-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

83.16 Channels preserve density matrices

lemma *unitary-conj-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*unitary-conj-channel U p* ϱ)
⟨*proof*⟩

lemma *bit-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-flip-channel p* ϱ)
⟨*proof*⟩

lemma *phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*phase-flip-channel p* ϱ)
⟨*proof*⟩

lemma *bit-phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-phase-flip-channel p* ϱ)
⟨*proof*⟩

83.17 Pauli product identities

With the Day 32 convention for Y: $XY = -iZ$, $YZ = -iX$, $ZX = -iY$.

lemma *Pauli-X-times-Pauli-Y*: *Pauli-X* $*$ *Pauli-Y* $= -\mathrm{i} \cdot_m$ *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Y-times-Pauli-Z*: *Pauli-Y* $*$ *Pauli-Z* $= -\mathrm{i} \cdot_m$ *Pauli-X*
⟨*proof*⟩

lemma *Pauli-Z-times-Pauli-X*: *Pauli-Z* $*$ *Pauli-X* $= -\mathrm{i} \cdot_m$ *Pauli-Y*
⟨*proof*⟩

end

theory *Unitary-Tensor-Algebra*
imports *Day02-Trace-Projections*
begin

84 Unitary algebra

Library-level consequences of unitarity ($U^\dagger U = I$ and $UU^\dagger = I$ in the library's definition) used throughout; Paulsen's Day 3 characterisations of unitaries are in theory Day03_Unitary_Matrices.

lemma *unitary-square-dim*:
  assumes *uA*: *unitary A*
  shows *dim-row A* = *dim-col A*
⟨*proof*⟩

lemma *unitary-inv-l-dimcol*:
  assumes *uA*: *unitary A*
  shows $(A^\dagger) * A = 1_m$ (*dim-col A*)
  ⟨*proof*⟩

lemma *unitary-inv-r-dimrow*:
  assumes *uA*: *unitary A*
  shows $A * (A^\dagger) = 1_m$ (*dim-row A*)
  ⟨*proof*⟩

lemma *unitary-inv-l-dimrow*:
  assumes *uA*: *unitary A*
  shows $(A^\dagger) * A = 1_m$ (*dim-row A*)
  ⟨*proof*⟩

lemma *unitary-inv-r-dimcol*:
  assumes *uA*: *unitary A*
  shows $A * (A^\dagger) = 1_m$ (*dim-col A*)
  ⟨*proof*⟩

lemma *unitary-carrier-square-col*:
  assumes *uA*: *unitary A*
  shows *A* ∈ *carrier-mat* (*dim-col A*) (*dim-col A*)
  ⟨*proof*⟩

lemma *unitary-carrier-square-row*:
  assumes *uA*: *unitary A*
  shows *A* ∈ *carrier-mat* (*dim-row A*) (*dim-row A*)
  ⟨*proof*⟩

lemma *unitary-dagger*:
  assumes *uA*: *unitary A*
  shows *unitary* $(A^\dagger)$
⟨*proof*⟩

lemma *unitary-mult*:
  assumes *UA*: *unitary A*
    and *UB*: *unitary B*
    and *dimA*: *A* ∈ *carrier-mat n n*
    and *dimB*: *B* ∈ *carrier-mat n n*
  shows *unitary* (*A* ∗ *B*)
  ⟨*proof*⟩

lemma *unitary-mult2*:
  assumes *UA*: *unitary A*
    and *UB*: *unitary B*
    and *dims*: *dim-row A* = *dim-row B*

shows *unitary* $(A * B)$
⟨*proof*⟩

85 Tensor product infrastructure

Kronecker products of matrices, $A \bigotimes B$: the tensor of identities, the adjoint $(R \otimes T)^* = R^* \otimes T^*$, the mixed-product property $(R_1 \otimes T_1)(R_2 \otimes T_2) = R_1 R_2 \otimes T_1 T_2$ (both from Day 9), and unitarity of tensors of unitaries.

85.1 Tensor of Identities

lemma *tensor-mat-id*:
Tensor.tensor-mat $(1_m\ n)\ (1_m\ m) = 1_m\ (n * m)$
⟨*proof*⟩

85.2 Tensor Adjoint

85.3 Mixed-Product Property

lemma *tensor-mat-mult*:
assumes *dimAC*: *dim-col A* = *dim-row C*
and *dimBD*: *dim-col B* = *dim-row D*
shows *Tensor.tensor-mat A B* ∗ *Tensor.tensor-mat C D* =
Tensor.tensor-mat $(A * C)\ (B * D)$
⟨*proof*⟩

85.4 Corollaries of the Mixed-Product Property

lemma *tensor-adjoint-mult*:
$(Tensor.tensor\text{-}mat\ (A^\dagger)\ (B^\dagger)) * (Tensor.tensor\text{-}mat\ A\ B) =$
$Tensor.tensor\text{-}mat\ ((A^\dagger) * A)\ ((B^\dagger) * B)$
⟨*proof*⟩

lemma *tensor-mult-adjoint*:
$(Tensor.tensor\text{-}mat\ A\ B) * (Tensor.tensor\text{-}mat\ (A^\dagger)\ (B^\dagger)) =$
$Tensor.tensor\text{-}mat\ (A * (A^\dagger))\ (B * (B^\dagger))$
⟨*proof*⟩

85.5 Tensor Unitarity

lemma *tensor-mat-unitary-carrier*:
assumes *dimA*: $A \in$ *carrier-mat n n*
and *dimB*: $B \in$ *carrier-mat m m*
and *uA*: *unitary A*
and *uB*: *unitary B*
shows *unitary* (*Tensor.tensor-mat A B*)
⟨*proof*⟩

lemma *tensor-mat-unitary*:
assumes *uA*: *unitary A*
and *uB*: *unitary B*
shows *unitary* (*Tensor.tensor-mat A B*)
⟨*proof*⟩

lemma *tensor-mat-unitary-one-left* [*simp*]:
assumes *uA*: *unitary A*
shows *unitary* $(Tensor.tensor\text{-}mat\ (1_m\ n)\ A)$

⟨*proof*⟩

lemma *tensor-mat-unitary-one-right* [*simp*]:
assumes *uA*: *unitary A*
shows *unitary* (*Tensor.tensor-mat A* (1_m *n*))
⟨*proof*⟩

lemma *one-smult-mat*: (*1*::*complex*) $\cdot_m$ *M* = *M*
⟨*proof*⟩

lemma *mult-index-lt*:
fixes *i j n m* :: *nat*
assumes *i* < *n* **and** *j* < *m*
shows *i* ∗ *m* + *j* < *n* ∗ *m*
⟨*proof*⟩

lemma *mult-index-div-mod*:
fixes *i l m* :: *nat*
assumes *l* < *m*
shows (*i* ∗ *m* + *l*) *div m* = *i* (*i* ∗ *m* + *l*) *mod m* = *l*
⟨*proof*⟩

lemma *hs-inner-prod-left-add*:
assumes *dim-row A1* = *dim-row B* **and** *dim-col A1* = *dim-col B*
and *dim-row A2* = *dim-row B* **and** *dim-col A2* = *dim-col B*
shows ⟨⟨ *A1* + *A2* | *B* ⟩⟩ = ⟨⟨ *A1* | *B* ⟩⟩ + ⟨⟨ *A2* | *B* ⟩⟩
⟨*proof*⟩

end
theory *QC-Prelims*
imports *Day23-ChoiExtremeUCP*
begin

86 Small matrix and vector lemmas shared by the gap-closure theories

Index case splits, trace facts, entrywise vector sums, rank-one identities and rectangular *foldr* sums. Nothing here is specific to a lecture; the theories *Day12-Counting-Probability*, *Day18-Positive-Maps*, *Day22-Kraus-Independence*, *Day28-Horodecki-Peres*, *Day30-BitFlip-Code* and *Day32-Pauli-Expansion* build on it.

lemma *cnj-mult-self-cmod*: *cnj z* ∗ *z* = *complex-of-real* ((*cmod z*)^2)
⟨*proof*⟩

lemma *Re-cnj-mult-self*: *Re* (*cnj z* ∗ *z*) = (*cmod z*)^2
⟨*proof*⟩

lemma *Tr-hermitian-Im-zero*:
assumes *Pc*: *P* ∈ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Im* (*Tr P*) = *0*
⟨*proof*⟩

lemma *Tr-hermitian-real*:
assumes *Pc*: *P* ∈ *carrier-mat n n* **and** *herm*: *hermitian P*

shows *Tr P = complex-of-real (Re (Tr P))*
⟨*proof*⟩

A positive semidefinite matrix of trace zero is zero (the notes: "given a non-zero $P \geq 0$, $\mathrm{Tr}(P) \neq 0$").

lemma *psd-Tr-zero-imp-zero*:
assumes *Pc*: *P ∈ carrier-mat n n* **and** *pos*: *psd P* **and** *tr0*: *Tr P = 0*
shows *P = 0$_m$ n n*
⟨*proof*⟩

lemma *smult-smult-mat*:
fixes *A* :: *complex Matrix.mat*
shows *a ·$_m$ (b ·$_m$ A) = (a * b) ·$_m$ A*
⟨*proof*⟩

lemma *zero-smult-mat-carrier*:
fixes *A* :: *complex Matrix.mat*
assumes *A ∈ carrier-mat m k*
shows *0 ·$_m$ A = 0$_m$ m k*
⟨*proof*⟩

Finite sums of vectors, entrywise.

definition *vec-sum* :: *nat ⇒ 'c set ⇒ ('c ⇒ complex Matrix.vec) ⇒ complex Matrix.vec* **where**
vec-sum n Js w = Matrix.vec n (λi. sum (λJ. w J $ i) Js)

lemma *vec-sum-dim* [*simp*]: *dim-vec (vec-sum n Js w) = n*
⟨*proof*⟩

lemma *vec-sum-index* [*simp*]: *i < n ⟹ vec-sum n Js w $ i = sum (λJ. w J $ i) Js*
⟨*proof*⟩

lemma *vec-sum-cong*:
assumes *⋀J. J ∈ Js ⟹ w J = w' J*
shows *vec-sum n Js w = vec-sum n Js w'*
⟨*proof*⟩

lemma *mult-mat-vec-vec-sum*:
assumes *Ac*: *A ∈ carrier-mat m n* **and** *fin*: *finite Js*
and *wd*: *⋀J. J ∈ Js ⟹ dim-vec (w J) = n*
shows *A *$_v$ vec-sum n Js w = vec-sum m Js (λJ. A *$_v$ w J)*
⟨*proof*⟩

lemma *inner-prod-vec-sum-right*:
assumes *ud*: *dim-vec u = n*
and *wd*: *⋀J. J ∈ Js ⟹ dim-vec (w J) = n*
shows *⟨u | vec-sum n Js w⟩ = sum (λJ. ⟨u | w J⟩) Js*
⟨*proof*⟩

lemma *mult-mat-vec-smult-vec*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *Ac*: *A ∈ carrier-mat m n* **and** *vd*: *dim-vec v = n*
shows *A *$_v$ (c ·$_v$ v) = c ·$_v$ (A *$_v$ v)*
⟨*proof*⟩

lemma *inner-prod-unit-vec-right*:
assumes *ud*: *dim-vec u = n* **and** *j*: *j < n*
shows *⟨u | unit-vec n j⟩ = cnj (u $ j)*
⟨*proof*⟩

lemma *outer-prod-self-psd*:
assumes *dim-vec* $v = n$
shows *psd* $(v \oslash v)$
⟨*proof*⟩

Conjugating a rank-one matrix: $W|\varphi\rangle\langle\varphi|W^* = |W\varphi\rangle\langle W\varphi|$.

lemma *conj-outer-prod*:
fixes W :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *Wc*: $W \in$ *carrier-mat m n* **and** φd: *dim-vec* $\varphi = n$
shows $W * (\varphi \oslash \varphi) * (W^\dagger) = (W *_v \varphi) \oslash (W *_v \varphi)$
⟨*proof*⟩

lemma *outer-prod-zero-vec*: $(0_v\ n$ :: *complex Matrix.vec*$) \oslash (0_v\ n) = 0_m\ n\ n$
⟨*proof*⟩

lemma *idx8*: $k < (8$::*nat*$) \Longrightarrow k = 0 \vee k = 1 \vee k = 2 \vee k = 3 \vee k = 4 \vee k = 5 \vee k = 6 \vee k = 7$
⟨*proof*⟩

lemma *dagger-mult-diag-inner*:
fixes Y :: *complex Matrix.mat*
assumes *Yc*: $Y \in$ *carrier-mat p q* **and** *j*: $j < q$
shows $((Y^\dagger) * Y)$ \$\$ $(j,j) = \langle$*Matrix.col Y j* | *Matrix.col Y j*$\rangle$
⟨*proof*⟩

lemma *foldr-add-mat-carrier-rect*:
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat p q*
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q) \in$ *carrier-mat p q*
⟨*proof*⟩

lemma *foldr-add-mat-index-rect*:
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat p q* **and** $i < p$ **and** $j < q$
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q)$ \$\$ $(i,j) = (\sum x \leftarrow xs.\ f\ x$ \$\$ $(i,j))$
⟨*proof*⟩

end

theory *Matrix-Typed*
imports
HOL−Hahn-Banach.Hahn-Banach
Jordan-Normal-Form.Matrix
HOL−Library.Cardinality
Unitary-Tensor-Algebra
begin

87 Typed-dimension matrix wrapper

The type $('a, 'm ::$ *finite*$, 'n ::$ *finite*$)$ *lmat* represents matrices of size $CARD('m) \times CARD('n)$ with entries in $'a$, by typedef of the JNF matrix type at the appropriate dimension.

typedef (**overloaded**) $('a, 'm ::$ *finite*$, 'n ::$ *finite*$)$ *lmat* =
$\{M ::$ *'a Matrix.mat*. *dim-row* $M = CARD('m) \wedge$ *dim-col* $M = CARD('n)\}$
morphisms *unwrap-lm wrap-lm*
⟨*proof*⟩

setup-lifting *type-definition-lmat*

lemma *unwrap-lm-dim-row* [*simp*]:
dim-row (*unwrap-lm* (*M* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)) = *CARD*('*m*)
⟨*proof*⟩

lemma *unwrap-lm-dim-col* [*simp*]:
dim-col (*unwrap-lm* (*M* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)) = *CARD*('*n*)
⟨*proof*⟩

lemma *unwrap-lm-carrier* [*simp*]:
unwrap-lm (*M* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
∈ *carrier-mat* (*CARD*('*m*)) (*CARD*('*n*))
⟨*proof*⟩

88 Typeclass instances

The unique zero matrix of the right shape.

instantiation *lmat* :: (*zero*, *finite*, *finite*) *zero*
begin
lift-definition *zero-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
is 0_m (*CARD*('*m*)) (*CARD*('*n*)) ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-zero* [*simp*]:
unwrap-lm (*0* :: ('*a* :: *zero*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
= 0_m (*CARD*('*m*)) (*CARD*('*n*))
⟨*proof*⟩

instantiation *lmat* :: (*plus*, *finite*, *finite*) *plus*
begin
lift-definition *plus-lmat* ::
('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
is (+) ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-plus* [*simp*]:
unwrap-lm ((*A* :: ('*a* :: *plus*, '*m* :: *finite*, '*n* :: *finite*) *lmat*) + *B*)
= *unwrap-lm A* + *unwrap-lm B*
⟨*proof*⟩

instantiation *lmat* :: (*uminus*, *finite*, *finite*) *uminus*
begin
lift-definition *uminus-lmat* ::
('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
is *uminus* ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-uminus* [*simp*]:
unwrap-lm (− (*A* :: ('*a* :: *uminus*, '*m* :: *finite*, '*n* :: *finite*) *lmat*))
= − *unwrap-lm A*
⟨*proof*⟩

instantiation *lmat* :: (*minus*, *finite*, *finite*) *minus*

begin
lift-definition *minus-lmat* ::
(*'a*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ (*'a*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ (*'a*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
is (−) ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-minus* [*simp*]:
unwrap-lm ((*A* :: (*'a* :: *minus*, *'m* :: *finite*, *'n* :: *finite*) *lmat*) − *B*)
= *unwrap-lm A* − *unwrap-lm B*
⟨*proof*⟩

Lift the additive structure laws via ⟦⋀*i j*. ⟦*i* < *dim-row ?B*; *j* < *dim-col ?B*⟧ ⟹ *?A* $$ (*i*, *j*) = *?B* $$ (*i*, *j*); *dim-row ?A* = *dim-row ?B*; *dim-col ?A* = *dim-col ?B*⟧ ⟹ *?A* = *?B* on the underlying JNF matrices.

instance *lmat* :: (*semigroup-add*, *finite*, *finite*) *semigroup-add*
⟨*proof*⟩

instance *lmat* :: (*ab-semigroup-add*, *finite*, *finite*) *ab-semigroup-add*
⟨*proof*⟩

instance *lmat* :: (*monoid-add*, *finite*, *finite*) *monoid-add*
⟨*proof*⟩

instance *lmat* :: (*comm-monoid-add*, *finite*, *finite*) *comm-monoid-add*
⟨*proof*⟩

instance *lmat* :: (*group-add*, *finite*, *finite*) *group-add*
⟨*proof*⟩

instance *lmat* :: (*ab-group-add*, *finite*, *finite*) *ab-group-add*
⟨*proof*⟩

89 Real scalar multiplication and Hahn-Banach overloading

HOL's (·) is the abstract real-scalar multiplication used by the Hahn-Banach locales. We overload it on the type (*complex*, *'m*, *'n*) *lmat* as multiplication by the embedded real, lifted from JNF's ($\cdot_m$).

overloading
prod-lmat ≡
Vector-Space.prod ::
real ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
⇒ (*complex*, *'m*, *'n*) *lmat*
begin
definition *prod-lmat* ::
real ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
⇒ (*complex*, *'m*, *'n*) *lmat*
where
prod-lmat r M = *wrap-lm* (*complex-of-real r* $\cdot_m$ *unwrap-lm M*)
end

lemma *unwrap-lm-prod* [*simp*]:
unwrap-lm (*Vector-Space.prod r* (*M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*))
= *complex-of-real r* $\cdot_m$ *unwrap-lm M*
⟨*proof*⟩

90 Real-vectorspace locale on the full type

Every (*complex*, *'m*, *'n*) *lmat* with the universe set is a real vector space in HOL/Hahn_Banach's sense.

lemma *vectorspace-UNIV-lmat*:
Vector-Space.vectorspace (*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*)
⟨*proof*⟩

91 The Hilbert-Schmidt norm on (*complex*, *'m*, *'n*) *lmat*

The HS / Frobenius norm: $\|M\|_H S = sqrt\ (Re\ \langle\langle M \mid M\rangle\rangle)$. We define it on (*complex*, *'m*, *'n*) *lmat* using the underlying matrix's Hilbert–Schmidt inner product of Day 1.

definition *hs-norm-lmat* ::
(*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
where
hs-norm-lmat M = *sqrt* (*Re* (⟨⟨ *unwrap-lm M* | *unwrap-lm M* ⟩⟩))

lemma *hs-norm-lmat-nonneg*: *0* ≤ *hs-norm-lmat M*
⟨*proof*⟩

lemma *hs-norm-lmat-sq*:
(*hs-norm-lmat M*)^2 = *Re* (⟨⟨ *unwrap-lm M* | *unwrap-lm M* ⟩⟩)
⟨*proof*⟩

lemma *hs-norm-lmat-smult*:
hs-norm-lmat (*Vector-Space.prod a M*) = |*a*| ∗ *hs-norm-lmat M*
⟨*proof*⟩

92 Triangle inequality for the HS norm

Cauchy-Schwarz on the HS inner product (real form), used in the triangle inequality proof.

lemma *hs-inner-prod-real-self*:
⟨⟨ *A* | *A* ⟩⟩ = *complex-of-real* (*Re* (⟨⟨ *A* | *A* ⟩⟩))
⟨*proof*⟩

lemma *hs-cauchy-schwarz-re*:
fixes *A B* :: *complex Matrix.mat*
assumes *dr*: *dim-row A* = *dim-row B* **and** *dc*: *dim-col A* = *dim-col B*
shows (*Re* (⟨⟨ *A* | *B* ⟩⟩))^2 ≤ *Re* (⟨⟨ *A* | *A* ⟩⟩) ∗ *Re* (⟨⟨ *B* | *B* ⟩⟩)
⟨*proof*⟩

lemma *hs-norm-lmat-triangle*:
fixes *M N* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *hs-norm-lmat* (*M* + *N*) ≤ *hs-norm-lmat M* + *hs-norm-lmat N*
⟨*proof*⟩

93 Subspace, seminorm, normed-vectorspace instances

HS norm is a seminorm on the whole real vector space of (*complex*, *'m*, *'n*) *lmat*.

lemma *seminorm-UNIV-hs-norm-lmat*:
seminorm (*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

HS norm is a (real) norm: it vanishes only on the zero matrix.

lemma *hs-norm-lmat-zero-iff*:
fixes M :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows (*hs-norm-lmat* $M = 0$) $\longleftrightarrow$ ($M = 0$)
⟨*proof*⟩

lemma *norm-UNIV-hs-norm-lmat*:
Normed-Space.norm
(*UNIV* :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

lemma *normed-vectorspace-UNIV-lmat*:
normed-vectorspace
(*UNIV* :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

A complex-linear subspace V of (*complex*, $'m$, $'n$) *lmat* (i.e. nonempty, closed under addition and complex-scalar multiplication) is in particular a real-linear subspace, hence a *subspace* of *UNIV* in HOL/Hahn_Banach's sense.

definition *cmat-subspace-lmat* ::
(*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set* ⇒ *bool*
where
cmat-subspace-lmat $V \equiv$
$V \neq \{\} \wedge$
$(\forall x \in V.\ \forall y \in V.\ x + y \in V) \wedge$
$(\forall x \in V.\ \forall c :: complex.$
wrap-lm ($c \cdot_m$ *unwrap-lm* x) $\in V$)

lemma *subspace-of-cmat-subspace-lmat*:
fixes V :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat set*
assumes *cmat-subspace-lmat* V
shows *Subspace.subspace* V (*UNIV* :: (*complex*, $'m$, $'n$) *lmat set*)
⟨*proof*⟩

94 Complex scalar multiplication on lmat

Complex scalar multiplication on (*complex*, $'m$, $'n$) *lmat*, extending the *Vector-Space.prod* overload (which is only by reals).

definition *cmult-lmat* ::
complex ⇒ (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
⇒ (*complex*, $'m$, $'n$) *lmat*
where
cmult-lmat c M = *wrap-lm* ($c \cdot_m$ *unwrap-lm* M)

lemma *unwrap-lm-cmult* [*simp*]:
unwrap-lm (*cmult-lmat* c (M :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*))
= $c \cdot_m$ *unwrap-lm* M
⟨*proof*⟩

lemma *cmult-lmat-real*:
cmult-lmat (*complex-of-real* a) M = *Vector-Space.prod* a M
⟨*proof*⟩

lemma *cmult-lmat-add*:
fixes x y :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *cmult-lmat* c ($x + y$) = *cmult-lmat* c x + *cmult-lmat* c y
⟨*proof*⟩

lemma *cmult-lmat-scalar-add*:
fixes x :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *cmult-lmat* $(c + d)$ x = *cmult-lmat* c x + *cmult-lmat* d x
⟨*proof*⟩

lemma *cmult-lmat-scalar-mult*:
fixes x :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *cmult-lmat* $(c * d)$ x = *cmult-lmat* c (*cmult-lmat* d x)
⟨*proof*⟩

lemma *cmult-lmat-one*:
fixes x :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *cmult-lmat* 1 x = x
⟨*proof*⟩

lemma *cmult-lmat-uminus-one*:
fixes x :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *cmult-lmat* $(-\ 1)$ x = $-\ x$
⟨*proof*⟩

lemma *cmult-lmat-i-sq*:
fixes x :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *cmult-lmat* i (*cmult-lmat* i x) = $-\ x$
⟨*proof*⟩

lemma *hs-norm-lmat-cmult*:
fixes M :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *hs-norm-lmat* (*cmult-lmat* c M) = *cmod* c * *hs-norm-lmat* M
⟨*proof*⟩

lemma *hs-norm-lmat-cmult-i* [*simp*]:
fixes M :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *hs-norm-lmat* (*cmult-lmat* i M) = *hs-norm-lmat* M
⟨*proof*⟩

end

theory *Essential-Functional-Analysis*
imports
Matrix-Typed
Day10-Quantum-Gates
Jordan-Normal-Form.Schur-Decomposition
Jordan-Normal-Form.DL-Rank
begin

95 Overview

The notes take the analysis of finite-dimensional spaces for granted ("it is easy to see that it is closed", Day 22; "C is a closed cone", Day 28). This theory supplies those facts for the matrix space M_n with the Hilbert–Schmidt norm $\|A\| = \sqrt{\mathrm{Tr}(A^* A)}$: the norm axioms, the parallelogram law, entrywise bounds, completeness, closedness of the sets of Hermitian and of positive semidefinite matrices, and Carathéodory's theorem in the form needed on Day 28: a non-negative combination of product matrices $\sigma \otimes \tau$ in M_{mp} can be rewritten using at most $2(mp)^2 + 1$ of them.

96 HS inner-product algebra

lemma *hs-inner-prod-self-zero-iff*:
fixes A :: *complex Matrix.mat*
shows $\langle\langle\ A \mid A\ \rangle\rangle = 0 \longleftrightarrow A = 0_m\ (\textit{dim-row}\ A)\ (\textit{dim-col}\ A)$
⟨*proof*⟩

lemma *Re-hs-inner-prod-right-smult-real*:
fixes $A\ B$ :: *complex Matrix.mat* **and** t :: *real*
assumes *dim-row* A = *dim-row* B **and** *dim-col* A = *dim-col* B
shows $\textit{Re}\ (\langle\langle\ A \mid \textit{complex-of-real}\ t \cdot_m B\ \rangle\rangle) = t * \textit{Re}\ (\langle\langle\ A \mid B\ \rangle\rangle)$
⟨*proof*⟩

lemma *Re-hs-inner-prod-swap*:
fixes $A\ B$ :: *complex Matrix.mat*
assumes *dim-row* A = *dim-row* B **and** *dim-col* A = *dim-col* B
shows $\textit{Re}\ (\langle\langle\ A \mid B\ \rangle\rangle) = \textit{Re}\ (\langle\langle\ B \mid A\ \rangle\rangle)$
⟨*proof*⟩

97 Trace lemmas

lemma *Tr-smult-mat*:
fixes M :: *complex Matrix.mat* **and** c :: *complex*
assumes *M-c*: $M \in \textit{carrier-mat}\ n\ n$
shows $\textit{Tr}\ (c \cdot_m M) = c * \textit{Tr}\ M$
⟨*proof*⟩

lemma *Re-hs-inner-eq-Re-Tr-dagger*:
fixes $A\ B$ :: *complex Matrix.mat*
assumes *A-c*: $A \in \textit{carrier-mat}\ n\ n$
and *B-c*: $B \in \textit{carrier-mat}\ n\ n$
shows $\textit{Re}\ (\langle\langle\ A \mid B\ \rangle\rangle) = \textit{Re}\ (\textit{Tr}\ ((A^\dagger) * B))$
⟨*proof*⟩

98 HS norm definition and basic facts

HS / Frobenius norm on raw complex matrices. Definition duplicates the one in *Matrix-Typed* for the typed wrapper, but here for the raw matrices.

definition *hs-norm* :: *complex Matrix.mat* ⇒ *real* **where**
$\textit{hs-norm}\ A \equiv \textit{sqrt}\ (\textit{Re}\ (\langle\langle\ A \mid A\ \rangle\rangle))$

lemma *hs-norm-nonneg*: $\textit{hs-norm}\ A \geq 0$
⟨*proof*⟩

lemma *hs-norm-sq*:
$(\textit{hs-norm}\ A)^2 = \textit{Re}\ (\langle\langle\ A \mid A\ \rangle\rangle)$
⟨*proof*⟩

lemma *hs-norm-zero*: $\textit{hs-norm}\ (0_m\ m\ n) = 0$
⟨*proof*⟩

lemma *hs-norm-smult*:
$\textit{hs-norm}\ (c \cdot_m A) = \textit{cmod}\ c * \textit{hs-norm}\ A$
⟨*proof*⟩

Triangle inequality for the HS norm via Cauchy-Schwarz.

lemma *hs-norm-triangle*:
fixes *A B* :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat n n*
and *B-c*: $B \in$ *carrier-mat n n*
shows *hs-norm* $(A + B) \leq$ *hs-norm A* + *hs-norm B*
⟨*proof*⟩

HS-norm-squared of $U + (-t) \cdot V$, expanded along the bilinear structure. We extract this into a stand-alone lemma so the parameter names do not clash with let-bound names in callers.

lemma *hs-norm-sq-diff-expand*:
fixes *U V* :: *complex Matrix.mat* **and** *t* :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *V-c*: $V \in$ *carrier-mat n n*
shows $(\textit{hs-norm}\ (U + \textit{complex-of-real}\ (-t) \cdot_m V))^2$
$= \textit{Re}\ (\langle\langle U \mid U \rangle\rangle) - 2 * t * \textit{Re}\ (\langle\langle U \mid V \rangle\rangle)$
$+ t^2 * \textit{Re}\ (\langle\langle V \mid V \rangle\rangle)$
⟨*proof*⟩

HS norm satisfies the parallelogram identity.

lemma *hs-norm-parallelogram*:
fixes *U V* :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *V-c*: $V \in$ *carrier-mat n n*
shows $(\textit{hs-norm}\ (U + V))^2 + (\textit{hs-norm}\ (U - V))^2$
$= 2 * (\textit{hs-norm}\ U)^2 + 2 * (\textit{hs-norm}\ V)^2$
⟨*proof*⟩

lemma *hs-norm-zero-iff*:
fixes *A* :: *complex Matrix.mat*
shows *hs-norm A* $= 0 \longleftrightarrow A = 0_m$ (*dim-row A*) (*dim-col A*)
⟨*proof*⟩

Every entry of a matrix is bounded in modulus-squared by the HS-norm squared. Useful for transferring HS-norm convergence to entry-wise convergence.

lemma *cmod-entry-sq-le-hs-norm-sq*:
fixes *A* :: *complex Matrix.mat* **and** *i j* :: *nat*
assumes *A-c*: $A \in$ *carrier-mat n n*
and *i-lt*: $i < n$ **and** *j-lt*: $j < n$
shows $(\textit{cmod}\ (A\ \$\$\ (i,j)))^2 \leq (\textit{hs-norm}\ A)^2$
⟨*proof*⟩

Modulus form of the entry bound: $|A_{i,j}| \leq \|A\|_H S$.

lemma *cmod-entry-le-hs-norm*:
fixes *A* :: *complex Matrix.mat* **and** *i j* :: *nat*
assumes *A-c*: $A \in$ *carrier-mat n n*
and *i-lt*: $i < n$ **and** *j-lt*: $j < n$
shows *cmod* $(A\ \$\$\ (i,j)) \leq$ *hs-norm A*
⟨*proof*⟩

99 HS-norm convergence vs entry-wise convergence

HS-norm convergence implies entry-wise convergence: if (M_n) converges to L in HS-norm then for every (i, j) the entry sequence M_n \$\$ (i, j) tends to L \$\$ (i, j).

lemma *HS-lim-entry-lim*:
fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *r* :: *nat*

and $i\ j$ :: *nat* **and** L :: *complex Matrix.mat*
assumes *carrier*: $\forall\, n.$ *M-seq* $n \in$ *carrier-mat* $r\ r$
and *L-c*: $L \in$ *carrier-mat* $r\ r$
and *lim*: $(\lambda n.$ *hs-norm* (*M-seq* $n - L)) \longrightarrow 0$
and *i-lt*: $i < r$ **and** *j-lt*: $j < r$
shows $(\lambda n.$ *M-seq* n \$\$ $(i,j)) \longrightarrow L$ \$\$ (i,j)
⟨*proof*⟩

HS-Cauchy implies entry-wise Cauchy: the dual of *HS-lim-entry-lim*. Combined with the completeness of $\mathbb{C}$ it yields a candidate limit for every entry.

lemma *HS-Cauchy-entry-Cauchy*:
fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** r :: *nat*
and $i\ j$ :: *nat*
assumes *carrier*: $\forall\, n.$ *M-seq* $n \in$ *carrier-mat* $r\ r$
and *hs-cauchy*: $\forall\, \varepsilon{>}0.\ \exists\, N.\ \forall\, x \geq N.\ \forall\, y \geq N.$
hs-norm (*M-seq* $x -$ *M-seq* $y) < \varepsilon$
and *i-lt*: $i < r$ **and** *j-lt*: $j < r$
shows *Cauchy* $(\lambda n.$ *M-seq* n \$\$ $(i,j))$
⟨*proof*⟩

HS-norm-squared as a real double sum of entry-modulus squares. This is the entry-wise expansion of $\|A\|^2 = Re\ \langle\langle\ A \mid A\ \rangle\rangle$.

lemma *hs-norm-sq-as-entry-sum*:
fixes A :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat* $r\ r$
shows (*hs-norm* $A)^2$
$= (\sum i{\in}\{0..{<}r\}.\ \sum j{\in}\{0..{<}r\}.\ (cmod\ (A$ \$\$ $(i,j)))^2)$
⟨*proof*⟩

Pointwise entry bounds give an HS-norm-squared bound.

lemma *hs-norm-sq-le-from-entry-bounds*:
fixes A :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat* $r\ r$
and *entries*: $\bigwedge i\ j.\ i < r \Longrightarrow j < r \Longrightarrow (cmod\ (A$ \$\$ $(i,j)))^2 \leq M$
shows (*hs-norm* $A)^2 \leq$ *real* $r *$ *real* $r * M$
⟨*proof*⟩

Entry-cmod bound gives an HS-norm bound. Combines the *hs-norm-sq-le-from-entry-bounds* bound with *sqrt*.

lemma *hs-norm-le-from-entry-bounds*:
fixes A :: *complex Matrix.mat* **and** δ :: *real*
assumes *A-c*: $A \in$ *carrier-mat* $r\ r$
and *δ-nn*: $0 \leq \delta$
and *entries*: $\bigwedge i\ j.\ i < r \Longrightarrow j < r \Longrightarrow cmod\ (A$ \$\$ $(i,j)) \leq \delta$
shows *hs-norm* $A \leq$ *real* $r * \delta$
⟨*proof*⟩

Entry-wise convergence implies HS-norm convergence. This is the slow half of HS-completeness; we use the entry-bound helper to keep it tractable.

lemma *entry-lim-imp-HS-lim*:
fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *M-lim* :: *complex Matrix.mat*
and r :: *nat*
assumes *carrier*: $\forall\, n.$ *M-seq* $n \in$ *carrier-mat* $r\ r$
and *M-lim-c*: *M-lim* $\in$ *carrier-mat* $r\ r$
and *entry-lim*: $\bigwedge i\ j.\ i < r \Longrightarrow j < r$
$\Longrightarrow (\lambda n.$ *M-seq* n \$\$ $(i,j)) \longrightarrow$ *M-lim* \$\$ (i,j)

shows $(\lambda n.\ \textit{hs-norm}\ (\textit{M-seq}\ n - \textit{M-lim})) \longrightarrow 0$
⟨*proof*⟩

HS-completeness of the matrix space: every HS-Cauchy sequence in *carrier-mat r r* converges in HS norm to a matrix in *carrier-mat r r*. Combines *HS-Cauchy-entry-Cauchy* + completeness of $\mathbb{C}$ + *entry-lim-imp-HS-lim*.

lemma *HS-Cauchy-imp-convergent*:
fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *r* :: *nat*
assumes *carrier*: $\forall n.\ \textit{M-seq}\ n \in \textit{carrier-mat}\ r\ r$
and *hs-cauchy*: $\forall \varepsilon{>}0.\ \exists N.\ \forall x \geq N.\ \forall y \geq N.$
$\textit{hs-norm}\ (\textit{M-seq}\ x - \textit{M-seq}\ y) < \varepsilon$
shows $\exists\, \textit{M-lim}.\ \textit{M-lim} \in \textit{carrier-mat}\ r\ r$
$\wedge\ (\forall \varepsilon{>}0.\ \exists N.\ \forall n \geq N.\ \textit{hs-norm}\ (\textit{M-seq}\ n - \textit{M-lim}) < \varepsilon)$
⟨*proof*⟩

Symmetry of HS-distance: $\|A - B\| = \|B - A\|$.

lemma *hs-norm-sub-swap*:
fixes *A B* :: *complex Matrix.mat*
assumes *A-c*: $A \in \textit{carrier-mat}\ n\ n$
and *B-c*: $B \in \textit{carrier-mat}\ n\ n$
shows $\textit{hs-norm}\ (A - B) = \textit{hs-norm}\ (B - A)$
⟨*proof*⟩

HS-norm closedness of the Hermitian set.

lemma *hermitian-HS-closed*:
fixes *M-seq* :: *nat* ⇒ *complex Matrix.mat* **and** *L* :: *complex Matrix.mat*
and *r* :: *nat*
assumes *carrier*: $\forall n.\ \textit{M-seq}\ n \in \textit{carrier-mat}\ r\ r$
and *L-c*: $L \in \textit{carrier-mat}\ r\ r$
and *herm-n*: $\forall n.\ \textit{hermitian}\ (\textit{M-seq}\ n)$
and *lim*: $(\lambda n.\ \textit{hs-norm}\ (\textit{M-seq}\ n - L)) \longrightarrow 0$
shows *hermitian L*
⟨*proof*⟩

HS-norm closedness of the PSD set. The proof combines hermitian_HS_closed for the Hermitian part with continuity of the quadratic form $\sigma \mapsto \langle x \mid \sigma *_v x \rangle$ (linear in the entries of σ, hence HS-continuous via entry convergence).

lemma *psd-HS-closed*:
fixes σ-*n* :: *nat* ⇒ *complex Matrix.mat* **and** σ :: *complex Matrix.mat*
and *r* :: *nat*
assumes *carrier*: $\forall n.\ \sigma\text{-}n\ n \in \textit{carrier-mat}\ r\ r$
and σ-*c*: $\sigma \in \textit{carrier-mat}\ r\ r$
and *psd-n*: $\forall n.\ \textit{psd}\ (\sigma\text{-}n\ n)$
and *lim*: $(\lambda n.\ \textit{hs-norm}\ (\sigma\text{-}n\ n - \sigma)) \longrightarrow 0$
shows *psd* σ
⟨*proof*⟩

Reverse triangle inequality: $|\|A\| - \|B\|| \leq \|A - B\|$.

lemma *hs-norm-sub-triangle-ineq*:
fixes *A B* :: *complex Matrix.mat*
assumes *A-c*: $A \in \textit{carrier-mat}\ n\ n$
and *B-c*: $B \in \textit{carrier-mat}\ n\ n$
shows $|\textit{hs-norm}\ A - \textit{hs-norm}\ B| \leq \textit{hs-norm}\ (A - B)$
⟨*proof*⟩

100 Carathéodory's theorem for conic combinations

Carathéodory's theorem for cones: in a real vector space V of dimension d, every element of the conic hull of a set S (a finite non-negative combination of elements of S) is already a non-negative combination of at most d elements of S. We need it for V the real space of Hermitian $(mp)\times(mp)$ matrices, of real dimension $(mp)^2$, and S the product positive matrices $\sigma \otimes \tau$: it bounds the length of the decompositions of elements of the separable cone uniformly, which is what makes the cone closed (Separable_Cone). HOL-Analysis has Carathéodory for *euclidean-space*; using it directly would require a Euclidean-space instance on the Hermitian matrices, so the theorem is proved here directly for lists: any $d+1$ vectors are linearly dependent, and renormalising the weights along a dependence relation removes one term while keeping all weights non-negative.

Below is a fold-based weighted sum of the cone-style list, suitable for stating Carathéodory at the matrix level.

fun *cone-list-sum* ::
  nat ⇒ *nat* ⇒
   (*real* × *complex Matrix.mat* × *complex Matrix.mat*) *list* ⇒
   complex Matrix.mat
where
  cone-list-sum m p [] = 0_m (*m*∗*p*) (*m*∗*p*)
| *cone-list-sum m p* ((*w*, *σ*, *τ*) # *ts*) =
     complex-of-real w ·$_m$ (*σ* ⨂ *τ*) + *cone-list-sum m p ts*

lemma *cone-list-sum-carrier*:
  cone-list-sum m p ts ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

cone-list-sum is additive over list-append.

lemma *cone-list-sum-append*:
  cone-list-sum m p (*xs* @ *ys*)
    = *cone-list-sum m p xs* + *cone-list-sum m p ys*
⟨*proof*⟩

A zero-weight singleton over a properly-sized matrix pair contributes the zero matrix.

lemma *cone-list-sum-zero-singleton*:
  assumes *σ-c*: *σ* ∈ *carrier-mat m m* **and** *τ-c*: *τ* ∈ *carrier-mat p p*
  shows *cone-list-sum m p* [(*0*, *σ*, *τ*)] = 0_m (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

Dropping a zero-weight entry from a cone list (with properly typed matrix part) preserves the cone sum.

lemma *cone-list-sum-drop-zero*:
  assumes *i-lt*: *i* < *length xs*
     and *zero-w*: *fst* (*xs* ! *i*) = *0*
     and *typing*: ∀(*w*, *σ*, *τ*) ∈ *set xs*.
                 σ ∈ *carrier-mat m m* ∧ *τ* ∈ *carrier-mat p p*
  shows *cone-list-sum m p* (*take i xs* @ *drop* (*Suc i*) *xs*)
        = *cone-list-sum m p xs*
⟨*proof*⟩

Linearity of *cone-list-sum* in the weight. Two lists with matching matrix parts but weights related by $w_k = {w_k}^1 + \mu * r_k$ satisfy the corresponding sum-equation.

lemma *cone-list-sum-weight-split*:
  fixes *f g h* :: *nat* ⇒ *real*
   and *σf τf* :: *nat* ⇒ *complex Matrix.mat*

```
  and μ :: real and L :: nat
 assumes typing: ⋀k. k < L ⟹
                   σf k ∈ carrier-mat m m ∧ τf k ∈ carrier-mat p p
    and split: ⋀k. k < L ⟹ f k = g k + μ ∗ h k
 shows cone-list-sum m p (map (λk. (f k, σf k, τf k)) [0..<L])
     = cone-list-sum m p (map (λk. (g k, σf k, τf k)) [0..<L])
       + complex-of-real μ ·_m
          cone-list-sum m p (map (λk. (h k, σf k, τf k)) [0..<L])
⟨proof⟩
```

Entry-wise expression for *cone-list-sum*: each entry is a HOL sum over the list indices of weight times tensor entry.

```
lemma cone-list-sum-entry:
 fixes m p :: nat
    and ts :: (real × complex Matrix.mat × complex Matrix.mat) list
 assumes typing: ∀(w, σ, τ) ∈ set ts.
                   σ ∈ carrier-mat m m ∧ τ ∈ carrier-mat p p
    and i-lt: i < m ∗ p and j-lt: j < m ∗ p
 shows cone-list-sum m p ts $$ (i, j)
        = (∑ k<length ts.
            complex-of-real (fst (ts!k))
              ∗ (fst (snd (ts!k)) ⊗ snd (snd (ts!k))) $$ (i, j))
⟨proof⟩
```

An entry-wise zero criterion: if every entry is 0, the matrix is 0_m.

```
lemma matrix-eq-zero-via-entries:
 fixes M :: complex Matrix.mat
 assumes M-c: M ∈ carrier-mat (m∗p) (m∗p)
    and zero: ⋀i j. i < m∗p ⟹ j < m∗p ⟹ M $$ (i, j) = 0
 shows M = 0_m (m∗p) (m∗p)
⟨proof⟩
```

Top-level linear-algebra core. A list of more than $2\ (m\ p)^2$ complex $(m\ p) \times (m\ p)$ matrix pairs admits a non-trivial real linear combination of their tensor products that vanishes.

The proof is the standard JNF *vec-space TYPE(real)* $(2\ (m\ p)^2)$ *li-le-dim* contrapositive applied to a real-flatten of the tensor products. Admitted here as a single named lemma so it can be discharged independently of the cone Carathéodory reduction machinery (which is fully proved below modulo this one lemma).

```
lemma cone-list-real-lin-dep:
 fixes m p :: nat
    and pairs :: (complex Matrix.mat × complex Matrix.mat) list
 assumes m-pos: 0 < m and p-pos: 0 < p
    and typing: ∀(σ, τ) ∈ set pairs.
                   σ ∈ carrier-mat m m ∧ τ ∈ carrier-mat p p
    and long: length pairs > 2 ∗ (m∗p)^2
 shows ∃r :: nat ⇒ real.
          (∃k < length pairs. r k ≠ 0) ∧
          (∀k ≥ length pairs. r k = 0) ∧
          cone-list-sum m p
            (map (λk. (r k, fst (pairs ! k), snd (pairs ! k)))
                 [0..<length pairs])
           = 0_m (m∗p) (m∗p)
⟨proof⟩
```

One reduction step: a length-L cone list with $L > (m\ p)^2 + 1$ admits a length-$(L-1)$ list with the same cone sum, weights non-negative, and matrix pairs inherited from the original list.

NOTE on the conclusion: the standard Carathéodory reduction *reweights* rather than *selects*, so we require only that the *matrix pair* (σ, τ) of each new entry occurs in the original list the weight may differ. This is what the downstream closedness argument actually needs (typing + non-negativity inherited from the original cone).

lemma *caratheodory-matrix-cone-step*:
  fixes *m p* :: *nat*
    and *ts* :: (*real* × *complex Matrix.mat* × *complex Matrix.mat*) *list*
  assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
    and *ts-typing*: ∀ (*w*, *σ*, *τ*) ∈ *set ts*.
                  σ ∈ *carrier-mat m m* ∧ *τ* ∈ *carrier-mat p p*
    and *ts-nn*: ∀ (*w*, -, -) ∈ *set ts*. *0* ≤ *w*
    and *ts-long*: *length ts* > *2* ∗ (*m*∗*p*)^2 + *1*
  shows ∃ *ts′*. *length ts′* = *length ts* − *1*
          ∧ (∀ (*w*, *σ*, *τ*) ∈ *set ts′*.
              σ ∈ *carrier-mat m m* ∧ *τ* ∈ *carrier-mat p p* ∧ *0* ≤ *w*
              ∧ (∃ *w*$_0$. (*w*$_0$, *σ*, *τ*) ∈ *set ts*))
          ∧ *cone-list-sum m p ts* = *cone-list-sum m p ts′*
⟨*proof*⟩

The Carathéodory bound follows from the reduction step by strong induction on *length ts*. This part is now closed unconditionally; only the two focused sub-sorries inside *caratheodory-matrix-cone-step* remain.

lemma *caratheodory-matrix-cone*:
  fixes *m p* :: *nat*
    and *ts* :: (*real* × *complex Matrix.mat* × *complex Matrix.mat*) *list*
  assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
    and *ts-typing*: ∀ (*w*, *σ*, *τ*) ∈ *set ts*.
                  σ ∈ *carrier-mat m m* ∧ *τ* ∈ *carrier-mat p p*
    and *ts-nn*: ∀ (*w*, -, -) ∈ *set ts*. *0* ≤ *w*
  shows ∃ *ts′*. *length ts′* ≤ *2* ∗ (*m*∗*p*)^2 + *1*
          ∧ (∀ (*w*, *σ*, *τ*) ∈ *set ts′*.
              σ ∈ *carrier-mat m m* ∧ *τ* ∈ *carrier-mat p p* ∧ *0* ≤ *w*
              ∧ (∃ *w*$_0$. (*w*$_0$, *σ*, *τ*) ∈ *set ts*))
          ∧ *cone-list-sum m p ts* = *cone-list-sum m p ts′*
⟨*proof*⟩

How the bound is used (Separable_Cone): a sequence in the separable cone has decompositions of length at most $K = 2(mp)^2+1$; padding with zero-weight entries and normalising traces makes the weights bounded, so coordinatewise Bolzano–Weierstrass extracts a convergent subsequence of weights and factors; tensoring and addition are continuous, so the limit lies in the cone. The compactness of the unit-trace positive matrices is obtained from the Euclidean-space instance of Lmat_Euclidean (real dimension $2(mp)^2$) rather than from the sequential-compactness lemma *HS-Cauchy-imp-convergent* proved above.

end

theory *Lmat-Euclidean*
  imports
    Quantum-Computing-by-VP-3-10.Matrix-Typed
    HOL−Analysis.Euclidean-Space
    HOL−Analysis.L2-Norm
    HOL−Analysis.Elementary-Metric-Spaces
    HOL−Analysis.Topology-Euclidean-Space
begin

The HOL/Hahn_Banach development imports a locale also called *norm* under the qualifier

Normed-Space. Hide its short names so that, inside our instantiation blocks, *norm* resolves to the class method from *HOL.Real-Vector-Spaces*.

hide-const (**open**) *Normed-Space.norm*
hide-const (**open**) *Normed-Space.seminorm*

101 Overview

Nothing in this theory is stated in the notes; it is the bridge that lets the HOL-Analysis theorems on convex sets (separating hyperplanes, compactness) be applied to matrices. ('*a*, '*m*, '*n*) *lmat* is given the typeclass tower *real-vector*, *metric-space*, *real-normed-vector*, *real-inner*, *euclidean-space*, *heine-borel*; for '*a* = *complex* the norm is the Hilbert–Schmidt norm, the inner product its real part, and the dimension is $2\,\mathrm{CARD}('m)\,\mathrm{CARD}('n)$.

102 Real scalar multiplication and *real-vector* instance

For '*a* :: *real-vector*, lift element-wise scalar multiplication via *map-mat* (*scaleR a*).

instantiation *lmat* :: (*real-vector*, *finite*, *finite*) *scaleR*
begin

lift-definition *scaleR-lmat* ::
real ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
is λ*r M*. *map-mat* (λ*x*. *r* ∗$_R$ *x*) *M* ⟨*proof*⟩

instance ⟨*proof*⟩

end

lemma *unwrap-lm-scaleR* [*simp*]:
unwrap-lm ((*r* :: *real*) ∗$_R$ (*M* :: ('*a* :: *real-vector*, '*m* :: *finite*, '*n* :: *finite*) *lmat*))
= *map-mat* (λ*x*. *r* ∗$_R$ *x*) (*unwrap-lm M*)
⟨*proof*⟩

lemma *unwrap-lm-scaleR-index* [*simp*]:
fixes *M* :: ('*a* :: *real-vector*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *unwrap-lm* (*r* ∗$_R$ *M*) $$ (*i*, *j*) = *r* ∗$_R$ (*unwrap-lm M* $$ (*i*, *j*))
⟨*proof*⟩

instance *lmat* :: (*real-vector*, *finite*, *finite*) *real-vector*
⟨*proof*⟩

103 Auxiliary: pointwise unwrapping (used by later instances)

lemma *unwrap-lm-plus-index* [*simp*]:
fixes *M N* :: ('*a* :: *plus*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *unwrap-lm* (*M* + *N*) $$ (*i*, *j*) = *unwrap-lm M* $$ (*i*, *j*) + *unwrap-lm N* $$ (*i*, *j*)
⟨*proof*⟩

lemma *unwrap-lm-minus-index* [*simp*]:
fixes *M N* :: ('*a* :: *ab-group-add*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *unwrap-lm* (*M* − *N*) $$ (*i*, *j*) = *unwrap-lm M* $$ (*i*, *j*) − *unwrap-lm N* $$ (*i*, *j*)
⟨*proof*⟩

lemma *unwrap-lm-zero-index* [*simp*]:
fixes $i\ j :: nat$
assumes $i < CARD('m :: finite)\ j < CARD('n :: finite)$
shows *unwrap-lm* $(0 :: ('a :: zero, 'm, 'n)\ lmat)\ \$\$\ (i, j) = 0$
⟨*proof*⟩

lemma *lmat-eq-iff-entries*:
fixes $M\ N :: ('a, 'm :: finite, 'n :: finite)\ lmat$
shows $M = N \longleftrightarrow$
$(\forall\, i\ j.\ i < CARD('m) \longrightarrow j < CARD('n)$
$\longrightarrow$ *unwrap-lm* $M\ \$\$\ (i, j) =$ *unwrap-lm* $N\ \$\$\ (i, j))$
⟨*proof*⟩

104 Distance, uniformity, open: *metric-space* instance

Distance is the L2 sum of entry-wise distances, indexed by $\{0..<CARD('m)\} \times \{0..<CARD('n)\}$. Uniformity and *open* are defined via the standard metric-induced filter so that the *open-uniformity* axiom is discharged by reflexivity.

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *dist*
begin

definition *dist-lmat* :: $('a, 'm :: finite, 'n :: finite)\ lmat$
$\Rightarrow ('a, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$ **where**
dist-lmat $M\ N =$ *L2-set* $(\lambda(i, j).\ dist\ (unwrap\text{-}lm\ M\ \$\$\ (i, j))\ (unwrap\text{-}lm\ N\ \$\$\ (i, j)))$
$(\{0..<CARD('m)\} \times \{0..<CARD('n)\})$

instance ⟨*proof*⟩

end

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *uniformity-dist*
begin

definition *uniformity-lmat* ::
$(('a, 'm :: finite, 'n :: finite)\ lmat \times ('a, 'm, 'n)\ lmat)\ filter$ **where**
(*uniformity-lmat* :: $(('a, 'm, 'n)\ lmat \times ('a, 'm, 'n)\ lmat)\ filter$)
$= (INF\ e \in \{0<..\}.\ principal\ \{(x, y).\ dist\ x\ y < e\})$

instance ⟨*proof*⟩

end

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *metric-space*
begin

definition *open-lmat* :: $('a, 'm :: finite, 'n :: finite)\ lmat\ set \Rightarrow bool$ **where**
open-lmat $U = (\forall\, x \in U.\ \forall_F\ (x', y)\ in\ uniformity.\ x' = x \longrightarrow y \in U)$

instance ⟨*proof*⟩

end

105 Norm and *real-normed-vector* instance

instantiation *lmat* :: (*real-normed-vector*, *finite*, *finite*) *real-normed-vector*

begin

definition *norm-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ *real* **where**
norm-lmat M = *L2-set* (λ(*i*, *j*). *norm* (*unwrap-lm M* $$ (*i*, *j*)))
({*0*..<*CARD*('*m*)} × {*0*..<*CARD*('*n*)})

definition *sgn-lmat* ::
('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m*, '*n*) *lmat* **where**
sgn-lmat (*M* :: ('*a*, '*m*, '*n*) *lmat*) = *scaleR* (*inverse* (*norm M*)) *M*

instance ⟨*proof*⟩

end

106 Real inner product

For '*a* :: *real-inner*, the lmat inner product is the entry-wise sum of inner products. When '*a* = *complex* with the standard real-inner instance *inner z w* = *Re* (*cnj z* ∗ *w*), this is the real part of the Hilbert-Schmidt inner product.

instantiation *lmat* :: (*real-inner*, *finite*, *finite*) *real-inner*
begin

definition *inner-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
⇒ ('*a*, '*m*, '*n*) *lmat* ⇒ *real* **where**
inner-lmat M N = (∑ (*i*, *j*) ∈ {*0*..<*CARD*('*m*)} × {*0*..<*CARD*('*n*)}.
inner (*unwrap-lm M* $$ (*i*, *j*)) (*unwrap-lm N* $$ (*i*, *j*)))

instance ⟨*proof*⟩

end

107 Basis: matrix units carrying a basis vector

definition *lmat-axis* :: *nat* ⇒ *nat* ⇒ '*a* :: *zero*
⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* **where**
lmat-axis i j u = *wrap-lm* (*Matrix.mat* (*CARD*('*m*)) (*CARD*('*n*))
(λ(*r*, *s*). *if r* = *i* ∧ *s* = *j then u else 0*))

lemma *unwrap-lmat-axis* [*simp*]:
unwrap-lm (*lmat-axis i j u* :: ('*a* :: *zero*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
= *Matrix.mat* (*CARD*('*m*)) (*CARD*('*n*))
(λ(*r*, *s*). *if r* = *i* ∧ *s* = *j then u else 0*)
⟨*proof*⟩

lemma *lmat-axis-index* [*simp*]:
fixes *u* :: '*a* :: *zero*
assumes *r* < *CARD*('*m* :: *finite*) *s* < *CARD*('*n* :: *finite*)
shows *unwrap-lm* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*) $$ (*r*, *s*)
= (*if r* = *i* ∧ *s* = *j then u else 0*)
⟨*proof*⟩

Inner product against a single matrix-unit basis vector picks out one matrix entry.

lemma *inner-lmat-axis*:
fixes *u* :: '*a* :: *real-inner*
and *M* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *ij*: *i* < *CARD*('*m*) *j* < *CARD*('*n*)

shows *inner M* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*)
= *inner* (*unwrap-lm M* $$ (*i*, *j*)) *u*
⟨*proof*⟩

Two matrix-unit basis vectors are equal iff their entries and positions match (or both vectors are zero).

lemma *lmat-axis-eq-iff*:
fixes *u v* :: '*a* :: *zero*
assumes *i* < *CARD*('*m* :: *finite*) *j* < *CARD*('*n* :: *finite*)
and *i'* < *CARD*('*m*) *j'* < *CARD*('*n*)
shows (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*) = *lmat-axis i' j' v*
⟷ (*u* = *v* ∧ *i* = *i'* ∧ *j* = *j'*) ∨ (*u* = *0* ∧ *v* = *0*)
⟨*proof*⟩

Inner product between two matrix-unit basis vectors: 1 if same position and basis component, 0 otherwise.

lemma *inner-lmat-axis-axis*:
fixes *u v* :: '*a* :: *real-inner*
assumes *ij*: *i* < *CARD*('*m* :: *finite*) *j* < *CARD*('*n* :: *finite*)
and *ij'*: *i'* < *CARD*('*m*) *j'* < *CARD*('*n*)
shows *inner* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*) (*lmat-axis i' j' v*)
= (*if i* = *i'* ∧ *j* = *j' then inner u v else 0*)
⟨*proof*⟩

108 Basis set

definition *Basis-lmat-set* :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat set*
where *Basis-lmat-set* = (⋃ *i* ∈ {*0*..<*CARD*('*m*)}.
⋃ *j* ∈ {*0*..<*CARD*('*n*)}.
⋃ *u* ∈ *Basis*. {*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*})

lemma *Basis-lmat-set-finite*:
finite (*Basis-lmat-set* :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat set*)
⟨*proof*⟩

lemma *Basis-lmat-set-nonempty*:
Basis-lmat-set ≠ ({} :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat set*)
⟨*proof*⟩

lemma *Basis-lmat-orthonormal*:
fixes *u v* :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *uB*: *u* ∈ *Basis-lmat-set* **and** *vB*: *v* ∈ *Basis-lmat-set*
shows *inner u v* = (*if u* = *v then 1 else 0*)
⟨*proof*⟩

lemma *axis-mem-Basis-lmat-set*:
fixes *u* :: '*a* :: *euclidean-space*
assumes *i-lt*: *i* < *CARD*('*m* :: *finite*)
and *j-lt*: *j* < *CARD*('*n* :: *finite*)
and *uB*: *u* ∈ *Basis*
shows (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*) ∈ *Basis-lmat-set*
⟨*proof*⟩

lemma *Basis-lmat-perp-imp-zero*:
fixes *x* :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes ∗: ∀ *u* ∈ *Basis-lmat-set*. *inner x u* = *0*
shows *x* = *0*

⟨*proof*⟩

lemma *zero-inner-Basis-lmat*:
fixes u :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
shows *inner* (*0* :: ('*a*, '*m*, '*n*) *lmat*) u = *0*
⟨*proof*⟩

lemma *Basis-lmat-perp-zero*:
fixes x :: ('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
shows (∀ u ∈ *Basis-lmat-set*. *inner* x u = *0*) ⟷ x = *0*
⟨*proof*⟩

109 Euclidean space instance

instantiation *lmat* :: (*euclidean-space*, *finite*, *finite*) *euclidean-space*
begin

definition *Basis-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat set* **where**
Basis-lmat = *Basis-lmat-set*

instance ⟨*proof*⟩

end

110 Heine-Borel

instance *lmat* :: (*euclidean-space*, *finite*, *finite*) *heine-borel* ⟨*proof*⟩

111 Dimension

Two distinct (i, j, u) with u ∈ *Basis-*'*a* give distinct matrix-axis vectors.

lemma *lmat-axis-inj-on-Basis-indices*:
fixes u v :: '*a* :: *euclidean-space*
assumes *ij*: i < *CARD*('*m* :: *finite*) j < *CARD*('*n* :: *finite*)
and *ij'*: i' < *CARD*('*m*) j' < *CARD*('*n*)
and *uB*: u ∈ *Basis* **and** *vB*: v ∈ *Basis*
shows (*lmat-axis* i j u :: ('*a*, '*m*, '*n*) *lmat*) = *lmat-axis* i' j' v
⟷ i = i' ∧ j = j' ∧ u = v
⟨*proof*⟩

lemma *DIM-lmat*:
DIM(('*a* :: *euclidean-space*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
= *CARD*('*m*) ∗ *CARD*('*n*) ∗ *DIM*('*a*)
⟨*proof*⟩

112 Component-wise bounds via the L2 norm

lemma *norm-lmat-index-le*:
fixes M :: ('*a* :: *real-normed-vector*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes i < *CARD*('*m*) j < *CARD*('*n*)
shows *norm* (*unwrap-lm* M $$ (i, j)) ≤ *norm* M
⟨*proof*⟩

lemma *dist-lmat-index-le*:
fixes M N :: ('*a* :: *real-normed-vector*, '*m* :: *finite*, '*n* :: *finite*) *lmat*

assumes $i < CARD('m)$ $j < CARD('n)$
shows *dist* (*unwrap-lm M $$ (i, j)*) (*unwrap-lm N $$ (i, j)*) $\leq$ *dist M N*
⟨*proof*⟩

113 Convergence: lmat sequence convergence implies entry convergence

lemma *tendsto-unwrap-lm-index*:
fixes $f :: 'b \Rightarrow ('a :: real\text{-}normed\text{-}vector, 'm :: finite, 'n :: finite)$ *lmat*
and $L :: ('a, 'm, 'n)$ *lmat*
assumes *lim*: $(f \longrightarrow L)$ *F*
and *ij*: $i < CARD('m)$ $j < CARD('n)$
shows $((\lambda x.$ *unwrap-lm* $(f\ x)$ *$$* $(i, j)) \longrightarrow$ *unwrap-lm L $$* $(i, j))$ *F*
⟨*proof*⟩

lemma *LIMSEQ-unwrap-lm-index*:
fixes $f :: nat \Rightarrow ('a :: real\text{-}normed\text{-}vector, 'm :: finite, 'n :: finite)$ *lmat*
and $L :: ('a, 'm, 'n)$ *lmat*
assumes $f \longrightarrow L$ $i < CARD('m)$ $j < CARD('n)$
shows $(\lambda n.$ *unwrap-lm* $(f\ n)$ *$$* $(i, j)) \longrightarrow$ *unwrap-lm L $$* (i, j)
⟨*proof*⟩

end

theory *Separable-Cone*
imports
Lmat-Euclidean
Quantum-Computing-by-VP-3-10.Day10-Quantum-Gates
HOL−Analysis.Convex
HOL−Analysis.Convex-Euclidean-Space
HOL−Analysis.Polytope
HOL−Analysis.Starlike
begin

HOL-Hahn_Banach defines a locale also called *continuous*. Hide its short name so that, in the proofs below, *continuous* resolves to the topological-space predicate from *HOL.Topological-Spaces*.

hide-const (**open**) *Function-Norm.continuous*

114 Overview (Days 27–28 of the notes)

This theory proves the results restated lecture by lecture in theory Day24_Separability. The notes argue in the real vector space of Hermitian matrices with a closed-cone separation theorem; formally we work in the typed matrix space $(complex, 'm \times 'p, 'm \times 'p)$ *lmat*, a real inner-product space (Lmat_Euclidean), and use HOL-Analysis' *separating-hyperplane-closed-point*. Steps: the positive matrices form a closed set; the unit-trace positive matrices form a compact set; the tensor product is continuous, so the product density matrices *Sprime* form a compact set not containing 0; by Carathéodory (Essential_Functional_Analysis) every element of the separable cone is a bounded combination of at most $2(mp)^2 + 1$ generators, and the cone is closed. Separation then yields a real functional, which is converted into a Hermitian witness W with $\Re\,\mathrm{Tr}(W\rho) < 0 \leq \Re\,\mathrm{Tr}(W\sigma)$ on the cone.

115 Typed PSD predicate

definition *psd-lmat* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* ⇒ *bool* **where**
psd-lmat M ⟷ *psd* (*unwrap-lm M*)

lemma *psd-lmat-unwrap* [*iff*]:
psd-lmat (*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*) ⟷ *psd* (*unwrap-lm M*)
⟨*proof*⟩

116 Typed tensor product

Two square matrices with type-level dimensions $'m$, $'p$ tensor to a square matrix with type-level dimension $'m \times 'p$.

lemma *tensor-mat-carrier-lmat*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat* (*CARD*(*'m* :: *finite*)) (*CARD*(*'m*))
and *B*: *B* ∈ *carrier-mat* (*CARD*(*'p* :: *finite*)) (*CARD*(*'p*))
shows *A* ⨂ *B* ∈ *carrier-mat* (*CARD*(*'m* × *'p*)) (*CARD*(*'m* × *'p*))
⟨*proof*⟩

definition *tensor-lmat* ::
(*complex*, *'m* :: *finite*, *'m*) *lmat* ⇒ (*complex*, *'p* :: *finite*, *'p*) *lmat*
⇒ (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat* **where**
tensor-lmat σ τ = *wrap-lm* (*unwrap-lm σ* ⨂ *unwrap-lm τ*)

lemma *unwrap-lm-tensor-lmat* [*simp*]:
fixes *σ* :: (*complex*, *'m* :: *finite*, *'m*) *lmat*
and *τ* :: (*complex*, *'p* :: *finite*, *'p*) *lmat*
shows *unwrap-lm* (*tensor-lmat σ τ* :: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*)
= *unwrap-lm σ* ⨂ *unwrap-lm τ*
⟨*proof*⟩

117 The separable cone

The set of *separable matrices*: all nonneg combinations of tensor products of typed PSD matrices. Equivalently, the convex conic hull of the product PSD set.

definition *separable-lmat-gen* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*,
'm × *'p*) *lmat set* **where**
separable-lmat-gen =
{*tensor-lmat σ τ* | *σ τ*.
(*psd-lmat σ* :: *bool*) ∧ (*psd-lmat τ* :: *bool*)}

definition *separable-cone-lmat* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*,
'm × *'p*) *lmat set* **where**
separable-cone-lmat = *convex-cone hull separable-lmat-gen*

Sanity: the cone contains 0, is convex, and is conic.

lemma *separable-cone-lmat-convex-cone*:
convex-cone (*separable-cone-lmat*
:: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*)
⟨*proof*⟩

lemma *zero-in-separable-cone-lmat*:
(*0* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*)
∈ *separable-cone-lmat*
⟨*proof*⟩

118 Step 1: PSD set is closed

Sub-lemmas for the closedness of PSD: the Hermitian half and the nonneg-quadratic-form half.

The entry-projection $M \mapsto$ *unwrap-lm M $$ (i, j)* is continuous on lmat for valid index (i, j).

lemma *isCont-unwrap-lm-index*:
fixes *x* :: (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
assumes *ij*: *i* < *CARD*('m) *j* < *CARD*('n)
shows *isCont* (λ*M*. *unwrap-lm M $$ (i, j)*) *x*
⟨*proof*⟩

lemma *continuous-on-unwrap-lm-index*:
fixes *S* :: (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
assumes *ij*: *i* < *CARD*('m) *j* < *CARD*('n)
shows *continuous-on S* (λ*M*. *unwrap-lm M $$ (i, j)*)
⟨*proof*⟩

lemma *closed-entry-hermitian-eq*:
fixes *i j* :: *nat*
assumes *ij*: *i* < *CARD*('k :: *finite*) *j* < *CARD*('k)
shows *closed* {*M* :: (*complex*, *'k*, *'k*) *lmat*.
unwrap-lm M $$ (i, j) = *cnj* (*unwrap-lm M $$ (j, i)*)}
⟨*proof*⟩

lemma *closed-hermitian-unwrap-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *hermitian* (*unwrap-lm M*)}
⟨*proof*⟩

lemma *closed-nonneg-quad-form-lmat*:
fixes *x* :: *complex Matrix.vec*
assumes *xdim*: *dim-vec x* = *CARD*('k :: *finite*)
shows *closed* {*M* :: (*complex*, *'k*, *'k*) *lmat*.
Re (⟨*x* | *unwrap-lm M* ∗$_v$ *x*⟩) ≥ *0*}
⟨*proof*⟩

lemma *closed-psd-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *psd-lmat M*}
⟨*proof*⟩

119 Trace on lmat

definition *Tr-lmat* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* ⇒ *complex* **where**
Tr-lmat M = (∑ *i* = *0*..<*CARD*('k). *unwrap-lm M $$ (i, i)*)

lemma *continuous-on-Tr-lmat*:
fixes *S* :: (*complex*, *'k* :: *finite*, *'k*) *lmat set*
shows *continuous-on S Tr-lmat*
⟨*proof*⟩

lemma *Tr-lmat-add*:
fixes *M N* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
shows *Tr-lmat* (*M* + *N*) = *Tr-lmat M* + *Tr-lmat N*
⟨*proof*⟩

lemma *Tr-lmat-scaleR*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat* **and** *a* :: *real*
shows *Tr-lmat* (*a* ∗$_R$ *M*) = *a* ∗$_R$ *Tr-lmat M*

⟨*proof*⟩

lemma *Tr-lmat-zero* [*simp*]:
Tr-lmat (*0* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*) = *0*
⟨*proof*⟩

lemma *Tr-lmat-sum-scaleR*:
fixes *T* :: *'a set* **and** *u* :: *'a* ⇒ *real*
and *f* :: *'a* ⇒ (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *finite T*
shows *Tr-lmat* (∑ *t* ∈ *T*. *u t* ∗*R f t*) = (∑ *t* ∈ *T*. *u t* ∗*R Tr-lmat* (*f t*))
⟨*proof*⟩

lemma *closed-unit-trace-lmat*:
closed {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *Tr-lmat M* = *1*}
⟨*proof*⟩

120 Compactness of the unit-trace PSD slice

For PSD σ with *Tr* σ = *1*: ${\sigma^2}_{HS} = \sum {\lambda_i}^2 \leq (\sum \lambda_i)^2$ = *1* where λ_i are the (nonneg) eigenvalues.

lemma *psd-entry-bound-by-diag*:
fixes *M* :: *complex Matrix.mat*
assumes *Mcar*: *M* ∈ *carrier-mat n n*
and *Mpsd*: *psd M*
and *i*: *i* < *n* **and** *j*: *j* < *n*
shows (*norm* (*M* $$ (*i*, *j*)))^2 ≤ *Re* (*M* $$ (*i*, *i*)) ∗ *Re* (*M* $$ (*j*, *j*))
⟨*proof*⟩

lemma *norm-lmat-le-of-psd-unit-trace*:
fixes *M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
assumes *psd*: *psd-lmat M* **and** *tr*: *Tr-lmat M* = *1*
shows *norm M* ≤ *1*
⟨*proof*⟩

lemma *compact-unit-trace-psd-lmat*:
compact {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*. *psd-lmat M* ∧ *Tr-lmat M* = *1*}
⟨*proof*⟩

121 Tensor continuity

The typed tensor product is jointly continuous in both factors.

lemma *continuous-on-tensor-lmat*:
fixes *T* :: ((*complex*, *'m* :: *finite*, *'m*) *lmat*
× (*complex*, *'p* :: *finite*, *'p*) *lmat*) *set*
shows *continuous-on T* (λ(*σ*, *τ*). *tensor-lmat σ τ*
:: (*complex*, *'m* × *'p*, *'m* × *'p*) *lmat*)
⟨*proof*⟩

122 The unit-trace product slice

definition *Sprime* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat set*
where
Sprime =
{*ϱ*. ∃(*σ* :: (*complex*, *'m*, *'m*) *lmat*) (*τ* :: (*complex*, *'p*, *'p*) *lmat*).
ϱ = *tensor-lmat σ τ*

$\wedge$ *psd-lmat* $\sigma \wedge$ *Tr-lmat* $\sigma = 1$
$\wedge$ *psd-lmat* $\tau \wedge$ *Tr-lmat* $\tau = 1\}$

lemma *Sprime-image*:
(*Sprime* :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat set*)
= $(\lambda(\sigma, \tau).$ *tensor-lmat* $\sigma\ \tau)$ ‘
$(\{\sigma$ :: (*complex*, $'m$, $'m$) *lmat*. *psd-lmat* $\sigma \wedge$ *Tr-lmat* $\sigma = 1\}$
$\times \{\tau$ :: (*complex*, $'p$, $'p$) *lmat*. *psd-lmat* $\tau \wedge$ *Tr-lmat* $\tau = 1\})$
⟨*proof*⟩

lemma *compact-Sprime*:
compact (*Sprime* :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat set*)
⟨*proof*⟩

Reindex $\{0..<n*q\}$ via (div, mod) into $\{0..<n\} \times \{0..<q\}$.

lemma *sum-div-mod-split*:
fixes f :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ $'a$ :: *comm-monoid-add*
assumes *q-pos*: $0 < q$
shows $(\sum a = 0..<n.\ \sum b = 0..<q.\ f\ a\ b)$
$= (\sum i = 0..<n * q.\ f\ (i\ div\ q)\ (i\ mod\ q))$
⟨*proof*⟩

lemma *Tr-lmat-tensor*:
fixes σ :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*
and τ :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*
shows *Tr-lmat* (*tensor-lmat* $\sigma\ \tau$ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
= *Tr-lmat* σ $*$ *Tr-lmat* τ
⟨*proof*⟩

lemma *zero-not-in-Sprime*:
(0 :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*) $\notin$ *Sprime*
⟨*proof*⟩

123 Closedness of the separable cone

Proof strategy (uses the typeclass tower from *Lmat-Euclidean*):
Let $S' = \{$*tensor-lmat* $\sigma\ \tau \mid \sigma\ \tau.$ *psd-lmat* $\sigma \wedge$ *psd-lmat* $\tau \wedge$ *Tr* (*unwrap-lm* σ) $= 1 \wedge$ *Tr* (*unwrap-lm* τ) $= 1\}$ be the unit-trace product slice.

1. **S’ is compact.**
   - $\{\sigma$ :: *lmat*. *psd-lmat* $\sigma\}$ is closed (entry-wise lift of the Hermitian condition + entry-wise lift of the quadratic-form nonnegativity, both via *tendsto-unwrap-lm-index*).
   - $\{\sigma$ :: *lmat*. *Re* (*Tr* (*unwrap-lm* σ)) $= 1\}$ is closed (trace is continuous in entries, via *tendsto-unwrap-lm-index*).
   - PSD + unit-trace is bounded: *norm* $\sigma^2 =$ *Re Tr* $(\sigma^\dagger\ \sigma) \leq$ *Tr* $\sigma \cdot \lambda_{max} \leq 1$.
   - Closed + bounded in finite-dimensional *real-inner* = compact (*bounded-closed-imp-seq-compact* + heine_borel).
   - *tensor-lmat* is continuous (entries are bilinear in the entries of the factors, lifts via the typeclass).
   - S' is the image of (compact) $\times$ (compact) under *tensor-lmat*, hence compact.
2. **convex hull S’ is compact** via *compact-convex-hull* (applies because every euclidean_space is a real_inner finite-dimensional space).

3. **0 ∉ convex hull S'.** Every element of *convex hull S'* has trace 1 (linear combinations of trace-1 matrices with nonneg coefficients summing to 1), while 0 has trace 0.

4. **separable_lmat_gen = conic hull S'.** Each *tensor-lmat σ τ* with *σ*, *τ* PSD equals (*Tr σ · Tr τ*) · *tensor-lmat* (*σ*/*Tr σ*) (*τ*/*Tr τ*) when traces are positive, and equals *0* when one trace is 0 (in which case the corresponding factor is itself 0 because PSD + Tr = 0 ⇒ 0).

5. **Putting it together.** *separable-cone-lmat* = *convex-cone hull separable-lmat-gen* = *convex-cone hull* (*conic hull S'*) = *convex-cone hull S'* = *conic hull* (*convex hull S'*) (the last by *convex-cone-hull-separate-nonempty*). Apply *closed-conic-hull* with the (compact, *0* ∉ ...) clause.

The infrastructure for all of these steps is now available (*norm*, *inner*, *dist*, *compact*, *convex*, *convex-cone*, etc. all defined for typed lmat via the euclidean_space + heine_borel instances).

The convex cone hulls of *separable-lmat-gen* and *Sprime* coincide: every *tensor-lmat σ τ* with *σ*, *τ* PSD can be rescaled into *Sprime* by dividing by traces (when nonzero) or replaced by 0 when a trace is zero.

The hard direction depends on three facts that each lift a spectral / bilinearity property:
psd_lmat_scaleR PSD preserved under nonneg real scaling psd_lmat_zero_trace_zero PSD with Tr = 0 is the zero matrix tensor_lmat_scaleR_left,right tensor is bilinear over real scaling psd_lmat_Tr_real_nonneg Tr of PSD is a real nonneg complex.
These reduce *tensor σ τ* with *σ*, *τ* PSD to (*Tr σ* ∗ *Tr τ*) ∗$_R$ *tensor* (*σ*/*Tr σ*) (*τ*/*Tr τ*) when traces are positive, or to 0 when one is zero both forms in *convex-cone hull Sprime*.

lemma *Sprime-subset-separable-lmat-gen*:
(*Sprime* :: (*complex*, 'm :: *finite* × 'p :: *finite*, 'm × 'p) *lmat set*)
⊆ *separable-lmat-gen*
⟨*proof*⟩

The four spectral / bilinearity helpers needed for the hard direction. Each is a one-line lift of a JNF fact, but the JNF facts (spectral closure and tensor scaleR) live outside the parent heap so they are stated as targeted sorries here for the moment.

lemma *psd-lmat-Tr-real-nonneg*:
fixes *M* :: (*complex*, 'k :: *finite*, 'k) *lmat*
assumes *M-psd*: *psd-lmat M*
shows *Im* (*Tr-lmat M*) = *0* ∧ *0* ≤ *Re* (*Tr-lmat M*)
⟨*proof*⟩

lemma *psd-lmat-zero-trace-zero*:
fixes *M* :: (*complex*, 'k :: *finite*, 'k) *lmat*
assumes *M-psd*: *psd-lmat M* **and** *tr-zero*: *Tr-lmat M* = *0*
shows *M* = *0*
⟨*proof*⟩

lemma *unwrap-lm-scaleR-eq*:
fixes *M* :: (*complex*, 'm :: *finite*, 'n :: *finite*) *lmat* **and** *a* :: *real*
shows *unwrap-lm* (*a* ∗$_R$ *M*) = *complex-of-real a* ·$_m$ *unwrap-lm M*
⟨*proof*⟩

lemma *psd-lmat-scaleR-nonneg*:
fixes *M* :: (*complex*, 'k :: *finite*, 'k) *lmat* **and** *a* :: *real*
assumes *M-psd*: *psd-lmat M* **and** *a-nn*: *0* ≤ *a*
shows *psd-lmat* (*a* ∗$_R$ *M*)

⟨*proof*⟩

lemma *tensor-lmat-scaleR-left*:
fixes a :: *real*
and σ :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*
and τ :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*
shows (*tensor-lmat* (a $*_R$ σ) τ
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
$= a$ $*_R$ *tensor-lmat* σ τ
⟨*proof*⟩

lemma *tensor-lmat-scaleR-right*:
fixes a :: *real*
and σ :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*
and τ :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*
shows (*tensor-lmat* σ (a $*_R$ τ)
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*)
$= a$ $*_R$ *tensor-lmat* σ τ
⟨*proof*⟩

lemma *tensor-lmat-zero-left*:
fixes τ :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*
shows (*tensor-lmat* (*0* :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*) τ
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*) = *0*
⟨*proof*⟩

lemma *tensor-lmat-zero-right*:
fixes σ :: (*complex*, $'m$ :: *finite*, $'m$) *lmat*
shows (*tensor-lmat* σ (*0* :: (*complex*, $'p$ :: *finite*, $'p$) *lmat*)
:: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*) = *0*
⟨*proof*⟩

lemma *convex-cone-hull-separable-eq-Sprime*:
(*convex-cone hull*
(*separable-lmat-gen*
:: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat set*))
= *convex-cone hull Sprime*
⟨*proof*⟩

theorem *separable-cone-lmat-closed*:
closed (*separable-cone-lmat*
:: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat set*)
⟨*proof*⟩

124 The lmat inner product as a real-trace inner product

The real inner product on (*complex*, $'m$, $'n$) *lmat* is the real part of the JNF Hilbert-Schmidt inner product on the underlying matrices. For square matrices it is also *Re* (*Tr* ($M^\dagger * N$)).

lemma *inner-lmat-eq-re-hs-inner-prod*:
fixes M N :: (*complex*, $'m$ :: *finite*, $'n$ :: *finite*) *lmat*
shows *inner* M N = *Re* ⟨⟨ *unwrap-lm* M | *unwrap-lm* N ⟩⟩
⟨*proof*⟩

lemma *inner-lmat-eq-re-tr-dagger*:
fixes M N :: (*complex*, $'k$ :: *finite*, $'k$) *lmat*
shows *inner* M N = *Re* (*Tr* ((*unwrap-lm* M)† $*$ *unwrap-lm* N))
⟨*proof*⟩

125 Separating hyperplane for non-separable states

The closedness of the separable cone, combined with HOL-Analysis *separating-hyperplane-closed-point*, yields the geometric Hahn-Banach witness: for every ϱ outside the separable cone there is a linear functional (an HS inner product against some matrix a) that is strictly negative on ϱ and nonneg on every separable element. Because the separable set is a CONE, the strict inequality *inner a x* $>$ *b* on the cone forces $b < 0$ and *inner a x* ≥ 0 on the cone.

theorem *separable-cone-lmat-separating-hyperplane*:
  fixes ϱ :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
  assumes *notin*: $\varrho \notin$ *separable-cone-lmat*
  shows $\exists\, a$ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*.
      inner a $\varrho < 0$
    $\wedge$ ($\forall\, \sigma \in$ *separable-cone-lmat*. $0 \leq$ *inner a* σ)
⟨*proof*⟩

126 Entanglement witness in real-trace form

Reformulating the separating hyperplane via the *inner* $=$ *Re Tr* identity gives a JNF matrix W whose adjoint times ϱ has strictly negative real trace, while $W^\dagger * \sigma$ has nonneg real trace for every σ in the separable cone.

theorem *entanglement-witness-re-tr*:
  fixes ϱ :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
  assumes *notin*: $\varrho \notin$ *separable-cone-lmat*
  shows $\exists\, W$ :: *complex Matrix.mat*.
      $W \in$ *carrier-mat* (*CARD*($'m \times 'p$)) (*CARD*($'m \times 'p$))
    $\wedge$ *Re* (*Tr* (W $*$ *unwrap-lm* ϱ)) < 0
    $\wedge$ ($\forall\, \sigma$ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*.
        $\sigma \in$ *separable-cone-lmat*
        $\longrightarrow$ $0 \leq$ *Re* (*Tr* (W $*$ *unwrap-lm* σ)))
⟨*proof*⟩

127 Hermitian entanglement witness

For ϱ outside the separable cone with hermitian unwrap, the witness can be chosen hermitian. Direct entrywise construction of $W = (A + A^\dagger)/2$ using *Matrix.mat*; we avoid lemmas about *dagger* distributing over sums or scalar multiplication.

theorem *entanglement-witness-hermitian*:
  fixes ϱ :: (*complex*, $'m$ :: *finite* $\times$ $'p$ :: *finite*, $'m \times 'p$) *lmat*
  assumes *notin*: $\varrho \notin$ *separable-cone-lmat*
    and ϱ*-herm*: *hermitian* (*unwrap-lm* ϱ)
  shows $\exists\, W$ :: *complex Matrix.mat*.
      $W \in$ *carrier-mat* (*CARD*($'m \times 'p$)) (*CARD*($'m \times 'p$))
    $\wedge$ *hermitian* W
    $\wedge$ *Re* (*Tr* (W $*$ *unwrap-lm* ϱ)) < 0
    $\wedge$ ($\forall\, \sigma$ :: (*complex*, $'m \times 'p$, $'m \times 'p$) *lmat*.
        $\sigma \in$ *separable-cone-lmat*
        $\wedge$ *hermitian* (*unwrap-lm* σ)
        $\longrightarrow$ $0 \leq$ *Re* (*Tr* (W $*$ *unwrap-lm* σ)))
⟨*proof*⟩

128 Hermitian closure of the separable cone

Every element of *separable-cone-lmat* has hermitian unwrap. Generators *tensor-lmat* σ τ with σ, τ PSD are hermitian, and the hermitian set is closed under + and nonneg real scaling.

lemma *hermitian-zero-mat*:
hermitian (0_m *n n* :: *complex Matrix.mat*)
⟨*proof*⟩

lemma *hermitian-add-mat*:
fixes *P Q* :: *complex Matrix.mat*
assumes *Pcar*: *P* ∈ *carrier-mat n n* **and** *Qcar*: *Q* ∈ *carrier-mat n n*
and *Pherm*: *hermitian P* **and** *Qherm*: *hermitian Q*
shows *hermitian* (*P* + *Q*)
⟨*proof*⟩

lemma *hermitian-smult-real-mat*:
fixes *M* :: *complex Matrix.mat* **and** *c* :: *real*
assumes *Mcar*: *M* ∈ *carrier-mat n n* **and** *Mherm*: *hermitian M*
shows *hermitian* (*complex-of-real c* ·$_m$ *M*)
⟨*proof*⟩

Tensor product of hermitians is hermitian (entrywise via *index-tensor-mat*).

lemma *hermitian-tensor-mat*:
fixes *P Q* :: *complex Matrix.mat*
assumes *Pcar*: *P* ∈ *carrier-mat m m* **and** *Qcar*: *Q* ∈ *carrier-mat p p*
and *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
and *Pherm*: *hermitian P* **and** *Qherm*: *hermitian Q*
shows *hermitian* (*P* ⨂ *Q*)
⟨*proof*⟩

Generators of *separable-cone-lmat* have hermitian unwrap.

lemma *separable-lmat-gen-hermitian*:
fixes σ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes σ ∈ *separable-lmat-gen*
shows *hermitian* (*unwrap-lm* σ)
⟨*proof*⟩

The hermitian set in lmat is a convex cone, hence contains the hull of the generators.

lemma *hermitian-lmat-set-convex-cone*:
convex-cone {*M* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*.
hermitian (*unwrap-lm M*)}
⟨*proof*⟩

theorem *separable-cone-lmat-hermitian*:
fixes σ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes σ ∈ *separable-cone-lmat*
shows *hermitian* (*unwrap-lm* σ)
⟨*proof*⟩

Final form of the entanglement witness theorem: the side condition "σ has hermitian unwrap" is automatic for elements of the separable cone.

theorem *entanglement-witness-hermitian'*:
fixes ϱ :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *notin*: ϱ ∉ *separable-cone-lmat*
and ϱ*-herm*: *hermitian* (*unwrap-lm* ϱ)
shows ∃ *W* :: *complex Matrix.mat*.

$W \in \mathit{carrier\text{-}mat}\ (\mathit{CARD}('m \times 'p))\ (\mathit{CARD}('m \times 'p))$
$\wedge\ \mathit{hermitian}\ W$
$\wedge\ \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \varrho)) < 0$
$\wedge\ (\forall \sigma :: (\mathit{complex}, 'm \times 'p, 'm \times 'p)\ \mathit{lmat}.$
$\sigma \in \mathit{separable\text{-}cone\text{-}lmat}$
$\longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \sigma)))$

⟨*proof*⟩

129 Main separability characterisation

The Horodecki-style witness characterisation: for hermitian ϱ, membership in the separable cone is equivalent to the nonneg-real-trace property holding against every hermitian witness that is itself nonneg-real-trace on the cone.

theorem *separable-iff-witness*:
fixes $\varrho :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat}$
assumes *ϱ-herm*: $\mathit{hermitian}\ (\mathit{unwrap\text{-}lm}\ \varrho)$
shows $\varrho \in \mathit{separable\text{-}cone\text{-}lmat}$
$\longleftrightarrow$
$(\forall W :: \mathit{complex\ Matrix.mat}.$
$W \in \mathit{carrier\text{-}mat}\ (\mathit{CARD}('m \times 'p))\ (\mathit{CARD}('m \times 'p))$
$\wedge\ \mathit{hermitian}\ W$
$\wedge\ (\forall \sigma :: (\mathit{complex}, 'm \times 'p, 'm \times 'p)\ \mathit{lmat}.$
$\sigma \in \mathit{separable\text{-}cone\text{-}lmat}$
$\longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \sigma)))$
$\longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \varrho)))$

⟨*proof*⟩

130 Density-matrix specialisation

For an actual quantum density matrix ϱ (PSD with unit trace), the conditions $\mathit{psd\text{-}lmat}\ \varrho$ and $\mathit{Tr\text{-}lmat}\ \varrho = 1$ imply $\mathit{hermitian}\ (\mathit{unwrap\text{-}lm}\ \varrho)$, so the witness theorem and the characterisation specialise cleanly.

theorem *entanglement-witness-for-density*:
fixes $\varrho :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat}$
assumes *ϱ-psd*: $\mathit{psd\text{-}lmat}\ \varrho$
and *ϱ-tr*: $\mathit{Tr\text{-}lmat}\ \varrho = 1$
and *ϱ-not-sep*: $\varrho \notin \mathit{separable\text{-}cone\text{-}lmat}$
shows $\exists W :: \mathit{complex\ Matrix.mat}.$
$W \in \mathit{carrier\text{-}mat}\ (\mathit{CARD}('m \times 'p))\ (\mathit{CARD}('m \times 'p))$
$\wedge\ \mathit{hermitian}\ W$
$\wedge\ \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \varrho)) < 0$
$\wedge\ (\forall \sigma :: (\mathit{complex}, 'm \times 'p, 'm \times 'p)\ \mathit{lmat}.$
$\sigma \in \mathit{separable\text{-}cone\text{-}lmat}$
$\longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \sigma)))$

⟨*proof*⟩

The witness is also nonneg on every separable density matrix (*Sprime*) since *Sprime* $\subseteq$ *separable-cone-lmat*.

theorem *entanglement-witness-for-density-on-Sprime*:
fixes $\varrho :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat}$
assumes *ϱ-psd*: $\mathit{psd\text{-}lmat}\ \varrho$
and *ϱ-tr*: $\mathit{Tr\text{-}lmat}\ \varrho = 1$
and *ϱ-not-sep*: $\varrho \notin \mathit{separable\text{-}cone\text{-}lmat}$
shows $\exists W :: \mathit{complex\ Matrix.mat}.$

$W \in \mathit{carrier\text{-}mat}\ (\mathit{CARD}('m \times 'p))\ (\mathit{CARD}('m \times 'p))$
$\wedge\ \mathit{hermitian}\ W$
$\wedge\ \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \varrho)) < 0$
$\wedge\ (\forall \sigma :: (\mathit{complex}, 'm \times 'p, 'm \times 'p)\ \mathit{lmat}.$
$\sigma \in \mathit{Sprime} \longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \sigma)))$
⟨*proof*⟩

The full Horodecki characterisation, specialised to density matrices.

theorem *separable-density-iff-witness*:
fixes $\varrho :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat}$
assumes *ϱ-psd*: *psd-lmat ϱ*
and *ϱ-tr*: *Tr-lmat ϱ = 1*
shows $\varrho \in \mathit{separable\text{-}cone\text{-}lmat}$
$\longleftrightarrow$
$(\forall W :: \mathit{complex\ Matrix.mat}.$
$W \in \mathit{carrier\text{-}mat}\ (\mathit{CARD}('m \times 'p))\ (\mathit{CARD}('m \times 'p))$
$\wedge\ \mathit{hermitian}\ W$
$\wedge\ (\forall \sigma :: (\mathit{complex}, 'm \times 'p, 'm \times 'p)\ \mathit{lmat}.$
$\sigma \in \mathit{separable\text{-}cone\text{-}lmat}$
$\longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \sigma)))$
$\longrightarrow 0 \leq \mathit{Re}\ (\mathit{Tr}\ (W * \mathit{unwrap\text{-}lm}\ \varrho)))$
⟨*proof*⟩

The trace is identically *1* on the convex hull of *Sprime*.

lemma *Tr-lmat-convex-hull-Sprime-eq-one*:
fixes $\varrho :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat}$
assumes $\varrho \in \mathit{convex\ hull\ Sprime}$
shows *Tr-lmat ϱ = 1*
⟨*proof*⟩

A density matrix is separable iff it is a convex combination of separable pure-product states. This is the standard textbook definition of separability for mixed states.

theorem *separable-density-iff-in-convex-hull-Sprime*:
fixes $\varrho :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat}$
assumes *ϱ-psd*: *psd-lmat ϱ*
and *ϱ-tr*: *Tr-lmat ϱ = 1*
shows $\varrho \in \mathit{separable\text{-}cone\text{-}lmat} \longleftrightarrow \varrho \in \mathit{convex\ hull\ Sprime}$
⟨*proof*⟩

Topology of the separable density-matrix set.

lemma *compact-convex-hull-Sprime*:
compact (*convex hull*
$(\mathit{Sprime} :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat\ set}))$
⟨*proof*⟩

lemma *closed-convex-hull-Sprime*:
closed (*convex hull*
$(\mathit{Sprime} :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat\ set}))$
⟨*proof*⟩

lemma *bounded-convex-hull-Sprime*:
bounded (*convex hull*
$(\mathit{Sprime} :: (\mathit{complex}, 'm :: \mathit{finite} \times 'p :: \mathit{finite}, 'm \times 'p)\ \mathit{lmat\ set}))$
⟨*proof*⟩

Strict separation in the inner-product form: by closedness of *convex hull Sprime* we get a strictly negative-inner-product hyperplane. The Re-trace strict-form is then an immediate corollary once the hermitian witness construction is replayed.

lemma *separable-density-separating-hyperplane*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* ϱ = *1*
and ϱ-*not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows $\exists\, a\ b.$
inner a ϱ < *b*
$\wedge$ $(\forall\, \sigma$ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *convex hull Sprime* $\longrightarrow$ *b* < *inner a* σ)
⟨*proof*⟩

The inner-product-to-Re-trace bridge for hermitian arguments, extracted as a separate lemma so we can reuse it for the strict separation theorem. This is the same hermitisation construction that drives *entanglement-witness-hermitian*.

lemma *hermitian-witness-from-lmat*:
fixes *a* :: (*complex*, $'k$ :: *finite*, $'k$) *lmat*
shows $\exists$ *W* :: *complex Matrix.mat*.
W ∈ *carrier-mat* (*CARD*($'k$)) (*CARD*($'k$))
$\wedge$ *hermitian W*
$\wedge$ $(\forall$ *M* :: (*complex*, $'k$, $'k$) *lmat*.
hermitian (*unwrap-lm M*)
$\longrightarrow$ *Re* (*Tr* (*W* ∗ *unwrap-lm M*)) = *inner a M*)
⟨*proof*⟩

The strict Re-trace form of the entanglement witness.

theorem *entanglement-witness-strict*:
fixes ϱ :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes ϱ-*psd*: *psd-lmat* ϱ
and ϱ-*tr*: *Tr-lmat* ϱ = *1*
and ϱ-*not-sep*: $\varrho \notin$ *separable-cone-lmat*
shows $\exists$ (*W* :: *complex Matrix.mat*) *c*.
W ∈ *carrier-mat* (*CARD*($'m$ × $'p$)) (*CARD*($'m$ × $'p$))
$\wedge$ *hermitian W*
$\wedge$ *Re* (*Tr* (*W* ∗ *unwrap-lm* ϱ)) < *c*
$\wedge$ $(\forall\, \sigma$ :: (*complex*, $'m$ × $'p$, $'m$ × $'p$) *lmat*.
$\sigma \in$ *convex hull Sprime*
$\longrightarrow$ *c* < *Re* (*Tr* (*W* ∗ *unwrap-lm* σ)))
⟨*proof*⟩

end